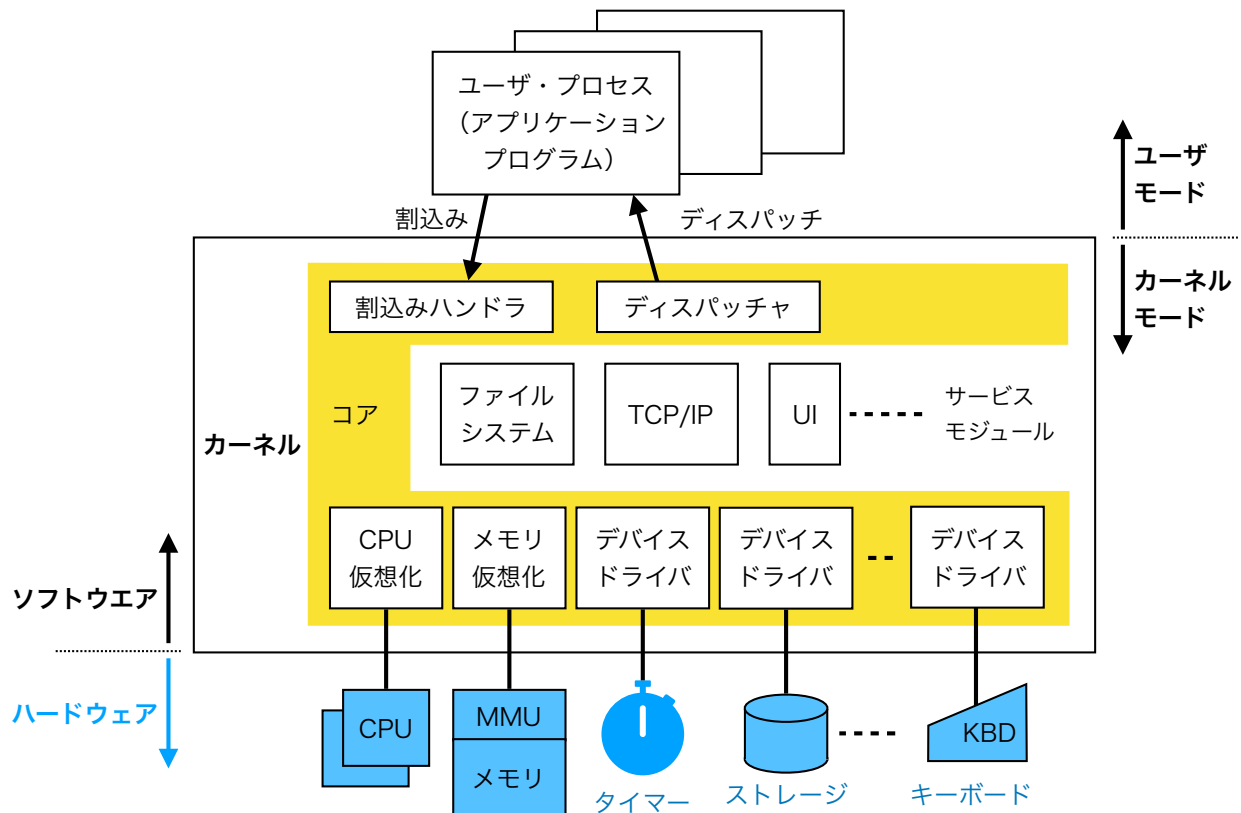


オペレーティングシステム Operating System



オペレーティングシステム (PDF 版)
初版 (Ver. 1.0.13)

徳山工業高等専門学校
情報電子工学科

<https://github.com/tctsigemura/OSTextBook/>

Copyright © 2018 - 2024 by
Dept. of Computer Science and Electronic Engineering,
Tokuyama College of Technology, JAPAN

本書は JSPS 科研費 22500833 および 16K0099 の助成を受けて作成しました.

本書は CC-BY-SA 4.0 ライセンスによって許諾されています。

本書の最新版は，以下からダウンロード可能です.

<https://github.com/tctsigemura/OSTextBook/blob/master/os.pdf>

本書の講義用スライドは，以下からダウンロード可能です.

<https://github.com/tctsigemura/OSTextBook/tree/master/Sld>

本書は CC-BY-SA 3.0 de, CC-BY-SA 4.0 ライセンスで許諾された著作物を含みます.

関係ライセンスの全文は,

<https://creativecommons.org/licenses/by-sa/3.0/de/> (CC-BY-SA 3.0 de),

<https://creativecommons.org/licenses/by-sa/4.0/deed.ja> (CC-BY-SA 4.0)

で確認できます.

本書を授業などにご使用の節は，次のアドレスにご連絡いただければ幸いです.

sigemura@tokuyama.kosen-ac.jp

目次

第 I 部	OS とは	1
第 1 章	オペレーティングシステムとは	3
1.1	オペレーティングシステムの役割	3
1.2	オペレーティングシステムの歴史	5
1.3	まとめ	12
第 2 章	前提知識	15
2.1	コンピュータのハードウェア構成	15
2.2	CPU の構成	17
2.3	最近のコンピュータの実際の構成	17
2.4	割込み	18
2.5	オペレーティングシステムの構造	20
2.6	カーネルの構成方式	22
2.7	もう一つの仮想マシン	24
2.8	実装例	25
2.9	まとめ	25
第 II 部	CPU 管理	27
第 3 章	CPU の仮想化	29
3.1	時分割多重	29
3.2	プロセスの状態	30
3.3	プロセスの切換え (コンテキストスイッチ)	31
3.4	PCB (Process Control Block)	35
3.5	スレッド (Thread)	37
3.6	CPU 仮想化の実装例	42
3.7	まとめ	42

第 4 章	CPU スケジューリング	45
4.1	評価基準	45
4.2	システムごとの目標	46
4.3	プロセスの振舞	47
4.4	スケジューリング方式	48
4.5	スケジューラの実装例	52
4.6	まとめ	52
第 5 章	プロセス同期	53
5.1	競合 (Race Condition, Competition)	53
5.2	クリティカルセクション (Critical Section)	54
5.3	相互排除 (Mutual Exclusion)	54
5.4	セマフォ (Semaphore)	58
5.5	セマフォの実装例	65
5.6	まとめ	65
第 6 章	プロセス間通信	67
6.1	共有メモリ	67
6.2	メッセージ通信	72
6.3	メッセージ通信機構の実装例	78
6.4	まとめ	78
第 7 章	モニタ	79
7.1	概要	79
7.2	構成要素	79
7.3	相互排除問題の解	80
7.4	生産者と消費者問題の解	81
7.5	Java のセマフォクラスによるモニタの実装	84
7.6	Java のモニタ風機構による生産者と消費者問題の解	87
7.7	まとめ	89
第 III 部	メモリ管理	91
第 8 章	主記憶 (メモリ)	93
8.1	ハードウェア構成	93
8.2	メモリ保護機構	94
8.3	プログラムの再配置	96
8.4	アドレス空間の仮想化	98
8.5	まとめ	100

第 9 章	メモリ割付け方式	101
9.1	固定区画方式	101
9.2	可変区画方式	102
9.3	可変区画方式の空き領域選択方式	103
9.4	空き領域の管理方式	104
9.5	実装例	106
9.6	まとめ	106
第 10 章	セグメンテーション	107
10.1	リロケーションレジスタ方式の問題点	107
10.2	セグメント	107
10.3	セグメント番号	108
10.4	セグメンテーション機構	109
10.5	セグメンテーション機構による仮想記憶	111
10.6	セグメントの共用	112
10.7	セグメンテーションの利点・欠点	112
10.8	まとめ	114
第 11 章	ページング	115
11.1	基本概念	115
11.2	ページング機構	117
11.3	ページの共用	120
11.4	ページテーブルの編成方法	121
11.5	まとめ	126
第 12 章	仮想記憶	129
12.1	基本概念	129
12.2	デマンドページング	130
12.3	Copy on Write	133
12.4	メモリマップドファイル	134
12.5	ページ置き換えアルゴリズム	139
12.6	フレーム割り付けアルゴリズム	144
12.7	まとめ	144
第 IV 部	ファイル管理	147
第 13 章	二次記憶装置（ストレージ）	149
13.1	記憶装置の階層	149
13.2	接続方式	149

13.3	記憶媒体	150
13.4	ハードディスク	151
13.5	フォーマット	153
13.6	ブートストラップ	155
13.7	実装例	156
13.8	まとめ	156
第 14 章	ファイルシステムの概念	159
14.1	ファイルの名前付け	159
14.2	ファイルの別名	160
14.3	ボリュームのマウント	162
14.4	ファイルの属性	163
14.5	アクセス制御	164
14.6	ファイルの種類	164
14.7	ファイルシステムの操作	165
14.8	ファイルシステムの健全性	167
14.9	まとめ	169
第 15 章	FAT ファイルシステム	171
15.1	特徴	171
15.2	ボリューム内部の配置	172
15.3	ディレクトリエントリ	173
15.4	FAT (File Allocation Table)	174
15.5	ディレクトリファイル	175
15.6	FAT ファイルシステムの全体像を示す例	175
15.7	実装例	177
15.8	まとめ	178
第 16 章	UNIX ファイルシステム	179
16.1	概要	179
16.2	ボリューム内部の配置	179
16.3	i-node (index node)	180
16.4	ディレクトリファイル	182
16.5	パス名と i-node の対応付け	183
16.6	まとめ	184
第 17 章	ZFS	187
17.1	特徴	187
17.2	ZFS のソフトウェア構成	189

17.3	ストレージプールの構造 (概要)	190
17.4	ストレージプールの更新	191
17.5	ストレージプールの構造	192
17.6	スナップショットとクローン	195
17.7	まとめ	198
 第 V 部 TacOS の実装例		 201
第 18 章 TaC と TacOS		203
18.1	ハードウェア構成	204
18.2	TacOS	205
18.3	まとめ	206
 第 19 章 TacOS の CPU 仮想化		 207
19.1	PCB	207
19.2	実行可能列	209
19.3	メモリ配置	210
19.4	割込み処理	211
19.5	プロセス切換えプログラム	212
19.6	スケジューラ	214
19.7	まとめ	216
 第 20 章 TacOS のセマフォ		 217
20.1	データ構造	217
20.2	使用例	218
20.3	割当て	219
20.4	P 操作ルーチン	221
20.5	V 操作ルーチン	222
20.6	setPri() 関数	224
20.7	まとめ	224
 第 21 章 TacOS のメッセージ通信		 227
21.1	メッセージ通信機構	227
21.2	リンク構造体	228
21.3	リンクの作成	228
21.4	サーバ用のメッセージ通信ルーチン	228
21.5	サーバプロセスの例	229
21.6	クライアント用のメッセージ通信ルーチン	230
21.7	クライアントプロセスの例	231

21.8	まとめ	232
第 22 章	TacOS のメモリ管理	233
22.1	データ構造の初期化	233
22.2	メモリの割り付け	235
22.3	メモリの解放	236
22.4	まとめ	237
第 23 章	TacOS のファイルシステム	239
23.1	ファイルシステムサーバ	239
23.2	fs クラス	239
23.3	fatSys クラス	242
23.4	File クラス (file クラス)	246
23.5	dirAccess クラス	247
23.6	blkFile クラス	249
23.7	mmcsapi クラス	254
23.8	まとめ	256
第 VI 部	資料と文献	257
付録 A	TaC に関する資料	259
A.1	CPU の概要	259
A.2	メモリマップと I/O マップ	262
付録 B	TacOS のファイルフォーマット	265
B.1	.o 形式ファイル	265
B.2	.exe 形式ファイル	267
B.3	.bin 形式ファイル	268
参考文献		271

第 I 部

OS とは

第 1 章

オペレーティングシステムとは

オペレーティングシステム (Operating System : OS) は, Windows, macOS, Linux, FreeBSD, Android, iOS 等である. 皆さんは, これらを使用した経験を持っているだろう. そして, これらが次のソフトウェアから構成されていることを何となく感じているのではないだろうか.

1. カーネル (OS の本体)
2. ライブラリ (プログラムが使用するサブルーチン, DLL)
3. ユーザインタフェース (GUI, CLI)
4. ユーティリティソフトウェア (ファイル操作, 時計, シェル, システム管理 ...)
5. プログラム開発環境 (エディタ, コンパイラ, アセンブラ, リンカ, インタプリタ)

広義では上に列挙した全て^{*1}がオペレーティングシステムの一部である. 逆に狭義では「カーネル」だけをオペレーティングシステムと考える. 本書では狭義のオペレーティングシステムの仕組みを勉強する.

1.1 オペレーティングシステムの役割

オペレーティングシステムの重要な役割は次に述べる二つである.

1.1.1 拡張マシンとしてのオペレーティングシステム

OS はハードウェアの機能を抽象化した便利な拡張マシンを提供する. 次に抽象化と拡張マシンの例を示す.

例 1 二次記憶装置の抽象化 (ファイルシステム)

ハードディスク, USB メモリ, CD-ROM 等の二次記憶装置は, どれもデータを記録する機能を持ったハードウェアである. しかし, それらの制御方法や記録されるデータの構造は全く異なる. オペレーティングシステムは, 二次記憶装置をファイルの集合 (ファイルシステム) として抽象化してユーザプログラム (アプリケーションプログラム) に提供する.

例 2 コンピュータそのものの抽象化 (プロセス)

^{*1} 上に挙げたソフトウェアの中で「プログラム開発環境」は, Linux や FreeBSD では OS に含まれているが, それ以外では別にインストールする必要がある OS の一部とは言い難くなっている.

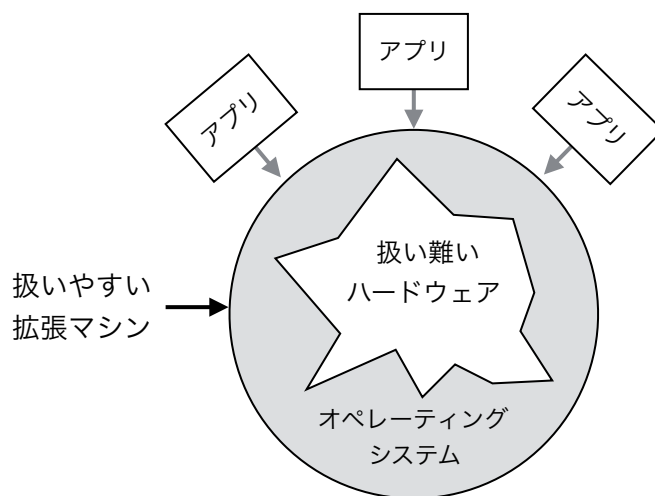


図 1.1: 抽象化

プロセスはプロセス専用の仮想 CPU と仮想メモリを持つ。システムコールを通じて入出力も可能である。プロセスは CPU、メモリ、入出力を持っているので1台のコンピュータと考えることもできる。プロセスはコンピュータを**抽象化**したものとも言える。（プロセス＝仮想コンピュータ）

例 3 拡張されたコンピュータ（システムコール）

オペレーティングシステムを備えたコンピュータ上では、アプリケーションプログラムがシステムコールを発行できる。システムコールを追加命令を考えると、オペレーティングシステムを備えたコンピュータは追加命令を実行可能な**拡張マシン**だと言える。（拡張マシンの命令＝機械語命令＋システムコール）

オペレーティングシステムが拡張マシンをアプリケーションプログラムに提供するイメージを図 1.1 に示す。ハードウェアの複雑で統一されていない凸凹のインタフェースは、オペレーティングシステムによってスッキリした円弧のインタフェース（使いやすい**抽象化**されたインタフェース）に変換される。

オペレーティングシステムの円がハードウェアの外側にあるのは、オペレーティングシステムによって機能が拡張されたことを示す。ハードウェアを含む円全体が拡張マシンを表している。

1.1.2 ハードウェア管理プログラムとしてのオペレーティングシステム

オペレーティングシステムはハードウェア資源を管理・制御し、アプリケーションプログラムにシステムコール等のサービスを提供する。図 1.2 はカーネルの役割を説明している。

オペレーティングシステムは、管理するハードウェア資源をアプリケーションプログラムに割り当てる。複数のアプリケーションプログラムに割り付けるために資源を**仮想化**して必要な数だけ作り出す。例えば、CPU は時間を区切って複数のプロセスが共有する（**時分割多重**による**仮想化**）。メモリはアドレスで区切って複数のプロセスが共有する（**空間分割多重**による**仮想化**）。



図 1.2: コンピュータシステムの構成

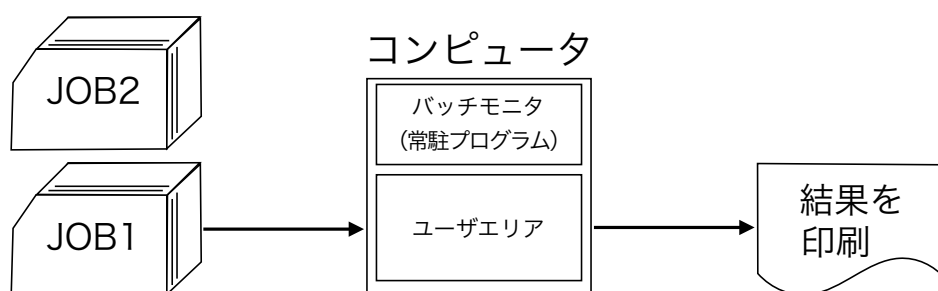


図 1.3: バッチ処理

1.2 オペレーティングシステムの歴史

1.2.1 第1世代（1945～1955，真空管の時代）

初期のコンピュータはコンソールパネルを通して操作する、巨大な TeC[1] のようなものだった。OS は存在せず、プログラマは TeC と同様なプログラミングとデバッグを行っていた。しかし、当時のコンピュータは TeC と異なり大変高価な装置であった。その高価なコンピュータを一人のプログラマが長時間にわたって独占使用することになる。プログラマがバグの原因を考えている間、とても高価なコンピュータが遊んでしまい勿体ないものであった。

1.2.2 第2世代（1955～1965，トランジスタの時代）

コンピュータがトランジスタ回路で製作されるようになり、**メインフレーム**と呼ばれる大型コンピュータが、大企業、政府機関や大学等で実用的に使用されるようになった。メインフレームは数百万ドルと高価だったので、ハードウェアを遊ばせること無く使用することが優先課題であった。そこで人手を介すること無く自動的に次々と連続して処理を行う「コンピュータの自動運転」が行われるようになった。この処理方式は**バッチ処理**と呼ばれた。図 1.3 にバッチ処理の概要を示す。

プログラマは図 1.4 に示す紙カードにプログラムやデータを一行ずつ打込む。100 行のプログラムは 100 枚の紙カードを使用して記録する。このようにして出来た紙カードの束が一つの処理単位（**ジョブ**）になる。コンピュータでは**バッチモニタ**と呼ばれる常駐プログラムが実行される。バッチモニタは紙カードからジョブを読み込み実行させる。ジョブが終了するとバッチモニタに制御が戻り、次のジョブが自動的に実行される。バッチモニタが発展してやがて OS になる。



ウィキメディア / Bundesarchiv, B 145 Bild-F038812-0014 / Schaack, Lothar / CC-BY-SA 3.0 de

図 1.6: フォルクスワーゲンで使われている System/360

3. システムコール

ユーザプログラムが直接に入出力装置等にアクセスすることは、バッチ処理を継続できなくする恐れがあるので許されない。例えばユーザプログラムがハードウェアのモードを切り換えてしまうと、以降のジョブが正常に実行されなくなる恐れがある。そこで、ユーザプログラムはバッチモニタに依頼（システムコール）して入出力を行う必要がある。

プログラムが終了する時は、カーネルモードに切り換えてバッチモニタに戻る必要がある。カーネルモードに切り替える機械語命令をユーザプログラムが実行可能だと、実行モードが無意味になるので許可すべきではない。システムコールを使用してプログラムを終了する。

4. 記憶保護

ユーザプログラムのバグでバッチモニタが破壊されないように、ユーザモードで実行中は主記憶のバッチモニタ領域に書き込みができないようにする。

1.2.3 第3世代（1966～1980, IC とマルチプログラミングの時代）

1960年代のコンピュータはIC（Integrated Circuit）を用いて作られるようになり価格性能比が随分改善された。第3世代と呼ばれる当時のオペレーティングシステムの中には、現代のオペレーティングシステムの先祖であったり、強い影響を与えたものがある。図 1.13 に第3世代から現代に至るまでの系統図を示す。

IBMが開発した System/360（図 1.6）は、高価で大型のものから安価で小型のものまでで同じオペレーティングシステムが使用でき、同じユーザプログラムを実行できるシリーズ化を行い商業的に大成功をおさめた [28]。System/360 はそれ以前のものとは異なり科学技術計算にも事務処理にも使用できる。System/360 のオペレーティングシステムは、1966年にデビューした OS/360 である。図 1.13 に示すように、OS/360 の子孫である z/OS が現代でも使用されている [2]。

OS/360 を含む第3世代のオペレーティングシステムが実現した重要な新しい機能を紹介する。

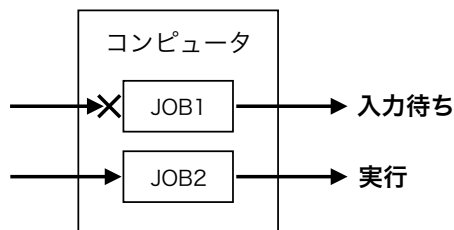


図 1.7: マルチプログラミングシステム

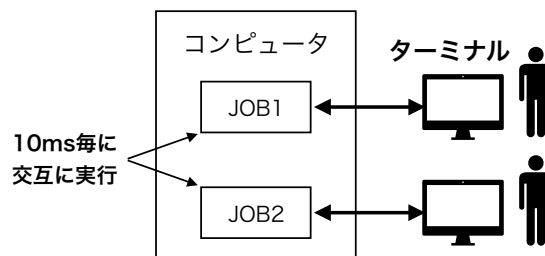


図 1.8: タイムシェアリングシステム



写真: <http://commons.wikimedia.org/wiki/File:Televideo925Terminal.jpg> (パブリックドメイン)

図 1.9: ターミナル

- 仮想記憶

主記憶を仮想化し実際より大きい主記憶があるように見せる。実際の主記憶より大きいプログラムが実行可能になる。

- マルチプログラミング

図 1.7 のように複数のプログラム（ジョブ）を主記憶にロードしておき、その中で実行可能なものを選んで実行する。入出力待ち等で実行できなくなったら他のプログラムを実行する。高価な CPU が入出力待ちで停止する可能性を低くすることができた。

- タイムシェアリング (TSS: Time Sharing System)

マルチプログラミングの一種である。図 1.8 のように、複数のターミナルをコンピュータに接続し複数のユーザが同時にコンピュータを使用できるようにする。短時間（例えば 10ms）で処理するジョブを次々に切り換えることで、ユーザは自分がコンピュータを独占しているように感じることができる。なお、ターミナルは図 1.9 のような、キーボードと表示装置だけを備えた安価な装置である。

この時代のオペレーティングシステムやコンピュータシステム、そして、それらの開発プロジェクトの中で、その後のオペレーティングシステムに多くの影響を与えた有名なものを紹介する。

- OS/360

世界初の本格的な商用オペレーティングシステムである。メインフレームの主流 OS となり子孫は現在でも使用されている [2].

- MULTICS (MULTiplexed Information and Computing Service) プロジェクト [28]

MIT, ベル研究所, General Electric が共同で始めた巨大で強力なコンピュータシステムを構築するプロジェクトである。強力な一台のコンピュータで都市一つ分のコンピュータサービスを提供する構想だった。完成までに長い期間を要し（その間にベルと GE が脱落し）、商業的には失敗であったが、その後のオペレーティングシステムに影響を与える多くのアイデアが出てきた。

- UNIX (ユニックス)

MULTICS プロジェクトから抜けたベル研の Ken Thompson らにより開発された [6]。図 1.13 に示すように、現代のオペレーティングシステムの多くが UNIX を起源にしている。子孫ではないものも UNIX の影響を強く受けている。Linux は UNIX 互換のオペレーティングシステムを作ろうとして開発が始まった [20]。Android の中身は Linux である [21]。z/OS は UNIX 互換環境を備えている [5]。Windows にも UNIX 互換環境 (POSIX サブシステム) を利用可能なものがある [23]。

- Dynabook (ダイナブック：OS だけでなくコンピュータ全体を指す) [32]

アラン・ケイが 1972 年に著した「A Personal Computer for Children of All Ages」[30, 31] に登場する理想のパーソナルコンピュータである。アラン・ケイがゼロックスのパロアルト研究所に在籍中の 1970 年代に開発した Alto 上の「暫定ダイナブック環境」(図 1.10) は既に GUI やマウスを使用していた。スティーブ・ジョブスが Alto を見たことが LISA 開発のきっかけになったと言われている [17]。

1.2.4 第 4 世代 (1980～現代, PC の時代)

1970 年代に単一の LSI に CPU 全体を集積したマイクロプロセッサが登場した。1970 年代中頃にはマイクロプロセッサを用いて個人向けのコンピュータであるパーソナルコンピュータ（当時はマイクロコンピュータと呼んでいた）を作ることが可能になった。それに伴いパーソナルコンピュータ用のオペレーティングシステムが登場した。

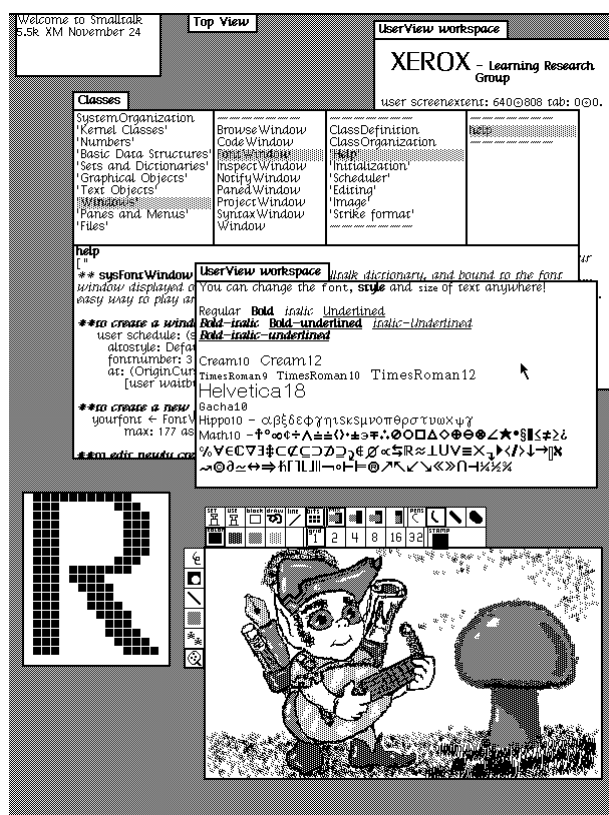
1. 8bit マイクロコンピュータの時代

1977 年に Digital Research 社が CP/M (Control Program for Microcomputer) と呼ばれる 8bit マイクロコンピュータ用の簡単なオペレーティングシステムを開発し成功した。しかしこのオペレーティングは 16bit パーソナルコンピュータの時代には早々に消え去ってしまった [29]。

2. 16bit パーソナルコンピュータの時代

IBM が 1981 年に 16bit パーソナルコンピュータ IBM PC [24] (図 1.11) を発売した。IBM PC は現在の Windows PC の先祖である。IBM PC の子孫は改良や拡張を続けながら現在まで高いシェアを維持し続けている。IBM PC のオペレーティングシステムとして開発されたのが、Microsoft 社の MS-DOS (MicroSoft Disk Operating System) [22] である。バージョン 2 からは UNIX のような階層ディレクトリやパイプ、リダイレクト等の機能を持っている。図 1.13 に示すように、MS-DOS は Windows に置き換わり Windows ME までバージョンアップが繰り返された。

Apple 社は 1984 年に Macintosh (図 1.12) を発売した。Macintosh の OS である Mac OS は



ウィキメディア / SUMIM.ST /

Alto や NoteTaker で動作したアラン・ケイ達の暫定 Dynabook 環境 (Smalltalk-76、同-78 の頃) / CC-BY-SA

4.0

図 1.10: Alto (Alto エミュレータ) のスクリーンショット



ウィキメディア / Bundesarchiv, B 145 Bild-F077948-0006 / Engelbert Reineke / CC-BY-SA 3.0 de

図 1.11: IBM PC



ウィキメディア / w:User:Grm wnr / File:Macintosh 128k transparency.png /GFDL

図 1.12: 初代 Macintosh

LISA を経て Dynabook[30, 31] の影響を受けていると言われている [29]. 図 1.13 に示すように、当初の Mac OS は Mac OS 9[16] まで改良が続けられた。

3. 32bit パーソナルコンピュータの時代

1990 年頃には 32bit のマイクロプロセッサが、パーソナルコンピュータでも使用されるようになった。32bit のマイクロプロセッサは実行モードを備え、またメモリ管理ユニットも利用可能であった。つまり、第3世代の本格的なオペレーティングシステムが必要とするハードウェアが、パーソナルコンピュータでも利用可能になった。

そこで、従来ワークステーションやミニコンで使用されていた UNIX を安価なパーソナルコンピュータ（特に IBM PC 互換機）で動くようにする人たちが現れ、オープンソースソフトウェアとして Linux や FreeBSD 等の開発が始まった。また、もともとパーソナルコンピュータ用の Windows や Mac OS も 32bit マイクロプロセッサの機能を使いこなす本格的なオペレーティングシステムに生まれ変わった。

- Linux

1991 年に開発が始まった Linux は、UNIX 互換のオペレーティングシステムをパーソナルコンピュータ（IBM PC 互換機）用に独自に作成したものである [20]。Linux は改良され続け、現在ではパーソナルコンピュータだけでなく、スーパーコンピュータ「京」のオペレーティングシステム [33] から、スマートフォンのオペレーティングシステムである Android[21]、テレビ等の組込みシステムのオペレーティングシステムまで、広く使われるようになっている。

- BSD 系の UNIX

386BSD[12] は BSD UNIX を Intel 80386 CPU を搭載したパーソナルコンピュータ（IBM PC 互換機）で動作するようにしたものである。386BSD は FreeBSD 等に受継がれるが UNIX のライセンス問題が発生する [6]。ライセンス問題が片付き安心して使用できるようになった 4.4BSD-Lite Release 2[6] をベースに FreeBSD, NetBSD, OpenBSD 等の多くの BSD 系 PC-UNIX が開発された。

その後、FreeBSD は Mac OS X に取り込まれている。また、FreeBSD に ZFS が移植された [34] のでファイルサーバ用に特化した FreeNAS[14] にも使用されている。なお、徳山工業高等専門学校・情報電子工学科のパソコン室では 1993 年 10 月に 386BSD の利用を開始して以来、2014 年 3 月まで FreeBSD を学生用 PC やサーバのオペレーティングシステムとして使用してきた [26]。

- System V 系の UNIX

System V の流れを汲む Solaris[7] は、RISC マイクロプロセッサ SPARC を搭載するサーバやワークステーションでも、パーソナルコンピュータ（IBM PC 互換）でも使用できる。

- 従来のパーソナルコンピュータ用オペレーティングシステム

従来の Windows や Mac OS は CPU の実行モード等を使用していなかったため、アプリケーションプログラムのバグによりシステム全体が停止するトラブルを防ぐことができなかった。そこで、32bit マイクロプロセッサの使用を前提に新しく作り直された。

新しく作り直された 32bit の Windows NT 系列の製品は、徐々に従来の Windows を置換えた。（図 1.13 参照）。現在（2017 年 10 月）の最新版は Windows 10 である。

Mac OS は、2001 年に UNIX の流れを汲み安定して動作する OPENSTEP ベースの Mac OS X[18] に置き換わった（図 1.13 参照）。その後、名称が OS X、macOS と変更されたが、これらは Mac OS X の改良版である。現在（2017 年 10 月）の最新版は macOS 10.13 High Sierra である。iPhone の iOS は Mac OS X をタッチパネル用に再構成したものである [19]。

1.2.5 インターネット世代

現在のオペレーティングシステムは TCP/IP 機構が組み込まれインターネットに接続することができる。今ではパーソナルコンピュータやスマートフォンの使用をインターネット抜きに考えることができない。オペレーティングシステムにとってインターネットに接続できることは重要なことである。

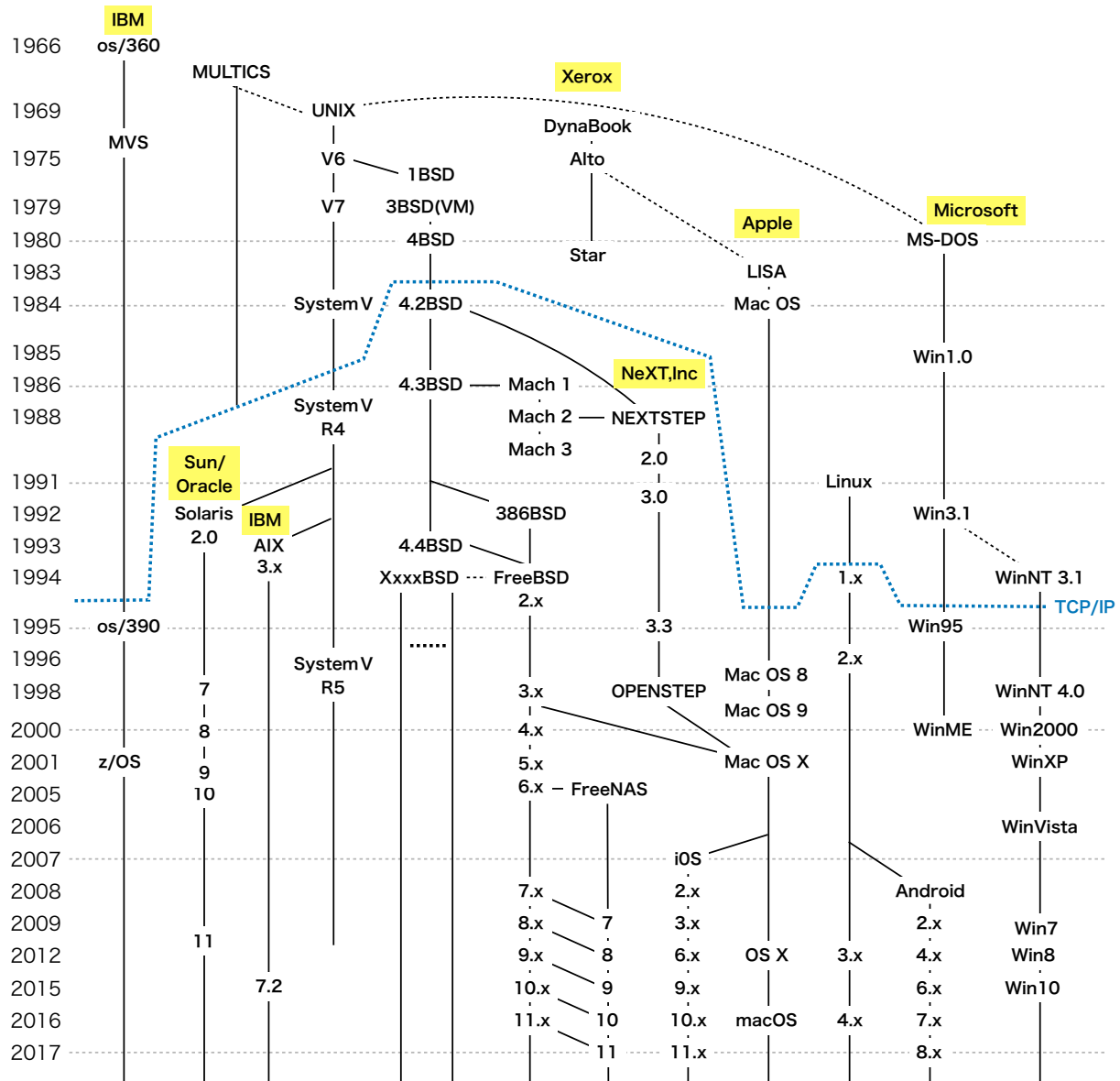
TCP/IP を実装した 4.2BSD が 1984 年に公開された [11]。以来、4.2BSD の子孫はインターネットに対応している。1988 年に公開された System V R4 は BSD 起源の TCP/IP の実装を含んでいた [25]。これの子孫もインターネットに対応している。Linux も 1.0 の頃には TCP/IP の実装を含んでいた [27]。Windows は Windows 95 から TCP/IP を標準装備している [23]。Mac OS は Mac OS 8 が発表されるまでにはインターネット対応がされていた [16]。メインフレームの世界でも OS/390 はインターネットに対応した [4]。

このようにして 1990 年代の後半には多くのオペレーティングシステムがインターネット対応を完了させた。インターネット対応を完了させたオペレーティングシステムを「インターネット世代のオペレーティングシステム」と言うことができる。

1.3 まとめ

狭義のオペレーティングシステムはカーネルのことを指す。本書は狭義のオペレーティングシステムについて述べている。

オペレーティングシステムの重要な役割は、コンピュータの資源を抽象化することと仮想化することである。オペレーティングのユーザは、使いやすい抽象化されたインタフェースを通して資源を利用



系統図は [2, 3, 4, 5, 6, 7, 8, 9, 11, 10, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23] の内容を総合して作成した。

図 1.13: オペレーティングシステムの系統図

できる。また、ユーザは仮想化された資源を必要な数だけ独占して使用することができる。

オペレーティングシステムは、1950 年代に出現したバッチモニタから進化してきた。現在では、スーパーコンピュータから組み込み用コンピュータまで、非常に広い範囲のコンピュータが本格的なオペレーティングシステムを搭載している。

練習問題

1.1 次の言葉の意味を説明しなさい.

- (a) オペレーティングシステム
- (b) カーネル
- (c) 拡張マシン
- (d) 抽象化
- (e) 仮想化
- (f) 時分割多重
- (g) 空間分割多重
- (h) プロセス
- (i) バッチモニタ
- (j) 実行モード
- (k) 記憶保護
- (l) 仮想記憶
- (m) マルチプログラミング
- (n) タイムシェアリング

1.2 抽象化の例をいくつか挙げなさい.

1.3 仮想化の例をいくつか挙げなさい.

1.4 自分がいつも使用しているコンピュータやスマートフォンのオペレーティングシステムの種類を調べなさい.

第 2 章

前提知識

本書で想定しているコンピュータのハードウェアやソフトウェアの構成について解説する。

2.1 コンピュータのハードウェア構成

本書は、コンピュータのハードウェア構成が図 2.1 のようになっていることを前提にしている。複数の CPU（Central Processing Unit）がメモリを共有し、また、全ての CPU は同じ機能を持ち優劣が無い。このような方式を *SMP（対称型マルチプロセッシング：Symmetric Multiprocessing）* と呼ぶ。メモリは CPU だけでなく、I/O コントローラ（図 2.1 ではアダプタやコントローラ）にも共有される。

1. CPU

CPU はコンピュータの頭脳である。図は CPU が二つの構成になっているが、実際は一つの場合も、もっと多い場合もある。

2. メモリ（主記憶装置）

プログラムやデータを記憶し、プログラム実行する際に CPU が直接使用する記憶装置である。

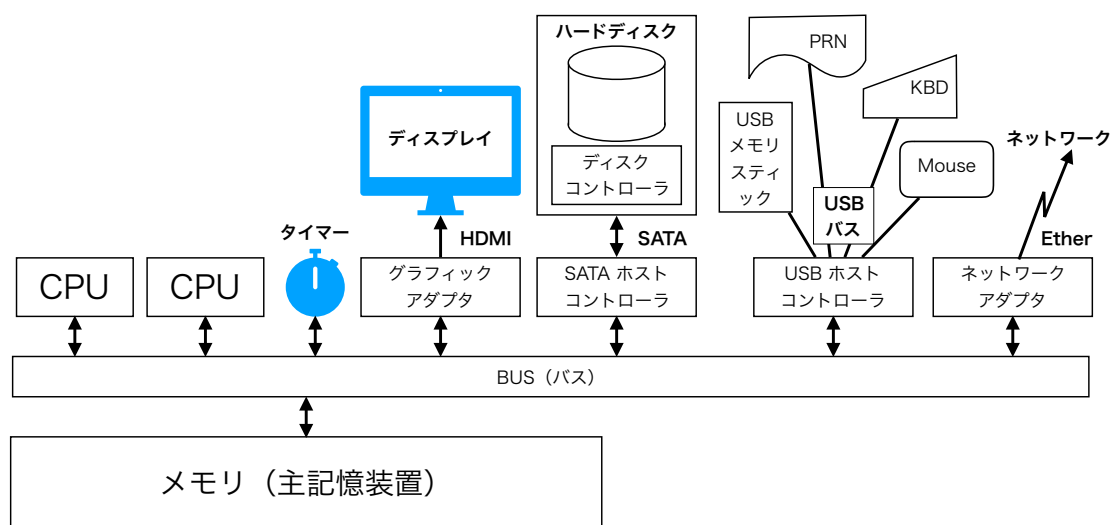


図 2.1: ハードウェア構成

3. タイマー

一定間隔で繰り返し CPU に割込みを発生するインターバルタイマーである。

4. グラフィックアダプタ

ディスプレイを接続するためのアダプタである。表示内容を記憶するメモリを独自に持つ場合と、主記憶装置を使用する場合がある。最近のパーソナルコンピュータでは、グラフィックアダプタに GPU(Graphics Processing Unit) が組込まれている。

5. SATA ホストコントローラ

SATA (Serial Advanced Technology Attachment) は、パーソナルコンピュータと二次記憶装置 (ハードディスクや CD-ROM) を接続するためのインタフェース規格である。SATA ホストコントローラは次のような動作をする。

- (a) CPU が SATA ホストコントローラにコマンドを書き込む。コマンドは、「読み／書き」、「セクタアドレス」、「セクタ数」、「メモリアドレス」を含んだものである。
- (b) SATA ホストコントローラは、ディスクコントローラと通信しハードディスクにコマンドを渡す。
- (c) ハードディスクの読み・書きが可能になったら、ホストコントローラはハードディスクとメモリの間でデータ転送を行う。このような CPU を介さないデータ転送のことを、*DMA (Direct Memory Access)* と呼ぶ。
- (d) SATA ホストコントローラは CPU に割込み信号を送り、データの転送が完了したことを知らせる。(I/O 完了割込み)

CPU は、SATA ホストコントローラにコマンドを送ってから割込みが発生するまでの間、他の仕事を行うことができる。ハードディスクの操作 (I/O 操作) と CPU の計算は並列実行される。

6. USB ホストコントローラ

USB (Universal Serial Bus) は、パーソナルコンピュータと周辺装置を手軽に接続できるインタフェースである。USB メモリスティックやプリンタ、キーボード、マウス等、多くの周辺装置が USB を通して接続できる。USB コントローラも SATA ホストコントローラのように DMA 機能を備えている。

7. ネットワークアダプタ

パーソナルコンピュータのネットワークアダプタは、GbE (Gigabit Ethernet) 規格のものが普及している。これも SATA ホストコントローラのように DMA 機能を備えている。

8. BUS (バス)

パーソナルコンピュータのハードウェアを構成する装置の間でデータをやり取りするための配線である。CPU だけでなく DMA を使用するコントローラやアダプタが大量のデータ転送を行うので、バスのデータ転送能力がパーソナルコンピュータの性能向上のボトルネックになる。

そのため後で説明するように、実際の物理的な接続は図 2.1 とはかなり異なった構成になっている。しかし、オペレーティングシステムが意識しなければならない論理的な接続は図 2.1 のものである。

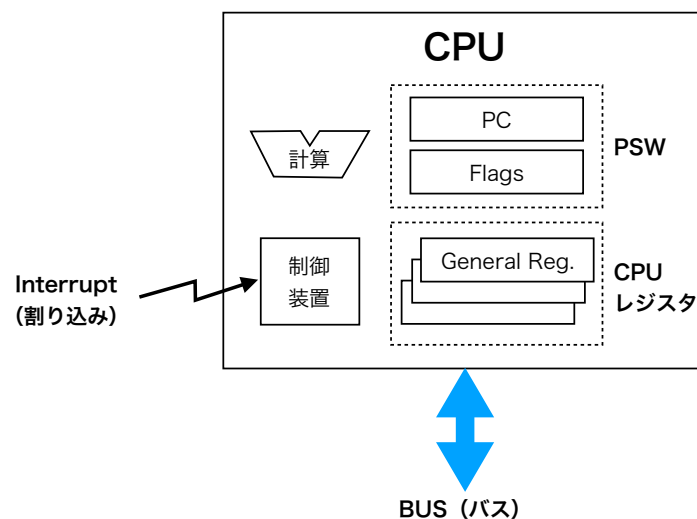


図 2.2: CPU の構成

2.2 CPU の構成

本書では、CPU は図 2.2 の部品で構成され则认为する。図 2.1 に示したように、CPU は BUS を通して他の装置と接続される。CPU は、一つの機械語命令の実行が終わり次の命令の実行を開始する前に、他の装置から割り込みを受け付けることができる^{*1}。

1. PSW (*Program Status Word*)

PSW は、PC (Program Counter) と Flags (フラグ) から構成されるものとする。PC は CPU が実行中のプログラムの命令アドレスを保持するカウンタである。Flags には計算の結果によって変化するビットの他に、割り込み許可／不許可を表現するビット、実行モード (ユーザモード／カーネルモード) を表現するビット等が含まれる。

2. CPU レジスタ

計算に使用する CPU の汎用レジスタのことである。TeC では G0, G1, G2, SP のこと、情報処理技術者試験の COMET では GR0, GR1, GR2, GR3, GR4 のことである。

PSW と CPU レジスタは、機械語命令を実行する度に値が変化・確定しプログラムが意識している^{*2}ので、CPU を仮想化し、実行するプロセスを切替える際に保存・復旧の対象となる。

2.3 最近のコンピュータの実際の構成

Intel 社の CPU を使用したデスクトップ・パーソナルコンピュータとサーバコンピュータの構成を説明する。バスがボトルネックにならないように、CPU とメモリが直接に接続してある。

^{*1} 例外的に、メモリ管理に関する一部の割り込みは機械語命令の途中で発生する。

^{*2} 一方で CPU 内部にはプログラムから見えないレジスタもある。

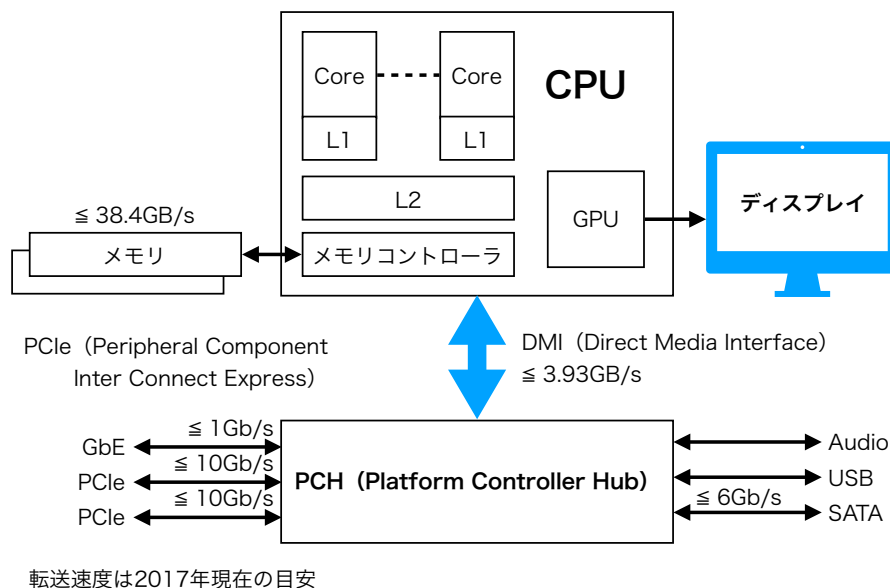


図 2.3: デスクトップ PC の構成

2.3.1 デスクトップ・パーソナルコンピュータ

図 2.3 は Intel 社の CPU を使用した近年のデスクトップ・パーソナルコンピュータの構成を表している。Intel 社の用語では、これまで「CPU」と呼んでいたものを「Core (コア)」と呼ぶ。「CPU」は複数のコアを含んだ LSI のことを指している。デスクトップ・パーソナルコンピュータ用の CPU には 1～4 個のコアが集積されている。

コアに隣接している L1 はレベル 1 キャッシュ (Level 1 cache) を表している。L2 は複数のコアにシェアされるレベル 2 キャッシュ (Level 2 cache) を表している。メモリとのデータ転送量が多い Core と GPU が CPU に集積され、I/O 装置のホストコントローラやアダプタは PCH に集積されている。CPU と PCH は DMI と呼ばれる専用のインタフェースを用いて接続される。

2.3.2 サーバコンピュータ

より強力な処理能力が必要なサーバ用コンピュータでは、図 2.4 のように多くのコアを内蔵する CPU を複数個使用する。現在 (2017 年秋) 最新の Intel Xeon Processor Scalable Family の場合、CPU 同士は UPI と呼ばれる高速な専用インタフェースで接続される。最大の構成は、28 コアの CPU を 8 個使用し合計 224 コアのものである。PCH もサーバ用のものは、より多くのストレージやネットワークを接続できる。

2.4 割込み

通常、コンピュータはユーザ・プロセスを実行し目的の仕事をしている。何かイベントが発生すると割込みにより CPU に通知される。CPU はカーネルモードに切り替わり、カーネル内部の割込みハンドラに制御を移す。CPU がユーザ・プロセスの実行からカーネルの実行に移行するのは、**割込みが発生した時だけ**である。

カーネルへ実行を移すには割込みを発生する以外に方法がない。割込みが発生する原因には以下のも

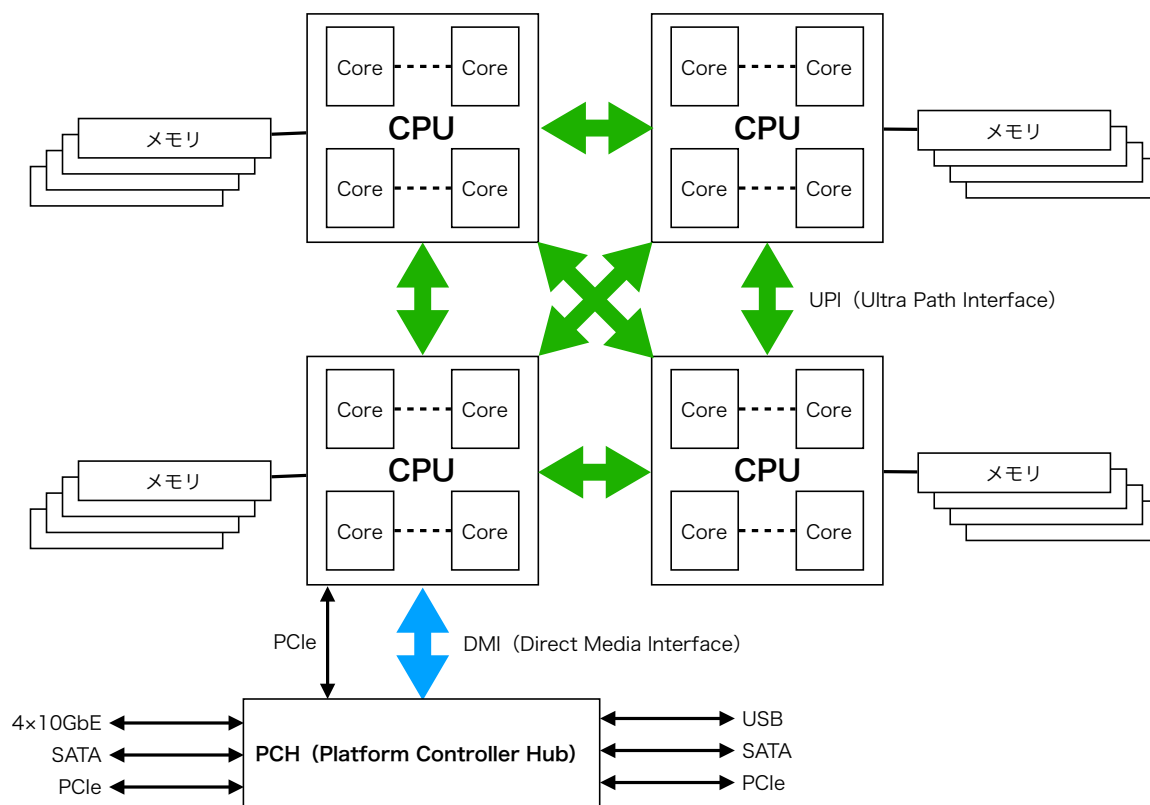


図 2.4: サーバ PC の構成

のがある。システムコール以外はユーザ・プロセスが意図しない間に発生する。

1. I/O 完了・タイマー

ホストコントローラやネットワークアダプタ、タイマー等のハードウェアが、コマンドの実行完了等を CPU に知らせるために発生する。

2. システムコール

ユーザ・プロセスは、割込みを発生する特殊な機械語命令である *SVC (Supervisor Call)* 命令^{*3}を用いてシステムコールを発行する。カーネルは SVC 命令実行時の CPU レジスタの値などからシステムコールの種類やパラメータを知ることができる。

3. 保護違反

ユーザ・プロセスが、ユーザ・モードでは実行が許可されない特権命令を実行したり、アクセスが許可されないメモリ領域をアクセスした場合に発生する。

4. ソフトウェアのエラー

ユーザ・プロセス実行中に計算でオーバーフローが発生した場合等に発生する。

5. ハードウェアのエラー

ハードウェアの故障や電源の異常を検知した時に発生する。

^{*3} CPU によっては TRAP 命令、INT 命令と呼ばれることもある。

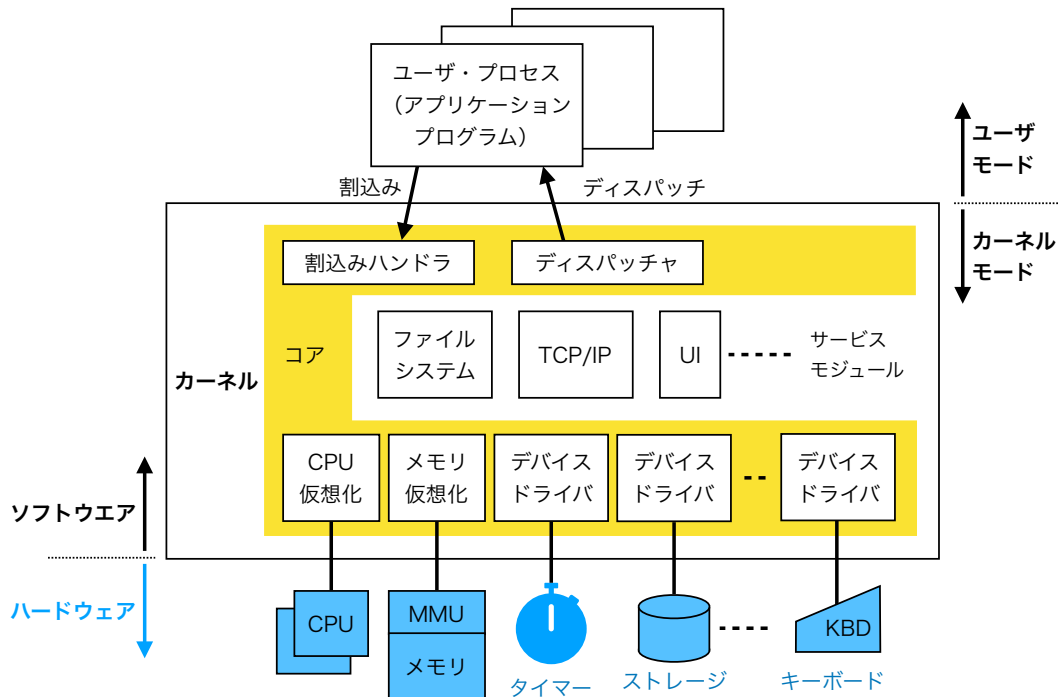


図 2.5: オペレーティングシステムの構造

2.5 オペレーティングシステムの構造

図 2.5 にオペレーティングシステムの構造を示す。カーネルは図 2.5 中央部分のソフトウェアである。ユーザプロセスはユーザモードで、カーネルはカーネルモードで実行される。

2.5.1 カーネルの構成

カーネルは以下のモジュールから構成される。

1. 割込みハンドラ

割込みが発生した時に自動的に実行される割込み処理ルーチンである。割込みが発生した原因を判断し、必要なモジュールを呼び出す。例えば、タイマーからの割込みならタイマーのデバイスドライバを呼び出す。

2. ディスパッチャ

カーネルの処理が終了した時、実行可能なプロセスの中から一つを選んで実行を再開する。

3. コア

コアは、資源の仮想化を行うために、必ずカーネルモードで実行する必要がある。

4. サービスモジュール

サービスモジュールは、ハードウェアを抽象化した便利なコンピュータをユーザ・プロセスに提供するためのプログラムである。

5. デバイスドライバ

ハードウェアを制御するソフトウェアである。ハードウェアの差異を吸収し抽象化したインタ

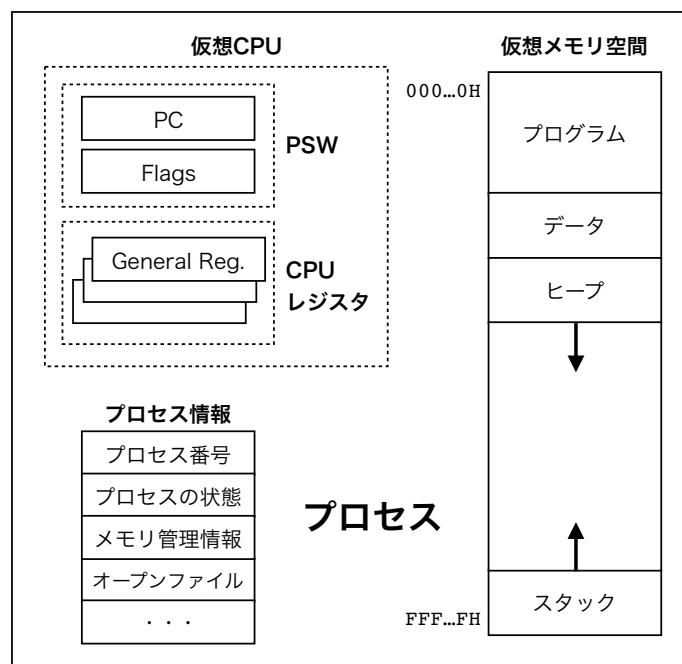


図 2.6: プロセスの構造

フェースをカーネルに提供する。例えば、同じキーボードでも、PS/2、USB などの接続方式や機種などにより制御方法は大幅に異なる。一方でカーネルが必要としている API は一定である。デバイスドライバがキーボードの機種ごとの差異を吸収する。

割込みが発生するとカーネル・モードに切り換わり割込みハンドラに制御が移る。その後、カーネル内では以下の手順で処理がされる。

1. 割込みハンドラは後でプロセスの実行を再開できるように、プロセスの CPU の状態（コンテキスト：PSW，CPU レジスタ）を保存する。
2. 割込みハンドラは割込み原因を調べ、原因に応じたカーネル内のサービスモジュールやデバイスドライバに制御を渡す。例えばファイル操作のシステムコールならファイルシステムへ制御を渡す。
3. サービスモジュールやデバイスドライバの処理が終了したらディスパッチャに制御が渡される。ディスパッチャは実行可能なプロセスの一つを選び、コンテキストを復旧しプロセスの実行を再開する。

2.5.2 プロセスの構造

図 2.5 のユーザ・プロセス部分を詳しく描いたものを図 2.6 に示す。

1. 仮想 CPU

CPU を仮想化し、プロセス毎に CPU が存在するように見せることで、マルチプログラミングを可能にする。プロセスが CPU を使用する時間を区切り、次々に切替える時分割多重により CPU の仮想化は達成される。

他のプロセスが CPU を使用している間に、プロセスのコンテキストを保存する領域を仮想 CPU と呼ぶことにする。ハードウェアの実 CPU に対応して PSW と CPU レジスタの保存先が必要である。前の節で説明したように、プロセスからカーネルに制御が移る時にプロセスのコンテキストを保存する。プロセス実行時にはコンテキストが実 CPU にロードされる。

2. 仮想メモリ空間

メモリを仮想化しプロセス毎に専用のメモリ空間が存在するように見せかける。実現方法は第 III 部「メモリ管理」で詳しく学ぶ。仮想メモリ空間は次の部分から構成される。

(a) プログラム

機械語プログラムがここに配置される。C 言語で記述されたプログラムの場合、関数の実行文（式文、if 文、for 文、while 文など）が翻訳された機械語が該当する。

(b) データ

プログラムの変数部分がここに配置される。C 言語ではグローバル変数が該当する。

(c) ヒープ

プログラム実行時に動的に拡大される領域である。C 言語の `malloc()` 関数はヒープに新しい領域を確保する。`malloc()` 関数が使用される度にヒープ領域は後ろに向かって拡大する。

(d) スタック

プログラム実行時にメモリ空間の最後から前に向かって伸びる領域である。サブルーチン・コール時に戻りアドレスを保存したり、C 言語のローカル変数や関数引数を置いたりする。

3. プロセス情報

名前にあたる「プロセス番号」、実行中／実行可能／待ちのどの状態なのかを表す「プロセスの状態」、使用しているメモリの大きさ等を表す「メモリ管理情報」、CPU を使用した時間を表す「CPU 時間」等の情報のことである^{*4}。その他に、プロセスが現在オープンしているファイルに関する情報や、親プロセス、子プロセス、シグナルハンドラの登録状況、プロセスの優先度など、様々な情報がここに記録される。

2.6 カーネルの構成方式

カーネルが動作不良を起こすと実行中の全てのユーザ・プロセスを巻き込んでシステムが停止するので、カーネルには非常に高い信頼性が要求される。しかし、カーネルは非常に大きなプログラムになりがちであり^{*5}、高い信頼性を確保するにはカーネルの構成方法に工夫が必要である。

2.6.1 単層カーネル（モノリシック・カーネル）

最も一般的な構成方法である。図 2.5 のカーネルは単層カーネルの例になっている。カーネル内の全てのモジュールがリンクされ、一つのプログラムになる。カーネル内でモジュールの呼び出しは CALL 機械語命令を用いて行うので効率が良い。しかし、モジュール同士が密にリンクされているので、モジュール間で情報の隠蔽が難しくバグが入りやすい。また、全てのモジュールがカーネル・モードで実行されるので、一つのモジュールのバグが致命的な結果を引き起こす。Linux や FreeBSD は、この方

^{*4} これらは UNIX の `ps` コマンドで表示することができる。

^{*5} Linux や Windows のカーネルのソースコードは 500 万行にもなる [35]。

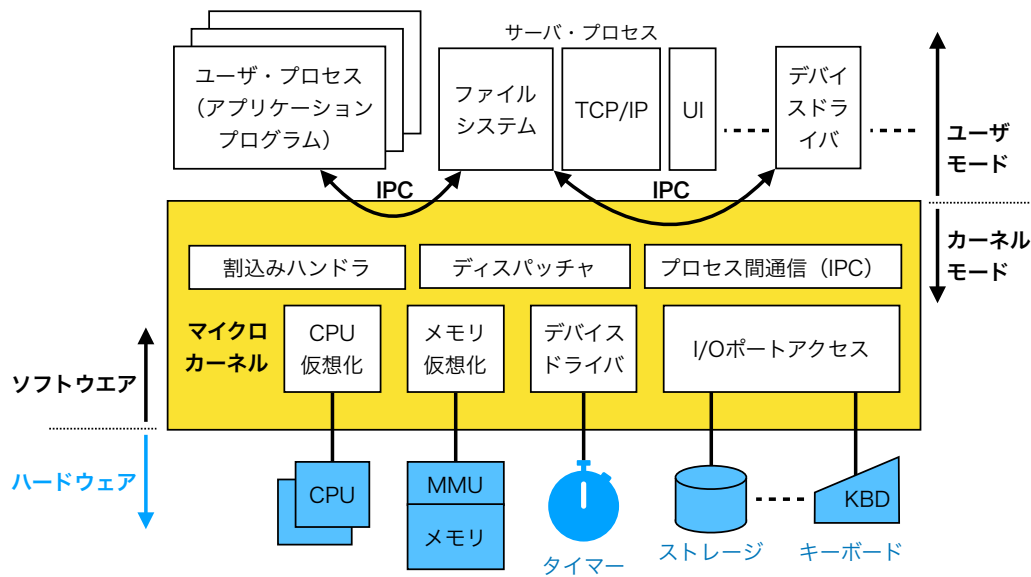


図 2.7: マイクロカーネル方式

式のカーネルを持つ。

2.6.2 マイクロカーネル（micro-kernel）

図 2.5 の「コア」からデバイスドライバを取り除き^{*6}，カーネル（マイクロカーネル）とし構成する方式である。図 2.7 にマイクロカーネル方式の概要を示す。カーネル・モードで実行されるのはマイクロカーネルだけである。

サービスモジュールはカーネルから独立したサーバ・プロセスとし、権限の低いユーザ・モードで実行される。ユーザ・プロセスは、マイクロカーネルが提供する IPC（プロセス間通信：Inter-Process Communication）を用いて、サーバ・プロセスにサービスを要求する。サーバ・プロセス同士、サーバ・プロセスとデバイスドライバ・プロセスも IPC を用いて通信する。

デバイスドライバは I/O ポートにアクセスするのでカーネル・モードで実行される必要があると考えられるが、I/O ポートへのアクセスをマイクロカーネルのシステムコールに置換えることで、デバイスドライバもユーザ・プロセスとして実装することが可能である。この場合は、アクセスしても良い I/O アドレスの範囲内かどうか、マイクロカーネルがチェックすることが可能である。

マイクロカーネル方式は、サービスモジュールやデバイスドライバが権限の低いプロセスとして実行されるので、これらのバグでシステム全体が停止する危険性が低い。また、サービスモジュールやデバイスドライバ毎に独立したプログラムになりモジュール化が徹底しやすいので、巨大な単一プログラムであるモノリシックカーネルと比較してバグが発生しにくい。信頼性の高いオペレーティングシステムを構成するために有利である。しかし、IPC とプロセス切り換えのオーバーヘッドが大きいので性能が低くなる。多くの場合、信頼性と性能はトレードオフの関係にある。

^{*6} タイマーのデバイスドライバは CPU の仮想化に必要なので、マイクロカーネルに残す。

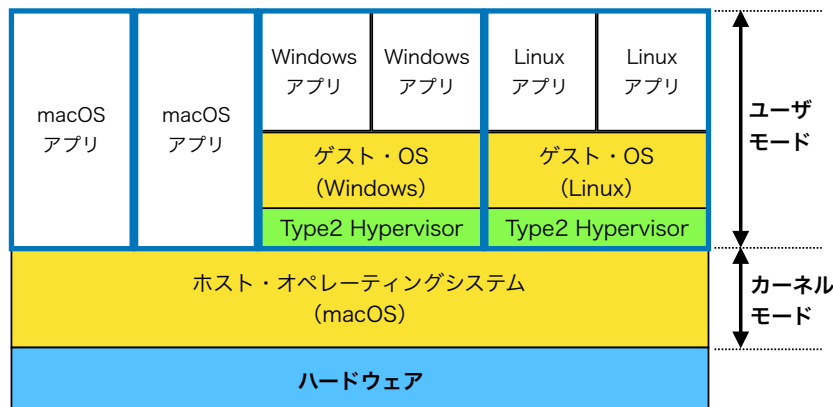


図 2.8: Type 2 ハイパーバイザ

2.7 もう一つの仮想マシン

1.1 で述べたように、オペレーティングシステムは抽象化され便利な拡張マシン（仮想マシン）を、必要な数だけ提供する。ここで述べた仮想マシンは、単一のユーザ・プロセスを実行する環境のことである。同じ「仮想マシン」と言う用語が、オペレーティングシステムを実行することが可能な、よりハードウェアを忠実に再現した仮想マシンを指す場合もある。この節では、一台のコンピュータ上で複数のオペレーティングシステムを実行可能な、もう一つの仮想マシンについて紹介する。

2.7.1 Type 2 ハイパーバイザ

例えば、Mac を使用している人が Windows でしか動作しないアプリケーションを使用する場合を想像してしてみる^{*7}。予め Mac のハードディスクに macOS とは別に Windows もインストールしておき、電源投入時に macOS と Windows を選んでブートする方法もあるが、オペレーティングシステムを切替える度にコンピュータを再起動するのは不便である。また、macOS のアプリケーションと Windows のアプリケーションを同時に実行したい場合もある。

そこで、図 2.8 に示す「Type 2 ハイパーバイザ (Type 2 Hypervisor)」を用いた仮想化が用いられる。ハイパーバイザはホスト・オペレーティングシステムの一つのユーザプロセスとして実行され、コンピュータ一台の機能をエミュレーションする。ハイパーバイザがエミュレーションするコンピュータの中で、ゲスト・オペレーティングシステムが稼働する。エミュレーションはソフトウェアだけで完全に行うのではなく^{*8}、ハードウェアの支援を受けて行うので高速に行うことができる [36]。Type 2 ハイパーバイザとして有名は製品は、VMware Workstation, VMware Fusion, VirtualBox^{*9}等である。

2.7.2 Type 1 ハイパーバイザ

メインフレーム上で 1960 年代から使用されている方式である。現在では PC サーバの仮想化にも使用されている。Type 1 ハイパーバイザはホスト・オペレーティングシステム無しにハードウェア上で

^{*7} 徳山高専情報電子工学科のパソコン室では、Windows や Linux でしか動作しない Xilinx ISE WebPACK を Mac で使用している。

^{*8} 完全にソフトウェアで行う場合もある。

^{*9} 徳山高専情報電子工学科のパソコン室では macOS 上の VirtualBox で Windows を動作させている。

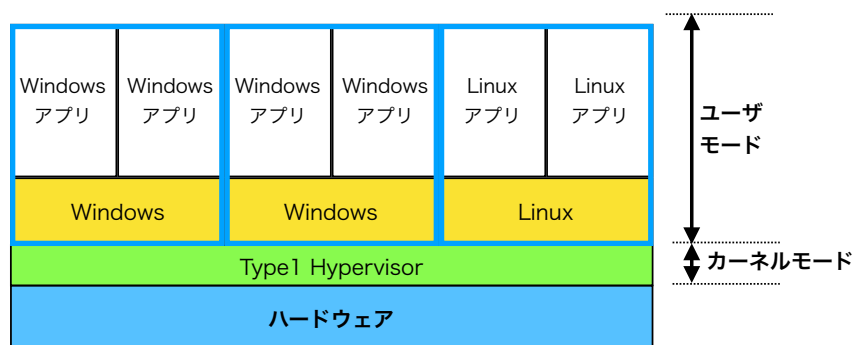


図 2.9: Type 1 ハイパーバイザ

直接実行される．Type 1 ハイパーバイザとして有名な製品は，IBM z/VM, VMware vSphere, Xen, Hyper-V 等である．

サーバ向けの製品が主流であり，例えば VMware vSphere は実行中のゲストを他の物理サーバに移動する等，非常に高度な機能を持っており [37]，一台のサーバ上に効率よく多数の仮想マシンを動かすことができる．徳山高専情報電子工学科のパソコン室でも，2 台のサーバ上に 50 台の仮想デスクトップマシンを動かしていたことがある．

2.7.3 仮想アプライアンス

ゲスト・オペレーティングシステムとアプリケーションまでインストールし，すぐに使用できる状態で配布される仮想マシンである．例えば，メールフィルタソフトをインストールした仮想マシンを入手しハイパーバイザで実行するだけで，すぐにメールフィルタリングを開始できる．

同じ手法で，すぐに使用できるパーソナルコンピュータ用のデスクトップ・オペレーティングシステムが配布されている場合もある．Linux の一種である Ubuntu の場合，VirtualBox で実行できるディスクイメージがダウンロードできる [38]．仮想アプライアンスは，ソフトウェアの新しい流通手法である．

2.8 実装例

本書では，TaC (Tokuyama Advanced educational Computer) と，TaC のオペレーティングシステム TacOS を実装例として参照する．第 18 章で TaC と TacOS の概要を紹介する．TaC は，学生が限られた時間で仕組みを理解できるように設計された，単純で小さな 16 ビットパーソナルコンピュータである．

2.9 まとめ

本書は *SMP* (対称型マルチプロセッシング: *Symmetric Multiprocessing*) のコンピュータを前提にしている．CPU は *PSW* (*Program Status Word*) と *CPU レジスタ*を含んでいる．最近の Intel 社の CPU では，従来の CPU を *Core* (コア)，複数のコアを含んだ LSI のことを CPU と呼ぶ．SMP では複数の CPU (コア) がメモリを共有する．更に，ホストコントローラやアダプタも *DMA* (*Direct Memory Access*) 方式によりメモリを共有する．

オペレーティングシステムのカーネルは、割込みハンドラ、ディスパッチャ、サービスモジュール、デバイスドライバ等から構成される。ユーザ・プロセスからカーネルへの切換え原因は**割込み**だけである。ユーザ・プロセス毎に**仮想 CPU**、**仮想メモリ空間**、管理情報等を持っている。

カーネルの構成方式には、**単層カーネル（モノリシック・カーネル）**方式と**マイクロカーネル（micro-kernel）**方式の二種類があった。マイクロカーネル方式ではサービスモジュールをサーバ・プロセスとし、**IPC（プロセス間通信）**を用いてサービスを要求する。サービスモジュール間の独立性が高くなり高信頼性のシステムを構成可能であるが、IPC はオーバーヘッドが大きい。信頼性と性能はトレードオフの関係にある。

Type1, Type2 ハイパーバイザは、PC のハードウェア全体をエミュレーションする仮想マシンを提供する。この仮想マシンの上で、Windows や Linux 等のオペレーティングシステムを実行することができる。

練習問題

2.1 次の言葉の意味を説明しなさい。

- (a) CPU・ホストコントローラ・バス
- (b) DMA
- (c) SMP（対象型マルチプロセッシング）
- (d) PSW・CPU レジスタ
- (e) 割込み
- (f) SVC 命令
- (g) ディスパッチャ
- (h) サービスモジュール
- (i) デバイスドライバ
- (j) カーネルのコア
- (k) コンテキスト
- (l) 仮想 CPU
- (m) 仮想メモリ空間
- (n) 単層カーネル（モノリシック・カーネル）・マイクロカーネル
- (o) IPC（プロセス間通信）
- (p) Type 1 ハイパーバイザ・Type 2 ハイパーバイザ

2.2 自分がいつも使用しているパーソナルコンピュータのハードウェア構成を調べなさい。

- (a) CPU の種類（名称，メーカー，クロック，コア数（CPU 数））
- (b) メモリの大きさ
- (c) 二次記憶装置（ストレージ）の種類（ハードディスク？，SSD？）
- (d) 二次記憶装置（ストレージ）の大きさ
- (e) グラフィックアダプタの種類
- (f) キーボードやマウスの接続方式（USB？，Bluetooth？）

第 II 部

CPU 管理

第 3 章

CPU の仮想化

オペレーティングシステムは、ハードウェアを抽象化した使いやすい拡張マシン（仮想マシン）を必要な数だけ提供する。数に限りがある資源は、必要な数だけあるように見せるために仮想化が行われる。CPU 資源も仮想化し、各プロセスが自分専用の CPU を持っているように見せかける。

3.1 時分割多重

CPU を仮想化するためには時分割多重が用いられる。ハードウェアである実 CPU の数は限られているので、時間を区切って実 CPU を使用するプロセスを次々に切替えていく。図 3.1 に CPU 仮想化の原理を示す。

実 CPU は図 2.2 のような構造をもつハードウェアである。プロセスの構造は図 2.6 に示した通りであり、仮想 CPU を含んでいる。実 CPU が短時間（例えば 10ms）に次々と実行するプロセスを切替えていくことで、複数のプロセスが夫々に専用の CPU を持ち並行して実行されているように見せかける。

CPU が実行するプロセスを切り換えるには、まず、実 CPU のコンテキストを現在のプロセスの仮想 CPU 領域に保存する。次に、新しく実行するプロセスの仮想 CPU 領域から実 CPU にコンテキストを読み込み、新しいプロセスの実行を再開する。一つのプロセスから別のプロセスに切替える処理をコンテキストスイッチと呼ぶ。また、実 CPU にコンテキストを読み込んで実行を再開することをディスパッチ、ディスパッチを行うプログラムをディスパッチャと呼ぶ。図 2.5 にもディスパッチャは描かれていた。

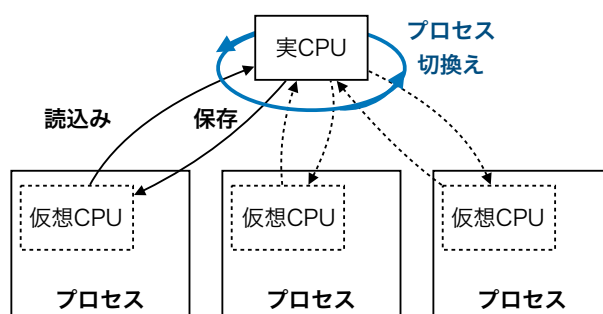


図 3.1: 時分割多重による CPU の仮想化

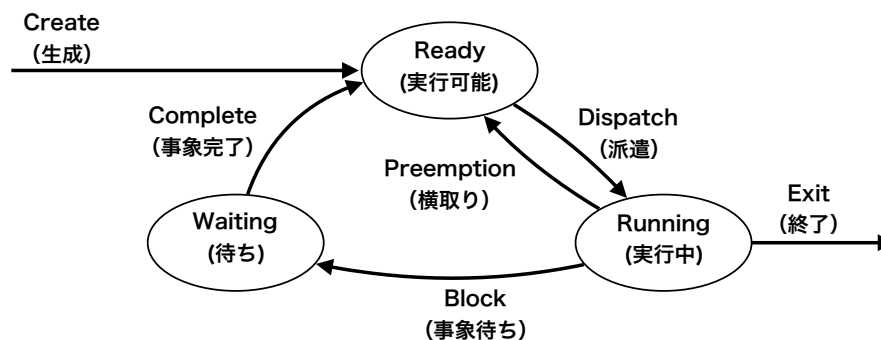


図 3.2: プロセスの状態遷移

3.2 プロセスの状態

プロセスは、キーボード等の入出力装置からの入力を待つ状態になったり、時間が経過するのを待つ状態になったりする。待ち (*Waiting*) 状態のプロセスには CPU を割当てて必要がない。このようにプロセスは幾つかの状態を持っている。プロセスの状態は UNIX では `ps` コマンドで確認できる。プロセスを模式的に示した図 2.6 では、「プロセス情報」の「プロセスの状態」のことである。

3.2.1 基本的な三つの状態

図 3.2 にプロセスの状態遷移図を示す。この図は最も簡単なものであり、実際のオペレーティングシステムでは、もっと状態数が多くなる^{*1}。図に示された三つの状態を説明する。

- *Ready* (実行可能)

CPU を割当てれば実行を開始できる状態のことである。プロセスは CPU が割当てられるのを待っている。

- *Running* (実行中)

CPU が割当てられ実行している状態のことである。CPU の数より多くのプロセスが同時に Running になることはできない。

- *Waiting* (待ち)

シグナルの到着や入出力の完了等の事象 (イベント) を待っている状態である。プロセスは実行することができない。

3.2.2 状態遷移

図 3.2 に示された六つの状態遷移の意味は以下の通りである。

1. *Create* (クリエート, 生成)

新しいプロセスが生成されると Ready 状態になる。親プロセスが `fork` システムコール (UNIX の場合) や `CreateProcess` システムコール (Windows の場合) を実行すると、新しい子プロセスが

^{*1} macOS の `ps` コマンドのオンラインマニュアルで確認すると、macOS ではプロセスの状態が、I (Idle), R (Runnable), S (Sleep), T (sTopped), U (Uninterruptible wait), Z (Zombie) の六つであることが分かる。

生成される。

2. *Dispatch*（ディスパッチ、派遣）

Ready 状態のプロセスは、自分の順番が来たら CPU が割当てられ Running 状態に遷移し実行を開始する。

3. *Preemption*（プリエンプション、横取り）

Running 状態のプロセスは、決められた時間（クオンタムタイム）を使い切ったときや、より優先度の高いプロセスが Ready 状態になったとき、CPU を取り上げられ Ready 状態に遷移する。

4. *Block*（ブロック、事象待ち）

Running 状態のプロセスが、システムコールを発行して自ら Waiting 状態に遷移することがある。例えば入出力システムコール（open, read, write, close 等）や、シグナル待ちシステムコール（pause, wait, sleep 等）を発行した場合である。また、他のプロセスからシグナルを受信した場合も、Waiting 状態に遷移することがある。更に、仮想記憶の機能を持つオペレーティングシステムでは、プロセスが読み書きしようとした領域がメモリ上に存在しない時もこの遷移が起こり、メモリ領域を確保するための処理がカーネル内部で始まる。

5. *Complete*（コンプリート、事象完了）

Waiting 状態のプロセスは、入出力の完了やシグナルの発生等の事象（イベント）が発生すると Ready 状態に遷移する。Waiting 状態のプロセスは停止しているのでプロセスが事象が発生することはない。事象はプロセスの外部からもたらされる。

6. *Exit*（終了）

プロセスが自ら exit システムコール（UNIX の場合）や ExitProcess システムコール（Windows の場合）を用いて終了する場合、または、プロセスがシグナルを受ける等して終了させられる場合に、この遷移が起こる。シグナルはプロセス（他プロセス、自プロセス）から明示的に送信される場合と、自プロセスが保護違反などのエラーを起こして発信される場合がある。

3.3 プロセスの切換え（コンテキストスイッチ）

Running 状態のプロセスが Block 遷移または Preemption 遷移し CPU を取り上げられると、他の Ready 状態のプロセスが CPU を割付けられ Dispatch 遷移し実行を再開する。

3.3.1 切換えの原因

Running 状態のプロセスが状態遷移を起こす原因を以下にまとめ直す。

1. イベント

Running 状態のプロセスは、自ら「システムコールを発行」することで Block 遷移をすることがある。また、他のプロセスからの「干渉^{*2}を受け」Block 遷移することがある。

2. タイムスライシング

Running 状態のプロセスが長時間の実行を続けると Preemption 遷移をする。一度に実行しても良い時間（クオンタムタイム）を使い切ったためである。Ready 状態のプロセスが他にあれば、そ

^{*2} 干渉には、より優先順位の高いプロセスが実行可能になった、別のプロセスからシグナル等を受取った等がある。

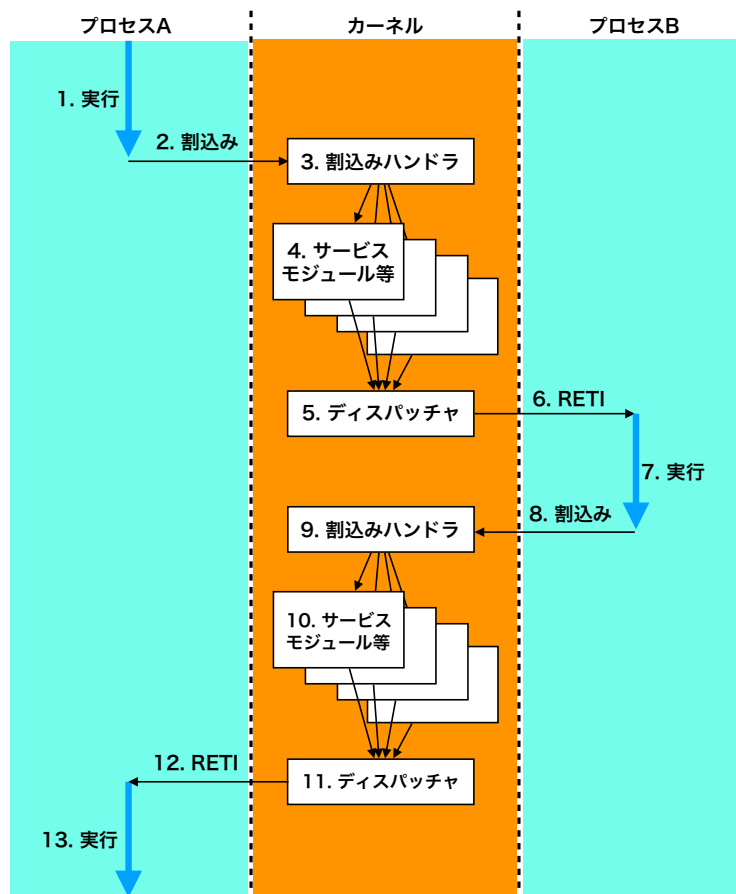


図 3.3: プロセスの切換え

のプロセスに実行が切替わる。他に実行すべきプロセスが無い場合は、再度、同じプロセスが実行される。

3.3.2 切換え手順

図 3.3 に二つのプロセス間で実行が切り換わる様子を示す。図では時間に従って上から下へ処理が進む。左側はプロセス A の実行を、右側はプロセス B に実行を、中央はカーネルの実行を表している。以下では、図の上半分でプロセス A からプロセス B に実行が切り替わる手順を説明する。図の下半分の説明は省略するが、上半分と同様な手順でプロセス B からプロセス A に切り替わる手順を示している。

1. 実行

日頃は CPU がユーザ・プロセスを実行している。

2. 割込み

割込みが発生し処理がプロセス A からカーネル内の割込みハンドラに移る。割込みの原因は 2.4 で述べた様々なものが考えられる。割込みが発生すると以下の処理が CPU のハードウェアにより自動的にされる。

(a) CPU の PSW (PC とフラグ) がスタックに保存される。

- (b) CPU の実行モードがカーネルモードに割込みは禁止に切り換わる（フラグが変更される）。
- (c) 割込みハンドラにジャンプする（PC が変更される）。

3. 割込みハンドラ

PSW（スタック上にある）と CPU レジスタ（図 2.2 参照）からなるプロセスのコンテキストをプロセスの仮想 CPU 領域（図 2.6 参照）に保存する。次に割込み原因を調べ、割込み原因に応じた処理（サービスモジュール等）にジャンプする。例えば、割込み原因が open システムコールなら、open システムコールの処理を行うファイルシステムのサービスモジュールにジャンプする。割込み原因が I/O 完了なら、完了した I/O に対応するデバイスドライバにジャンプする。

4. サービスモジュール等

サービスモジュールやデバイスドライバが割込み原因に応じた処理を行う。この過程でプロセスの状態が変化することがある。例えば、プロセスが発行したシステムコールが原因で Block 遷移する場合や、タイマーや I/O の完了割込により Waiting 状態だった別のプロセスが Complete 遷移する場合、タイマーの完了割込により現在のプロセスが Preemption 遷移する場合等が考えられる。サービスモジュール等の処理が完了するとディスパッチャにジャンプする。

5. ディスパッチャ

実行可能なプロセスの中から一つを選び、選んだプロセスの仮想 CPU 領域の内容を CPU レジスタにロードする。最後に PSW を復旧する機械語命令（RETI）を実行しプロセスの実行に戻る。

6. RETI

PSW を復旧する機械語命令として割込復帰用の *RETI* (*RETurn from Interrupt*) 命令を用いる。RETI 命令は単一の命令で PSW（PC とフラグ）を一度にスタックから復旧する。CPU の実行モードを表すフラグは PSW に含まれているので、PSW が復旧されることで実行モードがカーネルモードからユーザモードに切り換わる。

7. 実行

新しく選択されたユーザ・プロセスが実行される。

3.3.3 切換えの例

計算に長い時間を要する二つのプロセスだけがある時、クオンタムタイムを使い切ってもう一方のプロセスに切り換わり、交互に実行される様子を図 3.4 に示す。以下に手順を説明する。

1. 実行

プロセス A は計算処理を続けている。長い時間に渡ってシステムコールを発行することは無い。

2. タイマー割込み

タイマーは一定間隔で割込みを発生する。割込が発生すると CPU のハードウェアが自動的に PSW を保存し、割込みハンドラにジャンプする。オペレーティングシステムは、主に、この割込みを基準に時間の経過を認識する。

3. 割込みハンドラ

プロセスのコンテキストをプロセスの仮想 CPU に保存する。その後、割込原因を調べタイマーからの割込みなので、「タイマーに関する処理」を行うカーネル内のモジュールへジャンプする。

4. タイマーに関する処理

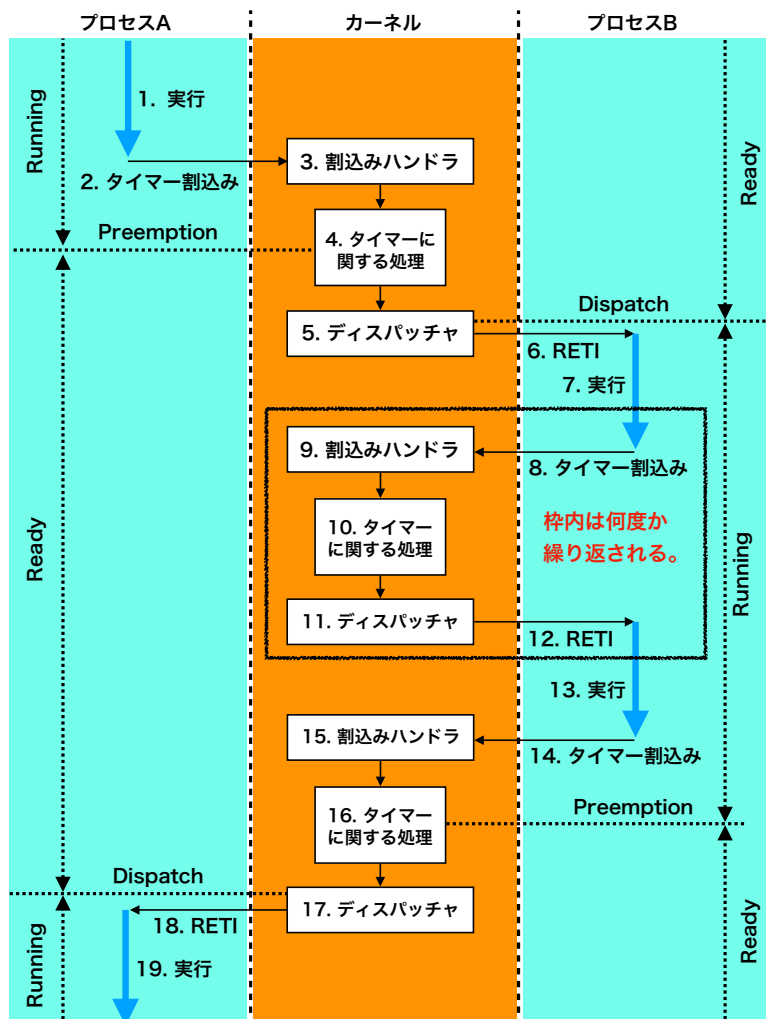


図 3.4: プロセスの切替えの例

一定間隔で発生するタイマーからの割込みを利用して、システムの時計を進めたり、リソース（CPU やメモリ等）の利用統計データを更新したりする。その間にプロセス A がクオンタムタイムを使い切ったことが判明すると、プロセス A を Preemption 遷移させる。この時点でプロセス A の状態が Ready に変化する。

5. ディスパッチャ

Ready 状態のプロセスの中から適切な一つを選び Dispatch 遷移させる。図 3.4 はプロセス B が選択された場合である。ディスパッチャはプロセス B の CPU レジスタを復旧する。

6. RETI

プロセス B の PSW を復旧し、プロセス B の実行を再開する。

7. 実行

プロセス B は計算処理を再開する。プロセス B も、長い時間に渡って計算を続けるプロセスとする。

8. タイマー割込み
計算を続けるうちにタイマーからの割込みが発生する。
9. 割込みハンドラ
プロセス B のコンテキストを保存する。
10. タイマーに関する処理
プロセス B は、まだ、クオンタムタイムを使い切っていないので、Preemption は発生しない。
11. ディスパッチャ
Preemption は発生しないので、プロセス B のコンテキストを復旧する。
12. RETI
プロセス B に戻る。
13. 実行
プロセス B は計算処理を再開する。
14. タイマー割込み
8.~13. を何度か繰り返し、クオンタムタイムを使い切った時のタイマー割込みである。
15. 割込みハンドラ
プロセス B のコンテキストを保存する。
16. タイマーに関する処理
クオンタムタイムを使い切ったので Preemption が発生する。
17. ディスパッチャ
Ready 状態のプロセス A を選択し Dispatch 遷移させる。プロセス A のコンテキストを復旧する。
18. RETI
プロセス A に戻る。
19. 実行
プロセス A は計算処理を再開する。

3.4 PCB (Process Control Block)

PCB はプロセスを表現する重要なカーネル内のデータ構造である。PCB はカーネル内のプロセステーブルに格納される。

3.4.1 PCB の内容

PCB は、図 2.6 の「仮想 CPU」と「プロセス情報」を合わせたものに相当する。PCB には以下のような情報が格納される。

- 仮想 CPU
- プロセス番号
- 状態 (Running, Waiting, Ready 等)
- 優先度
- 統計情報 (CPU 利用時間等)
- 次回のアラーム時刻

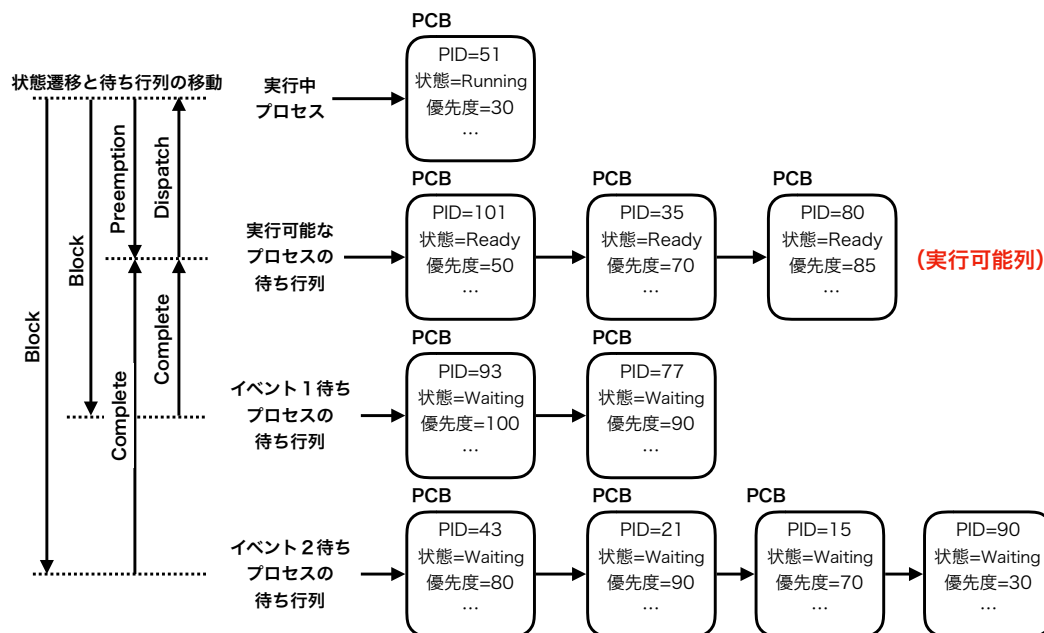


図 3.5: PCB のリスト

- ・ 親プロセス
- ・ 子プロセス一覧
- ・ シグナルハンドリング
- ・ 使用中のメモリ
- ・ オープン中のファイル
- ・ カレントディレクトリ
- ・ プロセス所有者のユーザ番号
- ・ PCB のリストを作るためのポインタ

3.4.2 PCB リスト

カーネル内では PCB がプロセスを表現する。例えば、優先順にソートされた Ready 状態のプロセスのリストは、優先度をキーにソートされた PCB の線形リスト（待ち行列）として表現される。この線形リストを**実行可能列**と呼ぶ。その様子を図 3.5 に示す。図は、数値が小さいほど優先度が高い意味になっている。

Ready 状態のプロセスだけでなく、Running 状態のプロセスや、Waiting 状態のプロセスも待ち行列で管理される。Waiting 状態のプロセスは、待ち合わせているイベント毎に待ち行列を作っている。イベント待ち行列のソート順はイベント毎にルールが決められる。

プロセスの状態遷移に合わせて PCB が待ち行列の間を移動する。図 3.5 の左側の「状態遷移と待ち行列の移動」が「どの待ち行列から、どの待ち行列に移動可能か」を表している。例えば、Running 状態（実行中）のプロセスが Preemption 遷移をすると、状態が Ready に変わるだけでなく、PCB が「実行可能なプロセスの待ち行列」に移動する。この移動ルールは図 3.2 の状態遷移と一致している。

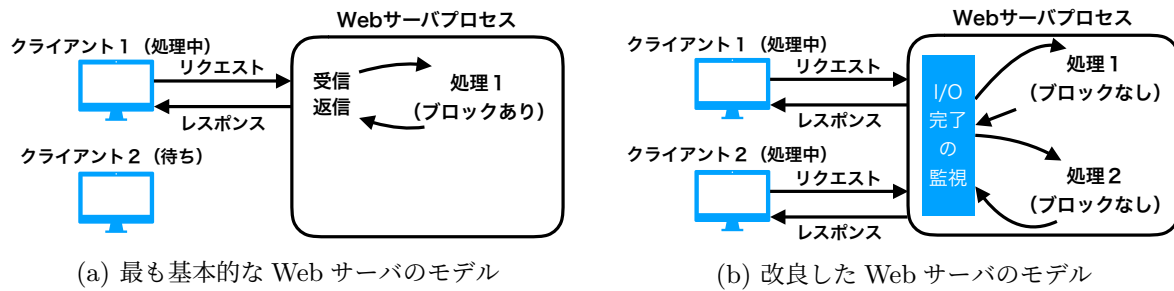


図 3.6: マルチプログラミングを用いない Web サーバ

3.5 スレッド (Thread)

ここまで、一つのプロセスが一つの仮想 CPU を持つモデルを考えてきた。しかし、実際のコンピュータハードウェアは CPU を複数持つ SMP の場合もある。これでは「ハードウェアの機能を抽象化した便利な**拡張マシン**」(1.1.1 参照)であるはずのプロセスが、「CPU が一つしかない**縮小マシン**」になっている。そこで、SMP に対応しプロセスが複数の仮想 CPU を持つモデルを導入する。これにより、一つのプロセスが並列実行する複数の処理の流れ(スレッド)を持つことが可能になる。

3.5.1 スレッドの役割

複数のプロセス(ジョブ)を主記憶にロードしておくことで CPU の利用効率を高くできることは既に説明した(8 ページ, マルチプログラミング参照)。マルチプログラミングの、もう一つのメリットは、プログラミングが簡単になる場合があることである。以下では Web サーバを例に、マルチプログラミングによる改善を紹介する。

- マルチプログラミングなし

図 3.6a に最も簡単なモデルを示す。Web サーバはリクエストを受信すると、それに対するレスポンスを返す。処理は 1 番目のクライアントから順に行われ、2 番目のクライアントは 1 番目の処理が終了するまで待たされる。このモデルの問題点は、処理中に Web サーバプロセスが I/O 待ち等でブロック(Block)する可能性があり、その間、他のクライアントへのサービスがされないことである。

2 番目以降のクライアントが長時間待たされないように、複数のクライアントの処理を並行してできるように改良したモデルが図 3.6b である。「I/O 完了の監視」は通信を含む複数の入出力を同時に監視し、どれかが読み書き可能になるのを待つ機能である。UNIX では select システムコールがこの機能を持つ。読み書き可能になったことを確認後に読み書きを行うのでプロセスがブロックすることが無くなり、複数のクライアントに対して同時にサービスを行うことができる。

しかし、Web サーバのプログラミングは難しくなる。一方のクライアントの処理が終わらないうちに、別のクライアントの処理を開始する必要があるからである。クライアント毎に処理がどこまで進んでいるのかを表す**状態**を持つ必要がある。また、CPU が複数存在する場合でも、同時には一つの CPU しか働かないことも問題である。

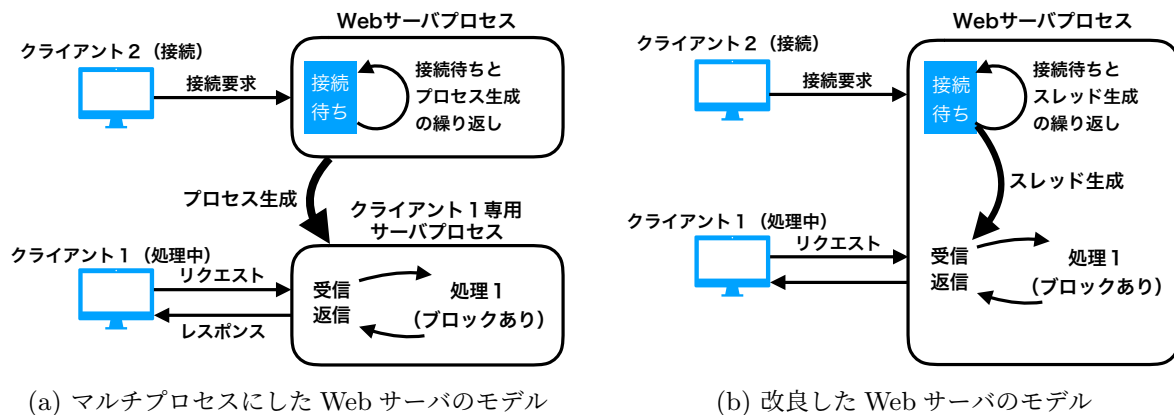


図 3.7: マルチプログラミングを用いる Web サーバ

- マルチプロセス

マルチプログラミングを用いることで前記の問題を解決したモデルを図 3.7a に示す。Web サーバプロセスは、まず、接続要求を待ちクライアント 1 からの接続を受け入れる。次に、クライアント 1 専用のサーバプロセスを生成し処理を任せる。Web サーバプロセスは、生成したプロセスの終了を待たずに、次の接続要求待ちになる。クライアント 2 からの接続要求があったらクライアント 2 専用のサーバプロセスを生成し、接続要求待ちに戻る。

このモデルなら、各クライアントの処理を別々のプロセスが行っているので、プロセスがブロックしても構わない。そのため、プログラミングは簡単になる。また、CPU が複数あればプロセスが真に並列に実行される。しかし、プロセスの生成はメモリ空間の確保や初期化を含み**重い処理**である。また、プロセスはメモリを共有していないのでプロセス間の情報共有には効率が悪い。

- マルチスレッド

複数のスレッドを使用したモデルを図 3.7b に示す。マルチプロセスの場合と良く似たプログラムであるが、クライアント毎に専用のプロセスを作る代わりに、クライアント毎に専用のスレッドを作る。スレッドの生成はプロセス生成より 10~100 倍速いと言われている [39]。また、スレッドはメモリを共有しているので情報共有には都合が良い。例えば、Web サーバが頻繁に参照されるページをメモリ上にキャッシュする場合、キャッシュをスレッドで共有できる。

3.5.2 スレッドの形式

読者は、「スレッドはカーネルが実現する」と暗黙のうちに考えていたかも知れない。しかし、ユーザプログラム（ライブラリ）内でスレッドを実現することもある。カーネルが実現するスレッドを**カーネルスレッド**、ユーザプログラム内で実現するスレッドを**ユーザスレッド**と呼ぶ。

- カーネルスレッド

カーネルスレッドの模式図を図 3.8a に示す。カーネルスレッドはプロセスの仮想 CPU を複数にし、仮想 CPU がプログラムを並行して実行する。「プロセス情報」から「プロセスの状態」は無くなり、代わりに仮想 CPU 毎に「仮想 CPU の状態」を管理するようになる。CPU が複数ある時、

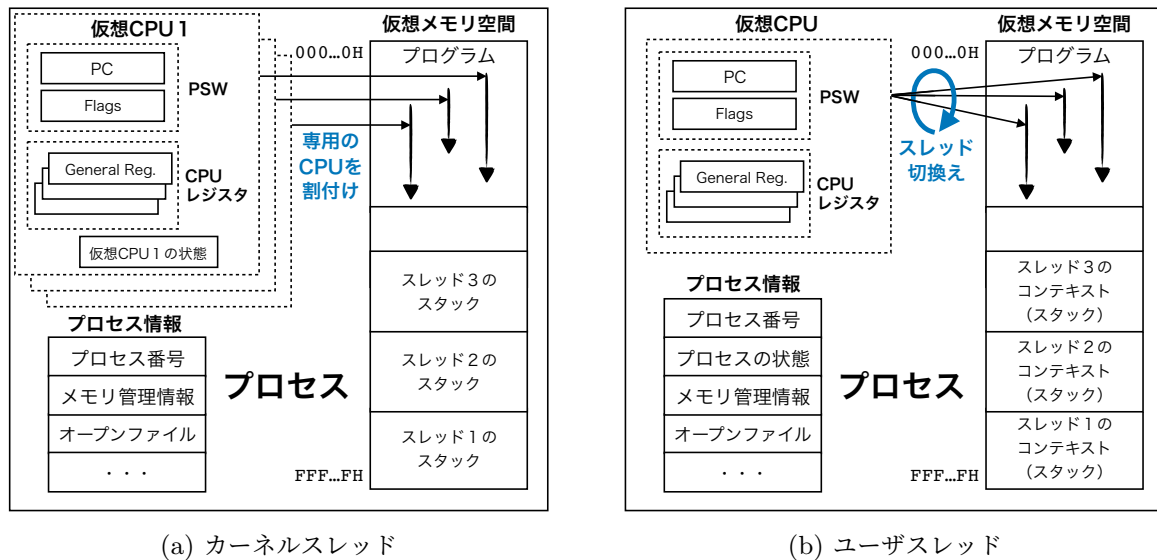


図 3.8: ユーザスレッドとカーネルスレッド

カーネルスレッドであれば、プロセス内を真に並列実行することが可能である。

• ユーザスレッド

ユーザスレッドの模式図を図 3.8b に示す。プロセスには単一の仮想 CPU しかない。ユーザスレッドは仮想 CPU を時分割多重して実現される。カーネルを経由しないでスレッドの生成や切換えをすることができるので、オーバーヘッドが非常に小さい。

以下に述べるように、両者を組合せた三つのスレッドモデルが使用される。

1. One-to-One Model

全てのスレッドがカーネルスレッドのモデルである。図 3.8a に相当する。プロセス内にカーネルが管理する仮想 CPU が複数あるので、複数プロセスと同等な並列実行が可能である。しかし、スレッドの生成や切換えにカーネルが介入するので、処理は重くなる。また、システムによっては生成できるスレッド数に制限がある。

2. Many-to-One Model

複数 (Many) のスレッドを一つ (One) のカーネルスレッドで実行するモデルである。図 3.8b に相当する。プロセス内にカーネルスレッドは一つしか存在しない。スレッドはユーザプログラム (ライブラリ) の工夫で単一のカーネルスレッドを複数に見せかけているだけなので、真の並列実行にはならない。また、何れかのスレッドがシステムコールでブロックすると、全てのスレッドが停止してしまう問題がある。

3. Many-to-Many Model

複数 (Many) のスレッドを複数 (Many) のカーネルスレッドで実行するモデルである。カーネルスレッドの数はスレッドの数より少なくても良い。前記二つのモデルの折衷案である。

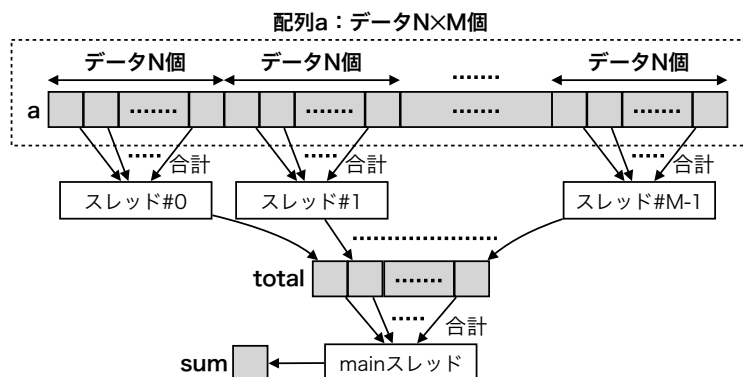


図 3.9: M 個のスレッドで手分けして合計を計算する様子

3.5.3 スレッドプログラミング

配列データの合計を求める処理をスレッドを用いて高速化する例を考えよう。図 3.9 に原理を示す。配列 a を M 分割し個別スレッドで（CPU が複数あれば）同時に小計を計算する。小計は配列 total に格納する。最後に main スレッドが total の合計を求めると全体の合計 sum が計算できる。

POSIX スレッドによる実装

このアイデアを POSIX スレッド^{*3}を用いた C 言語プログラムにしたものをリスト 3.1^{*4}に示す。

12 行の `thread()` 関数は M 個のスレッドで同時に並列実行される。配列 a の担当範囲等は引数 arg により指示される。関数の引数 (arg) やローカル変数 (args, sum, i) は、スレッドのスタック (図 3.8 参照) に割付けられるので、スレッド毎に別の実体を持つ。グローバル変数 a や total 等は全てのスレッドで共有される。

32 行の `pthread_attr_init()` は引数の `pthread_attr_t` 型変数をデフォルトのアトリビュート値で初期化する。33 行の `pthread_create()` がスレッドを生成する関数である。新しいスレッドの実行は引数で指定された `thread()` 関数から始まる。`pthread_create()` の引数 p は、`thread()` 関数が実行を開始する時に arg 引数に渡される。

38 行の `pthread_join()` はスレッドの終了を待つ関数である。スレッドの終了が確認できたら、39 行で小計を sum に足し込む。

実行時間の計測結果

リスト 3.1 のプログラムの実行時間を表 3.1 に、グラフにしたものを図 3.10 に示す^{*5 *6*7}。

スレッド数が 1 の時は、経過時間 (Real) とユーザ CPU 時間 (User) が、ほぼ、同じになる。一つのコア^{*8}が全力で合計を計算した結果である。

^{*3} POSIX スレッドは UNIX 系のオペレーティングシステムで利用できる。

^{*4} このプログラムは macOS High Sierra で動作確認をした。

^{*5} 実行時間の計測には OS X の `time` コマンドを用いた。

^{*6} 実行時間が短すぎて比較し難いので、プログラムの 14 行から 17 行を 10 万回繰り返すように改造した上で計測した。

^{*7} 計測に使用したコンピュータは OS X Yosemite をインストールした Mac Pro (Late 2013, 3.5GHz 6-Core Intel Xeon E5) である。C 言語コンパイラは Apple LLVM version 7.0.0 (clang-700.0.72) を使用した。

^{*8} 従来の CPU のこと。

リスト 3.1: M 個のスレッドで分担して配列データの合計を求めるプログラム

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #define N 1000                                // 1スレッドの担当データ数
5  #define M 10                                  // スレッド数
6  pthread_t tid[M];                             // M個のスレッドのスレッドID
7  pthread_attr_t attr[M];                       // M個のスレッドの属性
8  int a[M*N];                                  // このデータの合計を求める
9  int total[M];                                 // 各スレッドの求めた部分合
10 typedef struct { int no, min, max; } Args;      // スレッドに渡す引数の型定義
11
12 void *thread(void *arg) {                      // 自スレッドの担当部分のデータの合計を求める
13     Args *args = arg;                          // m番目のスレッド
14     int sum = 0;                                // 合計を求める変数
15     for (int i=args->min; i<args->max; i++) {    // a[N*m ... (N+1)*m] の
16         sum += a[i];                            // 合計を sum に求める.
17     }
18     total[args->no]=sum;                         // 担当部分の合計を記録
19     return NULL;                                // スレッドを正常終了する
20 }
21
22 int main() {                                    // mainスレッドの実行はここから始まる
23     // 擬似的なデータを生成する
24     for (int i=0; i<M*N; i++)                  // 配列 a を初期化
25         a[i] = i+1;
26     // M個のスレッドを起動する
27     for (int m=0; m<M; m++) {                  // 各スレッドについて
28         Args *p = malloc(sizeof(Args));        // 引数領域を確保
29         p->no = m;                              // m番目のスレッド
30         p->min = N*m;                           // 担当範囲下限
31         p->max = N*(m+1);                       // 担当範囲上限
32         pthread_attr_init(&attr[m]);            // アトリビュート初期化
33         pthread_create(&tid[m], &attr[m], thread, p); // スレッドを生成しスタート
34     }
35     // 各スレッドの終了を待ち, 求めた小計を合算する
36     int sum = 0;
37     for (int m=0; m<M; m++) {                  // 各スレッドについて
38         pthread_join(tid[m], NULL);            // 終了を待ち
39         sum += total[m];                       // 小計を合算する
40     }
41     printf("1+2+ ... +%d=%d\n", N*M, sum);
42     return 0;
43 }

```

表 3.1: スレッド数による実行時間比較

M N M*N	スレッド数 (M) ・ データ件数 (M*N)									
	1	2	3	4	5	6	7	8	9	10
	10,000	5,000	3,333	2,500	2,000	1,666	1,428	1,250	1,111	1,000
経過時間 (s)	1.881	0.980	0.657	0.493	0.406	0.339	0.335	0.332	0.319	0.312
ユーザ CPU 時間 (s)	1.879	1.953	1.959	1.958	2.009	2.011	2.244	2.462	2.679	2.846
システム CPU 時間 (s)	0.002	0.002	0.002	0.001	0.001	0.002	0.003	0.003	0.003	0.002

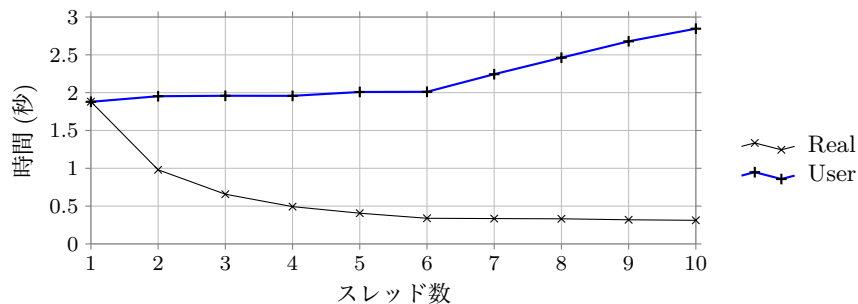


図 3.10: スレッド数による実行時間の変化

スレッド数が 1~6 の間は、経過時間がスレッド数に反比例して短くなる。合計の計算時間に相当するユーザ CPU 時間は、ほぼ一定である。使用したコンピュータが持つ六つのコアが、最大で六つのスレッドに割当てられ、真に並列実行された結果である。

スレッドの数が 6~10 に増加する間、経過時間は、ほぼ一定である。しかしユーザ CPU 時間が増加している。必要な計算量は一定なのに長い CPU 時間を必要とするので、コアの性能が悪化したように見える。

コアの性能が悪くなったように見えるのは、ハイパースレッディング・テクノロジー [40] により、コアの数が倍 (12 個) あるように見せかけているためである。ハイパースレッディング・テクノロジーは、単一スレッドを実行する場合は遊んでしまうコア内のユニットを、二つのスレッドを同時に実行することで効率よく使用する技術である。見かけ上コアの数が二倍になるが、合計の性能は二倍には達しないので、コアあたりの性能が下がったように見える。

3.6 CPU 仮想化の実装例

第 19 章に TacOS の CPU 仮想化の例を示す。この例は、C++ 言語で記述したカーネル内データ構造や、プロセス切換えプログラム、プロセススケジューラ等の実装を含む。また、プロセスのメモリ配置についても説明している。

3.7 まとめ

本章では、時分割多重による CPU の仮想化について学んだ。プロセスは幾つかの状態を持ち、実行できない状態の場合は CPU を割当てない。プロセスはイベントやタイムスライシングにより状態が変化する。CPU は、実行を中断するプロセスから次に実行するプロセスに、コンテキストスイッチを

行う。

PCB はプロセスを表現するカーネル内の重要なデータ構造である。例えばプロセスの待ち行列は PCB の待ち行列として表現されるし、プロセスの実行が中断する時は PCB にコンテキストが保存される。

スレッドを導入することで、SMP に対応したプロセスのモデルを表現できる。スレッドにはカーネルスレッドとユーザスレッドがあった。また、これらを組み合わせた三つのスレッドモデルがあった。POSIX スレッドを用いてデータの合計を計算する処理を高速化するプログラム例を紹介し、実行時間の計測結果を示した。スレッド数が CPU（コア）数以内の場合は、スレッド数に反比例して実行時間が短くなることが確認できた。

練習問題

3.1 次の言葉の意味を説明しなさい。

- (a) 時分割多重
- (b) コンテキストスイッチ
- (c) Dispatch（ディスパッチ）
- (d) Preemption（プリエンプション）
- (e) プロセスの状態
- (f) プロセスの状態遷移
- (g) RETI 命令
- (h) PCB
- (i) 待ち行列
- (j) 実行可能列
- (k) スレッド
- (l) カーネルスレッド
- (m) ユーザスレッド
- (n) One-to-One Model
- (o) Many-to-One Model
- (p) Many-to-Many Model

3.2 POSIX スレッドについて調査しなさい。

第 4 章

CPU スケジューリング

プロセス（スレッド）の実行順序を決めることをスケジューリング^{*1}と呼ぶ。システム内で最も貴重な資源である CPU の割当てを決める重要な機能である。

4.1 評価基準

スケジューリングの良し悪しを判断する評価基準には次のものがある。

- スループット (*Throughput*)

単位時間あたりに処理できるジョブ数のことである。大きい方が良い。

- ターンアラウンド時間 (*Turnaround time*)

プロセスが実行できるようになってから終了するまでの時間のことである。短いほうが良い。バッチ処理で、ユーザがジョブを提出してから実行結果の印刷物が届くまでの時間をイメージすると分かりやすい。

- レスポンス時間 (*Response time*)

対話的なシステム（TSS やデスクトップパソコン）において、ユーザが操作した影響で出力が変化し始めるまでの時間である。例えば、エンターキーを入力したあと画面が変化を始めるまでの時間である。対話的なアプリケーションの操作性に大いに影響がある。当然、短いほうが良い。

- 締め切り (*Deadline*)

制御用に用いられるリアルタイムシステム (*Real-time system*) では、決められた時刻（締め切り）までに結果を出すことが求められる。必ず時間を守らなければならない場合をハードリアルタイム (*Hard real time*)、できる限り時間を守らなければならない場合をソフトリアルタイム (*Soft real time*) と呼ぶ。オペレーティングシステムは、制御用プロセスが締め切りを守ることができるスケジューリングを行う必要がある。

- その他

システムの使用方法などにより様々な評価基準が考えられる。例えば、モバイルデバイスではバッテリーのために省エネルギーが評価基準になり得る。

^{*1} プロセスとスレッドの両方にあてはまることが多いので、この章ではプロセスのスケジューリングを前提に議論する。

表 4.1: スケジューリングの目標

コンピュータの種類	重視する性能
メインフレーム (バッチ処理)	スループット, ターンアラウンド時間
ネットワークサーバ	レスポンス時間, スループット
デスクトップパソコン	レスポンス時間
モバイルデバイス	レスポンス時間, 省エネルギー
組込み制御	締め切り

4.2 システムごとの目標

システムの種類によって、スケジューリングの目標は異なる。表 4.1 に概略をまとめる。

• メインフレーム

バッチ処理を行う場合はユーザとの対話的な処理ではないので、**スループット**を優先する。例えば、コンテキストスイッチにも処理時間が必要なので、プリエンプションを行わないスケジューリング方式を採用し、コンテキストスイッチの回数を少なくすること等が考えられる。また、ユーザが結果を早く受取ることができるように、**ターンアラウンド時間**にも気を使う必要がある。

• ネットワークサーバ

ネットワークに接続され、複数のクライアントから同時に多数の要求を受付けて処理する。この場合は、クライアントを操作しているユーザの操作性を損なわない**レスポンス時間**と、多数の要求を処理するための**スループット**が両立することが望まれる。両者のバランスが良いスケジューリングが求められる。

• デスクトップパソコン

一人のユーザが独占して使用するコンピュータである。ユーザは、複数の処理を同時にすることは少ない。ユーザの操作に素早く反応するために**レスポンス時間**が重要である。例えば、ユーザがワードプロセッサを操作している間にバックグラウンドでメールの着信チェックを行うプロセスが動く場合、ワードプロセッサが軽快に動くことを重視し、メールの着信チェックプロセスの性能が落ちて構わない。ユーザが直接操作するプロセスを優先するスケジューリングが求められる。

• モバイルデバイス

ノートパソコンやスマートフォンのようなシステムでは、基本的にはデスクトップパソコンと同じように**レスポンス時間**が重視される。しかし、バッテリーで駆動される場合は消費電力が少なくなる工夫も必要である。例えば、プロセスの切換え頻度を少なくすることで、エネルギーの消費を小さくするスケジューリングを採用することが考えられる。

• 組込み制御用のコンピュータ

締め切りまでに処理を完了することが重要である。例えば、時速 50km で走行するエレベータ^{*2}の制御コンピュータが、1 秒遅刻してブレーキを掛けたらどうなるだろうか。時速 50km は秒速 13m

^{*2} 高層ビルのエレベータの中にはもっと高速なものもある。

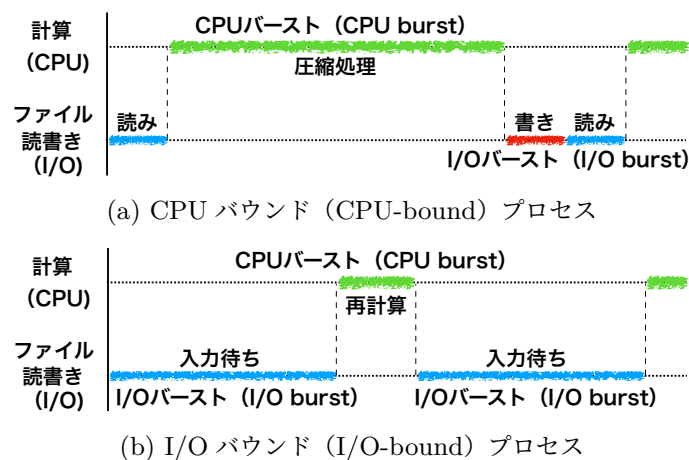


図 4.1: CPU バウンドと I/O バウンドプロセス

なので、エレベータは 13m 行き過ぎて停まることになる。最上階、または、最下階を目指しているとき 13m 行き過ぎるとエレベータは天井か床に激突してしまう。エレベータのブレーキ制御プロセスはハードリアルタイムに分類できる。同じエレベータでも、現在階数の表示はタイミングが少し遅れても大きな影響はない。エレベータの階数表示プロセスはソフトリアルタイムに分類できる。

4.3 プロセスの振舞

一般に、プロセスは計算と入出力を繰り返す。計算と入出力にかかる時間の割合に応じて、二種類のプロセスに分類できる。

4.3.1 CPU バウンドプロセス

例として、動画を圧縮するビデオエンコーディング・プロセスを考えてみよう。プロセスは、図 4.1a に示すように、次の三つの処理を繰り返す。

1. 未圧縮の動画ファイルを少し読む。
2. 圧縮処理を行う。
3. 結果を圧縮済み動画ファイル書込む。

ビデオエンコーディング・プロセスは CPU が行う圧縮処理に長い時間がかかり、入出力にかかる時間が短い。このように CPU 処理にかかる時間が相対的に長いプロセスのことを CPU バウンド (CPU-bound) プロセスと呼ぶ。また、CPU が使用される期間を CPU バースト (CPU burst)、I/O が使用される期間を I/O バースト (I/O burst) と呼ぶ。CPU バウンドプロセスは長い CPU バーストと短い I/O バーストを持つ。

4.3.2 I/O バウンドプロセス

二つ目の例としてスプレッドシート・プロセスを考えてみよう。スプレッドシート・プロセスは、まず、ユーザが何れかのセルにデータを入力するのを待つ。次に、入力されたデータを用いてスプレッド

シートの再計算を行い結果を表示する。ユーザがセルにデータを入力するたびに同様な処理を繰り返す。このプロセスは図 4.1b に示すように、ユーザ操作を待つ長い入力待ちと、再計算と表示を行う短い CPU 処理を行う。このような、長い I/O バーストと短い CPU バーストを持つプロセスを *I/O バウンド (I/O-bound) プロセス* と呼ぶ。

4.4 スケジューリング方式

いくつかの代表的なスケジューリング方式を紹介する。

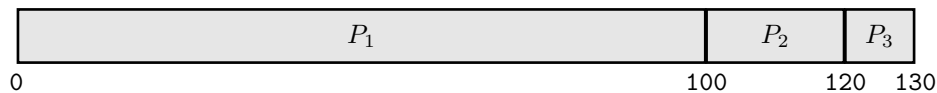
4.4.1 First-Come, First-Served (FCFS) スケジューリング

Ready 状態になった順（到着順）に実行する方式である。Running 状態になったらブロックするまで実行を継続する。プリエンプションはしない。以下の例では CPU バースト一回分の期間しか示さないが、実際は、図 4.1 に示すように CPU バースが繰り返し発生する。

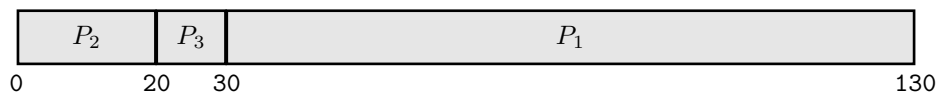
FCFS 方式は実行可能列を FIFO にするだけで実現できるが性能は良くない。例えば次の三つのプロセスが時刻 0 で、 P_1 , P_2 , P_3 の順に Ready 状態になったとする。

プロセス	到着時刻	CPU バースト時間 (ms)
P_1	0	100
P_2	0	20
P_3	0	10

この時、三つのプロセスの実行開始・終了の時刻を図で表すと次のようになる。



平均ターンアラウンド時間を計算すると、 $(100 + 120 + 130)/3 = 117$ ms となる。一方で、プロセスの到着順が P_2 , P_3 , P_1 の順だったとすると、三つのプロセスの実行開始・終了の時刻は図のようになる。



この場合の平均ターンアラウンド時間を計算すると、 $(20 + 30 + 130)/3 = 60$ ms となる。このように、FCFS では最悪な平均ターンアラウンド時間を選択することもある。プリエンプションをしないので、一旦、CPU バウンドなプロセスが実行を開始すると、他のプロセスは長い時間待たされる。

4.4.2 Shortest-Job-First (SJF) スケジューリング

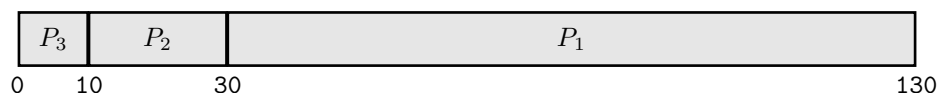
SJF 方式^{*3}は、平均ターンアラウンド時間を最小にするスケジューリング方式である。SJF 方式では CPU バースト時間が短いものを先に実行するようにスケジューリングする。実行可能列は CPU バースト時間が短い順にソートされている。

三つのプロセスがあった時、実行順に各プロセスの実行時間が T_1 , T_2 , T_3 とすると、平均ターンア

^{*3} SPT (Shortest Processing Time first) と呼ぶ。

ラウンド時間は、 $(T_1 + (T_1 + T_2) + (T_1 + T_2 + T_3))/3 = T_1 + T_2 * 2/3 + T_3/3$ となり、先に実行したプロセスの実行時間ほど結果に及ぼす影響が大きいことが分かる。実行時間が短いプロセスを先に実行するスケジューリング方式は、平均ターンアラウンド時間を最小にする。

前出の三つのプロセスを SJF 方式でスケジューリングした時の、実行開始・終了時刻は次の図のようになる。



この図より平均ターンアラウンド時間を求めると $(10 + 30 + 130)/3 = 57$ ms となり、これまでで最短である。しかし、次回の CPU バースト時間を知ることは一般には不可能なので、SJF 方式は現実的な方式ではない。次回の CPU バースト時間を予測することで擬似的な SJF 方式を実現する。

次回の CPU バースト時間を予測する方法として、指数平滑平均 (exponential average) を用いる例を紹介する。次回の予測時間を T_{n+1} 、前回の予測時間を T_n 、前回の実際の CPU バースト時間を t_n とすると、 $0 \leq \alpha \leq 1$ の時、指数平滑平均は次の式で表すことができる。

$$T_{n+1} = \alpha t_n + (1 - \alpha)T_n$$

この式から

$$T_{n+1} = \alpha t_n + (1 - \alpha)\alpha t_{n-1} + \cdots + (1 - \alpha)^j \alpha t_{n-j} + \cdots + (1 - \alpha)^{n+1} T_0$$

を得る。 $\alpha = 0.5$ の場合は、

$$T_{n+1} = 0.5t_n + 0.5^2 t_{n-1} + \cdots + 0.5^{j+1} t_{n-j} + \cdots + 0.5^{n+1} T_0$$

となる。この式は、過去の CPU バースト時間を、最近のものほど大きな重みを付けて平均したものになっている。つまり、次回の CPU バースト時間は、過去の CPU バースト時間と同程度であろうとの仮定に基づいた予測値を計算している。

4.4.3 Shortest-Remaining-Time-First (SRTF) スケジューリング

SRTF 方式^{*4}は、プリエンプシオン付きの SJF 方式である。プロセスが Ready 状態になるとき、このプロセスの CPU バースト時間と実行中プロセスの残り CPU バースト時間とを比較し、残り CPU バースト時間の方が長いときプリエンプシオンを起こす。次の例で SJF と SRTF を比較してみよう。

プロセス	到着時刻	CPU バースト時間 (ms)
P1	0	60
P2	10	40
P3	60	30

三つのプロセスを SJF でスケジューリングした場合は次の図のようになる。

^{*4} SRPT (Shortest Remaining Processing Time first) とも呼ぶ。



平均ターンアラウンド時間を計算すると、 $((60 - 0) + (90 - 60) + (130 - 10))/3 = 70$ ms となる。三つのプロセスを SRTF でスケジューリングした場合は次の図のようになる。



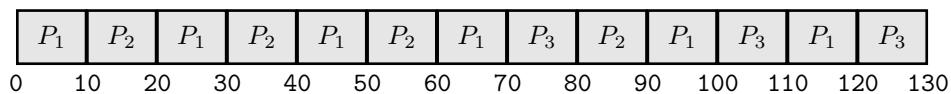
P_2 が到着した時、 P_2 の CPU バースト時間(40 ms)の方が P_1 の残り CPU バースト時間(60-10 = 50 ms) より短いので、 P_1 はプリエンプションし P_2 が先に実行される。 P_3 が到着した時も同様である。平均ターンアラウンド時間を計算すると、 $((130 - 0) + (50 - 10) + (90 - 60))/3 = 67$ ms となり、SJF よりも改善されている。

4.4.4 Round-Robin (RR) スケジューリング

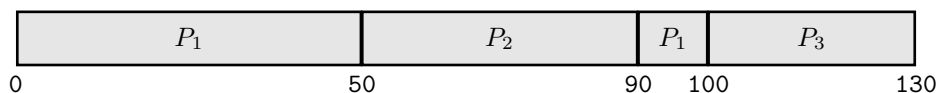
タイムシェアリングシステム (TSS) で使用された方式である。**クォンタム時間 (time quantum)**、または、**タイムスライス (time slice)** と呼ばれる 10 ms ~ 100 ms 程度の一定の時間が予め決められている。実行可能列は FIFO になっている。実行可能列の先頭のプロセスに CPU が割り付けられ Running 状態になる。プロセスの実行がクォンタム時間連続するとプリエンプションが発生し、プロセスは実行可能列の最後尾に付け加えられる。

クォンタム時間 (q) が短いと**レスポンス時間**が短くなり、対話的な処理が円滑に行える。例えば、10 個のプロセスが CPU を奪い合う状況でも、 $q = 10$ ms なら 100 ms に一度は全てのプロセスに CPU が割り付けられる。しかし、 q を小さくしすぎるとコンテキストスイッチの回数が多くなり、オーバーヘッドが大きくなる。逆に q が長いと FCFS と同じ結果になる。

前出の三つのプロセスを RR 方式 ($q = 10$ ms) でスケジューリングした例を次の図に示す。なお、新規プロセスと、クォンタム時間を使い切りプリエンプションしたプロセスが、同時に実行可能列に追加される場合は、新規プロセスを優先することにする。



平均ターンアラウンド時間を計算すると、 $((120 - 0) + (90 - 10) + (130 - 60))/3 = 90$ ms となる。次に $q = 50$ ms でスケジューリングした例を示す。



平均ターンアラウンド時間を計算すると、 $((100 - 0) + (90 - 10) + (130 - 60))/3 = 83$ ms となる。 $q = 50$ ms でスケジューリングした方が、平均ターンアラウンド時間が短くなった上に、コンテキストスイッチの回数が少ない。

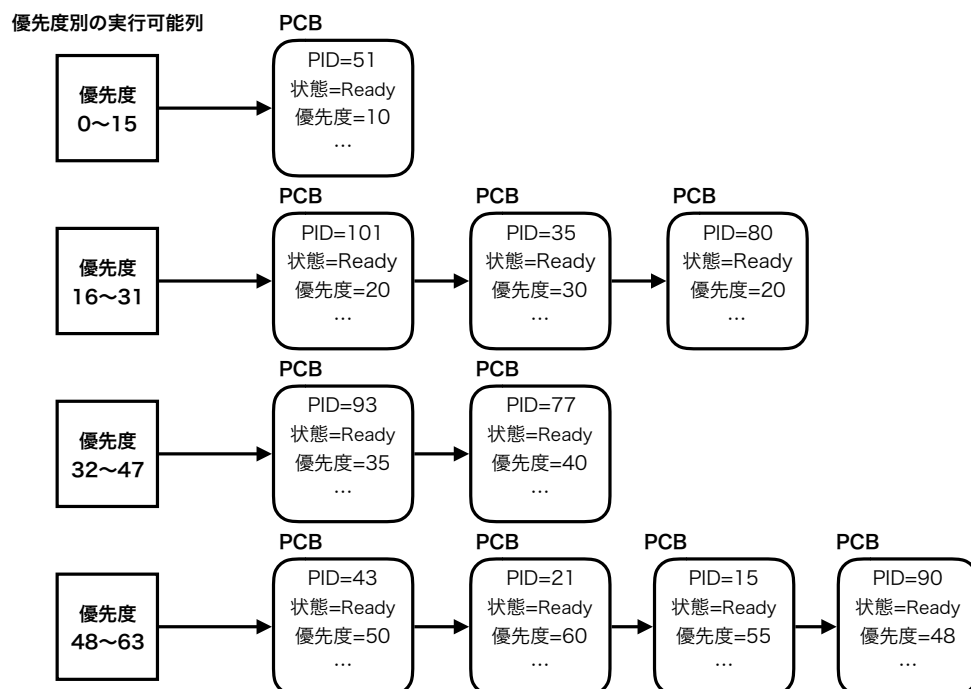


図 4.2: Multilevel Feedback Queue

4.4.5 Priority (優先度順) スケジューリング

プロセス毎に決められた**優先度**を基に行うスケジューリング方式である。実行中に優先度が変化する**動的優先度**を用いる方法と、プロセス生成時に決められ変化しない**静的優先度**を用いる方法がある。SRTF 方式は、残り CPU バースト時間が短い順の**動的優先度**方式と考えられる。

優先度順スケジューリング方式の問題点は、優先度の低いプロセスが全く実行されない**スタベーション** (starvation) が発生することである。これの対策として、実行可能列に留まるプロセスの優先度を徐々に高くしていく**エージング** (aging) が用いられる。実行可能列に長く留まるプロセスは優先度が高くなり、やがて実行される。

4.4.6 Multilevel Feedback Queue (FB) スケジューリング

Windows, macOS, UNIX 等で広く使用されているスケジューリング方式である。図 4.2 に示すように実行可能列を優先度別に複数設ける。優先度が近いプロセスが同じ実行可能列に登録される。同じ実行可能列では RR 方式でスケジューリングするので^{*5}、列内でプロセスの順番は優先度とは関係がない。CPU を割り付ける際は、優先度の高い実行可能列から順に調べ、最初に見つかった空ではない実行可能列を使用する。

プロセスの優先度は動的に変化する方式を用いる。CPU バウンドなプロセスの優先度は急激に引き下げられ、プロセスは下位の実行可能列に移動する。長く実行可能列に留まっているプロセスは、**エージング**により優先度を引き上げられ上位の実行可能列に移動する。実行中のプロセスより上位の実行可能列にプロセスが登録されるとプリエンプションが発生し、プロセスが切り換わる。これにより**スタ**

^{*5} 実行可能列ごとに、異なるスケジューリング方式を採用することも可能である。

バージョンを避ける。

4.5 スケジューラの実装例

19.6 に TacOS のスケジューラを示す。TacOS のスケジューリング方式は、**静的優先度**を用いる優先度順スケジューリング方式である。

4.6 まとめ

本章では CPU スケジューリングについて学んだ。スケジューリングの評価基準はコンピュータの種類などにより変化する。目的に合ったスケジューリング方式が採用される。

FCFS, SJF, SRTF, RR, 優先度順, FB 等のスケジューリング方式を紹介した。PC 等では FB 方式が用いられる。TacOS では優先度順のスケジューリングが用いられる。

練習問題

4.1 次の言葉の意味を説明しなさい。

- (a) スループット
- (b) ターンアラウンド時間・レスポンス時間
- (c) ハードリアルタイム・ソフトリアルタイム
- (d) CPU バウンドプロセス・I/O バウンドプロセス
- (e) FCFS スケジューリング・SJF スケジューリング・SRTF スケジューリング
- (f) RR スケジューリング・優先度順スケジューリング・FB スケジューリング
- (g) クォンタム時間
- (h) スタベーション
- (i) エージング

4.2 次の三つのプロセスの実行順をガントチャートで示しなさい。また、平均ターンアラウンド時間を計算しなさい。

プロセス名	到着時刻 (<i>ms</i>)	CPU バースト時間 (<i>ms</i>)
P_1	0	70
P_2	10	50
P_3	20	30

- (a) FCFS でスケジューリングした場合
- (b) SJF でスケジューリングした場合
- (c) SRTF でスケジューリングした場合
- (d) RR (但しクォンタム時間は 20ms とする.) でスケジューリングした場合
- (e) RR (但しクォンタム時間は 40ms とする.) でスケジューリングした場合
- (f) RR (但しクォンタム時間は 60ms とする.) でスケジューリングした場合

第 5 章

プロセス同期

これまで見てきたように、複数のプロセス（スレッド）が並行して実行される。複数の並行して実行されるプロセス（スレッド）が、決して競合することなく、必要に応じて協調して動作するために、プロセス（スレッド）間で同期をとる必要がある。この章ではプロセス（スレッド）間の同期について勉強する。

5.1 競合 (Race Condition, Competition)

複数のプロセス（スレッド）が資源を共有して処理を進めることがある。ここで言う資源とは「スレッド間で共有する変数」、「プロセス間で共有するメモリ」、「カーネル内部のデータ構造」、「ファイル」、「入出力装置」等が考えられる。共有する資源をプロセス（スレッド）がアクセスする時、きちんとした取り決めが無いと誤った結果になる場合がある。

例えば、銀行口座を管理する架空の例を考えてみよう。一つのプロセス内で、入金処理するスレッドと、引き落としを処理するスレッドが並行して実行されているとする。図 5.1 に、このプロセスの処理内容の一部を TeC 風のアセンブリ言語で示す。プロセスがほぼ同時に、給料 3 万円の振込とカード料金 2 万円の引き落としを受信したとする。二つのスレッドが競って `account` 変数の更新をする。処理前は `account` 変数に口座の残高 10 万円が記録されていたとする。

1. (1) → (2) → (3) → (a) → (b) → (c) の順で実行された場合
`account` 変数の値は 11 万円になり正しい結果になる。
2. (a) → (b) → (c) → (1) → (2) → (3) の順で実行された場合
`account` 変数の値は 11 万円になり正しい結果になる。
3. (1) → (2) → (a) → (b) → (c) → (3) の順で実行された場合
入金管理スレッドが途中でプリエンプションし、引き落としスレッドが実行された後、入金管理スレッドが再開された場合である。`account` 変数の値は 13 万円になる。
4. (1) → (a) → (2) → (b) → (3) → (c) の順で実行された場合
二つの CPU が並列にスレッドを実行した場合である。`account` 変数の値は 8 万円になる。

以上のように、スレッドの実行順序等により計算結果が間違ってしまうことがある。これは資源の利用について、**競合** (*Race Condition* または *Competition*) が発生しているからである。

		// スレッド間の共有変数	
	receipt DS	1	// 入金（3万円）
	payment DS	1	// 引き落とし（2万円）
	account DS	1	// 残高（10万円）
// 入金管理スレッド		// 引き落とし管理スレッド	
// 会社から給料（3万円）を受領し		// カード会社から引き落としを	
// receipt に金額を格納した。		// 受信し payment に金額を格納した。	
// 口座 account に足し込む		// 口座 account から差し引く	
(1)	LD	G0,account	(a) LD G0,account
(2)	ADD	G0,receipt	(b) SUB G0,payment
(3)	ST	G0,account	(c) ST G0,account
// 次の処理に進む		// 次の処理に進む	

図 5.1: 共有変数をアクセスする二つのスレッド

5.2 クリティカルセクション (Critical Section)

競合が発生するのは、一方のスレッドが自分の CPU レジスタにコピーした `account` の値を変更し書き戻すまでの間（変更中）に、もう一方のスレッドが `account` の値を自分の CPU レジスタにコピーすることが原因である。「変更中」の共有変数に他のスレッドがアクセスすることを禁止する必要がある。他のスレッドが共有変数にアクセスすることが許されないプログラムの区間を**クリティカルセクション (Critical Section)**、または、**クリティカルリージョン (Critical Region)** と呼ぶ。

図 5.1 の例で「(1) から (3)」と「(a) から (c)」は `account` 変数のクリティカルセクションであり、この区間をどれかのスレッドが実行している間は、他のスレッドが `account` 変数にアクセスしてはならない。クリティカルセクションの競合問題を効率よく解決するためには、次の三つの条件を満たす必要がある。

1. 二つ以上のプロセス（スレッド）が同時にクリティカルセクションに入らない。
2. クリティカルセクションに入っているプロセス（スレッド）がない時は、待たされることなくクリティカルセクションに入ることができる。
3. クリティカルセクションに入るために永遠に待たされることがない。

5.3 相互排除 (Mutual Exclusion)

複数のプロセス（スレッド）が同時にクリティカルセクションに入らないように制御することである。**排他制御**または**相互排他**とも呼ばれる。**相互排除**を達成するために、プロセス（スレッド）は、クリティカルセクションに入る際に権利を得る手続きを行う。これを行うプログラムの部分を**エントリー**

// 口座 account に足し込む			// 口座 account から差し引く		
	DI	// Entry Section		DI	// Entry Section
(1)	LD	G0,account	(a)	LD	G0,account
(2)	ADD	G0,receipt	(b)	SUB	G0,payment
(3)	ST	G0,account	(c)	ST	G0,account
	EI	// Exit Section		EI	// Exit Section

図 5.2: 割込み禁止による相互排除

セクション (Entry Section) と呼ぶ。クリティカルセクションを出る際に権利を返却する手続きを行う。これを行うプログラムの部分を**エグジットセクション (Exit Section)** と呼ぶ。

5.3.1 割込み禁止

シングルプロセッサ (CPU が一つしかない) システムでは、クリティカルセクションを実行するとき割込みを禁止することで目的を達成できる。図 5.2 に図 5.1 を改良したプログラムを示す。

エントリーセクションで DI (Disable Interrupt) 命令を実行し割込みを禁止する。エグジットセクションで EI (Enable Interrupt) 命令を実行し割込みを許可する。クリティカルセクションでは、CPU が割込みを受け付けない^{*1}のでプリエンプションは発生しない。クリティカルセクションの終わりまで CPU は連続して命令を実行する。また、CPU が一つしかないので、他の CPU が account 変数にアクセスすることもない。よって、account 変数の変更中に他のプロセス (スレッド) が account 変数にアクセスすることはない。

この方法は簡単に相互排除を行うことができるが、割込み禁止時間が長くならないように注意する必要がある。割込み禁止が長くなると、タイマーからの割込みを取りこぼし時計が正確に進まなくなったり、入出力装置の制御が間に合わなくなるなどの弊害が生じる^{*2}。また、DI 命令、EI 命令は特権命令なので、カーネル内だけで使用できる手法である。

5.3.2 専用命令を用いる方式

マルチプロセッサ (CPU が複数ある) システムでは、割込み禁止による方法では目的を達成することができない。クリティカルセクションでプリエンプションが発生しなくても、他の CPU によって実行されるプロセス (スレッド) がクリティカルセクションに入る可能性があるからである。

マルチプロセッサシステムとは、図 2.1 に示したメモリを共有する SMP システムのことである。複数の CPU によるメモリのアクセスはハードウェアにより順序付けされる。同じメモリアドレスへのアクセスが競合し、どちらの CPU が書き込んだ値とも異なる値になることはない。順序付けの結果、後になった書き込みの結果がメモリに残る。また機械語命令は、一部の例外を除いて、途中で割込まれることはない。このようなシステムでは、以下の機械語命令を相互排除の目的に使用できる。

- *TS (Test and Set) 命令*

^{*1} 再度、割込みが許可されるまで保留になる。プリエンプションはクリティカルセクションを出るまで遅延する。

^{*2} 割込み禁止期間に同じ割込みが複数回発生した場合、割込み許可になったとき割込みの種類につき一度だけ割込みが発生する。ハードウェアに、保留になった割込みのカウンタはない。

```

// エントリーセクション
L1      DI                // クリティカルセクションでプリエンブションしないように
      TS      GO, FLG // ゼロを取得できるプロセス（スレッド）は一時には一つだけ
      JZ      L2        // ゼロを取得できた場合だけクリティカルセクションに入れる
      EI                // ビジーウェイティングの間はプリエンブションのチャンスを作る
      JMP     L1        // クリティカルセクションに入れない場合はビジーウェイティング

// クリティカルセクション
L2      ...

// エグジットセクション
      LD      GO, #0
      ST      GO, FLG // フラグのクリアは普通のST命令でOK
      EI                // クリティカルセクション終了、プリエンブションしても良い

// 非クリティカルセクション
      ...

// メモリ上に置いたフラグ（CPUのフラグと混同しないこと）
FLG     DC      0        // 初期値ゼロ（TS命令により1に書き換えられる）

```

図 5.3: TS 命令の使用例

TS 命令は「(1) メモリの値を CPU レジスタにロード」し、「(2) 1 を同じメモリアドレスに書き込む」命令である。この二つを他の CPU のメモリアクセスに割込まれることなく、**アトミック** (*atomic*) に実行する。TS 命令 (TS R,M) の動作は、例えば次のようになる。

1. バスをロックする
2. $R \leftarrow [M]$
3. if (R==0) Zero $\leftarrow$ 1; else Zero $\leftarrow$ 0;
4. $[M] \leftarrow 1$
5. バスのロックを解除する

まず、他の CPU がメモリをアクセスしないようにバスをロックする。次に、メモリの指定番地から値を CPU レジスタにロードする。また、レジスタの値によって CPU の Zero フラグの値を決める。更に、メモリの指定番地に 1 をストアする。最後にバスのロックを解除する。ロードとストアで合計二回のメモリアクセスがあるが、バスがロックされているので、TS 命令の実行途中に他の CPU がメモリをアクセスすることはない。図 5.3 に TS 命令の使用例を示す。JZ 命令は Zero フラグが 1 の場合のみジャンプする。この例のように、エントリーセクションでループして待つ方式を**ビジーウェイティング** (*Busy Waiting*) と呼ぶ。

メモリのクリアは通常の ST 命令でできる^{*3}。TS 命令を用いる場合でも、クリティカルセクショ

^{*3} 通常の命令もメモリアクセスする度にバスをロックしている。

```

// エントリーセクション
L1      DI                // クリティカルセクションでプリエンプションしないように
        LD      G0, #1    // フラグに書き込む値
        SW      G0, FLG    // ゼロを取得できるプロセス（スレッド）は一時には一つだけ
        CMP     G0, #0    // ゼロを取得できたかテスト
        JZ      L2        // ゼロを取得できた場合だけクリティカルセクションに入れる
        EI                // ビジーウェイティングの間はプリエンプションのチャンスを作る
        JMP     L1        // クリティカルセクションに入れない場合はビジーウェイティング

// クリティカルセクション
L2      ...

// エグジットセクション
        LD      G0, #0
        ST      G0, FLG    // フラグのクリアは普通のST命令でOK
        EI                // クリティカルセクション終了，プリエンプションしても良い

// 非クリティカルセクション
        ...

// メモリ上に置いたフラグ（CPUのフラグと混同しないこと）
FLG     DC      0          // 初期値ゼロ（SW命令により1に書き換えられる）

```

図 5.4: SW 命令の使用例

ンは割込み禁止で実行する必要がある。クリティカルセクションでのプリエンプションを避けるためである。もしも、優先度の低いプロセス（スレッド）がクリティカルセクション内でプリエンプションすると、優先度の高いプロセス（スレッド）がエントリーセクションでビジーウェイティングを始めデッドロックに陥る可能性がある。この方式も、特権命令 DI, EI を使用するのでカーネル内でしか利用できない。

- **SW (Swap) 命令**

SW (Swap) 命令も SMP システムでの相互排除に使用できる。「SW R, M」は、以下を**アトミック (atomic)** に実行する。

1. バスをロックする
2. $T \leftarrow [M]$
3. $[M] \leftarrow R$
4. $R \leftarrow T$
5. バスのロックを解除する

ここで T は CPU 内部の一時的なレジスタ (T レジスタの存在はプログラムから見えない) である。図 5.4 に SW 命令の使用例を示す。使用例は TS 命令のものと似ているので解説は省略する。

- **CAS (Compare And Swap) 命令**

// 口座 account に足し込む			// 口座 account から差し引く		
L1	LD	G0,account	L2	LD	G0,account
	LD	G1,G0		LD	G1,G0
	ADD	G1,receipt		SUB	G1,payment
	CAS	G0,G1,account		CAS	G0,G1,account
	JNZ	L1		JNZ	L2

図 5.5: CAS 命令を用いた口座管理プログラムの例

CAS (Compare And Swap) 命令も SMP システムでの相互排除に使用できる。例えば「CAS R0, R1, M」は、以下を**アトミック (atomic)** に実行する。

1. バスをロックする
2. $T \leftarrow [M]$
3. if (T==R0) { $[M] \leftarrow R1$; $Zero \leftarrow 1$; } else { $R0 \leftarrow T$; $Zero \leftarrow 0$; }
4. バスのロックを解除する

CAS 命令を用いたエントリーセクション、エグジットセクションの作り方も、TS 命令と同様なのでここでは使用例を省略する。CAS 命令を用いると共有資源にロックを掛けない、**ロックフリー (Lock-free)** なアルゴリズムを実現できる。前出の銀行口座を管理する架空のプロセス (図 5.1) を CAS 命令を用いて書換えた例を図 5.5 に示す。

処理開始時の `account` の値を `G1` に保存しておく。計算結果を格納する際に、処理開始から `account` の値が変化していないことを確認してから書き込む。以前の例では、他のプロセスが共有資源にアクセスしないように、何らかのロックを掛けていた。この方式はロックを掛けずに「結果を書き込む時点で判断」するので、ロックフリーなアルゴリズムである。

5.3.3 フラグを用いる方式

アルゴリズムを工夫しソフトウェアだけで相互排他を実現する方式である。中でも 1981 年に G. L. Peterson が発表した *Peterson のアルゴリズム (Peterson's solution)* が有名なので紹介する。図 5.6 に Java 風の言語で書いた例を示す。

このアルゴリズムの特徴は次の通りである。

1. マルチプロセッサシステムでも使用できる。
2. 2 プロセス (スレッド) 以上に拡張可能だが複雑になる。
3. 最近のプロセッサと相性が悪い。(out-of-order 実行)

5.4 セマフォ (Semaphore)

これまでに紹介してきた相互排除は、主に**ビジーウェイティング**を用いるものであり、待っている間も CPU を使用し続ける。また、割込み禁止にする必要があるのでカーネル内でしか使用できない。これらは、カーネル内で短時間で終わる相互排除のために適しているが、長時間に渡る場合やユーザプロ

```

// スレッド間の共有変数
boolean flag[] = {false, false}; // クリティカルセクションに入りたい
int turn = 0;                     // 後でやってきたのはどちら

// スレッド0                                // スレッド1
...
// エントリーセクション                    // エントリーセクション
flag[0] = true;                             flag[1] = true;
turn = 0;                                   turn = 1;
while (turn==0 && flag[1]==true)             while(turn==1 && flag[0]==true)
    ; // ビジーウェイティング                ; // ビジーウェイティング

// クリティカルセクション                  // クリティカルセクション
...
// エグジットセクション                    // エグジットセクション
flag[0] = false;                             flag[1] = false;

// 非クリティカルセクション                // 非クリティカルセクション
...

```

図 5.6: Peterson のアルゴリズム

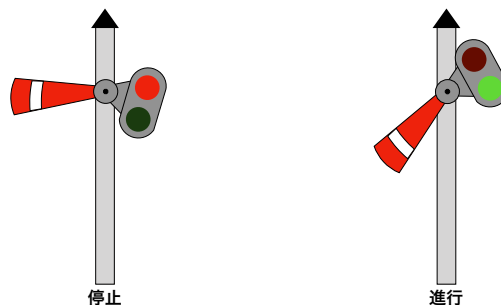


図 5.7: 腕木式信号機

グラムが直接使用する場合には適さない。そこで、オペレーティングシステムが提供するより洗練されたプロセス同期機構である**セマフォ**を紹介する。なお、これまでに紹介してきた相互排除は、セマフォを実現するためにも使用される。

5.4.1 概要

セマフォ (*Semaphore*: 腕木式信号機) は、1965 年に E. W. Dijkstra が提案したデータ型^{*4}である。語源となった腕木式信号機は、鉄道で使用する図 5.7 のような信号機である。

^{*4} C 言語なら構造体を用いてセマフォ型を宣言する。typedef struct { ... } Semaphore;

```

void P(Semaphore S) {
    if (S > 0) {
        S--;
    } else {
        プロセスを待ち(Waiting)状態にする;
        プロセスを S の待ち行列に追加する;
    }
}

```

(a) P 操作

```

void V(Semaphore S) {
    if (S の待ち行列は空) {
        S++;
    } else {
        一つのプロセスを待ち行列から取り出す;
        そのプロセスを実行可能(Ready)状態にする;
    }
}

```

(b) V 操作

図 5.8: セマフォのアルゴリズム

セマフォ型の変数は内部にカウンタ^{*5}を持ち、また、プロセスの待ち行列を作ることができる。セマフォ型 (Semaphore) の変数には、**P 操作** (*Proberen:try*) と **V 操作** (*Verhogen:raise*) を行うことができる。カーネルは P 操作と V 操作を、ユーザプロセスにシステムコールとして提供したり、カーネル内部のサービスモジュールやデバイスドライバにサブルーチンとして提供したりする。セマフォはプロセス (スレッド) の状態を**待ち (Waiting) 状態**に変える。**ビジーウェイティングでは無い**ので CPU を無駄遣いすることはない。

P 操作 (P(S)) セマフォ (S) の値が 1 以上の場合は、セマフォの値を 1 減らす。そうでない場合は、プロセス (スレッド) を待ち (Waiting) 状態にし、セマフォの待ち行列に追加する。アルゴリズムを C 言語風に記述したものを図 5.8a に示す。

V 操作 (V(S)) セマフォ (S) の待ち行列にプロセス (スレッド) がある場合は、それらの一つを起床させる。待っているプロセス (スレッド) が無い場合は、セマフォ (S) の値を 1 増やす。アルゴリズムを C 言語風に記述したものを図 5.8b に示す。

5.4.2 相互排除問題の解

初期値が 1 のセマフォを用いて相互排除問題の解を示すことができる。前出の架空の銀行口座管理プロセスの例を、セマフォを用いて解決したものをリスト 5.1 に示す。

1 行の `account` は相互排除が必要なスレッド間の共有変数である。2 行の `Semaphore` 型の変数 `accSem` が排他制御に使用するセマフォである。`accSem` は 1 で初期化される。クリティカルセクションに入るスレッドは、まず、6 行か 14 行で `accSem` に P 操作を行う。どちらか先にやって来たスレッドが P 操作を行った時点で `accSem` の値が 0 になる。

遅れてやって来たスレッドは `accSem` の値が 0 の間はクリティカルセクションに入ることができない。先のスレッドがクリティカルセクションを出て 8 行か 16 行で `accSem` に V 操作を行ったら、後のスレッドがクリティカルセクションに入ることができる。

^{*5} 腕木信号機の進行・停止のように二つの状態しか取らないものを**バイナリセマフォ**と呼ぶ。ここで取り上げるカウンタを持つものは**カウンティングセマフォ**と呼ぶ。カウンタの値は 0 以上の整数値である。

リスト 5.1: セマフォを用いた相互排除問題の解

```

1  int      account;                // スレッド間の共有変数 (残高)
2  Semaphore accSem = 1;            // 初期値1のセマフォaccSem (accountのロック用)
3  void receiveThread() {           // 入金管理スレッド
4      for ( ; ; ) {                // 入金管理スレッドは以下を繰り返す
5          int receipt = receiveMoney(); // ネットワークから入金を受信する
6          P( &accSem );              // account変数をロックするための P 操作
7          account = account + receipt; // account変数を変更する(クリティカルセクション)
8          V( &accSem );              // account変数をロック解除するための V 操作
9      }
10 }
11 void payThread() {                // 引落とし管理スレッド
12     for ( ; ; ) {                 // 引落とし管理スレッドは以下を繰り返す
13         int payment = payMoney(); // ネットワークから支払い額を受信する
14         P( &accSem );              // account変数をロックするための P 操作
15         account = account - payment; // account変数を変更する(クリティカルセクション)
16         V( &accSem );              // account変数をロック解除するための V 操作
17     }
18 }

```

5.4.3 生産者と消費者問題 (Producer-Consumer Problem) の解

生産者スレッドはデータを生産し有限な長さのリングバッファ (*ring buffer*) に書き込む。消費者スレッドはリングバッファからデータを読み出し消費する。この時、満杯のリングバッファに更に書き込んだり、空のリングバッファからデータを読み出したりしないように、スレッド間で歩調を合わせる (同期する) 必要がある。セマフォを用いた解をリスト 5.2 に示す。

リングバッファとセマフォ 1 行の `buffer` は大きさ `N` のリングバッファである。型は応用によって決まるので、リングバッファの型は仮に `Data` 型としている。2 行の `emptySem` はリングバッファの空きスロット数を表すセマフォである。最初は全てのスロットが空きなので初期値は `N` である。3 行の `fullSem` はリングバッファの使用スロット数を表すセマフォである。最初は使用中のスロットが無いので、初期値を `0` にしている。

生産者スレッド 4 行から始まる `producerThread` が、データを生産しリングバッファに書き込むスレッドである。5 行の変数 `in` はリングバッファの次回書き込み位置を表すローカル変数である。0,1,2,...,N-1,0,1,2,... の順に値が変化する。in はスレッドのローカル変数なので、相互排除をする必要がない。

`producerThread` は、7 行でデータを作り、8 行で空きスロット数が 1 以上なら `emptySem` の値を減らして、9 行でデータをリングバッファに書き込む。10 行で `in` の値を更新している。11 行で使用中スロット数 `fullSem` の値を増加させる。

消費者スレッド 14 行から始まる `consumerThread` は、データをリングバッファから読み出して消費するスレッドである。15 行の変数 `out` はリングバッファの次回読出し位置を表すローカ

リスト 5.2: セマフォを用いた生産者消費者問題の解

```

1 Data      buffer[N];           // スレッド間で共有するリングバッファ
2 Semaphore emptySem = N;        // リングバッファの空きスロット数を表すセマフォ
3 Semaphore fullSem  = 0;        // リングバッファの使用スロット数を表すセマフォ
4 void producerThread() {       // 生産者スレッド
5     int in = 0;                // リングバッファの次回格納位置
6     for ( ; ; ) {              // 生産者スレッドは以下を繰り返す
7         Data d = produce();     // 新しいデータを作る
8         P( &emptySem );         // リングバッファの空き数をデクリメント
9         buffer[ in ] = d;       // リングバッファにデータを格納
10        in = (in + 1) % N;      // 次回格納位置を更新
11        V( &fullSem );         // リングバッファのデータ数をインクリメント
12    }
13 }
14 void consumerThread() {       // 消費者スレッド
15     int out = 0;               // リングバッファの次回取り出し位置
16     for ( ; ; ) {              // 消費者スレッドは以下を繰り返す
17         P( &fullSem );         // リングバッファのデータ数をデクリメント
18         Data d = buffer[ out ]; // リングバッファからデータを取り出す
19         out = (out + 1) % N;    // 次回取り出し位置を更新
20         V( &emptySem );        // リングバッファの空き数をインクリメント
21         consume( d );          // データを使用する
22    }
23 }

```

ル変数である。out もスレッドのローカル変数なので、相互排除をする必要がない。

consumerThread は、17 行で空きスロット数が 1 以上なら fullSem の値を減らして、18 行でデータをリングバッファから読み出す。19 行で out の値を更新する。20 行で空きスロット数 emptySem の値を増加させる。21 行で読み出したデータを使用する。

5.4.4 複数生産者と複数消費者問題 (Producers-Consumers Problem) の解

前の問題で、関数 producerThread(), consumerThread() それぞれについて、複数のスレッドが存在する場合を考える。バッファに関する同期の他に、書き込み位置 (in)、取り出し位置 (out) に関する排他制御が必要になる。解をリスト 5.3 に示す。

リングバッファとセマフォ 1 行から 3 行に変更はない。

生産者スレッド 次回書き込み位置を表す in 変数を複数の producerThread で共有する必要がある。in 変数の宣言を 4 行に移動し、スレッド間の共有変数に変更した。また、in 変数を producerThread 間で相互排除するためのセマフォ inSem を 5 行に追加した。

producerThread では、in 変数の参照や書き換えを行う 11 行と 12 行が in 変数に関するクリティカルセクションである。10 行と 13 行に inSem を用いた相互排除機構を追加した。

リスト 5.3: セマフォを用いた複数生産者・複数消費者問題の解

```

1 Data      buffer[N];                // スレッド間で共有するリングバッファ
2 Semaphore emptySem = N;             // リングバッファの空きスロット数を表すセマフォ
3 Semaphore fullSem  = 0;             // リングバッファの使用スロット数を表すセマフォ
4 int        in = 0;                  // リングバッファの次回格納位置
5 Semaphore inSem = 1;                // in の排他制御用セマフォ
6 void producerThread() {            // 生産者スレッド (複数のスレッドで並列実行する)
7     for ( ; ; ) {                  // 生産者スレッドは以下を繰り返す
8         Data d = produce();         // 新しいデータを作る
9         P( &emptySem );             // リングバッファの空き数をデクリメント
10        P( &inSem );                // in にロックを掛ける
11        buffer[ in ] = d;           // リングバッファにデータを格納
12        in = (in + 1) % N;          // 次回格納位置を更新
13        V( &inSem );               // in のロックを外す
14        V( &fullSem );              // リングバッファのデータ数をインクリメント
15    }
16 }
17 int out = 0;                        // リングバッファの次回取り出し位置
18 Semaphore outSem = 1;               // out の排他制御用セマフォ
19 void consumerThread() {            // 消費者スレッド (複数のスレッドで並列実行する)
20     for ( ; ; ) {                  // 消費者スレッドは以下を繰り返す
21         P( &fullSem );             // リングバッファのデータ数をデクリメント
22         P( &outSem );              // out にロックを掛ける
23         Data d = buffer[ out ];     // リングバッファからデータを取り出す
24         out = (out + 1) % N;        // 次回取り出し位置を更新
25         V( &outSem );              // out のロックを外す
26         V( &emptySem );            // リングバッファの空き数をインクリメント
27         consume( d );              // データを使用する
28     }
29 }

```

消費者スレッド 次回読み出し位置を表す `out` 変数について、生産者スレッドと同様な相互排除機構を追加してある。

5.4.5 リーダ・ライタ問題 (Readers-Writers Problem) の解

資源 (共有データ) に対して、読み出しだけを行うリーダプロセス (リーダスレッド) と、読み書き両方を行うライタプロセス (ライタスレッド) の二種類がある場合に、単に資源をロックするより**並行性 (concurrency)** を高くすることができる。リーダスレッドは、値を読み出すだけなので、他のリーダスレッドと同時に共有データをアクセスしても良い。ライタスレッドは、値を書換えるので、他のライタスレッドともリーダスレッドとも同時に共有データをアクセスすることは許されない。セマフォによる解をリスト 5.4 に示す。

共有データとセマフォ 1 行の `records` が共有データである。2 行の `rwSem` は共有データの相互

リスト 5.4: セマフォを用いたリーダ・ライタ問題の解

```

1 Data      records;                // 共有するデータ
2 Semaphore rwSem = 1;              // リーダとライタの排他用セマフォ
3 void writerThread() {              // ライタスレッド (複数のスレッドで並列実行する)
4     for ( ; ; ) {                  // ライタスレッドは以下を繰り返す
5         Data d = produce();         // 新しいデータを作る
6         P( &rwSem );                // 共有データにロックを掛ける
7         writeRecores( d );          // データを書換える
8         V( &rwSem );                // 共有データのロックを外す
9     }
10 }
11 int      cnt = 0;                  // リーダ間の共有変数 (読出し中のリーダ数)
12 Semaphore cntSem = 1;              // cnt の排他制御用セマフォ
13 void readerThread() {              // リーダスレッド (複数のスレッドで並列実行する)
14     for ( ; ; ) {                  // リーダスレッドは以下を繰り返す
15         P( &cntSem );                // cnt にロックを掛ける
16         if ( cnt == 0 ) P( &rwSem ); // 自分が最初のリーダなら、代表してロックする
17         cnt = cnt + 1;              // cnt をインクリメント
18         V( &cntSem );                // cnt のロックを外す
19         Data d = readRecords();      // データを読み出す
20         P( &cntSem );                // cnt にロックを掛ける
21         cnt = cnt - 1;              // cnt をデクリメント
22         if ( cnt == 0 ) V( &rwSem ); // 自分が最後のリーダなら、代表してロックを外す
23         V( &cntSem );                // cnt のロックを外す
24         consume( d );                // データを使用する
25     }
26 }

```

排除用のセマフォである。これらは、全てのスレッドに関係がある。

ライタスレッド 3 行の `writerThread` は共有データを書き換えることがあるスレッドである。書き換え途中に他のスレッドが共有データをアクセスすることを禁止するために、`writerThread()` は 6 行で `rwSem` にロックを掛ける。8 行でロックを解除するまで、他のライタスレッドもリーダリーダも同時に共有データにアクセスすることはできない。このようなロックを**排他ロック** (*exclusive lock*) と呼ぶ。

リーダスレッド 13 行の `readerThread` は共有資源を読むことだけする。書き換え途中の不完全なデータを読み出さないように、`writerThread` と相互排除を行う必要がある。しかし、書き換え途中以外なら、他のリーダスレッドと同時にデータを読んでも構わない。

11 行の `cnt` 変数はリーダスレッド間で共有される。12 行の `cntSem` セマフォは `cnt` 変数の相互排除用である。リーダスレッドはこれらを使用し、`records` 共有データを読み出し中のリーダスレッドの数を管理する。16 行と 17 行、21 行と 22 行の二箇所が、`cnt` 変数に関するクリティカルセクションである。

16 行では最初に読み出しを始めるリーダスレッドを判断し、最初のリーダスレッドだけが代表して `rwSem` にロックを掛ける。二番目にやって来たリーダスレッドはロックを掛けないので、リーダスレッド相互は排他されない。しかし、排他ロックを用いるライタスレッドとは相互排除される。このようなロックを**共有ロック** (*shared lock*) と呼ぶ。22 行で最後に読み出しを終えるリーダスレッドを判断し、最後のリーダスレッドだけが代表して `rwSem` のロックを解除する。

リーダ・ライタ問題は、共有ロックと排他ロックを使用する問題の例になっている。共有ロックと排他ロックの考え方は、ここに示したスレッド間の共有変数の管理だけでなく様々な場面で使用される。例えば UNIX のシステムコール `flock` は、引数に定数 `LOCK_SH` を渡すと共有ロックを、定数 `LOCK_EX` を渡すと排他ロックをファイルに掛ける。また、UNIX の `open` システムコールは、引数に `O_SHLOCK` フラグを指定すると共有ロックを、引数に `O_EXLOCK` フラグを指定すると排他ロックを、ファイルのオープン時に自動的に掛ける。

5.5 セマフォの実装例

第 20 章に TacOS におけるセマフォの実装例を紹介する。TacOS のセマフォはカウンティングセマフォである。TacOS では、全てのプロセス間同期・排他はセマフォを使用して実現されている。

5.6 まとめ

この章ではプロセス間の同期に関する話題を取り上げた。**クリティカルセクション**を実行する時は、**競合**が発生しないようにプロセス間の**相互排除**をする必要がある。オペレーティングシステムのカーネル内部などで、短時間で終わるクリティカルセクションの相互排除を行う場合は、**割込み禁止**や**専用命令**を**ビジーウェーティング**と組み合わせて使用する方法が使用できる。専用命令として TS 命令、SW 命令、CAS 命令を紹介した。CAS 命令は**ロックフリー**なアルゴリズムを実現するために使用できる。

クリティカルセクションの実行に長い時間がかかる場合は、セマフォなど、プロセスの状態遷移を伴うオペレーティングシステムの機能を使用する。セマフォを用いた**相互排除問題**の解、**生産者と消費者問題**の解、**複数生産者と複数消費者問題**の解、**リーダ・ライタ問題**の解を学んだ。

練習問題

5.1 次の言葉の意味を説明しなさい.

- (a) 競合
- (b) クリティカルセクション
- (c) 相互排除
- (d) ビジーウェイティング
- (e) ロックフリーなアルゴリズム
- (f) セマフォ
- (g) 相互排除問題
- (h) 生産者と消費者問題
- (i) リーダライタ問題

5.2 なぜ割込みを禁止することで相互排除ができるか？

5.3 割込み禁止による相互排除がマルチプロセッサシステムでは不十分な理由は？

5.4 割込み禁止による相互排除はクリティカルセクションの三つの条件を満たしているか？

5.5 CPU が割込み禁止になっている間に発生した割込みはどのように扱われるか？

5.6 DI 命令や EI 命令が特権命令でなかったら、どのような不都合が生じるか？

5.7 シングルプロセッサシステムにおいて、機械語命令は**アトミック (atomic)** と言えるか？

5.8 マルチプロセッサシステムにおいて、機械語命令は**アトミック (atomic)** と言えるか？

5.9 TS 命令と SW 命令に共通な特長は何か？

5.10 図 5.3 のようなビジーウェイティングはシングルプロセッサシステムでも使用できるか？

5.11 セマフォを相互排除に使用する手順を説明しなさい.

5.12 生産者と消費者の問題において、二つのセマフォはどのような値に初期化されたか？

二つのセマフォは何の役割を持っていたか？

第 6 章

プロセス間通信

この章ではプロセス間通信（IPC：Inter-Process Communication）について学ぶ。5 章で学んだ、「生産者と消費者の問題」や「リーダー・ライタ問題」の具体的な解を得るためには、プロセス間で情報を共有する必要がある。プロセス間で情報を共有する代表的な機構として、**共有メモリとメッセージ通信**がある。複数のプロセスが情報を共有し協調して処理を進めることで、次のメリットが期待できる。

- 複数のプロセスが共通の情報へアクセスすることができる。
- 並列処理により、処理時間の短縮が期待できる。
- システムを見通しの良いモジュール化された構造で構築できる。

6.1 共有メモリ

共有メモリは図 6.1 に示すように、プロセス間で同じ物理メモリを共有する方式である。プロセス 1 とプロセス 2 は同じ物理メモリ（共有メモリ）を、それぞれの仮想メモリ空間に貼り付けている。

メモリ管理のハードウェア（Memory Management Unit：MMU）^{*1}を適切に設定することで、複数のプロセスの仮想メモリ空間に同じ物理メモリを貼り付ける。メモリを貼り付ける操作はシステムコールを用いて行う。貼り付けが完了した後は、システムコールを用いることなく情報の共有が可能であるが、プロセス間の同期機構は別に準備する必要がある。

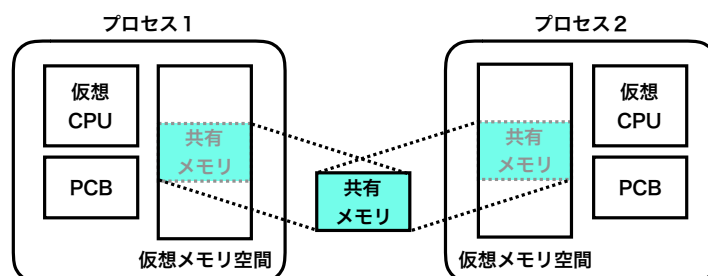


図 6.1: 共有メモリ

^{*1} MMU については「第 III 部 メモリ管理」で解説する。

共有メモリに関するライブラリとシステムコール

共有メモリなどの識別に使用するキーを生成（ライブラリ）

```
key_t ftok(const char *path, int id);
```

戻り値 : 引数から作成されるキー値

path : 実際に存在するファイルのパス

id : キーの作成に使用する追加の情報（同じpathから異なるキーを作る）

共有メモリセグメントの作成（システムコール）

```
int shmget(key_t key, size_t size, int flag);
```

戻り値 : 共有メモリセグメントID

key : キー

size : セグメントサイズ（バイト単位）

flag : 作成フラグとモード

共有メモリセグメントをプロセスの仮想アドレス空間に貼り付ける（システムコール）

```
void *shmat(int shmId, void *addr, int flag);
```

戻り値 : 共有メモリセグメントを配置したアドレス

shmId : 共有メモリセグメントID

addr : 貼り付けるアドレス（NULL(0)は、カーネルに任せる）

flag : 貼り付け方法等

共有メモリセグメントをプロセスの仮想アドレス空間から取り除く（システムコール）

```
int shmdt(void *addr);
```

戻り値 : 0=正常, -1=エラー

addr : 取り除く共有メモリセグメントのアドレス

共有メモリセグメントの制御（システムコール）

```
int shmctl(int shmId, int cmd, struct shmid_ds *buf);
```

戻り値 : 0=正常, -1=エラー

shmId : 共有メモリセグメントID

cmd : 削除(IPC_RMID)等のコマンド

buf : コマンドのパラメータ

図 6.2: UNIX の共有メモリ関連システムコールとライブラリ関数

6.1.1 UNIX の共有メモリ関連システムコール等

UNIX の共有メモリ関連のシステムコールとライブラリ関数を図 6.2 に紹介する。

ftok() ライブラリ関数 ftok() は、path と id の組合せからから、システム内で一意な key 値を生成する。

shmget システムコール key 値で識別される共有メモリセグメントの ID 返す。key 値で識別される共有メモリセグメントが存在しない場合は、size バイトのものを新しく作ることも可能である。flag の値は共有メモリのアクセス許可ビット（rw-rw-rw-）と、IPC_CREAT 等のフラ

[Terminal No.1]	[Terminal No.2]
\$./ipcUnixShearedMemoryServer	
sheared memory:initialization...	\$./ipcUnixShearedMemoryClient
sheared memory:initialization...	Enter a string: abcdefg
sheared memory:abcdefg	
sheared memory:abcdefg	\$./ipcUnixShearedMemoryClient
sheared memory:abcdefg	Enter a string: 1234567
sheared memory:1234567	\$./ipcUnixShearedMemoryClient
sheared memory:1234567	Enter a string: end
sheared memory:end	
\$	\$

図 6.3: UNIX のメモリ共有プログラム実行例

グである。

shmatt システムコール 共有メモリセグメントを ID (shmId) で指定し、プロセスの仮想アドレス空間に貼り付ける。

shmdt システムコール 共有メモリセグメントをアドレス (addr) で指定し、プロセスの仮想アドレス空間から取り除く。

shmctl システムコール 共有メモリセグメントを ID (shmId) で指定し操作する。共有メモリセグメントの削除等の操作ができる。

6.1.2 UNIX の共有メモリ使用例

共有メモリセグメントを作成し、そこから定期的にデータを読み出し表示するサーバプログラムの例をリスト 6.1 に示す^{*2}。また、サーバプログラムが作成した共有メモリセグメントにデータを書き込むクライアントプログラムの例をリスト 6.2 に示す。

サーバプログラムでは、28 行の `printf()` が共有メモリ (data) から文字列を読み出し表示する。文字列が `end` ならプログラムを終了する。クライアントプログラムでは、27 行の `fgets()` が共有メモリ (data) に文字列を書き込む。これらのプログラムでは、共有メモリが普通の文字配列のように `printf()` や `fgets()` に渡されている。共有メモリなので、`fgets()` が書き込んだ内容を `printf()` が読み出すことになる。

実行例は図 6.3 のようになる。図は二つのターミナルを開いて操作した状態を示している。左半分が第一のターミナル、右半分が第二のターミナルである。まず、左のターミナルでサーバプログラム (`ipcUnixSharedMemoryServer`) を起動する。これで共有メモリセグメントが準備された。次に、右のターミナルでクライアントプログラム (`ipcUnixSharedMemoryClient`) を起動する。この状態で右のターミナルに入力した文字列が、クライアントプログラムにより共有メモリに書き込まれる。左のターミナルで実行中のサーバプログラムは、共有メモリの内容を定期的に表示する。

ここに紹介した簡単なプログラムでは、クライアントプロセスがデータを書き換え中に、サーバプロ

^{*2} ここで示すプログラムは macOS 10.13.2 で動作確認してあるが、他の UNIX でも動作するはずである。

リスト 6.1: UNIX の共有メモリサーバ例

```
1 // 共有メモリサーバ (ipcUnixSharedMemoryServer.c) : 共有メモリからデータを読みだし表示する
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <unistd.h>
6 #include <sys/types.h>
7 #include <sys/ipc.h>
8 #include <sys/shm.h>
9 #define SHMSZ      512                                // 共有メモリのサイズ
10 int main() {
11     key_t key=ftok("shm.dat",'R');                    // キーを作る
12     if (key==-1) {                                     // エラーチェック
13         perror("shm.dat");
14         exit(1);
15     }
16     int shmid=shmget(key,SHMSZ,IPC_CREAT|0666);        // 共有メモリを作る
17     if (shmid<0) {                                     // エラーチェック
18         perror("shmget");
19         exit(1);
20     }
21     char *data=shmat(shmid,NULL,0);                   // 共有メモリを貼り付ける
22     if (data==(char *)-1) {                           // エラーチェック
23         perror("shmat");
24         exit(1);
25     }
26     strcpy(data, "initialization...\n");              // 共有メモリに書き込む
27     while(1) {                                         // 共有メモリの内容を
28         printf("sheared memory:%s",data);             //   5秒に1度メモリを表示
29         if (strcmp(data, "end\n") == 0) break;        //   "end"なら終了
30         sleep(5);
31     }
32     if (shmdt(data) == -1){                            // 共有メモリをアドレス空間
33         perror("shmdt");                               //   と切り離す
34         exit(1);
35     }
36     if (shmctl(shmid, IPC_RMID, 0) == -1){            // 共有メモリを廃棄する
37         perror("shmctl");
38         exit(1);
39     }
40     return 0;
41 }
```


リスト 6.2: UNIX の共有メモリクライアント例

```

1 // 共有メモリクライアント (ipcUnixSharedMemoryClient.c) : 共有メモリにデータを書き込む
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <errno.h>
5 #include <sys/types.h>
6 #include <sys/ipc.h>
7 #include <sys/shm.h>
8 #define SHMSZ      512                // メモリのサイズ
9 int main() {
10     int      shmid;
11     key_t    key;
12     char     *data, *s;
13     if ((key=ftok("shm.dat",'R')) == -1) { // サーバ側と同じキーを作る
14         perror("shm.dat");
15         exit(1);
16     }
17     if ((shmid=shmget(key,SHMSZ,0666))<0) { // 共有メモリを取得する
18         perror("shmget");
19         exit(1);
20     }
21     data=shmat(shmid,NULL,0);           // 共有メモリを貼り付ける
22     if (data == (char *)-1) {           // エラーチェック
23         perror("shmat");
24         exit(1);
25     }
26     printf("Enter a string: ");
27     fgets(data,SHMSZ,stdin);           // 共有メモリに直接入力する
28     if (shmdt(data)==-1){               // 共有メモリをメモリ空間と
29         perror("shmdt");               // 切り離す
30         exit(1);
31     }
32     return 0;
33 }

```

セスがデータを読み出す可能性がある。このようなプログラムを使用してはならない。実際に使用する場合は書き換え中のデータを読み出さないように、セマフォ等^{*3}を用いて相互排除を行う必要がある。原理の確認以外の目的に、このプログラムを使用してはならない。

^{*3} UNIX ではセマフォも使用できる。

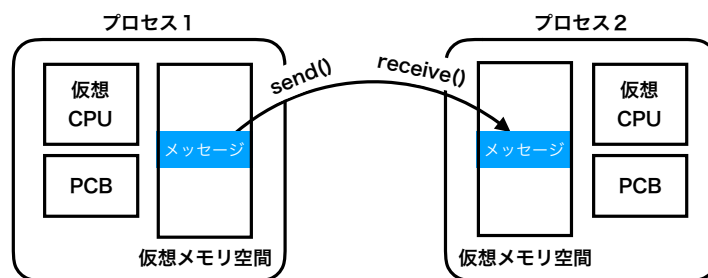


図 6.4: メッセージ通信

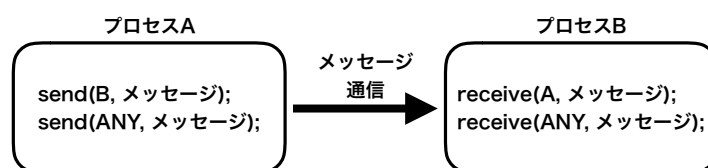


図 6.5: 直接指定方式

6.2 メッセージ通信

メッセージ通信は図 6.4 に示すように、システムコールを用いてプロセス間で情報をコピーする方式である。プロセス1は send システムコールを用いてプロセス2へメッセージを送る。プロセス2は receive システムコールを用いてプロセス1からメッセージを受取る。メッセージ通信は、データを送る度にシステムコールを使用するのでオーバーヘッドが大きいですが、プロセス間の同期機構としても働く。

6.2.1 通信相手の指定方式 (Naming)

メッセージの通信相手を指定する方式が二つある。

直接指定方式 相手プロセスを直接指定する方式である。図 6.5 は直接指定方式を表している。

send, receive システムコールの引数は、**相手プロセスとメッセージ**になる。**相手プロセス**として ANY のような記述を許すことで、多対多の通信も可能である。また、受信したメッセージをいくつか貯めることが可能な、バッファ付きの通信方式もあり得る。

間接指定方式 リンク（ポート、ソケット、チャネルとも呼ばれる）を作成し、通信先としてリンクの名前を用いる方式である。図 6.6 は間接指定方式を表している。send, receive システムコールの引数は、**リンクとメッセージ**になる。同じリンクを共有する複数のプロセスが存在すると、自然に多対多の通信方式が実現できる。リンクにメッセージをいくつか貯めるバッファ機能を持たせる。

6.2.2 バッファリング (Buffering)

直接指定方式か間接指定方式かに関わりなく、メッセージを格納するバッファを用意することができる。送信プロセスはバッファに空きがあれば待ち時間なしに send システムコールを完了できる。受信プロセスはバッファにメッセージがあれば待ち時間なしに receive システムコールを完了できる。

間接指定方式ではリンクがバッファを持つと考え、リンクを作成する時点でバッファの大きさを指定

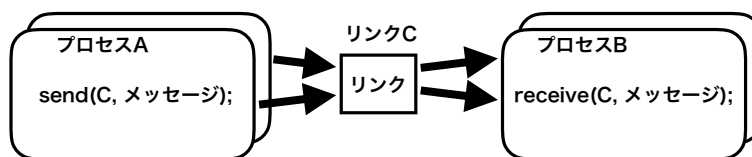


図 6.6: 間接指定方式

する人が多い。図 6.6 で「リンク」の位置にバッファがあると考えると分かりやすい。

6.2.3 メッセージの形式

通信に用いられるメッセージの形式には次の選択肢がある。

メッセージ長 固定長方式または可変長方式

メッセージ形式 タグ付きまたはタグなし

タグは種類を表すためにメッセージに付加されるデータのことである。タグ付きのメッセージ通信機構では、送信側はメッセージにタグを付加する。受信側はタグを指定してメッセージを選択的に受信することができる。

6.2.4 同期方式 (Synchronization)

非同期方式 (ノンブロッキング: *Nonblocking*) と **同期方式** (ブロッキング: *Blocking*) の二つがある。同期方式の特別な場合としてクライアント・サーバモデルに特化した**ランデブー方式**^{*4}も考えられる。

非同期方式 `send()` はバッファに空きがない場合エラーで終了する。`receive()` はバッファにメッセージがない場合エラーで終了する。

同期方式 `send()` はバッファに空きがない場合はブロックし、空きができるのを待つ。`receive()` はバッファにメッセージがない場合はブロックし、メッセージが届くのを待つ。

ランデブー方式 サーバプロセスはクライアントを待つ。クライアントプロセスはサーバに処理を依頼し待ち状態になる。サーバは処理を行い結果をクライアントに返信する。クライアントは処理結果を受信したら実行を再開する。

6.2.5 UNIX のメッセージ通信システムコール

UNIX では複数種類のメッセージ通信機構が利用可能である。ここでは、System V 系の UNIX を起源とする方式を紹介する。この方式は、**間接指定方式**、**バッファリングあり**、**可変長**、**タグ付き**の方式である。システムコールの引数によって、**同期方式**と**非同期方式**のどちらにも対応することができる。UNIX のメッセージ通信関連のシステムコール等を図 6.7 に示す。

msgBuf 構造体 ユーザが宣言する構造体である。必ず、long 型の `mtype` フィールドから始める必要がある。このフィールドが**タグ**の役割を持つ。`mtext` はメッセージの本体を格納する領域であり、ユーザが自由に大きさや用途を決めることができる。

msgget システムコール リンク (メッセージキューと呼ぶ) の ID を返す。`key` は、共有メモリの

^{*4} Ada のランデブ、ITRON のランデブポート機能のことである。TacOS のメッセージ通信もランデブー方式である。

メッセージ通信に関するシステムコールとデータ構造

メッセージ構造体（以下の構造体を自分で宣言して使用する）

```
struct msgbuf {
    long mtype;        // メッセージの型
    char mtext[N];     // メッセージの本体（Nバイト）
};
```

メッセージキューのIDを返す。

int msgget(key_t key, int msgflg); (システムコール)

戻り値 : メッセージキューID

key : キー (ftok())で作成したもの

msgflg : IPC_CREAT等のフラグとアクセス許可ビット

メッセージキューにメッセージを送信する (システムコール)

int msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg);

戻り値 : 0=正常, -1=エラー

msqid : メッセージキューID

msgp : メッセージ構造体のポインタ

msgsz : メッセージ本体のバイト数

msgflg : IPC_NOWAIT等のフラグ

メッセージキューからメッセージを受信する (システムコール)

int msgrcv(int msqid, const void *msgp, size_t msgsz, long msgtyp, int msgflg);

戻り値 : -1=エラー、受信したメッセージの本体バイト数

msqid : メッセージキューID

msgp : メッセージ構造体のポインタ

msgsz : メッセージ本体の最大バイト数

msgtyp : 受信するメッセージの型

msgflg : IPC_NOWAIT等のフラグ

メッセージキューの制御 (システムコール)

int msgctl(int msqid, int cmd, struct msgid_ds *buf);

戻り値 : -1=エラー、0<=コマンドにより異なる

msqid : メッセージキューID

cmd : 削除 (IPC_RMID) 等のコマンド

buf : コマンドのパラメータ

図 6.7: UNIX のメッセージ通信関連システムコールとデータ構造

リスト 6.3: UNIX のメッセージ通信プログラム例 (メッセージ構造体)

```

1 // ipcUnixMessage.h : メッセージ構造体の宣言
2 #define MAXMSG 100                                // メッセージ本体の長さ
3 struct msgBuf {                                     // メッセージ格納用構造体
4     long mtype;                                     // メッセージの型
5     char mtext[MAXMSG];                             // メッセージの本体
6 };

```

場合と同様に `ftok()` 関数を用いて生成した値である。メッセージキューを識別するために用いる。 `msgflg` に `IPC_CREAT` を指定すると、メッセージキューを新規に作成する。

msgsnd システムコール `msqid` で指定したメッセージキューにメッセージを送信する。 `msgp` に送信するメッセージを格納した `msgBuf` 構造体のポインタを渡す。メッセージは**可変長方式**なので `msgsz` で長さを指定する。 `msgsz` は構造体全体ではなく、構造体の `mtext` 部分のバイト数である。 `msgflg` に `IPC_NOWAIT` フラグを指定すると**非同期方式**になり、指定しないと**同期方式**になる。

msgrcv システムコール `msqid` で指定したメッセージキューからメッセージを受信する。 `msgp` に受信したメッセージを格納する `msgBuf` 構造体のポインタを渡す。 `msgsz` は受信可能な `mtext` の最大バイト数である。 `msgtyp` に受信したいメッセージの `mtype` を指定し、**タグ**が合致するメッセージを選択的に受信できる。 `msgflg` に `IPC_NOWAIT` フラグを指定すると**非同期方式**になる。

msgctl システムコール `msqid` で指定したメッセージキューに対して操作を行う。 `cmd` に操作の種類 (コマンド)、 `buf` にコマンドのパラメータを渡す。 `IPC_RMID` コマンドを指定するとメッセージキューの削除ができる。

6.2.6 UNIX のメッセージ通信プログラム例

メッセージを表現する構造体の例をリスト 6.3 に示す^{*5}。メッセージ本体の長さは `MAXMSG` に定義している。以下のプログラムは、メッセージ長をこの値に固定した例になっている。

リスト 6.4 にメッセージキューを作成しメッセージを書き込むプログラムの例を示す。このプログラムは入力した文字列をメッセージ本体に格納してメッセージキューに送信する。タグの役割を持つ `mtype` は常に 1 にしている。

リスト 6.5 に、メッセージキューからメッセージを読み込み内容を表示するプログラムの例を示す。22 行で `msgtyp` を 0 にして `msgrcv()` を実行している。 `msgtyp` が 0 の場合は、メッセージの `mtype` (タグ) を無視してメッセージキューの先頭から順にメッセージを受信する。26 行で `mtype` と `mtext` の内容を表示している。送信側のプログラムがメッセージキューを削除すると 22 行でエラーが発生し 24 行で終了する。

^{*5} ここで紹介するプログラムは macOS 10.13.2 で動作確認した。 macOS のオンラインマニュアルには、ここで紹介するメッセージ通信方式について記載がないが、試してみると使用できた。

リスト 6.4: UNIX のメッセージ通信プログラム例 (メッセージ送信側)

```

1 // メッセージ送信プログラム (ipcUnixMessageWriter.c) : メッセージキューを作成し送信する
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <sys/types.h>
5 #include <sys/ipc.h>
6 #include <sys/msg.h>
7 #include "ipcUnixMessage.h" // msgBuf 構造体の宣言
8 int main() {
9     struct msgBuf buf; // メッセージ領域
10    int msqid; // メッセージキュー ID
11    key_t key; // メッセージキューの名前
12    if ((key=ftok("msgq.dat", 'b'))== -1) { // ftok はファイル名から
13        perror("ftok"); // 重複のない名前(キー)を
14        exit(1); // 生成する
15    }
16    if ((msqid=msgget(key, 0644|IPC_CREAT))== -1) { // メッセージキューを作る
17        perror("msgget");
18        exit(1);
19    }
20    printf("Enter lines of text, ^D to quit:\n");
21    buf.mtype = 1; // メッセージの型
22    while (fgets(buf.mtext, MAXMSG, stdin) != NULL) { // キーボードから 1 行入力
23        if (msgsnd(msqid, &buf, MAXMSG, 0) == -1) { // メッセージを送信
24            perror("msgsnd");
25            break;
26        }
27    }
28    if (msgctl(msqid, IPC_RMID, NULL) == -1) { // メッセージキューを削除
29        perror("msgctl");
30        exit(1);
31    }
32    exit(0);
33 }

```

6.2.7 UNIX のメッセージ通信プログラムの実行例

メッセージ通信プログラムの実行例を図 6.8 に示す。図は二つのターミナルを開いて操作した状態を示している。左半分が第一のターミナル、右半分が第二のターミナルである。まず、左のターミナルで送信プログラム (ipcUnixMessageWriter) を起動する。これでメッセージキューが準備された。次に、右のターミナルで受信プログラム (ipcUnixMessageReader) を起動する。この状態で左のターミナルに入力した文字列が、メッセージ通信を用いて右のターミナルで実行中のプログラムに送信される。右のターミナルには受信したメッセージの mtype と mtext の内容が表示される。

リスト 6.5: UNIX のメッセージ通信プログラム例 (メッセージ受信側)

```

1 // メッセージ受信プログラム (ipcUnixMessageReader) : メッセージキューから受信する
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <sys/types.h>
5 #include <sys/ipc.h>
6 #include <sys/msg.h>
7 #include "ipcUnixMessage.h"
8 int main() {
9     struct msgBuf buf;
10    int msqid;
11    key_t key;
12    if ((key=ftok("msgq.dat", 'b'))== -1) {           // 送信側と同じキーを作る
13        perror("ftok");
14        exit(1);
15    }
16    if ((msqid=msgget(key, 0644))== -1) {             // ipcUnixMessageWriter が作った
17        perror("msgget");                             // メッセージキューを取得
18        exit(1);
19    }
20    printf("ready to receive messages.\n");
21    for(;;) {
22        if (msgrcv(msqid, &buf, MAXMSG, 0, 0)== -1) { // 先頭のメッセージを読み出す
23            perror("msgrcv");                         // メッセージキューが削除され
24            exit(1);                                   // エラーが発生したら終了
25        }
26        printf("%ld:%s", buf.mtype, buf.mtext);       // 受信したメッセージを表示
27    }
28    exit(0);
29 }

```

<pre> [Terminal No.1] \$./ipcUnixMessageWriter Enter lines of text, ^D to quit: abcdefg 1234567 ^D \$ </pre>	<pre> [Terminal No.2] \$./ipcUnixMessageReader ready to receive messages. 1:abcdefg 1:1234567 msgrcv: Identifier removed \$ </pre>
---	---

図 6.8: UNIX のメッセージ通信プログラム実行例

6.3 メッセージ通信機構の実装例

第 21 章に TacOS におけるメッセージ通信機構の実装例を紹介する。TacOS のメッセージ通信機構はランデブー方式であり、セマフォを利用して実装されている。

6.4 まとめ

この章ではプロセス間通信 (IPC) について学んだ。IPC には共有メモリとメッセージ通信の二種類があった。UNIX の共有メモリとメッセージ通信についてプログラム例を示した。

練習問題

6.1 次の言葉の意味を説明しなさい。

- (a) 共有メモリ
- (b) メッセージ通信
- (c) 直接指定方式
- (d) 間接指定方式
- (e) バッファリング
- (f) 同期方式
- (g) 非同期方式
- (h) ランデブー方式
- (i) メッセージのタグ

6.2 プロセス間の共有メモリとスレッド間の共有変数の違いは何か？

6.3 UNIX の共有メモリ使用例 (リスト 6.1, リスト 6.2) を実際に実行し動作確認しなさい。なお、ソースプログラムは以下から入手可能である。

<https://github.com/tctsigemura/OSTextBook/blob/master/SampleCode/UnixSharedMemory>

6.4 動作確認したプログラムでは、サーバプログラムは共有メモリが変更されたことを確認しないで、一定の時間間隔で共有メモリの内容を表示している。

- (a) どのような不都合が予想されるか？
- (b) クライアントとサーバで同期をする方法はあるか？

6.5 メッセージ通信でバッファを大きくすることのメリットは何か？

6.6 UNIX のメッセージ通信プログラム例 (リスト 6.3, リスト 6.4, リスト 6.5) を実際に実行し動作確認しなさい。なお、ソースプログラムは以下から入手可能である。

<https://github.com/tctsigemura/OSTextBook/blob/master/SampleCode/UnixMessage>

6.7 UNIX のメッセージ通信プログラム例は生産者と消費者の問題の解になっている。複数生産者と複数消費者の問題の解にもなっているか？

6.8 UNIX のメッセージ通信プログラム例が複数生産者と複数消費者の問題の解にもなっているか、動作確認する手順を説明しなさい。

第 7 章

モニタ

複数のプロセス（スレッド）で資源を共有する際に、プロセスの同期や相互排除にセマフォを用いることを既に学んだ。しかし、セマフォは基本的な機能を提供するだけで使い方はプログラマ任せなので、間違った使用がされる可能性が高い。その結果、タイミングに依存した発見の難しいバグを持ったプログラムが作成される。そこで、プログラミング言語と一体になり^{*1}、プログラマに同期機構を強制的に利用させる仕組みが考案された。そのような仕組みの一つとして**モニタ**（Monitor）を紹介する。

7.1 概要

モニタはリソース管理用の機能と制約を持った**抽象データ型** [41] である。C++ や Java などを読んだことのある人なら、「抽象データ型はクラスのこと」と言えば分かりやすいと思われる。

モニタは抽象データ型の一種であるが、プロセス（スレッド）間の同期を行う機構が予め組込まれ、その制約の下で使用するものである。モニタの特長を以下に箇条書きにする。

- プログラマが定義できる型である。（抽象データ型で一般的）
- データと操作をまとめて定義する。（抽象データ型で一般的）
- 同期のための機能が組込まれている。（モニタ独特）

なお、Java のクラスは同期のための機構も持っており、一定のルールに従って使用すればモニタに近い使用もできる。モニタをサポートするプログラミング言語は Concurrent Pascal が有名である。

7.2 構成要素

図 7.1 にモニタの模式図^{*2}を示す。モニタは以下で説明する構成要素を持つ抽象データ型である。

7.2.1 資源（データ、変数）

複数のスレッドによって共有される変数のことである。モニタの内部に必要な応じて名前付きで宣言する。モニタの外から直接アクセスすることはできない。

^{*1} 本章の話題はオペレーティングシステムではなくプログラミング言語である。

^{*2} 図 7.1 では、初期化プログラムが省略してある。

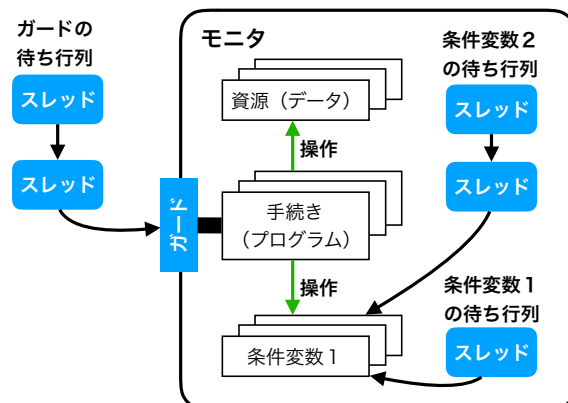


図 7.1: モニタの模式図

7.2.2 手続き（操作，メソッド）

外部から呼び出されるプログラムである。モニタの内部に必要なに応じて名前付きで定義する。モニタの外部から資源にアクセスできるインターフェースは手続きだけである。手続きの実行はガードの働きにより排他的に行われ、同時に実行される手続きは必ず一つ以内である。

7.2.3 ガード

モニタに一つのガードが存在し、手続きを排他的に実行するために用いられる。手続きを実行するときは自動的にガードにロックがかけられる。複数のスレッドが同時にモニタに入ることはいない。

7.2.4 条件変数

モニタの内部に必要なに応じて名前付きで宣言する。モニタの外部から直接アクセスすることはできない。条件変数には wait と signal の二つの操作ができる。wait 操作を行ったスレッドはガードを外して条件変数の待ち行列に入る。signal 操作は条件変数の待ち行列から一つのスレッドを選んで実行可能にする。実行可能になったスレッドはただちに実行を再開する。待ち行列にスレッドが複数ある時、どのスレッドが実行可能になるか明確な決まりはない。

7.2.5 初期化プログラム

モニタのインスタンスを作成する時に、初期化のために実行されるプログラムである。

7.3 相互排除問題の解

前出の架空の銀行口座管理プログラムの例をモニタに置換える。残高がスレッド間で共有される変数である。共有変数をスレッド間で安全に共有させるためにモニタを用いる。

7.3.1 共有変数の記述

リスト 7.1 に Java 風の仮想言語による銀行口座の記述を示す。本物の口座は他にも情報を持っているだろうが、ここでは口座は残高だけ持っていることにする。

2 行 この仮想言語では、Java の class 定義に似た monitor 定義ができるものとする。

3～4 行 資源の例である。この例では残高を表すスレッド間の共有変数（money）が資源である。

5～8 行 初期化プログラムの例である。モニタのインスタンス生成時に残高を引数で初期化する。

リスト 7.1: モニタによる相互排除の実現（仮想言語版）

```
1 // 銀行口座の残高を管理するモニタ
2 monitor MonAccount {
3     // 資源
4     int money; // スレッド間の共有変数（残高）
5     // 初期化プログラム
6     MonAccount(int m) {
7         money = m; // 口座の残高を初期化する
8     }
9     // 手続き
10    public void receive(int r) { // 入金手続き
11        money = money + r;
12    }
13    public void pay(int p) { // 引落とし手続き
14        money = money - p;
15    }
16 }
```

9～15 行 手続きの例である。手続きは共有資源を書換えるのでクリティカルセクションであるが、自動的にガードをロックし排他的に実行されるので明示的な相互排除を行う必要はない。

7.3.2 共有変数の利用

リスト 7.2 に、リスト 7.1 で定義した MonAccount モニタを利用した相互排除問題の解を示す。

2 行 リスト 7.1 に示した MonAccount モニタ型のインスタンスを生成する。

3～8 行 入金管理スレッドが実行するメソッドである。入金額を受信し口座に入金する。

9～13 行 引落とし管理スレッドが実行するメソッドである。支払い金額を受信し口座から引落とす。

14～17 行 プログラムは main() から実行を開始する。main() では「入金管理スレッド」と「引落とし管理スレッド」を起動する。これらのスレッドがそれぞれ、receiveThread() メソッドと payThread() メソッドを実行するものとする。

7.4 生産者と消費者問題の解

この問題で資源はデータを保管する FIFO 構造のバッファである。このバッファを以下ではキューと呼ぶことにする。キューとキューを操作する手続きは、全てモニタの中にまとめられる。キューを使用するユーザプログラムには、排他や同期に関わる難しいプログラムが含まれない。資源の操作に関する難しいプログラムが一箇所にまとめられることも、モニタを使用するメリットである。

以下では生産者と消費者問題の解を示すために、まず、データのバッファになるキューをモニタを用いて記述する。次に、キューを使用する生産者スレッドと消費者スレッドを作る。

7.4.1 キューの記述

リスト 7.3 に Java 風の仮想言語によるキューの記述例を示す。

リスト 7.2: モニタによる相互排除の利用 (仮想言語版)

```

1 class MonAccountMain {
2     static MonAccount account = new MonAccount(0); // 残高0円の口座を作る
3     static void receiveThread() {                  // 入金管理スレッド
4         for ( ; ; ) {                               // 以下を繰り返す
5             int receipt = receiveMoney();           // ネットワークから入金を受信
6             account.receive(receipt);                // 口座に入金する
7         }
8     }
9     static void payThread() {                       // 引落とし管理スレッド
10        for ( ; ; ) {                               // 以下を繰り返す
11            int payment = payMoney();                // ネットワークから支払いを受信
12            account.pay(payment);                    // 口座から引落とす
13        }
14    public static void main(String[] args) {
15        入金管理スレッドを起動;
16        引落とし管理スレッドを起動;
17    }
18 }

```

1 行 この仮想言語では、Java の class 定義に似た monitor 定義ができるものとする。

2～5 行 資源の例である。キューとして使用するリングバッファのデータ構造を宣言している。このモニタの目的は、資源であるキューを管理することである。手続きを介すること無しに資源にアクセスすることは禁止なので、モニタの外部からこれらのデータにアクセスできない。N がバッファの大きさ、buf がバッファ本体、first がバッファ中の次のデータ読みだし位置、last がバッファ中の次のデータ書き込み位置、cnt がバッファ中のデータ件数を表す。

6～8 行 条件変数の例である。この仮想言語では、Condition 型の変数として条件変数を宣言する。empty は、キューが空の時にデータを取り出そうとしたスレッドが、キューにデータが書き込まれるまで待つために使用する条件変数である。full は、キューが満杯の時にデータを書き込もうとしたスレッドが、キューに空きができるまで待つために使用する条件変数である。

9～14 行 初期化プログラムの例である。モニタのインスタンスを作る際に実行されるものとする。引数はバッファの大きさである。

15～30 行 手続きの例である。手続きはモニタの外部から呼び出すことができる。append() はキューにデータを追加する。remove() はキューからデータを取り出す。これらのプログラムが実行される時はガードによる排他制御がされる。

次にバッファが満杯で待ちが発生する例を考える。データをキューに追加するために append() を呼び出したスレッドは、コメントに (1) と記された行を実行し、キューが満杯の時 17 行の full.wait() で待ち状態になる。待ち状態になる時はガードを外すので、他のスレッドがモニタに入ることができる。他のスレッドがデータをキューから取り出すために remove() を呼び出すと、(2) の行が実行され

リスト 7.3: モニタによるキューの実現 (仮想言語版)

```
1 monitor BoundedBuffer {
2     // 資源 (リングバッファ)
3     int N;
4     int[] buf;
5     int first, last, cnt;
6     // 条件変数
7     Condition empty;
8     Condition full;
9     // 初期化
10    BoundedBuffer(int n) {
11        N = n;
12        buf = new int[N];
13        first = last = cnt = 0;
14    }
15    // 手続き
16    public void append(int x) {    // (1)
17        if (cnt==N) full.wait();  // (1)
18        buf[last] = x;           // (3)
19        last = (last + 1) % N;    // (3)
20        cnt++;                   // (3)
21        empty.signal();          // (3)
22    }
23    public int remove() {        // (2)
24        if (cnt==0) empty.wait(); // (2)
25        int x = buf[first];      // (2)
26        first = (first + 1) % N; // (2)
27        cnt--;                   // (2)
28        full.signal();           // (2)
29        return x;                // (4)
30    }
31 }
```

28 行の `full.signal()` まで進む. `full.signal()` が実行されると待ち状態のスレッドが一つ起床し, (3) の行がただちに実行される. (3) の実行が終了した後に (4) の行が実行される. (2) から (4) の実行の間, ガードは外さないなので他のスレッドがモニタに入ることはない.

7.4.2 生産者と消費者スレッドの記述

リスト 7.4 に Java 風の仮想言語による生産者と消費者問題の解を示す. このプログラムはリスト 7.3 で定義したキューを使用する.

2 行 リスト 7.3 に示した `BoundedBuffer` モニタ型のインスタンスである.

3~8 行 生産者スレッドが実行するメソッドである. 無限にデータを生産し `queue` に追加し続ける.

リスト 7.4: モニタによる生産者と消費者問題の解（仮想言語版）

```
1 class BoundedBufferMain {
2     static BoundedBuffer queue = new BoundedBuffer(10); // 大きさ10のキュー
3     static void producer() {                               // 生産者スレッドが実行
4         while(true) {
5             int x = データを作る();
6             queue.append(x);                               // キューにデータを追加
7         }
8     }
9     static void consumer() {                               // 消費者スレッドが実行
10        while(true) {                                     // キューから
11            int x = queue.remove();                         // データを取り出す
12            データを使用する(x);
13        }
14    }
15    public static void main(String[] args) {               // main から実行を開始
16        生産者スレッドを起動;
17        消費者スレッドを起動;
18    }
19 }
```

9～14行 消費者スレッドが実行するメソッドである。queue からデータを取り出し処理することを無限に繰り返す。

15～18行 プログラムはmain() から実行を開始する。main() では「生産者スレッド」と「消費者スレッド」を起動する。これらのスレッドがそれぞれ、producer() メソッドと consumer() メソッドを実行する。

7.5 Java のセマフォクラスによるモニタの実装

モニタの仕組みをより正確に理解するために、セマフォによるモニタの実装方法を考えてみる。リスト 7.3 の仮想言語で記述されたモニタを Java クラスに書換えたものをリスト 7.5 とリスト 7.6 に示す。

7.5.1 モニタ機能の Java による実装

リスト 7.5 は、モニタと同等な動作をする SemBoundedBuffer クラスである。

1行 java.util.concurrent パッケージの Semaphore クラスを使用する。Semaphore クラスはカウンティングセマフォ型である。

2行 セマフォを使用したキュークラスを SemBoundedBuffer と名付ける。

3行 セマフォ (guard) はモニタのガードの役割を持っている。スレッドは、モニタ (SemBoundedBuffer クラス) 内の手続き (メソッド) を実行する前に guard をロックする。

4～5行 モニタの条件変数に signal() 操作を行った時、条件変数で待っていたスレッドがあればただちに実行しなければならない。待っていたスレッドを先に実行させるために、signal() 操作を

リスト 7.5: モニタと同等なキューをセマフォで実現 (Java 版, 前半)

```

1 import java.util.concurrent.Semaphore;           // セマフォ型を利用可能にする
2 public class SemBoundedBuffer {
3     private Semaphore guard = new Semaphore(1); // ガード用のセマフォ
4     private Semaphore next = new Semaphore(0);  // signal時ブロック用セマフォ
5     private int nextCount = 0;                  // signal時ブロック・スレッド数
6     private class Condition {                   // 内部クラス'条件変数型'を定義
7         Semaphore sem = new Semaphore(0);       // 条件変数待ち用セマフォ sem
8         int count = 0;                          // 条件変数を待つスレッドの数
9         void await() {                          // 条件変数を待つメソッド
10            count++;                             // この条件変数待ちスレッドの数
11            if (nextCount>0) {                   // 起床後にawait()した場合なら
12                next.release();                  // signal()したスレッドを起床
13            } else {                             // 起こすスレッドがないなら
14                guard.release();                 // ガードを外してからブロック
15            }
16            sem.acquireUninterruptibly();         // 条件変数のセマフォで待つ
17            count--;                             // 待ちが完了
18        }
19        void signal() {                          // 条件変数で待つスレッドを起床
20            if (count>0) {                       // 待っているスレッドがあれば
21                nextCount++;                     // signal途中のスレッド数
22                sem.release();                   // 待ちスレッドを起こす
23                next.acquireUninterruptibly();   // 起きたスレッドを先に実行
24                nextCount--;                     // signal完了
25            }
26        }
27    }
28    private void exitProc() {                    // 手続きの出口処理
29        if (nextCount>0) {                       // signalされた後なら
30            next.release();                      // signalしたスレッドを起床
31        } else {                                 // そうでなければ
32            guard.release();                     // ガードを外す
33        }
34    }

```

行ったスレッドを待ちにするセマフォ (next) と next を待っているスレッドの数を記憶する変数 (nextCont) を準備する。

6行 条件変数型 (Condition) を内部クラスとして定義する。

7~8行 条件変数に wait 操作を行った時にスレッドを待ち状態にするためのセマフォ (sem) と待っているスレッドの数をカウントする変数 (count) である。

9行 await() は条件変数の wait 操作を行うメソッドである。Java の Object クラスに別の wait()

リスト 7.6: モニタと同等なキューをセマフォで実現 (Java 版, 後半)

```

35 // 資源 (リングバッファ)
36 private int N;
37 private int[] buf;
38 private int first, last, cnt;
39 // 条件変数
40 private Condition empty = new Condition();
41 private Condition full = new Condition();
42 // 初期化
43 public SemBoundedBuffer(int n) {
44     N = n;
45     buf = new int[N];
46     first = last = cnt = 0;
47 }
48 // 手続き
49 public void append(int x) {                // (1)
50     guard.acquireUninterruptibly();        // (1) ガードを取得
51     if (cnt==N) full.await();              // (1)
52     buf[last] = x;                        // (3)
53     last = (last + 1) % N;                 // (3)
54     cnt++;                                // (3)
55     empty.signal();                       // (3)
56     exitProc();                           // (3) 手続きの出口処理
57 }
58 public int remove() {                    // (2)
59     guard.acquireUninterruptibly();        // (2) ガードを取得
60     if (cnt==0) empty.await();             // (2)
61     int x = buf[first];                   // (2)
62     first = (first + 1) % N;               // (2)
63     cnt--;                                // (2)
64     full.signal();                        // (2)
65     exitProc();                           // (4) 手続きの出口処理
66     return x;                             // (4)
67 }
68 }

```

メソッドが定義されているので、名前を `await` にした。

11~15行 `release()` はセマフォに V 操作を行う。 `nextCnt` は `signal()` 中で待っているスレッドの数である。待っているスレッドがある場合は起こす。そうでなければモニタのガードを外す。

16行 `acquireUninterruptibly()` はセマフォに P 操作を行う。 `sem` は初期値 0 のセマフォなので、`await()` を呼び出したスレッドがここでブロックする。

19行 条件変数に `signal` 操作を行うメソッドである。

20～25 行 条件変数を待っているスレッドの数 (count) を調べ、1 以上なら 22 行で起床させる。自身は 23 行でブロックし、起床したスレッドが `exitProc()` を実行しモニタを出るのを待つ。

28 行 モニタ手続きの最後の行で呼び出すメソッドである。 `signal()` 中の 23 行でブロックしているスレッドがあれば起床させる。なければモニタのガードを外す。

7.5.2 モニタ機能の使用

リスト 7.6 に `SemBoundedBuffer` クラスの後半を示す。ここでは、`SemBoundedBuffer` クラスの前半で定義したモニタ機能を使用している。

35～38 行 資源 (リングバッファ) を表現するための変数を宣言する。モニタ (クラス) の外部から資源を隠蔽するために `private` 修飾子を付けて宣言する。

39～41 行 条件変数は前半で定義した `Condition` クラスのインスタンス変数である。

42～47 行 初期化はクラスのコンストラクタとして実装する。

48～67 行 手続きは仮想言語で定義したものに、50 行と 59 行の「ガード取得」、56 行と 65 行の「手続きの出口処理」が追加されている。

7.6 Java のモニタ風機構による生産者と消費者問題の解

Java 言語はモニタに似た同期機構をサポートしている。リスト 7.7 に Java による生産者と消費者問題の解を示す。Java には条件変数に相当するものが無い。

1 行 Java のモニタ風機構を利用したキュークラスを `MonBoundedBuffer` と名付ける。

2～5 行 資源 (リングバッファ) を表現するための変数を宣言する。クラスの外部から資源を隠蔽するために `private` 修飾子を付けて宣言する。

6～11 行 初期化をコンストラクタとして実装する。

13 行 クラスの内部だけで呼び出す `private` なメソッドである。 `await()` メソッドは条件変数の `wait()` に似た働きをする。

14 行 `await()` メソッドは `Object` クラスの `wait()` メソッドを呼び出す。Java オブジェクトは暗黙の条件変数があるような構造になっている。 `wait()` メソッドは暗黙の条件変数の待ち行列^{*3}にスレッドを入れる。 `wait()` メソッドは例外 (Exception) でも終了するので、`try-catch` 構文で使用する。

16, 23 行 外部から呼び出すことができるメソッドは `synchronized` 修飾子を付けて定義する。 `synchronized` メソッドはモニタの手続きと同様に、オブジェクトのガードをロックした^{*4}スレッドだけが実行できる。オブジェクトのガードをロックできない場合はガードの待ち行列に入る。

17 行 バッファが満杯の場合に `await()` を用いて暗黙の条件変数で待ち状態になる。 `await()` は例外でも終了するので、バッファに空きができるまで繰り返し `await()` を呼び出す。

21 行 `notify()` は暗黙の条件変数の `signal()` に相当する。暗黙の条件変数は一つしかないので、ス

^{*3} Java では待機セットと呼ぶ。

^{*4} Java ではモニタを所有すると言う。

リスト 7.7: Java のモニタ風機構による生産者と消費者問題の解

```

1 public class MonBoundedBuffer {
2     // 資源 (リングバッファ)
3     private int N;
4     private int[] buf;
5     private int first, last, cnt;
6     // 初期化
7     public MonBoundedBuffer(int n) {
8         N = n;
9         buf = new int[N];
10        first = last = cnt = 0;
11    }
12    // 手続き
13    private void await() {
14        try{wait();}catch(InterruptedException e){}
15    }
16    public synchronized void append(int x) {    // (1)
17        while (cnt==N) await();                // (1)
18        buf[last] = x;                        // (3)
19        last = (last + 1) % N;                 // (3)
20        cnt++;                                // (3)
21        if (cnt==1) notify();                  // (3)
22    }
23    public synchronized int remove() {        // (2)
24        while (cnt==0) await();                // (2)
25        int x = buf[first];                    // (2)
26        first = (first + 1) % N;               // (2)
27        cnt--;                                // (2)
28        if (cnt==N-1) notify();                // (2)
29        return x;                             // (2)
30    }
31 }

```

レッドが `remove()` で待ちになっている可能性がある場合（直前までバッファが空だった場合）だけ、`notify()` するようにしている。無条件に `notify()` を実行するようになると、`append()` で複数のスレッドが待ちになっている場合に、それらを起床させてしまう。

24 行 バッファが空の場合に `await()` を用いて暗黙の条件変数で待ち状態になる。

28 行 暗黙の条件変数は一つしかないので、スレッドが `append()` で待ちになっている可能性がある場合（直前までバッファが満杯だった場合）だけ、`notify()` するようにしている。

29 行 取り出したデータを呼び出し側に返す。16 行から 28 行の右端に書いてあるコメントはリスト 7.3 と同様に、生産者スレッドが 17 行でブロックした後、消費者スレッドが 28 行で生産者スレッドを起床させるときの実行順である。これまでの例と比較して 29 行が異なっている。Java の

`notify()` はモニタの `signal()` と異なり、`synchronized` メソッドの最後まで実行した後で、スレッドの切換えを起こすからである。

7.7 まとめ

モニタについて学んだ。モニタはスレッド間の同期と相互排除に使用できる「高級言語に組込まれた仕組み」である。モニタ内に資源と、資源を操作する手続きを記述する。資源の相互排除と同期に関わる難しい処理がプログラムのあちこちに分散しない。また、モニタ内の手続き（プログラム）の実行は自動的に相互排除されるので、クリティカルセクションを明示する必要もない。

モニタで記述した「生産者と消費者問題」の解を、セマフォを用いて実装し直す例を示した。この例をよく観察するとモニタの動作が細部まで理解できる。

Java 言語はモニタに似た機能をサポートする言語であるが、資源が外部からアクセスできないように `private` 修飾が必要なこと、条件変数がないこと、`wait()` が `signal()` 以外でも終了すること、`signal()` を実行したスレッドが手続きの最後まで実行されること等が異なる。

練習問題

- 7.1 次の言葉の意味を説明しなさい。
- (a) 抽象データ型
 - (b) 資源
 - (c) 手続き
 - (d) ガード
 - (e) 条件変数
- 7.2 抽象データ型の定義を調べなさい。
- 7.3 リスト 7.3 のプログラムにおいて、`cnt` なしにキュー（リングバッファ）を記述できるか？
- 7.4 リスト 7.3 のプログラムにおいて、キューが空のとき一つのスレッドが `remove()` を実行した。その後、別のスレッドが `append()` を実行した。この時の `append()`、`remove()` 内のプログラムが実行される順を答えなさい。
- 7.5 リスト 7.3 のプログラムは、複数生産者と複数消費者問題の解に使用できるか？
- 7.6 Java 風仮想言語のモニタを用いてリーダ・ライタ問題の解を示しなさい。
- 7.7 Java 風仮想言語のモニタを用いてセマフォを記述しなさい。
- 7.8 `semBoundedBuffer`（リスト 7.5、リスト 7.6）を実際に実行しなさい。メインルーチンを含むソースプログラムは以下から入手できる。
- <https://github.com/tctsigemura/OSTextBook/blob/master/SampleCode/SemBoundedBuffer/>
- 7.9 `signal()` は手続きの最後でしか使用できないことにすると、`semBoundedBuffer`（リスト 7.5、リスト 7.6）はどのように簡略化できるか。
- 7.10 `MonBoundedBuffer`（リスト 7.7）を実際に実行しなさい。メインルーチンを含むソースプログラムは以下から入手できる。
- <https://github.com/tctsigemura/OSTextBook/blob/master/SampleCode/MonBoundedBuffer/>
- 7.11 リスト 7.3 のプログラムと、リスト 7.7 でコメントに示すように実行順序が異なる。Java のモニタ風機構と従来のモニタのどのような違いによるものか？
- 7.12 その他に従来のモニタと Java のモニタ風機構の違いは何があるか？
- 7.13 モニタの `signal` と、セマフォの `V` 操作の違いは何があるか？

第 III 部

メモリ管理

第 8 章

主記憶（メモリ）

コンピュータシステムにおいて、主記憶（メモリ）^{*1}は CPU と同様に重要な装置である。CPU を仮想化し複数のプロセスを同時に実行可能にするには、主記憶も管理し複数のプロセスに適切に主記憶が割り振られ、かつ、プロセス同士が干渉しないように分離する必要がある。この章では主記憶と主記憶管理の基本的なアイデアについて学ぶ。

8.1 ハードウェア構成

主記憶は CPU がプログラムを実行する際に、プログラムの機械語やデータ、スタック領域等を置くメモリのことである。TeC の主記憶は 256 バイトの RAM 領域であったし、実験科目でよく使用される H8/3664 では 32KiB の ROM と 2KiB の RAM であった。現代の PC なら 4GiB から 16GiB 程度の大きさを持つ「メモリ」のことである。

本書で前提とするコンピュータのハードウェア構成は図 2.1 に示した。この章では CPU とメモリに着目するので、図を単純化し図 8.1a のようなモデルを用いる。この図は CPU がアドレスを指定してメモリのデータを読み書きすることを表している。

CPU は命令を実行する際に、次の手順でメモリをアクセスする。

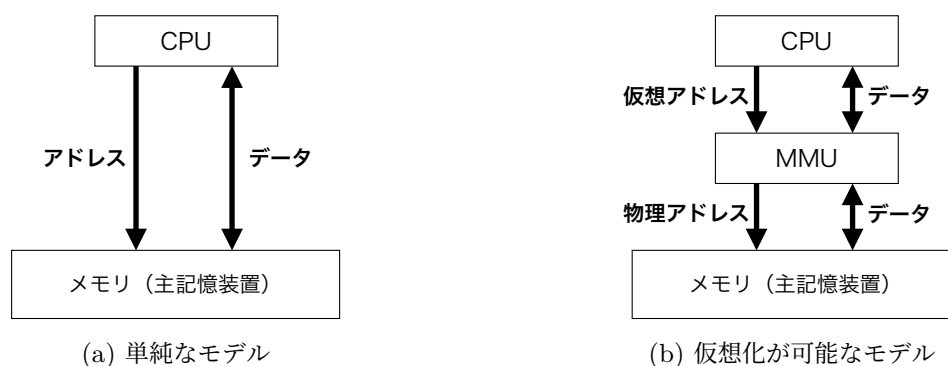


図 8.1: CPU とメモリの関係を表す単純なモデル

^{*1} 本章で「主記憶」と「メモリ」は同じ意味で用いられる。

1. 命令フェッチ (fetch)

PC の値をアドレスとして出力し主記憶からデータ（命令コード）を読む。

2. 命令デコード (decode)

フェッチした命令の種類を調べる。

3. 命令実行 (execution)

命令を実行する際に必要に応じてデータのアドレス（**実効アドレス**：*Effective Address(EA)*）を出力し主記憶のデータを**読み書きする**。

図 8.1a のモデルは、TeC や H8/3664 のようなマイクロコンピュータの様子を表すためには十分である。しかし、この単純なモデルは、本格的なオペレーティングシステムを作動させるには、次の点で不十分である。

1. メモリ保護機構がない。

ユーザプログラムのバグで、OS のカーネルや他のプロセスを破壊する可能性がある。

2. メモリの再配置機構がない。

同時に複数のプロセスが主記憶にロードされる環境では、プロセスの起動と終了が繰り返されるうちに使用できない小さなメモリの断片（フラグメント）ができる。フラグメントを解消するために、実行中プロセスをメモリ内で移動する機能が必要である。

3. 仮想記憶機構が実現できない。

メモリより大きなプログラムを実行するために、仮想記憶機構を導入したいができない。

そこで、図 8.1b のモデルを用いる。CPU とメモリの間に *MMU (Memory Management Unit: メモリ管理装置)* を追加する。MMU は CPU が出力した**仮想アドレス**を OS が指示したルールに則り**物理アドレス**に変換してメモリに送るハードウェアである。*OS の主記憶管理プログラムが MMU を制御することによって、使いやすく安全な仮想の主記憶をプロセスに提供する。*

8.2 メモリ保護機構

CPU を仮想化したことによって、複数のユーザプロセスをメモリに同時にロードし並列実行することが可能になった。これにより、CPU の使用効率が良くなるだけでなく、コンピュータの使い勝手が非常に良くなった。しかし、ユーザプログラムのバグや悪意によって、OS カーネルや他のユーザプログラムが破壊される可能性がでてきた。OS カーネルは、全てのメモリ領域にアクセスする必要がある。一方でユーザプロセスは自身に割当てられたメモリ以外にアクセスできない仕組みが必要である。

8.2.1 上限・下限レジスタ

プロセスがアクセスしても良いメモリアドレスの範囲をレジスタに設定し、メモリアクセスする度に CPU が出力するアドレスとレジスタの値を比較する。図 8.2a はプロセス 2 が実行中の上限・下限レジスタの状態を表している。図 8.2b はアドレスを比較するハードウェアの構成を示している。

1. OS カーネルはプロセスの実行を開始する前に、プロセスの上限アドレスと下限アドレスを上限・

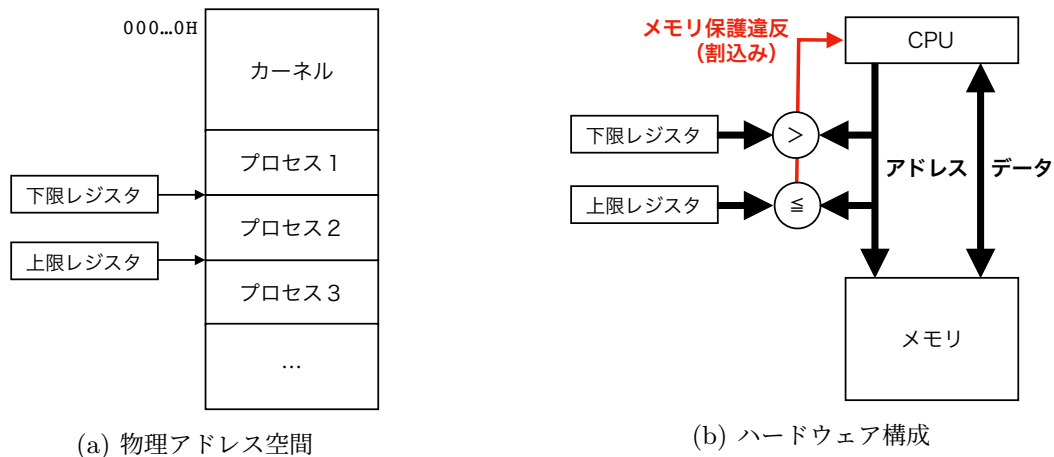


図 8.2: 上限・下限レジスタの仕組み

下限レジスタに設定する。上限・下限レジスタを操作できるのはカーネルモード*2で実行されるカーネルだけである。ユーザプロセスが、自身のアクセスできる領域を変更することはできない。

2. カーネルはプロセスの実行を開始させる。
3. プロセスはユーザモードで実行される。ユーザモードで実行中はハードウェアが CPU の出力するアドレスを上限・下限レジスタと比較する。
4. 上限・下限アドレスの範囲外へのアクセスの場合、ハードウェアがメモリアccessを阻止し CPU に割込みをかける。
5. 割込みが発生するとユーザプロセスの実行が打ち切れ、制御がカーネルに移る。

8.2.2 ロック／キー機構

主記憶をページに分割しページ毎にアクセス許可情報を持たせる。64KiB のメモリを 256 ページに分割した例を図 8.3a に示す。16bit のアドレスはページ番号を表す上位 8bit と、ページ内オフセットを表す下位 8bit に分割される。

図 8.3b に示すように CPU は、アドレス、アクセスキー、R/W/X を MMU に出力する。アクセスキーはプロセス毎に決まる数値*3、R/W/X はメモリアccessの種類を表す次のどれかである。R (Read) は読み込みを、W (Write) 書き込みを、X (eXecute) は命令のフェッチを意味する。

MMU は許可情報表を内蔵している。MMU は CPU が出力したアドレスからページ番号を求め表を引く。表のプロテクションキーがアクセスキーと一致していない場合、または、CPU の R/W/X が表のアクセスモードに含まれていない場合はメモリ保護違反の割込みを発生する。MMU を操作できるのは CPU の実行モードがカーネルモードの時だけ、MMU がメモリ保護違反の割込みを発生するのはユーザモード時だけである。特別なプロテクションキー (例えば 0) のページは全てのプロセスがアクセス可能とすれば、プロセス間の共有メモリを実現できる。

*2 実行モードは 1.2.2 で紹介したので忘れた人は再確認すること。

*3 プロセス番号でも良い。

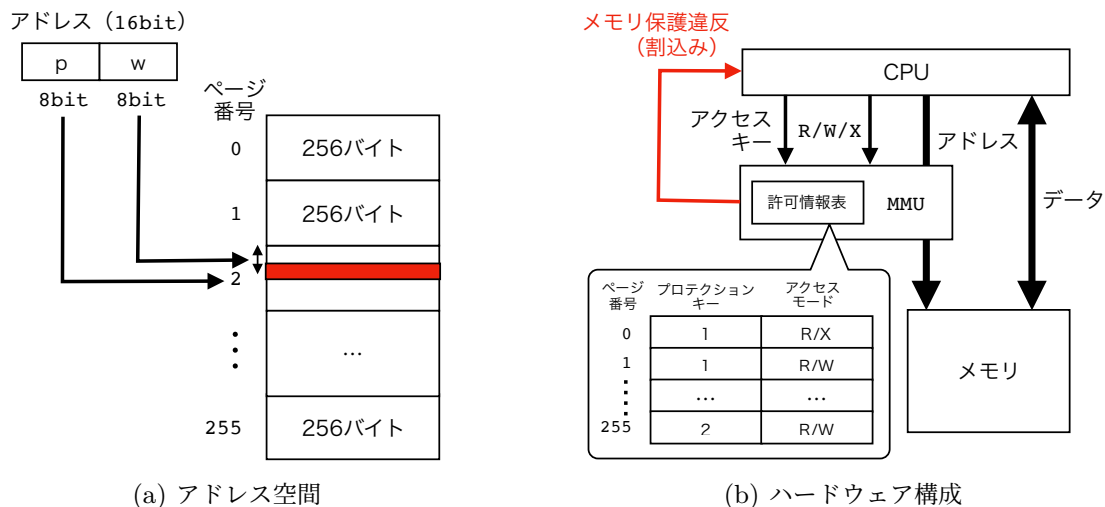


図 8.3: ロック／キー機構の仕組み

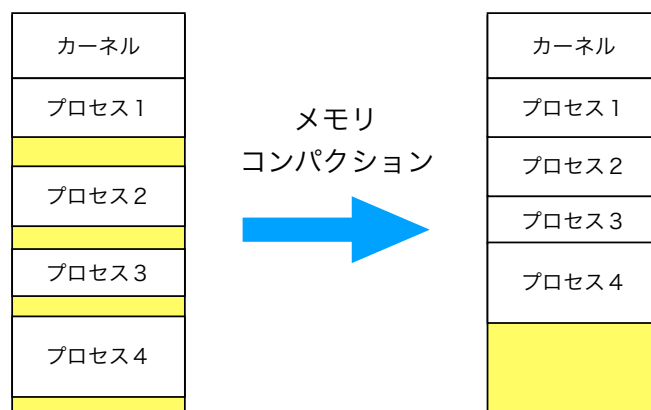


図 8.4: プログラムの動的再配置

8.3 プログラムの再配置

コンパイルされたプログラムはメモリにロードされる時にアドレスが確定する。ファイルに格納された実行可能形式プログラムは、ロード時にアドレスを変更できる必要がある。

また、実行中のプログラムをメモリ内で移動することがある。図 8.4 のようにメモリが多くの領域に分断され、領域の間に小さなメモリの断片（メモリフラグメント）が沢山できた場合は、プログラムの詰め合わせ（メモリコンパクション）を行う。実行中のプログラムを移動することを動的再配置と呼ぶ。

8.3.1 再配置可能オブジェクトファイル

プログラミング言語で記述されたプログラムは、コンパイルされ実行可能な機械語ファイルに変換される。しかしコンパイル時には、プログラムがメモリの何番地にロードされるか分からない。そこで、実行可能形式の機械語プログラムはジャンプ先アドレスや、データアドレスの確定をロード時に行うこ

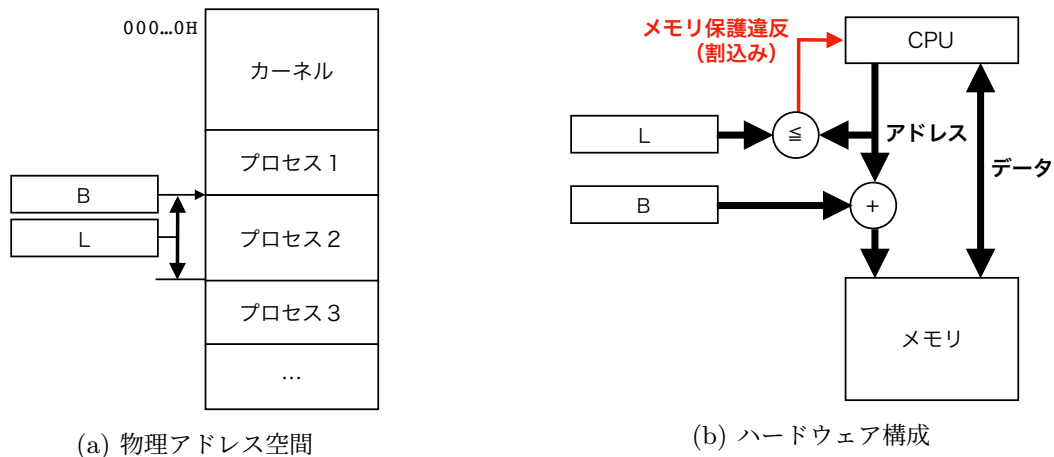


図 8.5: リロケーションレジスタ

とができなければならない。

ロードアドレスが確定しておらず、アドレスを変更可能な機械語プログラムは**再配置可能オブジェクト** (*relocatable object*) と呼ばれる。再配置可能オブジェクトファイルは、コンパイル済みの機械語プログラムの他に、プログラム中のどの部分がアドレスであるかを記録した**再配置表**も含む。プログラムを主記憶にロードする際に再配置表を参照し、プログラムやデータ中の全てのアドレス情報にロードアドレスを足し込む必要がある。例えばプログラムを 1234H 番地にロードすると、JMP 0100H の機械語は JMP 1334H に書換える必要がある。ロード時にアドレスを変換する方式を**静的再配置**と呼ぶ。再配置可能オブジェクトファイルの例として TacOS が使用するファイルのフォーマットを付録 B に示す。

8.3.2 リロケーションレジスタ

動的再配置を行うためには、実行中のプログラムがどこにアドレスを記憶しているか全て管理する必要がある。しかし、CPU レジスタやスタックに書き込まれたアドレス、リスト構造に含まれるポインタ等、すべてのアドレスデータを追跡することは困難である。

動的再配置を可能にするための一つのアイデアは、**リロケーションレジスタ**と呼ばれる特別なハードウェアを用いることである。図 8.5a に示すように^{*4}リロケーションレジスタは、プロセスのロードアドレス (B:Base) と大きさ (L:Limit) を記録するレジスタである。ディスパッチャがプロセスを実行する時に値を設定する。

図 8.5b に示すように、CPU が出力したアドレスはプロセスの大きさ (L) と比較される。アドレスが L 以上の場合は、プロセス領域の外なのでメモリ保護違反の割込みを発生する。CPU のアドレスにプロセスのロードアドレス (B) を足した値が、メモリのアドレスになる。

動的再配置を行うにはプロセスが Running 以外の状態の時に、主記憶上でプロセスのメモリ領域を新しい領域にコピーする。次回プロセスが実行される時、ディスパッチャは新しい領域のアドレスを B に設定する。ユーザプログラムは再配置されたことを知る必要はない。しかし、プロセス領域の移動は

^{*4} 図はプロセス 2 を実行するための設定を表している

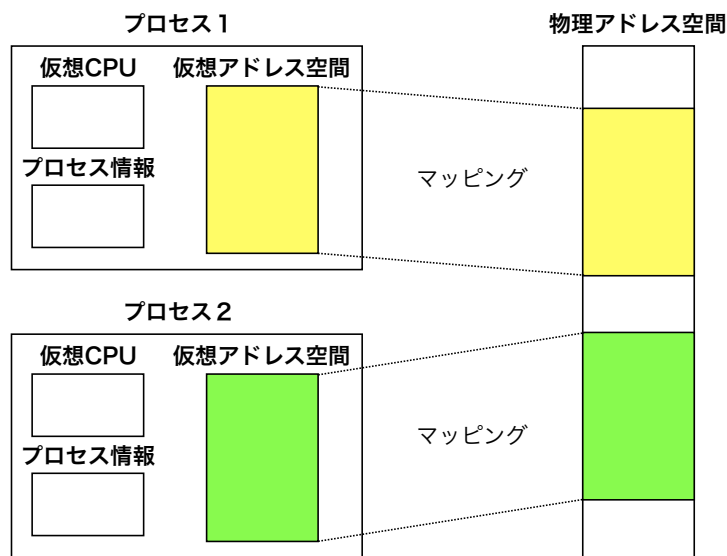


図 8.6: 仮想アドレス空間から物理アドレス空間へのマッピング

大量のメモリコピーを伴うので、オーバーヘッドが大きい処理である。

8.4 アドレス空間の仮想化

図 2.6 で示したように、プロセスは各々が専用の**仮想アドレス空間**（仮想メモリ空間）を持つ。仮想アドレス空間は仮想アドレスで番地付けされている。それに対しハードウェアとしてのメモリはシステム全体で一つしかない。ハードウェアメモリは物理アドレスで番地付けされており、**物理アドレス空間**を形成する。図 8.6 にプロセスの仮想アドレス空間が、物理アドレス空間にマッピングされる様子を示す。マッピングは、MMU による仮想アドレスから物理アドレスへの変換によってなされる。

8.4.1 単一仮想記憶

多重仮想記憶に移行する中間的な形式である。プロセスの仮想アドレスと物理アドレスが同じ方式である。メリットが少ないので通常は次に紹介する多重仮想記憶を用いる。

8.4.2 多重仮想記憶

アドレス空間が仮想化されることにより、全てのプロセスが独立したアドレス空間を持つことが可能になる。プロセス毎に独立したアドレス空間を持つ方式を**多重仮想記憶**と呼ぶ。

8.4.3 仮想アドレス空間の配置

仮想アドレス空間にプログラムや変数を配置する方法はオペレーティングシステムの種類により一定ではない。図 8.7 とリスト 8.1 に UNIX 上で C 言語プログラムが配置される様子を示す。

- 初期化済みのグローバル変数^{*5}は、初期化データ領域（*data* セグメント）に配置される。
- 初期化されないグローバル変数^{*6}は、非初期化データ領域（*bss* セグメント）に配置される。
- `main()` 関数は機械語に変換され、プログラム領域（*text* セグメント）に配置される。

^{*5} 正確には初期化済みの静的な変数。関数内で `static` 修飾した変数も含まれる。

^{*6} 正確には初期化されない静的な変数。関数内で `static` 修飾した変数も含まれる。

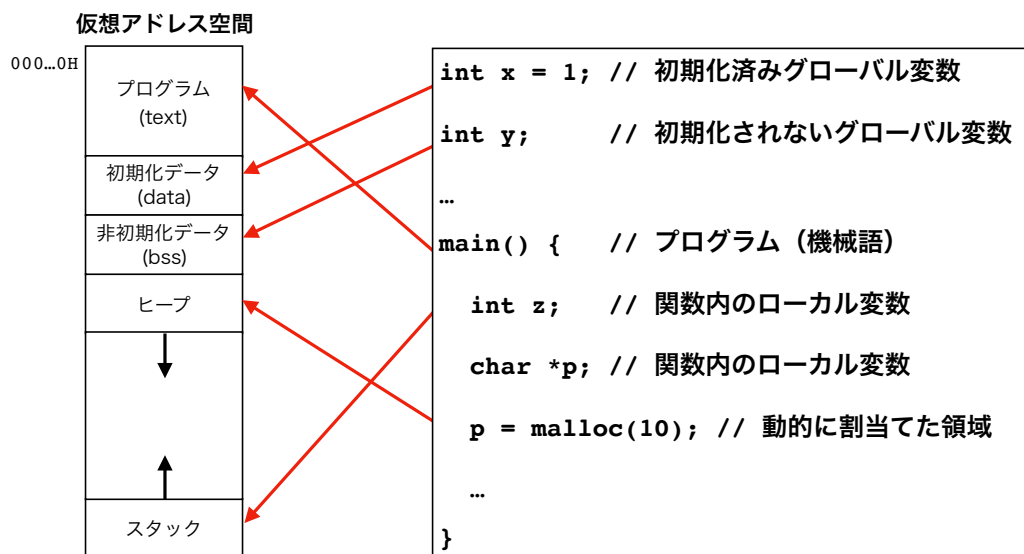


図 8.7: 仮想アドレス空間の配置例

リスト 8.1: C 言語プログラムを TaC の機械語に変換した例

_x	DW	1	// int x = 1;	DW命令はdataセグメントに出力
_y	WS	1	// int y;	WS命令はbssセグメントに出力
_main	PUSH	FP	// void main() {	機械語命令は
	LD	FP,SP	//	textセグメントに出力
	PUSH	G3	// int z;	
	PUSH	G4	// char *p;	
	LD	G0,#10	//	
	PUSH	G0	//	
	CALL	_malloc	// p = malloc(10);	
	ADD	SP,#2	//	
	LD	G4,G0	//	
	POP	G4		
	POP	G3		
	POP	FP		
	RET		// }	

- 関数のローカル変数^{*7}は、関数の実行開始時にスタックセグメントまたは CPU レジスタに割り付けられ、関数を終了する時に破棄される。同じスタックを関数呼び出しのために CALL 機械語命令も使用する。スタックは、必要に応じて仮想アドレス空間を 0 番地側に伸びる。
- malloc() 関数等を用いて動的に領域を割当てるとヒープセグメントが使用される。ヒープは必要に応じて仮想アドレス空間を 0 番地とは逆の方向に伸びる。

^{*7} 正確には自動変数。関数内で static 修飾した変数は含まれない。

8.5 まとめ

主記憶（メモリ）は、プログラムや変数をロードし、CPU がプログラムを実行する際に直接使用する記憶装置である。メモリ保護機構には**上限・下限レジスタ**、**ロック／キー機構**があった。プログラムの再配置は、プログラムをメモリにロードする時点で行う**静的再配置**と、プログラム実行中に行う**動的再配置**があった。静的再配置は**再配置可能オブジェクトファイル**に格納された**再配置表**を用いて行う。動的再配置には、ハードウェアの支援が必要である。このようなハードウェアの例は**リロケーションレジスタ**である。

メモリ保護やプログラムの動的再配置を行うために、MMU と呼ばれるハードウェアを CPU とメモリの間に配置する。MMU は**仮想アドレス**を**物理アドレス**にマッピングするアドレス変換器として働く。マッピング方法をプロセス毎に変更することで**多重仮想記憶**が実現できる。仮想アドレス空間の配置を UNIX を例に紹介した。UNIX プロセスの仮想アドレス空間は、text, data, bss, ヒープ, スタックセグメントからなる。

練習問題

8.1 次の言葉の意味を説明しなさい。

- (a) 主記憶（メモリ）
- (b) MMU
- (c) メモリ保護
- (d) 上限・下限レジスタ
- (e) 静的再配置
- (f) 動的再配置
- (g) 再配置可能オブジェクトファイル
- (h) リロケーションレジスタ
- (i) 仮想アドレス空間
- (j) 物理アドレス空間
- (k) 多重仮想記憶
- (l) text セグメント
- (m) data セグメント
- (n) bss セグメント
- (o) ヒープセグメント
- (p) スタックセグメント

8.2 自分がいつも使用している PC の主記憶（メモリ）は何 GiB か調べなさい。

8.3 上限・下限レジスタは、プログラムの動的再配置のために使用できるか？

8.4 付録 B に掲載した TacOS の再配置可能オブジェクトファイルの構造を理解しなさい。

8.5 リスト 8.1 のプログラムの場合、リロケーションレコードはいくつ必要か？

第 9 章

メモリ割付け方式

物理メモリの一部をプロセスに割付け，そこにプログラムをロードし実行する．物理メモリを複数のプロセスで分割し利用するために，幾つかの方式が考案されてきた．ここでは，固定区画方式と可変区画方式について解説する．

9.1 固定区画方式

予めメモリを大小数種類の区画に分割しておく．図 9.1 の例では利用可能なメモリを五つの区画に分割している．図 9.1a はプロセス 1 から 4 を実行する場合を示している．プロセスのサイズにより適切

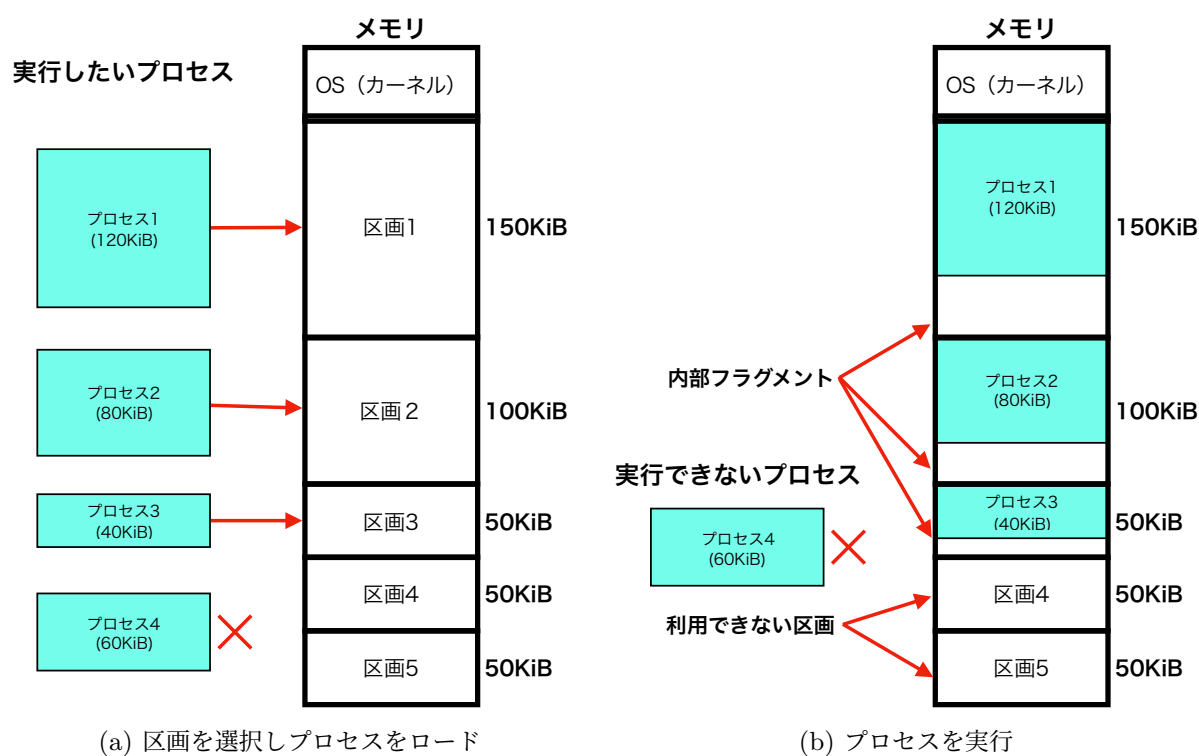


図 9.1: 固定区画方式

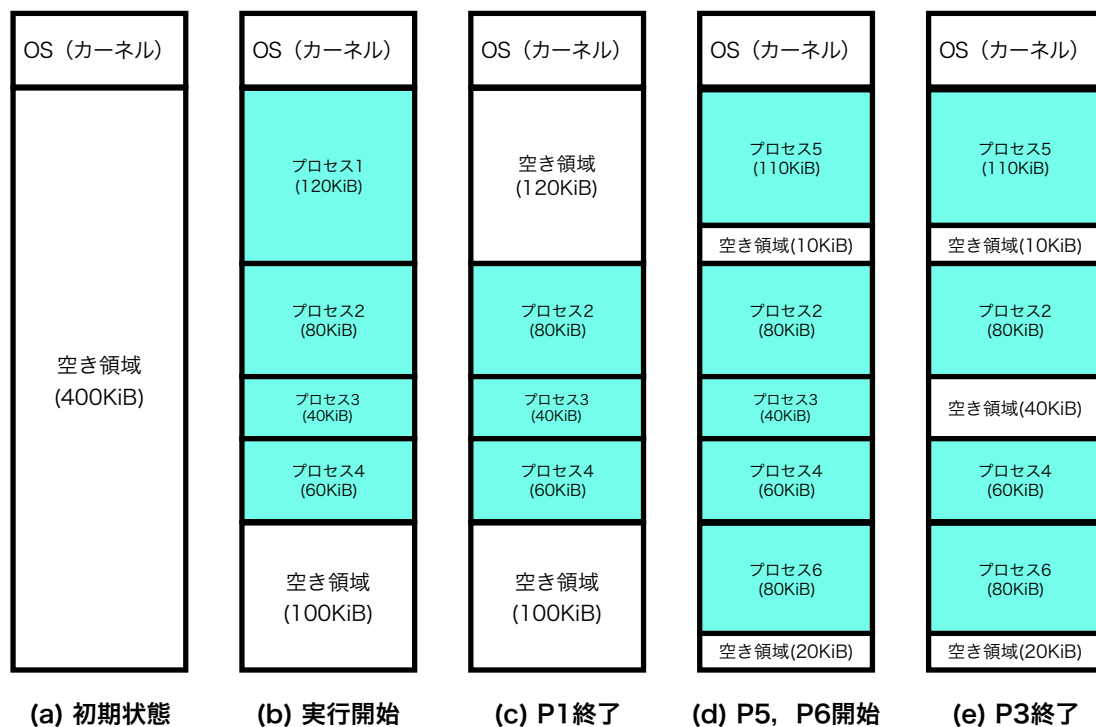


図 9.2: 可変区画方式

な区画を選択しプロセスをロードする。プロセス4はロード可能な区画が無いので実行できない。

図 9.1b の区画1 から 3 ように、区画の大きさとプロセスの大きさは一致するとは限らない。内部に使用されない領域（内部フラグメント）が生じる。また、区画4と区画5を合わせるとプロセス4をロード可能であるが、固定区画方式では区画を組合せて利用することはできない。仕組みは簡単だがメモリの利用率が低い。特徴を以下にまとめる。

1. 空き領域の管理が容易である。
2. 領域内部に無駄な領域（内部フラグメント）が生じる。
3. 小さな領域が複数空いても大きなプロセスは実行できない。
4. 実行可能なプロセスのサイズに強い制約がある。
（図の例では、151KiB のプロセスは実行できない。）
5. 同時に実行できるプロセスの数に制約がある。
（図の例では、同時に六つ以上のプロセスは実行できない。）

9.2 可変区画方式

空き領域に必要な応じたサイズの区画を割付ける方式である。図 9.2 に模式図を示す。

(a) 初期状態

メモリは、カーネル領域と一つの空き領域に分割される。

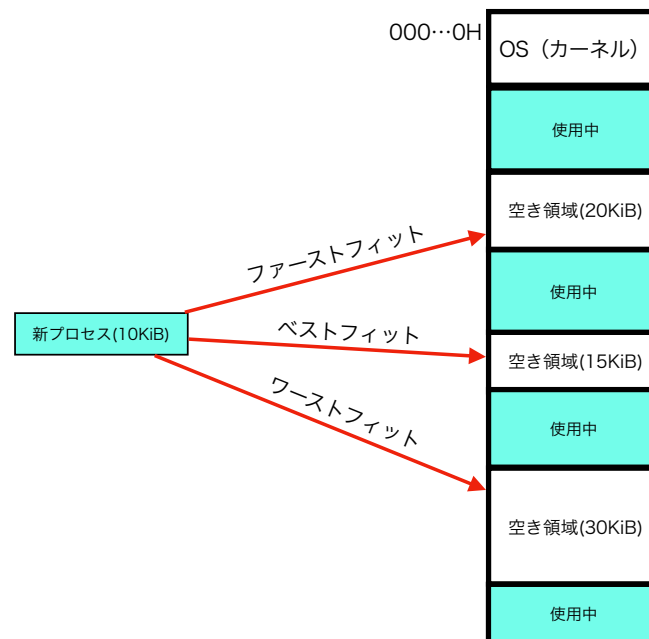


図 9.3: 空き領域の選択方式

(b) 実行開始

図 9.1 と同じ四つのプロセスがロードされ実行を開始した。図 9.1 の例では実行できなかった「プロセス 4」も実行できる。(メモリの利用効率は良い。)

(c) プロセス 1 (P1) 終了

終了したプロセスが利用していた領域は、再利用可能な空き領域になる。

(d) プロセス 5 (P5)、プロセス 6 (P6) 実行開始

120KiB の空き領域は、「110KiB の領域」と「10KiB の空き領域」に分割する。100KiB の空き領域は、「80KiB の領域」と「20KiB の空き領域」に分割する。プロセス 5 とプロセス 6 を新しい領域にロードし実行する。

(e) プロセス 3 (P3) 終了

プロセス 3 が利用していた領域は、再利用可能な 40KiB 空き領域になる。メモリ全体では、10KiB, 40KiB, 20KiB の空き領域ができた。

以上のように可変区画方式では、プロセスの開始と終了が繰り返されるに従い小さな空き領域ができる。このような区画の外にできる小さなメモリ領域を外部フラグメントと呼ぶ。

9.3 可変区画方式の空き領域選択方式

以下の三つの方式が知られている。図 9.3 に三つの方式で選択される空き領域の例を示す。

- ファーストフィット (first-fit) 方式

アドレス順に空き領域を探索し、最初に見つかった十分な大きさの領域を選択する。

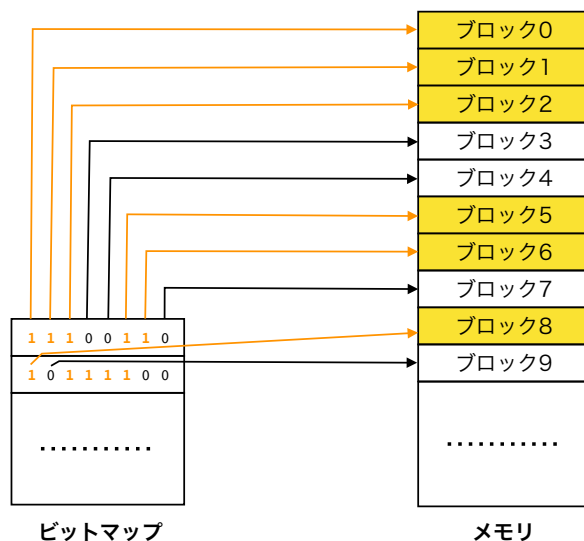


図 9.4: ビットマップ方式

- ベストフィット (*best-fit*) 方式

プロセスを格納可能な領域の中で最小のものを選択する。

- ワーストフィット (*worst-fit*) 方式

最も大きな領域を選択する。

シミュレーションの結果、メモリ利用率の点でワーストフィット方式は最も性能が劣るが、ファーストフィットとベストフィットの性能は互角だと言われている。しかし実行時間の点で、ファーストフィットがベストフィットより優れている [42]。

9.4 空き領域の管理方式

プロセスによって使用中のメモリ区画は、プロセスの PCB 等に記録しておけば見失う心配はない。しかし、どのプロセスにも属さない空き領域はメモリ管理側で記録しておく必要がある。

- ビットマップ (*bitmap*) 方式

図 9.4 のようにメモリを一定の大きさのブロックに分割し、1 ブロックをビットマップの 1 ビットに対応させる。ビットが 0 ならブロックが空き状態、ビットが 1 なら使用中の意味になる。

ビットマップはメモリ上に記録する。ビットマップの大きさは次のように計算できる。仮に 8GiB のメモリを 4KiB のブロックに分割して管理すると仮定すると、ブロックの総数は $8GiB \div 4KiB = (8 \times 2^{30}) \div (4 \times 2^{10}) = 2 \times 2^{20}$ 個となる。ビットマップの大きさはブロック数と同じ 2×2^{20} ビットになる。これをバイト単位に換算すると、 $(2 \times 2^{20}) \div 8 = 2^{18} = 256KiB$ となる。

ビットマップに使用するメモリは無視できるほど小さいものではない。ビットマップを小さくするにはブロックサイズを大きくすれば良い。しかし、ブロックサイズを大きくすると内部フラグメントが大きくなる。

- リスト (*linked-list*) 方式

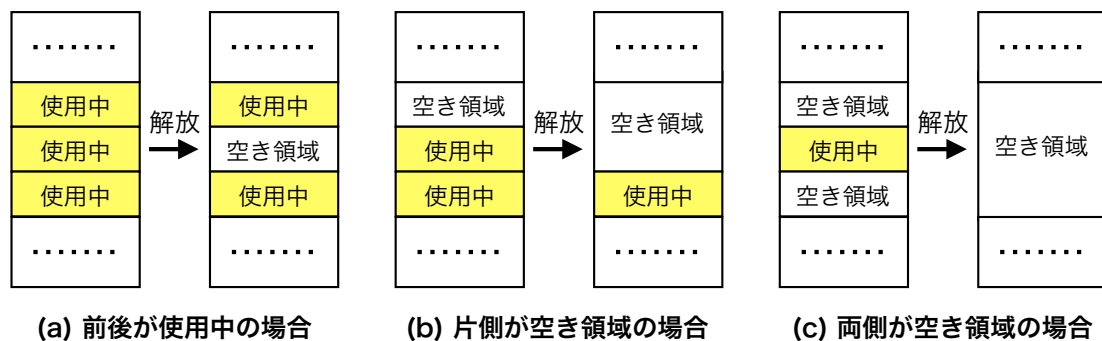


図 9.5: 領域開放時に空き領域を連結する様子

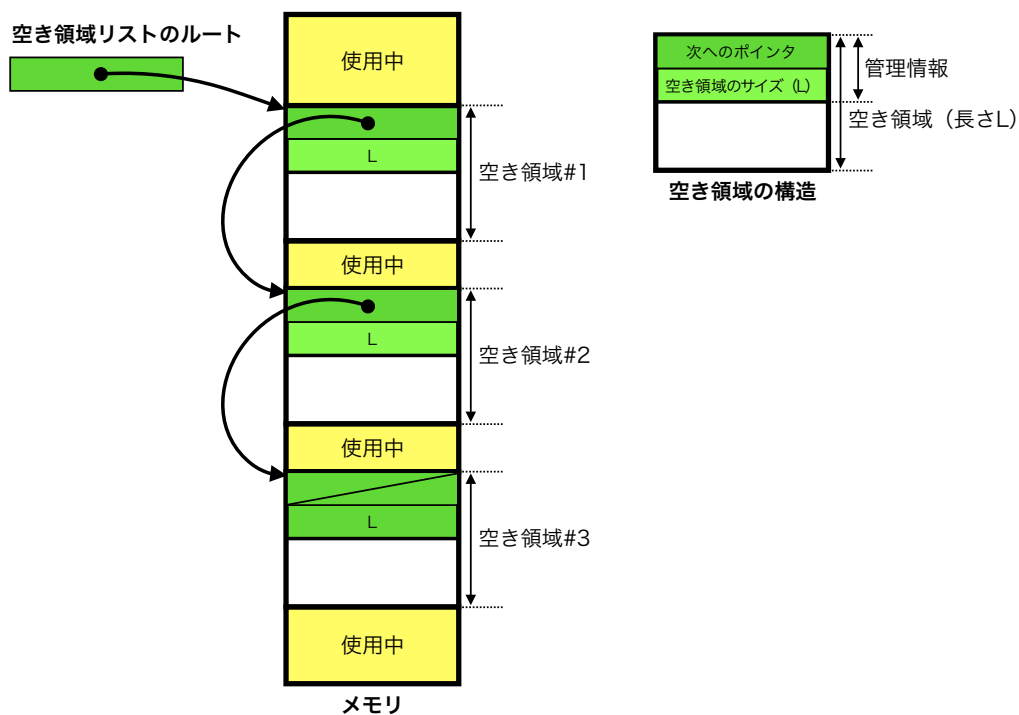


図 9.6: 空き領域リスト

空き領域をリストにして管理する方式である。使用中の領域が解放されると空き領域リストに追加される。解放される領域が、別の空き領域に隣接している場合は一つの空き領域になる。その様子を図 9.5 に示す。

リスト方式で用いるデータ構造の例を図 9.6 に示す。新しい空き領域と前後の空き領域をマージする処理が簡単に行えるように、空き領域はアドレス順にソートしてリストに挿入される。

アドレス順に領域がソートしてあると、ファーストフィット方式で領域を探索するためにも適している。ベストフィット方式の場合は領域サイズ順にソートしてあると良いが、前述の空き領域のマージ処理には適さない。

9.5 実装例

第22章に TacOS のメモリ割付けプログラムの例を示す。この例は、可変区画方式、ファーストフィット方式のメモリ管理プログラムを C--言語で実装したものである。

9.6 まとめ

物理メモリを分割しプロセスに割り付ける方式について学んだ。予めメモリを分割しておく固定区画方式と、必要に応じて分割する可変区画方式を紹介した。

可変区画方式における空き領域の選択方式には、ファーストフィット方式、ベストフィット方式、ワーストフィット方式があった。また、空き領域の管理方式には、ビットマップ方式、リスト方式があった。

練習問題

9.1 次の言葉の意味を説明しなさい。

- (a) 固定区画方式
- (b) 可変区画方式
- (c) 内部フラグメント
- (d) 外部フラグメント
- (e) ファーストフィット
- (f) ベストフィット
- (g) ワーストフィット
- (h) ビットマップ方式
- (i) リスト方式

9.2 可変区画方式で管理される 100KiB の空き領域がある時、次の順序で領域の割付け解放を行った。ファーストフィット方式を用いた場合とベストフィット方式を用いた場合について、実行後のメモリマップを図示しなさい。

- (a) 30KiB の領域を割付け
- (b) 40KiB の領域を割付け
- (c) 20KiB の領域を割付け
- (d) 先程割付けた 40KiB の領域を解放
- (e) 10KiB の領域を割付け

9.3 本章ではカーネルがプロセスにメモリを割り付ける方式について学んだ。プロセスのアドレス空間では `malloc()` 関数がヒープセグメントにメモリを割り付ける。`malloc()` 関数の仕組みを調査しなさい。

第 10 章

セグメンテーション

プロセスが使用するメモリ領域のサイズを動的に変化させたり、領域ごとに異なる性質を持たせたりすることが可能な、より高度なメモリ管理手法であるセグメンテーションを紹介する。

10.1 リロケーションレジスタ方式の問題点

ユーザは図 8.7 のように仮想アドレス空間にプログラムやデータを配置する。既に学んだリロケーションレジスタを用いた方式では、仮想アドレス空間は図 8.6 のようにメモリの連続領域にマッピングされる。この方式は以下の問題点を持っている。

- 必要なメモリの見積もりが難しい。

十分な大きさの仮想アドレス空間を準備しないと、実行時にヒープ領域やスタック領域が不足する可能性がある。しかし無闇に大きくするとヒープ領域とスタック領域の間が広くなりすぎメモリが無駄になる。実行前に必要なメモリの大きさを見積もる必要があり使い勝手が悪い。

- 領域の性質に応じたメモリ保護ができない。

リロケーションレジスタを用いて他プロセスやカーネルのメモリは保護可能である。しかし、プロセスが自身の領域を適切に使用することを強制できない。例えば、次のようなメモリ保護が望まれる。

- プログラム領域は機械語プログラムと定数データだけを格納しているので、読み出しと実行だけ許可する。
- データ、ヒープ、スタック領域に機械語プログラムは置かれないので、データの読み出しと書き込みだけ許可する。

10.2 セグメント

プロセスに複数のアドレス空間を持たせることで前記の問題を解決する。複数持つことができるアドレス空間のことをセグメントと呼ぶ。図 10.1 に複数のセグメントが存在する仮想アドレス空間の例を示す。プロセスの仮想アドレス空間は、セグメント番号とセグメント内アドレスの二つでアドレス付けされる。プロセスの仮想アドレス空間が二次元になった。

図 10.1 は以下のことを表している。プログラム、データ、ヒープ、スタックの領域を番号付けされ

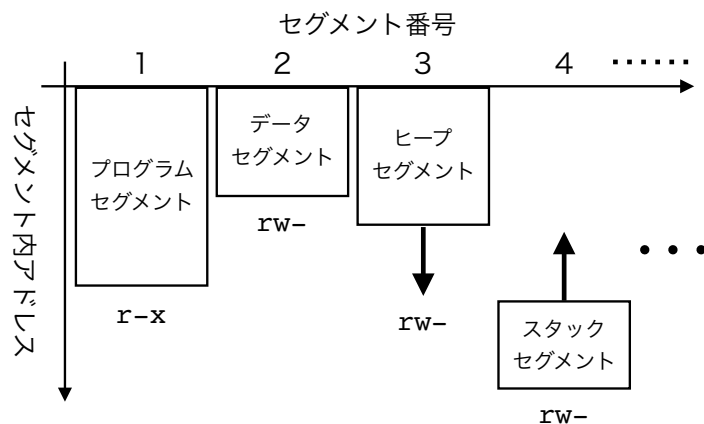


図 10.1: セグメントからなる仮想アドレス空間

たセグメントにした。セグメントに付記した `rw` はセグメントの保護モードを表している。セグメントの大きさは内容の大きさとぴったり同じサイズにできるので、内部フラグメントが生じない。ヒープ領域は実行時に必要に応じて長くすることができる。スタックが前に向かって伸びる場合は、前向きに伸びるセグメントが使いやすい。IA-32^{*1}は、前に向かって伸びるセグメントもサポートしている [43]。

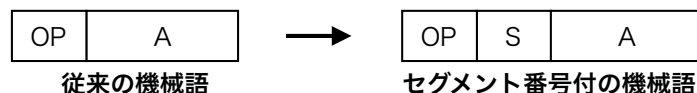
各領域を独立したセグメントにすることで、仮想アドレス空間内の領域配置の問題から解放された。

10.3 セグメント番号

仮想アドレスにセグメント番号が新たに必要になった。セグメント番号を供給するために CPU に変更が必要になる。以下では、セグメント番号を提供する方法を考える。

10.3.1 命令コード

機械語命令コードを変更し、セグメント番号を含める方法が考えられる。各命令がセグメント番号のために大きくなるので、プログラムサイズが大きくなる。



10.3.2 カレントセグメントレジスタ

CPU 内部に現在のセグメント番号を格納するレジスタを置く方式である。別のセグメントへプログラムをジャンプさせるセグメント間ジャンプ命令 (JMPS と仮に命名する)、セグメント間コール命令 (CALLS と仮に命名する)、セグメント間リターン命令 (RETS と仮に命名する) がカレントセグメントレジスタに新しいセグメント番号をロードする。

図 10.2 に模式図を示す。まず、セグメント 1 のプログラムが実行される。この時点ではカレントセグメントはセグメント 1 なので、LD GO,A はセグメント 1 内のデータ A を参照する。

CALLS 3:0 はセグメント 3 の 0 番地に配置されたサブルーチン呼び出す。その際、カレントセグメントレジスタの値とプログラムカウンタの値がスタックに保存される。その後、カレントセグメント

*1 32bit パーソナルコンピュータで広く使われてきたインテル社 CPU のアーキテクチャのことである。

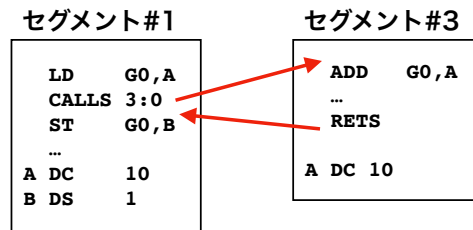


図 10.2: セグメント間のサブルーチンコール

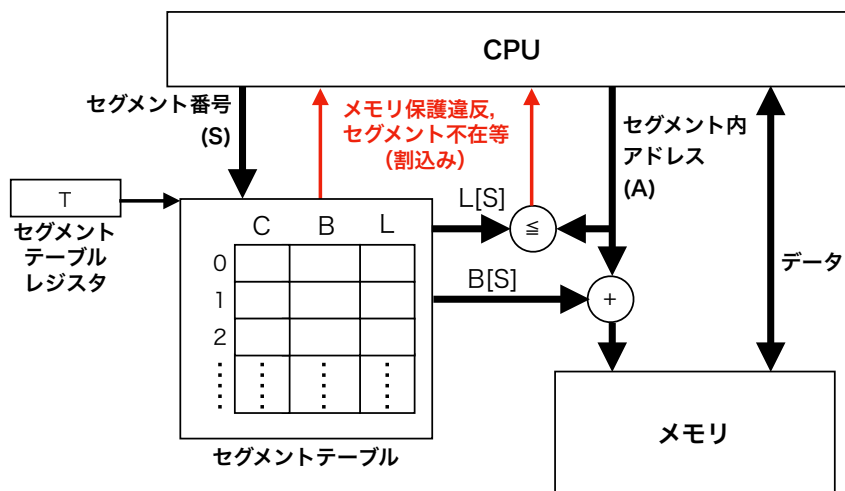


図 10.3: セグメンテーション機構

はセグメント 3 に、プログラムカウンタは 0 に変更される。サブルーチン実行中のカレントセグメントはセグメント 3 なので、サブルーチン中の `ADD GO, A` はセグメント 3 のデータ A を参照する。

`RETS` が実行されるとカレントセグメントレジスタとプログラムカウンタの値がスタックから復元され、セグメント 1 のプログラムの実行が再開される。

IA-32 は、プログラム用 (CS)、データ用 (DS)、スタック用 (SS) 等、数個のカレントセグメントレジスタを持つ。機械語命令のフェッチには CS、データのアクセスには DS、スタックの操作には SS が暗黙の内に使用される [44]。

10.4 セグメンテーション機構

セグメンテーション機構の模式図を図 10.3 に示す。CPU が出力したセグメント番号 (S) とセグメント内アドレス (A) の組を、セグメントテーブルを使用して物理アドレスに変換する。

10.4.1 セグメントテーブル

現在のプロセスが使用できるセグメントの一覧表である。表の一行 (エントリ) が一つのセグメントを表現する。B (Base) フィールドはセグメントの物理アドレス、L (Limit) フィールドはセグメントサイズであり、セグメントテーブルのエントリはリロケーションレジスタと同様な内容を含んでいる。

C (制御) フィールドは、表 10.1 のビットを含んでいる。V (Valid) ビットが 0 の場合、そのセグメ

表 10.1: セグメントテーブル C フィールドの例

名称	ビット数	意 味
V (Valid)	1	メモリにロードされている.
D (Dirty)	1	ロード後に変更された.
RWX (Read/Write/eXecute)	3	許されるアクセス方法.

ントはメモリ上に存在しない. 存在しないセグメントをアクセスしようとするすると**セグメント不在割込み**が発生する. D (Dirty) ビットはセグメントの内容がメモリにロードされた後に変更されたことを記録する. メモリが不足してセグメントをスワップアウトする際, D ビットが 0 ならセグメントを二次記憶装置へ書き戻す必要がない. RWX (Read/Write/eXecute) の三ビットはセグメントに対して行って良い操作を表す. CPU が許可されていない操作を行うとメモリ保護違反割込みが発生する.

図 10.3 では, セグメントテーブルが専用のハードウェアとして描かれているが, セグメントテーブルはメモリ上に置かれる. セグメントテーブルのアドレスは**セグメントテーブルレジスタ**が記憶している. プロセスを切り換える際は, そのプロセスのセグメントテーブルを指すようにセグメントテーブルレジスタを書き換える.

10.4.2 物理アドレスへの変換

CPU が出力したセグメント番号 (S) とセグメント内アドレス (A) の組は, 以下の手順で物理アドレスに変換される.

1. セグメントテーブルのエントリ読み出し

セグメントテーブルは, セグメントテーブルレジスタによって示されるメモリ上のアドレスに配置されている. セグメント番号をセグメントテーブルのインデクスとして使用し, セグメントテーブルの一つのエントリをメモリから読み出す.

2. C フィールドのチェック

読み出したエントリの C フィールドを調べ, セグメントがメモリにロードされていない場合や, 許可されていない種類 (RWX) のアクセスを CPU が行おうとしている場合は, 割込みが発生する.

3. セグメント内アドレスのチェック

読み出したエントリの L フィールド (L[S]) と CPU が出力したセグメント内アドレス (A) を比較する. L[S] はセグメントのサイズを表すので, A が L[S] 以上の場合はセグメント内アドレスがセグメントの後端を越えている. メモリアccessを阻止した上で割込みが発生する.

4. 物理アドレスの計算

読み出したエントリの B フィールド (B[S]) と CPU が出力したセグメント内アドレス (A) の和を求める. 和が物理アドレスである.

10.4.3 セグメントテーブルエントリのキャッシング

前記の物理アドレスへの変換手順では, メモリアccessの度にセグメントテーブルを参照していた. セグメントテーブルの参照はメモリアccessなので, メモリアccess回数が二倍になる. 他の CPU や I/O 装置もメモリを使用するので, メモリへのアクセスは混み合っている. メモリアccess回数は少な

セグメントレジスタ		裏レジスタ		
CS	セクタ	ベース	リミット	属性
DS	セクタ	ベース	リミット	属性
SS	セクタ	ベース	リミット	属性
ES	セクタ	ベース	リミット	属性
FS	セクタ	ベース	リミット	属性
GS	セクタ	ベース	リミット	属性

図 10.4: IA-32 のセグメントレジスタと裏レジスタ

くすべきである。

一方で、同時に使用されるセグメントの数は多くないので、必要なテーブルエントリを CPU や MMU にキャッシュすることは容易である。例えば IA-32 では、カレントセグメントレジスタ（CS, DS, SS 等）毎に、セグメントテーブルエントリのコピーを裏レジスタ [45] に持つ。カレントセグメントレジスタの値が変更された時、自動的に裏レジスタにエントリがコピーされる。図 10.4 にセグメントレジスタと裏レジスタの関係を示す。セグメントレジスタに格納されるセクタはセグメント番号に相当する。裏レジスタはハードウェアが自動的に使用しプログラムからは見えない。

10.5 セグメンテーション機構による仮想記憶

プログラム実行中に必要なセグメントだけをメモリにロードするようにする。これにより、全体がメモリに収まらない大きなプログラムも実行できる。メモリより大きなプログラムが実行できる点で、メモリが仮想化されたと言うことができる。仮想化されたメモリのことを**仮想記憶**と呼ぶ。

10.5.1 スワップイン

セグメントテーブルの V ビットが 0 のセグメントを参照すると、**セグメント不在割込み**が発生する。プロセスがセグメント不在割込みが発生するとオペレーティングシステムに制御が移る。

オペレーティングシステムは、まず、割込みの原因になったセグメントを二次記憶装置から読み込む（**スワップイン**する）。次に、セグメントテーブルを書き換える。最後に割込みが発生した命令の再実行から再開するようにプロセスをディスパッチする。

10.5.2 スワップアウト

オペレーティングシステムがセグメントをスワップインする際にメモリが不足するかも知れない。その場合、オペレーティングシステムは適切なセグメントを選択し二次記憶に追い出し（**スワップアウト**し）、メモリに空きを作らなければならない。今後、使用されそうに無いセグメントを選択すると良いが、どのセグメントが該当するか判断することは難しい問題である。

図 10.5 にセグメントがスワップアウト／スワップインされる様子を示す。図は新しくセグメント 3 が必要になりスワップインする様子を表している。セグメント 3 をロードするためにはメモリが不足するので、まず、使用頻度が低いセグメント 1 をスワップアウトし、次に、セグメント 3 をスワップインする。

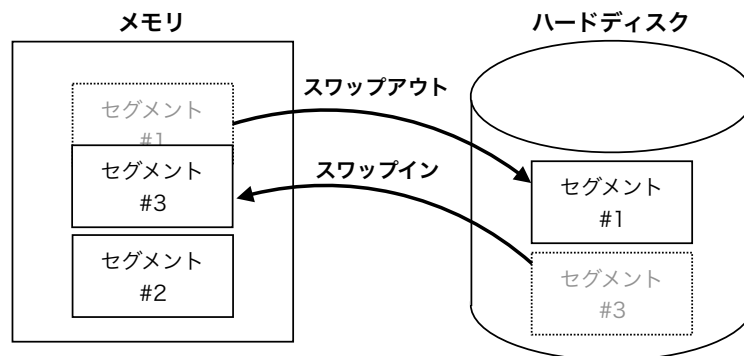


図 10.5: セグメントのスワッピング

10.6 セグメントの共用

プロセス間でセグメントを共用することでメモリの節約ができる。プログラムや定数等を格納し、書き込み禁止のセグメントは複数のプロセスで共用できる。逆に、書き込みが許可されているデータ、ヒープ、スタックセグメント等は共用できない。また、セグメントの共用を積極的に利用しプロセス間の共有メモリも実現できる。図 10.6 に三つのプロセスがセグメントを共用している様子を示す。

- C 言語ライブラリは、C 言語で使用する関数（`printf()` 等）を提供する。ライブラリはプログラムだけを格納し変更されないで、全てのプロセスで共用することができる^{*2}。
- プロセス 1 とプロセス 2 はどちらも emacs を実行している。プログラムは変更されないで、プロセス 1 とプロセス 2 で「emacs プログラムセグメント」を共用できる。
- プロセス 3 は a.out を実行している。プログラムは変更されないが、同じプログラムを実行中のプロセスが存在しないので「a.out プログラムセグメント」は共用できない。
- プロセスが書き換えるデータセグメントやスタックセグメントは、プロセス毎に内容が異なるので共用できない。別々のデータセグメントやスタックセグメントが必要になる。プロセス 1 とプロセス 2 はどちらも emacs を実行しているが、編集している文書が異なるのでデータセグメントの内容も異なるハズである。

10.7 セグメンテーションの利点・欠点

セグメンテーションの利点と欠点を以下にまとめる。

利点

- セグメントには、例えば「C 言語ライブラリセグメント」のような、論理的な意味を持たせることができる。
- セグメントの論理的な意味を反映したメモリ保護が可能である。
- プログラムやデータの共用が容易である。

^{*2} 本当は、ライブラリが使用するグローバル変数（例えば `errno`）をどうするか問題である。

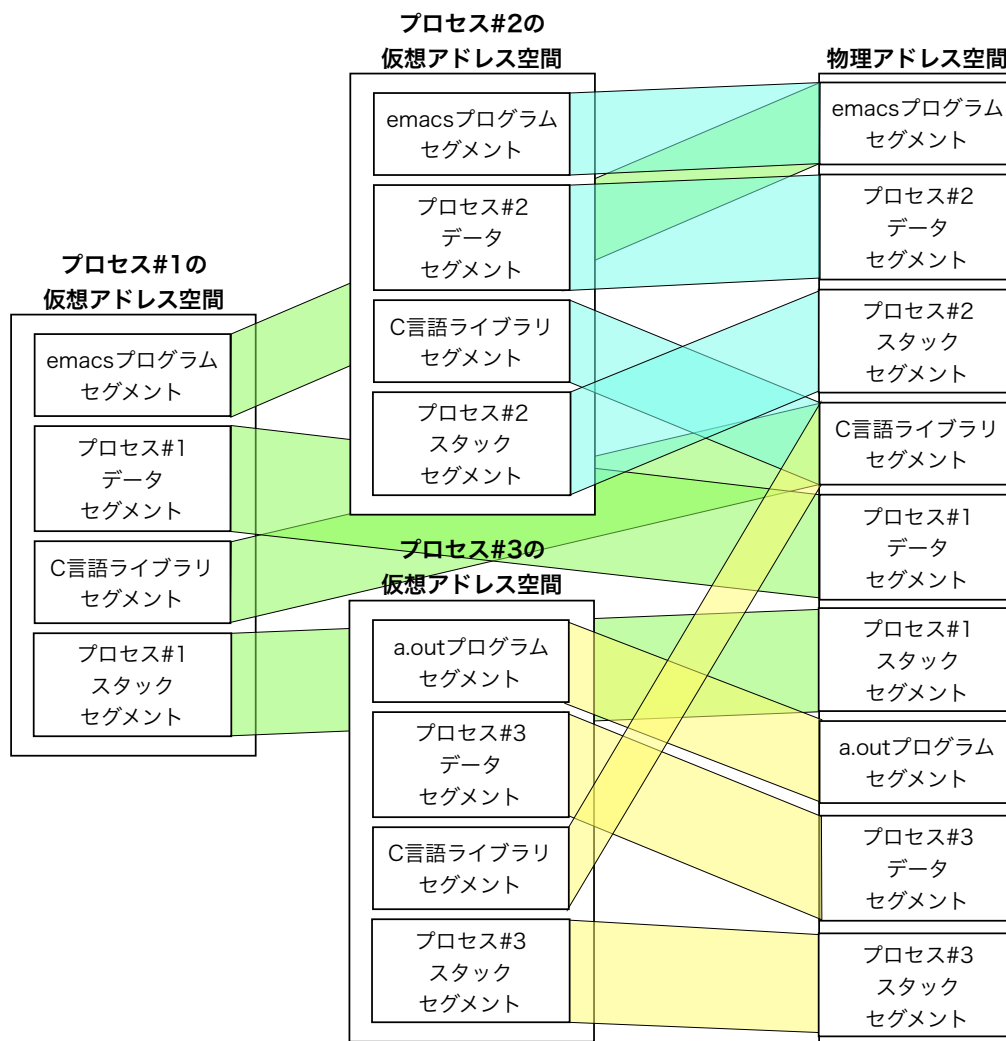


図 10.6: セグメントの共用

- セグメントの長さは自由に決められるので内部フラグメントが発生しない。
- セグメントの長さは動的に変化させることも可能である。
- セグメント単位のスワッピングを用いて仮想記憶を実現できる。

欠点

- 物理アドレス空間に外部フラグメントが生じる。
- 外部フラグメントの解消にはメモリコンパクションが必要である。
- 物理メモリ上に連続した領域が必要である。
- 物理メモリより大きいセグメントを作ることができない。

外部フラグメント問題と、物理メモリサイズによるセグメントサイズの制約を解決するために、次の章で紹介するページングと組合せてセグメンテーションを利用するシステムもある。例えば、IA-32はそうである。IA-32ではセグメンテーション機構が出力する一次元のアドレスをページング機構が物理

アドレスに変換する。

10.8 まとめ

本章では、領域の性格に合わせたメモリ保護ができる高度な管理機構であるセグメンテーションを紹介した。セグメントには論理的な意味付けをすることができる。論理的な意味付けに合わせてメモリ保護モードを設定したり、プログラム間で共有したりする。また、セグメントは可変長なので内部フラグメントを生じない。しかし、外部フラグメントを生じるのでメモリコンパクションを必要とする。

セグメントのスワッピングによる仮想記憶を実現できるが、物理メモリより大きなセグメントを作ることはいできない。そこで、ページングとセグメンテーションを組み合わせて利用するシステムがある。

練習問題

10.1 セグメントテーブルが次のような状態の時、以下の問に答えなさい。なお、仮想アドレスは「セグメント番号：物理アドレス」と表記する。また、物理アドレスは 8 ビットとする。

	C	B	L
0	V=1	0x30	0x20
1	V=1	0x80	0x30
2	V=1	0x00	0x20
3	V=0	0x50	0x20
...	...	...	...

(a) 次の仮想アドレスに対応する物理アドレスを答えなさい。但し、物理アドレスに変換できない場合はエラーと答えなさい。

- i. 0x0:0x10
- ii. 0x1:0x10
- iii. 0x1:0x40
- iv. 0x2:0x10
- v. 0x2:0x20
- vi. 0x3:0x10

(b) セグメントの配置を記入した物理アドレス空間のメモリマップを作成しなさい。

10.2 スタックセグメントを意識した前向きに伸びるセグメントも利用可能なセグメンテーション機構を設計しなさい。

- (a) セグメントテーブルに必要な変更は？
- (b) 図 10.3 に必要な変更は？
- (c) 他に必要な変更は？

第 11 章

ページング

メモリを一様なページに分割し、ページ単位で管理することで使いやすい仮想メモリを提供する。メモリより大きな仮想アドレス空間を使用でき、メモリコンパクションが不要なメモリ管理方式である。Windows, macOS, Linux 等の多くのオペレーティングシステムがページングを採用している。

11.1 基本概念

図 11.1 に示すように、プロセスの仮想アドレス空間は固定サイズのページに分割される。物理アドレス空間もページと同じサイズのフレーム^{*1}に分割される。ページはフレームにマッピングされる。

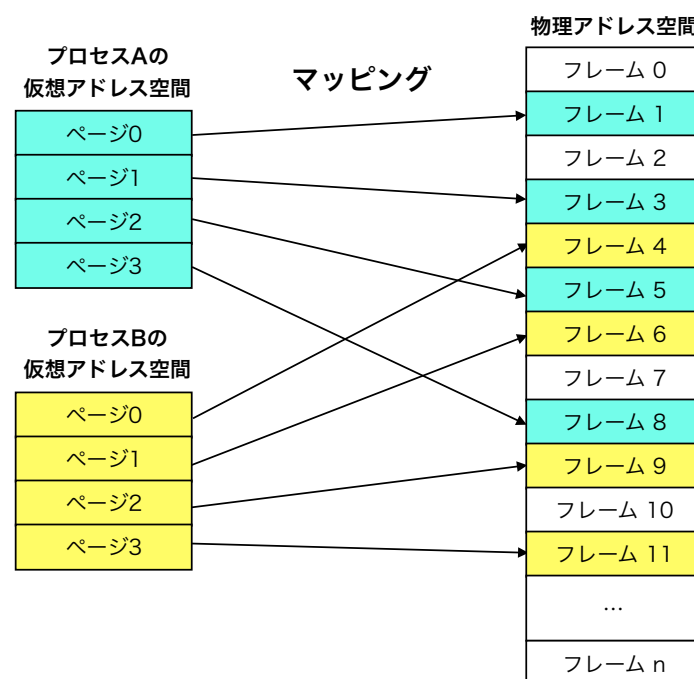


図 11.1: ページからフレームへのマッピング

^{*1} 「フレーム」は、「物理ページ」、「ページフレーム」と呼ばれることもある。

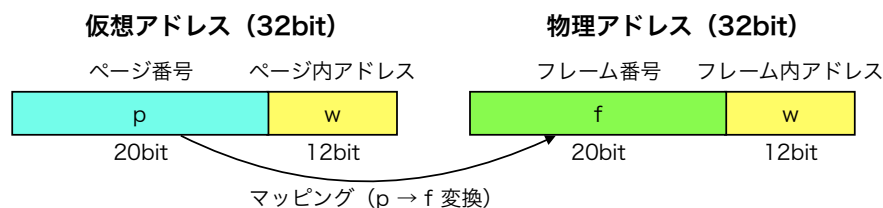


図 11.2: ページング使用時のアドレス例

11.1.1 ページとフレーム

ページのサイズは 2 の累乗にする。これにより、仮想アドレスの上位ビットをページ番号、下位ビットをページ内アドレスに分割して扱うことができる。物理アドレスでは上位ビットをフレーム番号、下位ビットをフレーム内アドレスに分割して扱う。図 11.2 に、32 ビットの仮想アドレスを 4KiB^{*2}のページに分割した例と、32 ビットの物理アドレス空間を 4KiB のフレームに分割した例を示す。4KiB のページ（フレーム）をバイト毎にアドレス付けするためには、次の計算から分かるように 12 ビットのページ内（フレーム内）アドレスが必要である。

$$4KiB = 4 \times 1KiB = 2^2 \times 2^{10}B = 2^{12}B$$

上位 20 ビットがページ（フレーム）番号を下位 12 ビットがページ内（フレーム内）アドレスを表現する。

仮想アドレスのページ番号（p）を物理アドレスのフレーム番号（f）に変換することで、ページがフレームにマッピングされる。図 11.2 の例では仮想アドレスも物理アドレスも同じ 32 ビットであるが、異なるサイズでも構わない^{*3}。図 11.1 は物理アドレス空間の方が広い例になっていた。

11.1.2 マッピング関数

仮想アドレス由来のページ番号（p）を、物理アドレスの一部であるフレーム番号（f）にマッピングする。マッピング関数はページテーブルと呼ばれる表として実装する。メモリ管理ハードウェア（MMU：Memory Management Unit）が実行時にページテーブルを参照し動的にマッピングを行う。

プロセス毎に異なる仮想アドレス空間をマッピングするので、プロセス毎に異なるマッピング関数（ページテーブル）が必要である。ディスパッチャはプロセスの実行を開始する前に MMU を操作し、新しいプロセスのマッピング関数を有効にする必要がある。図 11.1 の例では、プロセス A 実行時にはページ 0 がフレーム 1 へマッピングされる関数を使用するが、プロセス B 実行時にはページ 0 がフレーム 4 へマッピングされる関数に切り換える必要がある。

11.1.3 外部フラグメンテーション

全てのフレームは、任意のプロセスの任意のページにマッピング可能である。連続したフレームが存在しないと使用できない等の制約は無いのでフレームが無駄になることはない。ページングを用いることで割り付け単位（フレーム）の外にフラグメントが発生しなくなる。外部フラグメンテーション問題は解決しメモリコンパクションも不要になった。

^{*2} IA-32 のページサイズは基本的に 4KiB である。x86-64 でも基本は 4KiB である。

^{*3} 同じアーキテクチャですら、時期によって関係が変化することがある。IA-32 の物理アドレスは、当初は 32 ビットであったが途中から 36 ビットに変更された。この間、論理アドレスは 32 ビットのまま変更されていない。

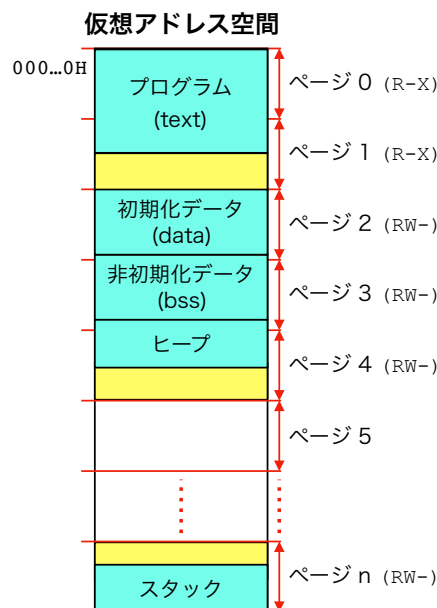


図 11.3: ページング使用時の仮想アドレス空間の例

11.1.4 内部フラグメンテーション

例えば UNIX プロセスの仮想アドレス空間は、図 11.3 のように配置される。プログラム領域はページ 0 からページ 1 の途中までを使用する。これらのページは読み出しと実行だけ (R-X) ができるようにメモリ保護を行う。データとヒープは読み出しと書き込みだけ (RW-) ができるようにするので、プログラムとは異なるページに配置する必要がある。そこで、ページ 1 の後半は使用しないで、ページ 2 からページ 4 にデータとヒープを配置する。ページ 5 からページ n-1 までは使用しないのでフレームを割り付けない。仮想アドレス空間に穴が空いた状態にする^{*4}。スタックはページ n に割り付ける。

以上のように配置すると、ページ 1 の後半、ページ 4 の後半、ページ n の前半に、フレームが割り付けられているにも係わらず使用されない領域ができる。このようにページ内部 (フレーム内部) に無駄な領域が発生することを**内部フラグメンテーション**と呼ぶ。フラグメント領域は使用されないはずだが、ユーザプログラムが誤ってアクセスするかもしれない。ページングではメモリ保護をページ単位で行うので、このような不正なアクセスを検知できない問題がある。

11.2 ページング機構

以上で説明したページングを実現するためのハードウェア機構について考える。

11.2.1 ページング機構の概要

図 11.4 にページング機構の模式図を示す。CPU が出力した仮想アドレスは、ページ番号 (p) とページ内アドレス (w) に分けられる。ページ番号は、ページテーブルから一つのエントリを選択するためのインデックスとして使用される。選択されたエントリのフレーム番号 (f) フィールドとページ内アドレス (w) を結合して物理アドレスを得る。

^{*4} *sparse address spaces* と呼ばれる。

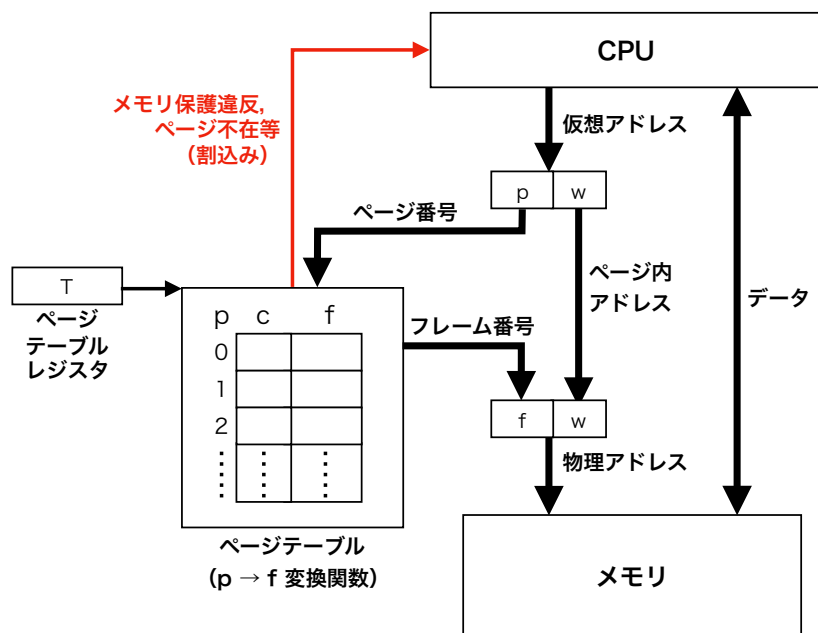


図 11.4: ページング機構の概要

表 11.1: ページテーブルの C フィールドの例

名称	ビット数	意味
V (Valid)	1	フレームが割り付けられている。
R (Reference)	1	ページの内容が参照された。
D (Dirty)	1	ページの内容が変更された。
RWX (Read/Write/eXecute)	3	許されるアクセス方法。

11.2.2 ページテーブルエントリ

ページテーブルのエントリはページ番号で選択する。エントリの内容は c（制御）と f（フレーム番号）フィールドである。f フィールドの内容がページテーブルの出力になる。

c フィールドの内容は、例えば表 11.1 のようなものである。ページにフレームが割り付けられていない場合は V ビットが 0 になっている。V ビットが 0 のページにアクセスした場合は、CPU に**ページ不在割込み**（*page fault*）を発生する。page fault が発生した時点でフレームを割当て、ページの内容をスワップインすることで仮想記憶が実現できる。R ビットはページが参照された時に 1 に変化する。R ビットはページの使用頻度を調べるために使用される^{*5}。D ビットはページが書き換えられた時に 1 に変化する。D ビットはページをスワップアウトする際に使用される。

11.2.3 ページテーブル

ページテーブルは、かなり大きな表である。例えば図 11.2 のようにページ番号が 20 ビットで表現されるなら、ページテーブルは $2^{20} = 1\text{Mi}$ エントリの大きさを持つことになる。また、メモリアクセス

^{*5} 詳しくは第 12 章で説明する。

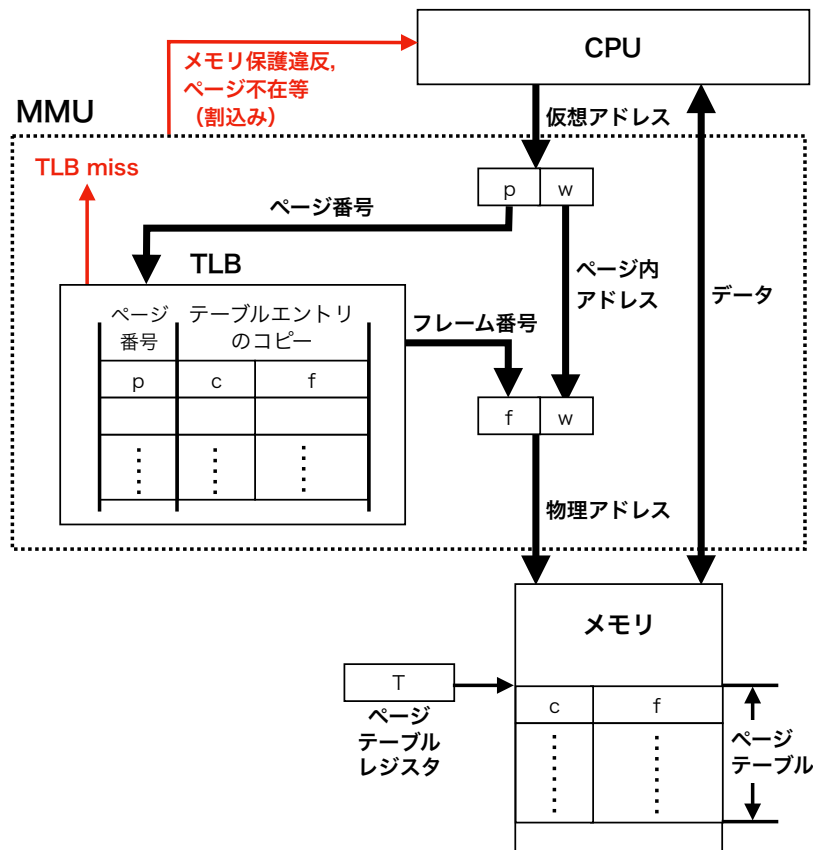


図 11.5: TLB を使用するページング機構

の度に参照されるので、通常のメモリと比較して桁違いに高速でなければならない。

図 11.4 ではページテーブルが専用のハードウェアのように描かれているが、このような大きくて高速な表を MMU の内部に持つことは困難である。また、プロセス毎にページテーブルが必要なので、プロセススイッチの度にページテーブル全体を MMU にロードし直すのも効率が悪い。そこで、ページテーブルはメモリ上に置くことになる。ページテーブルのメモリ上のアドレスはページテーブルレジスタが記憶している。

11.2.4 TLB (Translation Look-aside Buffer)

CPU がメモリをアクセスする度にメモリ上のページテーブルをアクセスすると、メモリのアクセス回数が二倍になる。そこで、変換結果を MMU 内の TLB と呼ばれる高速なメモリにキャッシュする。図 11.5 に TLB を使用したページング機構の模式図を示す。TLB はページ番号とフレーム番号の対応を記憶し、ページ番号をキーにして非常に高速に検索できる特殊なメモリである。このような記憶したキーで高速に検索できるメモリを連想メモリと呼ぶ。TLB のサイズは機種により異なり、数十エントリから数千エントリ程度である。

CPU が出力したページ番号を用いて TLB を検索し、見つければ TLB からフレーム番号が出力される。その際、TLB 上にコピーされた R (Reference) ビットや D (Dirty) ビットが操作されたり、RWX フィールドがチェックされる。チェックの結果、違反が見つければ CPU に割込みを発生する。

11.2.5 Page Table Walk

ページ番号が TLB に見つからない場合は、*TLB miss* になりメモリ上のページテーブルを検索する必要がある。ページテーブルを検索することを *page table walk* と呼ぶ。page table walk を行い TLB を更新する作業を MMU のハードウェアが自動的に行う機種と、CPU に割込みを発生しソフトウェアで行う機種がある。前者は page table walk の高速化を狙う。後者は MMU を単純にしたことで余ったチップ面積を TLB のエントリ数を増やすために使用し TLB miss の頻度を低くすることを狙う。

TLB に空きエントリが無い場合、新しいページテーブルエントリをロードする前に、どれかのエントリを TLB から捨てる必要がある。TLB 上のページテーブルエントリのコピーは、ロードされた後に R (Reference) ビットや D (Dirty) ビットが変更されている可能性がある。TLB のエントリを捨てる前にメモリ上のページテーブルに書き戻すことがある。

11.2.6 TLB エントリのクリア

プロセスは専用の仮想アドレス空間を持つので、プロセス毎に専用のページテーブルを持つことになる。プロセススイッチ時に、ディスパッチャが新しいプロセスのページテーブルのアドレスをページテーブルレジスタにロードする。

TLB は古いページテーブルの内容を反映しているので、ページテーブルを交換する際にクリアする。TLB の全てのエントリがクリアされると、直後に同じプロセスに戻ってきた場合や、カーネル領域などがプロセス間で共有される場合に効率が悪いので様々な工夫^{*6}が凝らされている場合もあるが、基本的にはプロセススイッチを行う際は TLB をクリアする。また、ページテーブルが変更された場合は、プロセススイッチが発生しなくても TLB をクリアする必要がある。

11.3 ページの共用

セグメンテーションではプロセスがセグメントを共用することができた。ページングではプロセス間でページを共用することができる。図 11.6 は、プロセス A とプロセス B が同じプログラム X を、プロセス C がプログラム C 実行している例である。各プロセスのページテーブルを適切に設定することで、図のようなマッピングをすることができる。

プログラム本体やライブラリの機械語は読み出しと実行専用 (R-X) になっており、プロセスが変更することは無いので共用することができる。プログラム X の機械語は第 1 フレームに格納されプロセス A とプロセス B で共用する。プログラム C の機械語は第 4 フレームと第 7 フレームを合わせた領域に格納される。プログラム C を実行しているのはプロセス C だけなので共用する必要がない。

ライブラリの機械語は第 3 フレームに格納されプロセス A、プロセス B、プロセス C が共用して利用する。ライブラリはプロセス A、B と、プロセス C で異なる仮想アドレスにマッピングされている。ライブラリ内の機械語プログラムは何番地にロードされても実行できる位置独立コード^{*7}でなければならない。セグメンテーションには、このような制約は無かった。

プロセス毎に内容が異なるデータやスタックは共用できない。プロセス毎に専用の領域を割当てる。

^{*6} TLB のエントリにプロセス番号も記録しクリア不要にする方式や、仮想アドレスを指定して特定のエントリだけクリアする方式が知られている。

^{*7} JMP や CALL 命令のアドレッシングは全てプログラムカウンタ相対で行う。データのアドレッシングは CPU レジスタに格納したアドレスを基準にした相対で行う。

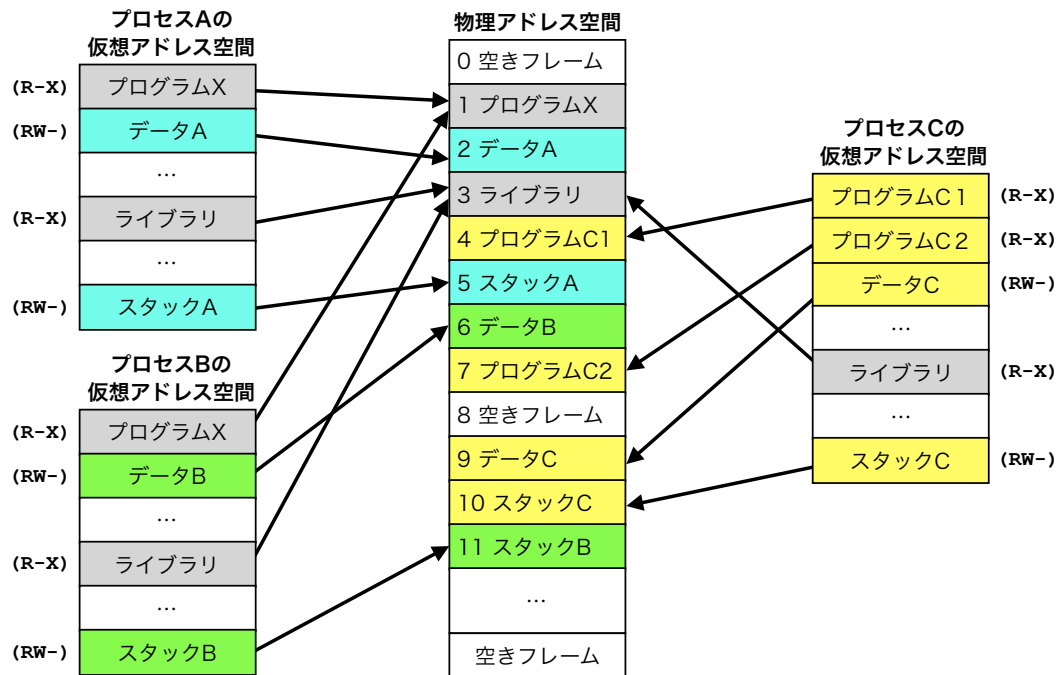


図 11.6: プロセス間でのページ共有

11.4 ページテーブルの編成方法

図 11.5 に示したようにページテーブルはメモリに置かれる。しかし、ページテーブルのサイズは無視できるほど小さなものではない。例えば、32 ビットマイクロプロセッサが PC に普及してきた 1990 年代の前半には、PC が備えるメモリは 4MiB から 16MiB 程度であった。IA-32 を用いる場合、ページテーブルの一つのエントリは 4 バイトなので、32 ビットの仮想アドレス空間を 4KiB のページで分割すると、次の計算のようにページテーブル全体では 4MiB になる。更に、ページテーブルはプロセス毎に必要である。メモリのほとんどをページテーブルに使用しても足りない。このままではページングは実用にならない。

$$2^{32}B \div 2^{12}B = 2^{20} = 1Mi \text{ エントリ}$$

$$1Mi \text{ エントリ} \times 4B = 4MiB$$

近年の 64 ビットマイクロプロセッサの場合も同様である。x86-64 では 48 ビットの仮想アドレス空間を 4KiB のページで分割する。ページテーブルの一つのエントリが 8 バイトなので、下の計算のようにページテーブルのサイズが 512GiB になる。これでは現代の PC にもページテーブルが大きすぎる。ページテーブルを小さくする必要がある。

$$2^{48}B \div 2^{12}B = 2^{36} = 64Gi \text{ エントリ}$$

$$64Gi \text{ エントリ} \times 8B = 512GiB$$

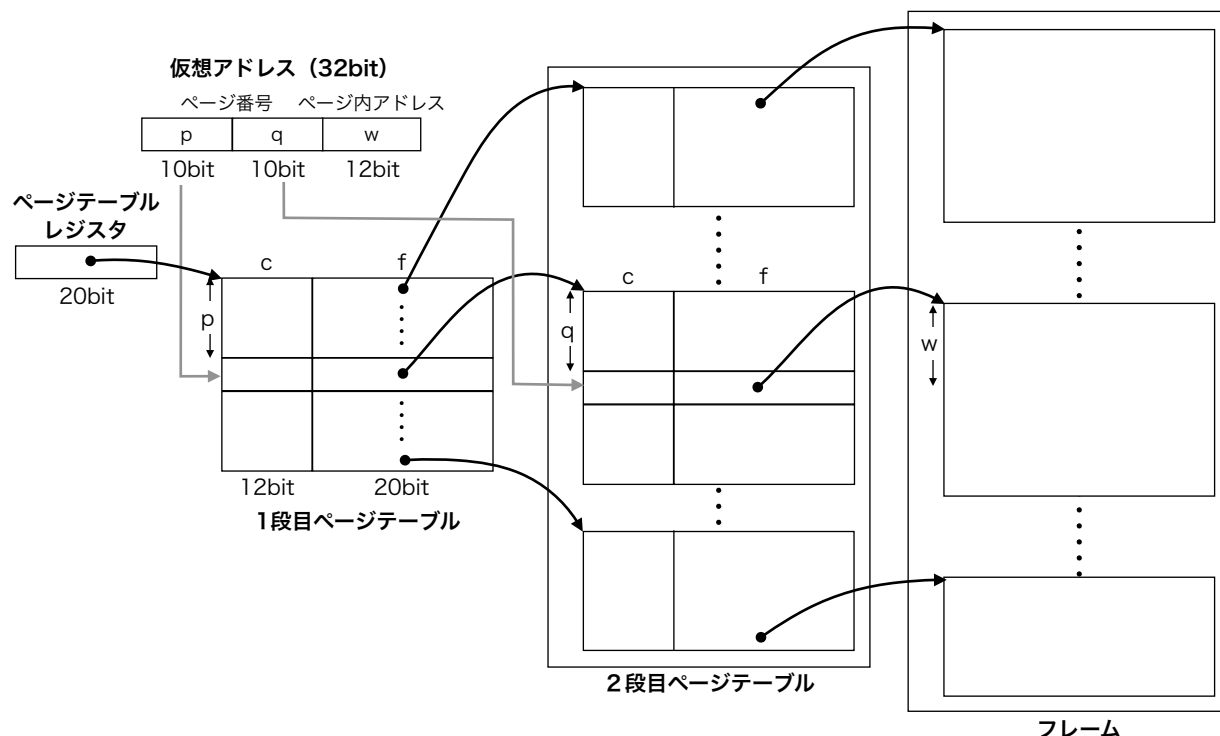


図 11.7: 二段ページテーブルの構造

11.4.1 二段のページテーブル

ページテーブルを二段にすることで、二段目に使用するメモリを節約することができる。図 11.7 に IA-32 で使用される二段のページテーブルの例を示す^{*8}。図の左上に示すように、32 ビットの仮想アドレスは 10 ビットのページ番号フィールド二つ (p , q) と、12 ビットのページ内アドレス (w) に分けられる。ページサイズは w が 12 ビットなので $2^{12} = 4KiB$ である。物理アドレスも 32 ビット^{*9}なので、フレーム番号は 20 ビットで表現する。

- *Page Table Walk*

まず、ページテーブルレジスタから一段目のページテーブルの位置を知る。次に、一段目のページテーブルの p 番目のエントリを参照することで二段目のページテーブルの位置を知る。最後に、二段目のページテーブルの q 番目のエントリを参照することでフレームの位置を知る。フレームの w バイト目が目的のアドレスである。このように二段のページテーブルを用いた page table walk を行うことで目的の物理アドレスに辿り着く。

- 一段目のページテーブル

一段目のページテーブルの大きさは、 p が 10 ビット、エントリサイズが 4 バイトより $2^{10} \times 4B = 4KiB$ となりフレームと同じである。そこで、どれか一つのフレームに一段目のページテーブルを格納することにする。ページテーブルレジスタはフレームの番号 (20 ビット) を記録すれば良い。

^{*8} 図 11.7 では IA-32 の用語ではなく、より一般的な用語を使用している。

^{*9} 初期の IA-32 の場合である。途中から 36 ビットに拡張された。

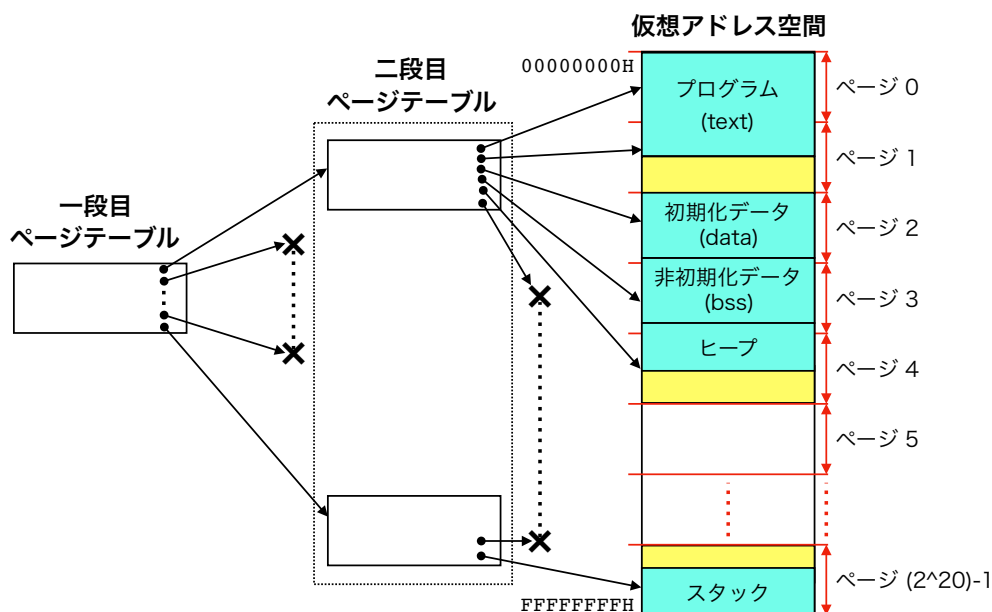


図 11.8: 穴空き仮想アドレス空間のページテーブル

・二段目のページテーブル

一段目のページテーブルの一つのエントリが、二段目のページテーブルの一つの区画を選択する。区画は 10 ビットの q を用いて参照されるので、大きさは一段目のページテーブルと同じ $4KiB$ になる。区画も一つのフレームに格納される。一段目のページテーブルの f フィールドには、二段目ページテーブルの一つの区画のフレーム番号 (20 ビット) を格納する。

・メモリの節約

図 11.3 や図 11.6 で示したように、プロセスの仮想アドレス空間には、フレームが割り付けられていない大きな穴が空いている。図 11.8 のように、穴の部分に二段目のページテーブルを割当てないことでメモリを節約できる。図の例では一段目と二段目合わせて 3 フレームしかページテーブルのために使用していない。もしも、二段目のページを全て割り付けたなら $2^{10} + 1 = 1,025$ フレームを使用するので効果は大きい。

11.4.2 多段ページテーブル

前の節では二段のページテーブルを紹介した。仮想アドレス空間が更に広い場合は、より段数の多いページテーブルが使用されることがある。例えば、x86-64 では 64 ビット (実質的には 48 ビット)*¹⁰ の広い仮想アドレス空間が使用できる。IA-32 では仮想アドレス空間が 32 ビットだったので 4GiB より大きなプロセス (セグメント) を作ることはできなかった。x86-64 ではより大きなプロセスを作ることができるので、4GiB の上限を気にしないでプログラミングできる。

しかし、二段のページテーブルを使用し続けると、ページテーブルの区画が大きくなりメモリの無駄

*¹⁰ 図 11.9 に示すように 64 ビットの上位 16 ビットは未使用なので、実質的な仮想アドレスは 48 ビットになる。 $2^{48}B = 2^8 \times 2^{40}B = 256TiB$ なので、48 ビットでも十分に広いアドレス空間である。

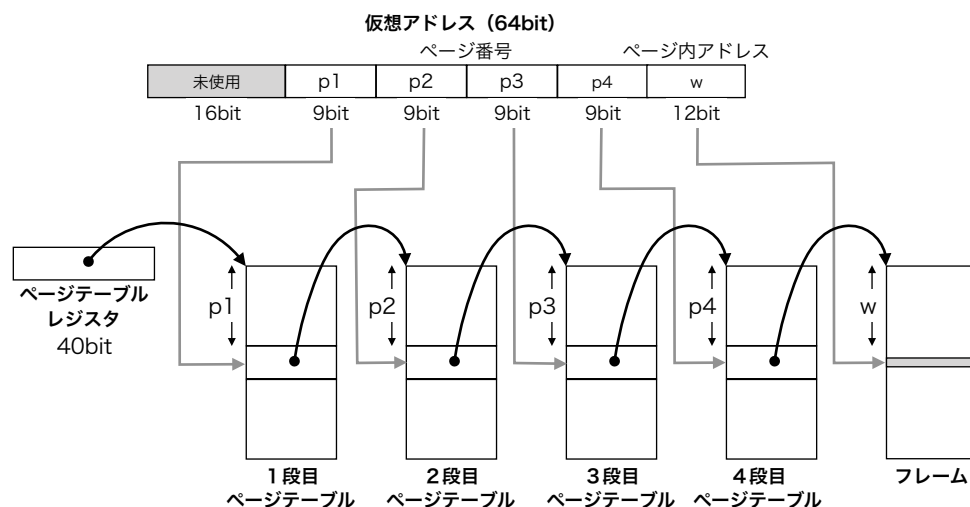


図 11.9: 四段のページテーブルの例

が多くなる。例えば、仮想アドレスが 48 ビット、二段のページテーブルを用い、エントリのサイズが 8 バイトと仮定する。48 ビットの仮想アドレスを、18 ビット (p)、18 ビット (q)、12 ビット (w) に分割して扱う場合、ページテーブル区画のサイズは $2^{18} \times 8\text{B} = 2\text{MiB}$ となる。図 11.8 と同様な考えで、最低限である 3 区画がページテーブルに割当てられたとするとプロセス当たり 6MiB になる。システム内にプロセスが 400 個^{*11}あったとすると、最低でも 2.4GiB のメモリがページテーブルのために消費されることになる。ページテーブルに使用されるメモリが多すぎる^{*12}。

ページテーブルの区画を小さくするために仮想アドレスをより小さく区切る。例えば、x86-64 では図 11.9 のように仮想アドレスのページ番号部分を四つに区切っている。ページ内アドレスが 12 ビットなので、ここでもページ（フレーム）サイズは 4KiB である。ページテーブルは 9 ビットの p1, p2, p3, p4 でインデクスされるので 512 エントリである。エントリサイズは x86-64 の場合 8 バイトなので、ページテーブルは 4KiB になりフレームサイズと同じになる。図 11.8 のように、最初と最後の数ページだけ使用している仮想アドレス空間をマッピングする場合、ページテーブルに使用するメモリは、1 段目に 1 フレーム、2 段目に 2 フレーム、3 段目に 2 フレーム、4 段目に 2 フレームの合計 7 フレーム（28KiB）で済む。プロセスが 400 個あったとしても、ページテーブルに使用するメモリの合計は約 11MiB にしかない^{*13}。

11.4.3 逆引きページテーブル

従来のページテーブルは、仮想アドレス空間の大きさにより大きくなるし、仮想アドレス空間の数だけ必要である。これは、仮想アドレスから物理アドレスに変換するために、仮想アドレス（ページ番号）をインデクスとし物理アドレス（フレーム番号）を内容とする表を、仮想アドレス空間毎に用いる自然な発想から生まれた。逆引きページテーブルは、従来のページテーブルとは逆に、物理アドレス（フレーム番号）をインデクスとし、仮想アドレス（ページ番号）を内容とする表をシステム全体で一つだ

*11 この原稿を書いている MacBook では、8GiB の物理メモリを搭載し、現在 354 個のプロセスが走っている。

*12 メモリが 8GiB と仮定すると、約 30% がページテーブルに消費されることになる。

*13 8GiB の約 0.13% しか使用しない。

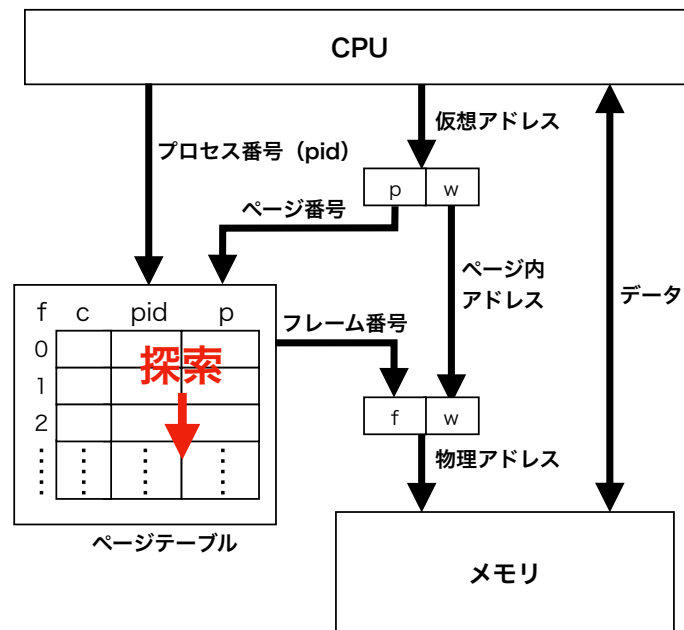


図 11.10: 逆引きページテーブルの概要

け用いる方式である。

- ページテーブルのサイズ

ページテーブルのエントリ数はフレーム数と等しくなるので、ページテーブルが使用する領域の大きさは、メモリ全体と比較して小さく、かつ、一定である。例えば 8GiB のメモリを管理するために、ページサイズが 4KiB なら次の計算のように 2Mi エントリである。エントリサイズが 8 バイトと仮定するとページテーブル全体で 16MiB となる^{*14}。

$$8GiB \div 4KiB = 2^{33} \div 2^{12} = 2^{21} = 2Mi \text{ エントリ}$$

- Page Table Walk

図 11.10 に逆引きページテーブルの模式図を示す。システムに一つだけの表なので、どのプロセスの仮想アドレス空間にフレームが割り付けられているか識別するために、プロセス番号 (pid) がページテーブルに格納されている。ページテーブルをプロセス番号 (pid) とページ番号 (p) で検索し、見つかったエントリのインデックス (f) をフレーム番号として出力する。

- ハッシュ表を用いた page table walk

ページテーブルを先頭から順に探索しては遅くて実用にならない。ハッシュ表を用いた検索を行う。図 11.11 に IBM 801 ミニコンピュータで用いられたメモリ管理方式 [46] を参考にした機構の模式図を示す。

チェーンのためのフィールド (next) を設け、ページテーブルをチェーンハッシュ表として扱う。ハッシュ表の大きさはハッシュ関数の作りやすさから二の累乗とする^{*15}。ハッシュ値はプロセス

^{*14} システム全体で 8GiB の 0.2% しか使用しない。

^{*15} ハッシュ表のサイズが二の累乗なら、ハッシュ値は計算結果の一部のビットを使用すれば良い。

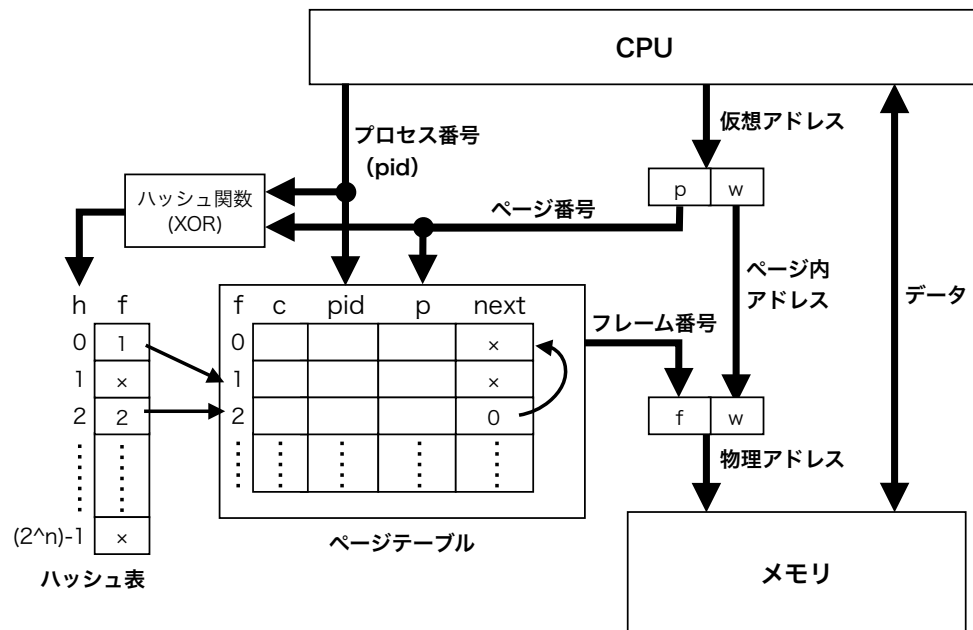


図 11.11: ハッシュを用いた逆引きページテーブルの構造

番号^{*16}とページ番号の XOR で計算する。ハッシュ表はページテーブルの一つのエントリのインデックスを格納する。エントリのプロセス番号 (pid) とページ番号 (p) が目的のものであれば、エントリの番号 (f) がフレーム番号として使用される。目的のものでない場合はチェーン (next) を使用して次のエントリに進む。チェーンの最後まで調べて見つからなければページ不在である。

- TLB

ハッシュ表やページテーブルはメモリに配置する。メモリアクセスの度に page table walk を行っている、メモリアクセスに時間がかかりすぎる。普段は TLB を用いる。

11.5 まとめ

この章ではページングについて学んだ。仮想アドレス空間のページを物理アドレス空間のフレームにマッピングする。マッピング関数はページテーブルによって実装される。プロセス毎にマッピング関数を準備することで、プロセス毎に独立した仮想アドレス空間を持つことができる (多重仮想記憶)。

全てのフレームは等価なので、メモリの割当て状態によって使用できないフレームが発生することはなく、外部フラグメンテーション問題は解決された。しかし、フレーム内に使用できない領域が発生する。内部フラグメンテーション問題は解決されない。

ページングのハードウェアは MMU に内蔵される。ページテーブルはメモリ上に配置し、そのアドレスをページテーブルレジスタが記憶する。ページテーブルエントリには、フレームが割当てられていることを表すビットやフレーム番号が格納される。フレームが割当てられないページを参照するとページ不在割込み (page fault) が発生する。これを積極的に使用することで仮想記憶が実現できる。

^{*16} IBM 801 ではセグメント ID であった。

ページ番号をフレーム番号に変換するために、ページテーブルを調べることを page table walk と言う。page table walk には手間がかかるので、変換結果は MMU 内の TLB と呼ばれる**連想メモリ**にキャッシュする。TLB に変換結果が見つからない場合を TLB miss と呼び、page table walk は TLB miss のときだけ行う。なお、page table walk は MMU のハードウェアが自動的に行う場合と、ソフトウェアにより行う場合がある。

ページテーブルのサイズは、かなり大きくなる。そこで、ページテーブルを小さくする工夫がされる。多段のページテーブルを用いる方法と、逆引きページテーブルを用いる方法を紹介した。

練習問題

11.1 次の言葉の意味を説明しなさい.

- (a) ページ
- (b) フレーム
- (c) 外部フラグメンテーション
- (d) 内部フラグメンテーション
- (e) ページテーブル
- (f) ページテーブルレジスタ
- (g) TLB
- (h) page table walk
- (i) TLB miss
- (j) ページ不在割込み (page fault)
- (k) 位置独立コード
- (l) 多段ページテーブル
- (m) 逆引きページテーブル

11.2 一回のメモリアクセス時間に 5ns, page table walk に 20ns かかるとする, TLB のヒット率が 50%, 90%, 95% の時の平均メモリアクセス時間を計算しなさい.

11.3 図 11.7 において, $p = 1$ の仮想アドレスの範囲を 8 桁の 16 進数で答えなさい.

11.4 図 11.7 において, $p = 1$, $q = 1$ の仮想アドレスの範囲を 8 桁の 16 進数で答えなさい.

11.5 逆引きページテーブルを用いる場合, TLB に格納すべき最低限の情報の範囲を考察しなさい.

11.6 図 11.11 に, $pid = 3$, $p = 2$ のページがフレーム 1 にマッピングされるページテーブルの状態を書き込みなさい.

11.7 逆引きページテーブルを用いるシステムで, プロセス間でページの共有が可能か考察しなさい.

第 12 章

仮想記憶

仮想記憶は、物理メモリよりも多くのメモリをプロセスが使用できるようにする。仮想記憶を実現するために使用できるメモリ管理機構として、第 10 章で学んだセグメンテーションと、第 11 章で学んだページングがある。既に第 10 章で、セグメンテーション機構による仮想記憶は簡単に説明した。しかし、セグメンテーションには、物理メモリより大きなセグメントを使用できない問題があった。

ページングを用いる方がメリットが多いので、現代のオペレーティングシステムはページングによる仮想記憶を使用している。この章ではページングに基づく仮想記憶方式について学ぶ。

12.1 基本概念

ページングでは、ページテーブルの V ビットを使用して仮想アドレス空間にフレームが割り付けられていない状態を表現していた。V ビットが 0 のページにアクセスするとページ不在割込み (*page fault*) が発生し、制御がオペレーティングシステムに移る。V ビットが 0 の状態を上手く使うことで、メモリより大きなプログラムでも実行できる仕組みを作る。

図 12.1 に示すように、ページの内容はフレームかディスク（バッキングストア）に格納する。ページの内容がフレームに置かれている時はページテーブルの対応するエントリの V ビットを 1 ($V=1$) にする。フレームに置かれていない時は $V=0$ とする。 $V=0$ のページにアクセスするとページ不在割込みが発生する。ページ不在の理由と対処方法は、例えば以下のようにまとめることができる。

1. 仮想アドレス空間の無効領域をアクセスし本当にエラーを起こした。
→ プロセスを終了する。
2. バッキングストアに内容が退避されているページにアクセスした。
→ フレームを割当て内容をディスクから復旧した後、プロセスを再開する。

プロセス生成時にバッキングストアに仮想アドレス空間のイメージを作成する。フレームは、まだ割当てない。プログラムが動作を開始するとページ不在割込みが発生し、その都度、必要なページをバッキングストアから読み込む。

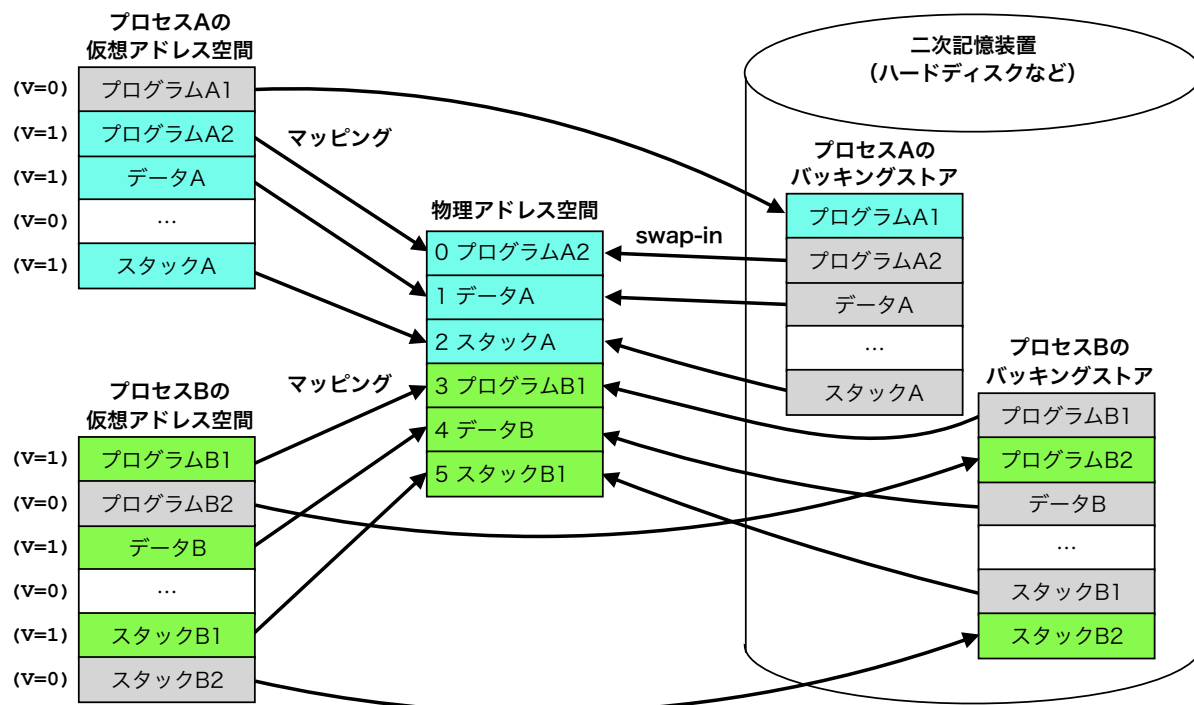


図 12.1: 仮想記憶の基本

12.2 デマンドページング

ページングによる仮想記憶を用いると、プロセス実行開始時にプログラム全体をメモリに読み込まなくても良い。原理上は、一ページも読み込まなくても実行を開始することができる。プログラムの最初の命令をフェッチする時点でページ不在割込みが発生し、オペレーティングシステムによりページが読み込まれる。このような、プログラムがアクセスした時点でページを読み込む方式を**デマンドページング (Demand Paging)**と呼ぶ。使用しないページをメモリに読み込むことがない点で無駄がない。現代の多くのオペレーティングシステムは、デマンドページング^{*1}をページ読み込み方式として採用している。

12.2.1 デマンドページングの手順

デマンドページングの手順を以下にまとめる。

1. プロセスが V=0 のページをアクセスする。
2. ページ不在割込みが発生しオペレーティングシステムに制御が移る。
3. オペレーティングシステムはプロセスがアクセスしたアドレスが正当なアドレスか調べる。
不正なアドレスならプロセスを終了させる。(処理終わり)
4. 空きフレームを捜す。

空きフレームが無い場合は何れかのフレームを選択しバックイングストアに書き出し (swap-out) 空

^{*1} デマンドページングの改良版も含む。

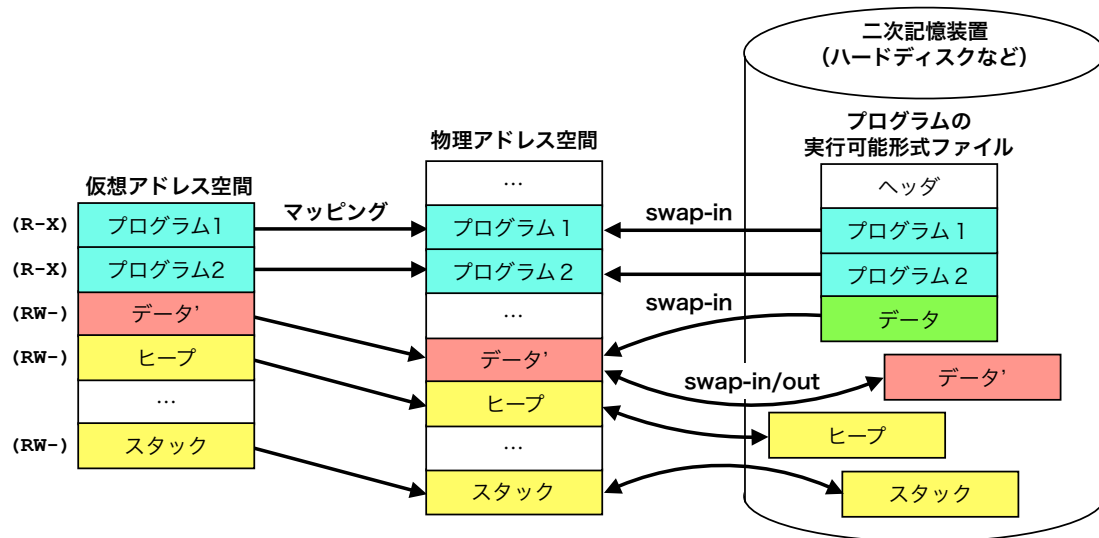


図 12.2: デマンドページングとプログラム実行

きフレームを作る。

5. 空きフレームをプロセスに割当てバックングストアからページの内容を読み込む (swap-in)。
6. ページテーブル等を新しい状態に更新する。
7. ページ不在割込みを発生した命令の再実行からプロセスを再開する。

12.2.2 プログラムファイルの直接 swap-in による実行

プログラムの機械語部分は変更されないので、swap-out 用のバックングストアを準備する必要はない。プログラムはデマンドページング方式で実行形式ファイルから直接 swap-in する。バックングストアに予めプログラムをコピーしたり、プログラムを予めフレームに読み込んだりしないので、プログラムの起動を素早く行うことができる。このアイデアを図 12.2 を用いて説明する。

1. 実行可能形式ファイルの構造

プログラムはデマンドページング用の実行可能形式ファイルに格納される。このファイルではデマンドページングで使用しやすいように、ページサイズの整数倍の境界からセグメントが配置されている。

- ヘッダはファイルがデマンドページング用の実行形式ファイルであることを示すマジックナンバーで始まり、ファイル内のセグメントの大きさなどを記述している。ヘッダのサイズはページサイズと同じである。
- ヘッダの次に、機械語プログラムを格納したセグメントが配置される。セグメントの大きさはページサイズの整数倍である。図 12.2 はプログラムのサイズが 2 ページの場合である。
- プログラムの次に初期化データが配置される。初期化データは初期値を明示したグローバル変数を集めた領域である。

2. プログラムの読み込み

仮想アドレス空間の「プログラム 1」ページがアクセスされページ不在割込みが発生する。オペ

レーティングシステムがフレームを割り付け、実行可能形式ファイルから「プログラム 1」領域を swap-in する。プログラム領域は読み出し実行 (R-X) だけが許可され、値が書き換えられることはない。

3. 初期化データの読み込み

仮想アドレス空間の「データ」ページがアクセスされ、ページ不在割込みが発生する。オペレーティングシステムがフレームを割り付け、実行可能形式ファイルから「データ」領域を swap-in する。データ領域は読み書き (RW-) が許可されるので値が変化する。

4. 非初期化データとヒープ領域の割り付け

非初期化データ領域とヒープ領域は連続しているので、ここではヒープ領域としてまとめて説明する。仮想アドレス空間の「ヒープ」ページがアクセスされページ不在割込みが発生する。オペレーティングシステムがフレームを割り付け内容をゼロでクリアする^{*2}。ヒープ領域は読み書き (RW-) が許可されるので値が変化する。

5. スタック領域の割り付け

仮想アドレス空間の「スタック」ページがアクセスされ、ページ不在割込みが発生する。オペレーティングシステムがフレームを割り付け、内容をゼロでクリアする^{*3}。スタック領域は読み書き (RW-) が許可されるので値が変化する。

12.2.3 プログラムの swap-out

プログラムを実行するに従い、デマンドページングにより、新しいページが次々とフレームに読み込まれる。他のプロセスも同じように振る舞うので、やがてフレームが枯渇する。使用頻度の低いフレームを解放し、再利用できるようにする必要がある。フレームを解放する際に内容を swap-out する場合がある。以下では実行中のプロセスの、各領域のフレームを解放する手順を簡単に述べる。

1. プログラム領域

機械語プログラムは読み出し実行 (R-X) だけ許可されたページに格納されるので、swap-in されてから書き換わることはない。再度ページが必要になった時は、実行可能形式ファイルから読み込めば良いので、解放するフレームの内容を swap-out する必要はない。バッキングストア領域と I/O トラフィックを小さくすることができる。

2. 初期化データ領域

初期化データ領域は、初期値が格納された状態で swap-in される。読み書き可能 (RW-) なのでプログラム実行中に書き換わる可能性がある。ページテーブルの D (Dirty) ビットが 0 の場合は読み込み時点から変更されていないので swap-out する必要はない。必要になったとき実行可能形式ファイルから読み出し直せば良い。ページテーブルの D (Dirty) ビットが 1 の場合は、フレームを再利用する前にバッキングストアに swap-out し、次回必要になった時に復元できるようにする。

3. 非初期化データ・ヒープ・スタック

これらの領域は、ゼロで初期化されてから使用が開始される。読み書き可能 (RW-) なのでプログ

^{*2} C 言語等の非初期化グローバル変数の初期値がゼロだと保証される。

^{*3} 以前フレームを使用したプロセスの機密情報が漏洩しないようにクリアする。

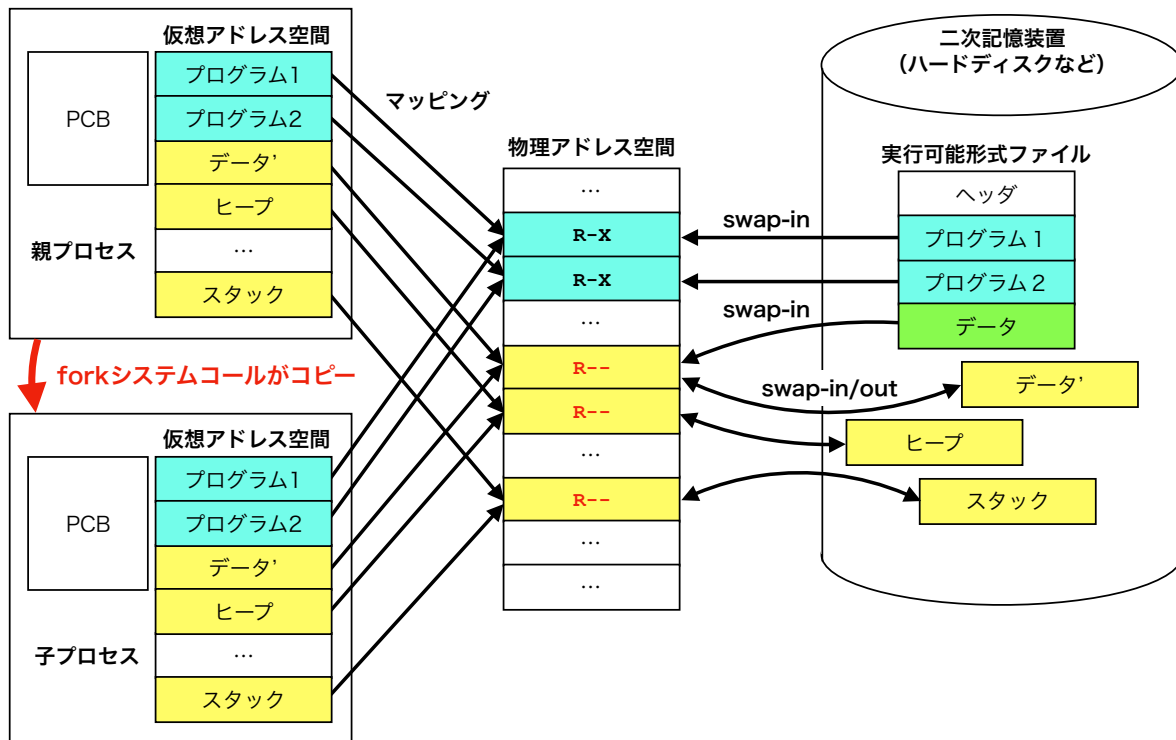


図 12.3: fork 直後の親子プロセス

ラム実行中に書き換わる可能性がある。ページテーブルの D (Dirty) ビットが 0 の場合は初期化時点（または、swap-in 時点）から変更されていないので、何もしないでフレームを解放しても良い。ページテーブルの D (Dirty) ビットが 1 の場合は、フレームを再利用する前にバッキングストアに swap-out し、次回必要になった時に復元できるようにする。

12.3 Copy on Write

UNIX の fork システムコールはプロセスのコピー（子プロセス）を作る。多くの場合、子プロセスはすぐに execve システムコールを発行し新しいプログラムの実行を開始するので、せっかくコピーした仮想アドレス空間はあまり活用されないまま廃棄される。これでは効率が悪いのでアドレス空間を親子で共有する vfork システムコールが提供されるようになった。vfork システムコールを用いる場合、子プロセスが execve するまで親プロセスは待ち状態になり、共有したアドレス空間を破壊し合わない工夫がされた。その後、Copy on Write と呼ばれるアドレス空間のコピーを遅らせる技術が用いられるようになり、fork システムコールを用いても無駄なメモリコピーが起こらなくなった。

fork 直後の様子 Copy on Write を用いる場合、図 12.3 に示すように fork 直後は親子プロセスでフレームを共有する。この時点ではメモリのコピーはしない。その代わりに、書き込み可能であるはずのデータ、ヒープ、スタック領域のメモリ保護を読み出し専用 (R--) に設定する。

Copy on Write の手順 どちらかのプロセスがスタックを書き換えようとした場合を例に、Copy on

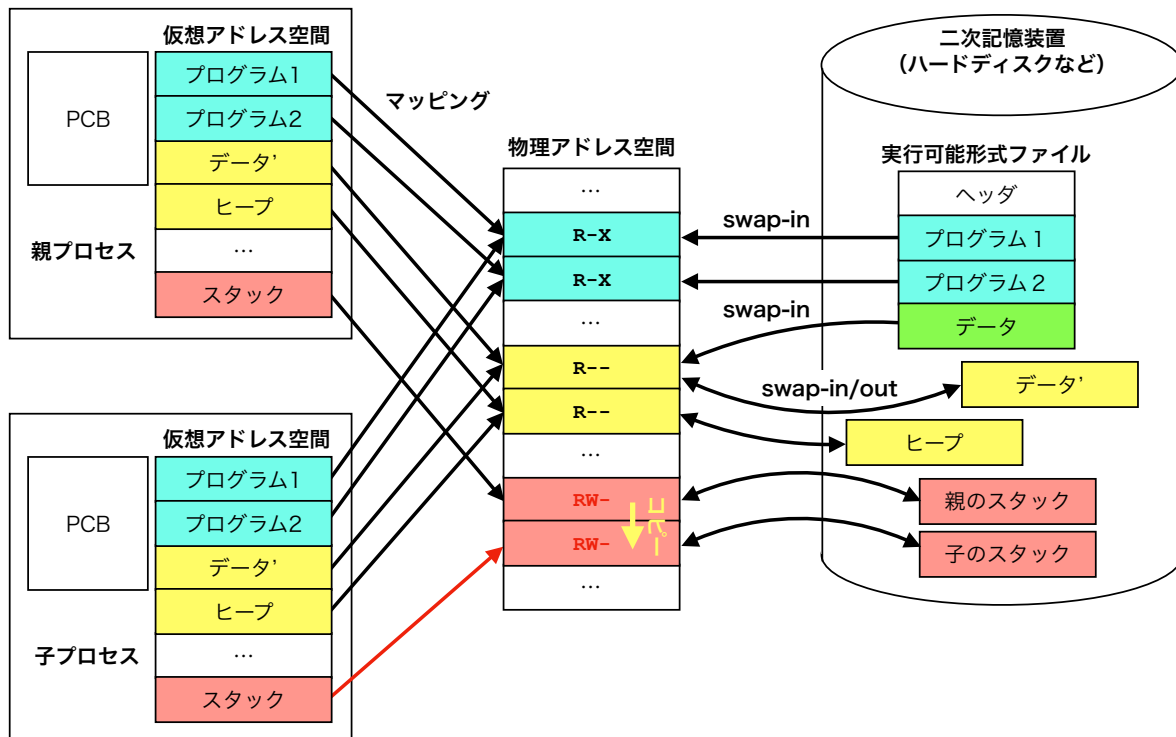


図 12.4: スタックで Copy on Write が発生した時の親子プロセス

Write が働く手順を説明する。プロセスがスタック領域を書き換えようとするときメモリ保護違反割込みが発生しオペレーティングシステムに切り換わる。その後、オペレーティングシステムが次に述べる操作を行い、図 12.4 に示す状態になる。

1. 新しいフレームを割当て、スタック領域フレームの内容をコピーする。
2. 片方のプロセスのスタック領域に新しいフレームをマッピングする。もう一方のプロセスのスタック領域は古いフレームをマッピングしたままにする。
3. 両プロセスのスタック領域の保護情報を読み書き (RW-) に変更する。
4. 割込みを発生した命令の再実行からプロセスを再開する。

以上のように書き込みが起こった時点でメモリがコピーされるので Copy on Write と呼ばれる。

12.4 メモリマップドファイル

仮想記憶機構を用いてファイルを読み書きする手段を提供する。仮想アドレス空間にファイルをマッピングすることで、ユーザプログラムがメモリ（配列）を操作する手順でファイルを読み書きできる。ファイル操作の度にシステムコールを発行しない軽いファイル操作手段である。また、複数のプロセスが同じファイルをマッピングすることで、プロセス間の広帯域のデータ共有手段にもなる。

12.4.1 UNIX のメモリマップドファイル

メモリマップドファイルの使用例を示す。UNIX では `mmap` システムコール^{*4} を用いて仮想アドレス空間とファイルに関連付ける。ファイルを仮想アドレス空間上の配列としてアクセスすることができる。以下に、`mmap` システムコールのプロトタイプ宣言と簡単な解説を掲載する。

```
void * mmap(void *addr, size_t len, int prot, int flags, int fd, off_t offset);
```

戻り値： マップされた領域の先頭アドレスが返される。アドレスはページサイズの倍数になる。

addr： マップしたい仮想アドレス空間の先頭アドレスを渡す。

len： マップする領域の大きさを渡す。大きさはページサイズの倍数にする。

prot： 保護モード (protection) を表す値を渡す。ページの保護モード (RWX) が決まる。

flags： ファイルをマップする (`MAP_FILE`) かファイルに関係づけたい名無しメモリをマップする (`MAP_ANON`) か、変更をプロセス間で共用する (`MAP_SHARED`) かプロセスにプライベートにする (`MAP_PRIVATE`) か等を表すフラグを渡す。

fd： オープン済みファイルのファイルディスクリプタを渡す。

offset： ファイルの `offset` バイトから始まる `len` バイトをマッピングする。offset はページサイズの倍数にする。

リスト 12.1 にファイルの内容を配列のように書き換えるプログラムの例を掲載する。このプログラムを実行すると予め作成しておいた `a.txt` ファイルの最初の 4KiB が英大文字で上書きされる。

8 行 予め作成しておいた 4KiB のファイルを開く。プロセスがメモリマップを通してファイルに読み書き両方ができるためには、`open` システムコールのフラグに `O_RDWR` を渡す必要がある。

13 行 仮想アドレス空間に 8 行でオープンしたファイルをマッピングする。マッピングするアドレスの決定をカーネルに任せるので第 1 引数は `NULL` にする。ファイルをマッピングし書き込んだ内容を反映するために、`MAP_FILE` フラグと `MAP_SHARED` フラグを指定する。

18 行 `mmap` システムコールが完了したらファイルはクローズして構わない。

19~21 行 `mmap` が返した領域を文字型の配列と見做して文字を書き込む。値をファイルに反映するために特別な操作をする必要はない。

12.4.2 メモリマップドファイルの仕組み

図 12.5 に、二つのプロセスの仮想アドレス空間に同じファイルの同じ部分をマップした例を示す。UNIX の `mmap` システムコールで `MAP_FILE` フラグと `MAP_SHARED` フラグを使用した場合に相当する。この例では、ファイルは読み書きの両方ができるようにマップされている。二つのプロセスは同じフレームを共用し、共有メモリを持った状態でもある。

図 12.5 は、ファイルの内容がフレームに読み込まれた状態を表している。しかし、`mmap` システムコール実行直後は、図とは異なり、フレームが割当てられていない。`mmap` はページとファイルに関連付けるが、実際にファイルを読み書きするのは仮想記憶の仕組みによる。以下に、メモリマップドファ

^{*4} Windows では `CreateFileMapping()` 関数が利用できる。

リスト 12.1: メモリマップドファイルの使用例

```

1 #include <stdio.h>                // perrorのために必要
2 #include <fcntl.h>                // open のために必要
3 #include <unistd.h>               // close のために必要
4 #include <sys/mman.h>             // mmap のために必要
5 int main() {
6     int fd;
7     char *p, *fname="a.txt";
8     fd = open(fname, O_RDWR);      // 予め作成してある4KiBのファイルを開く
9     if (fd<0) {
10         perror(fname);
11         return 1;
12     }
13     p = mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_FILE|MAP_SHARED, fd, 0);
14     if (p==MAP_FAILED) {
15         perror("mmap");
16         return 1;
17     }
18     close(fd);                    // マップしたらクローズして良い
19     for (int i=0; i<4096; i++) {  // ファイルにA~Zを繰り返し書き込む
20         p[i] = 'A' + (i % 26);
21     }
22     return 0;
23 }

```

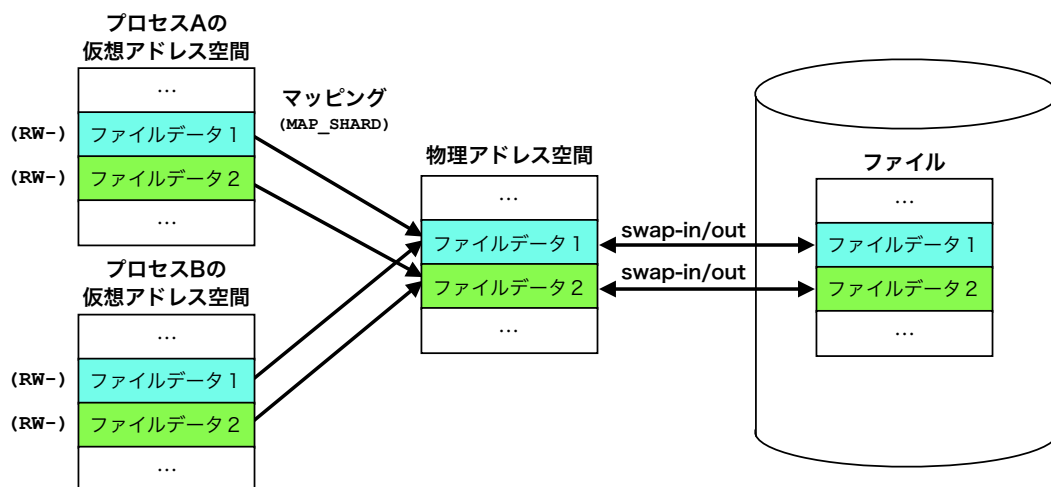


図 12.5: プロセス間で共有したメモリマップドファイル

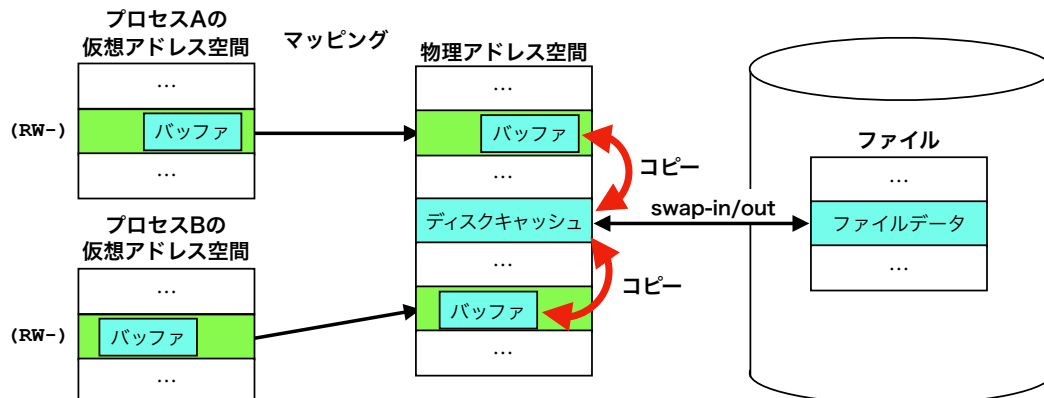


図 12.6: read/write システムコールのデータコピー

イルが読み書きされる仕組みを説明する。

1. ファイルの読み込み

マップされたアドレスをプロセスがアクセスした時点で、ファイルの該当箇所がデマンドページングの要領でフレームに読み込まれる。

2. ファイルの書き込み

定期的に変更のあった（Dirty）ページをファイルに書き戻す。また、プロセスが終了したりマッピングが解除された時も Dirty なページをファイルに書き戻す。

12.4.3 read/write システムコールとの比較

メモリマップドファイルの場合、フレーム上のデータが参照されたり変更される度にファイルの読み書きが起こるわけではなく、効率の良いファイルの参照が可能である。

一方で read/write システムコールの場合は、ディスクキャッシュを用いて二次記憶装置のアクセス回数を少なくする工夫がされる。しかし、read/write システムコールの引数として渡されたバッファとディスクキャッシュの間でメモリコピーをする必要がある。図 12.6 に、read/write システムコールを使用する場合のデータの流れを模式的に示す。メモリコピーは一般に重い処理である。

また、read/write の場合はデータの読み書きの度にシステムコールを発行する。一方でメモリマップドファイルの場合、mmap システムコールを用いてマッピングを完了してしまえば、システムコールを発行する必要がない。システムコールも一般に重い処理である。

12.4.4 プロセスにローカルなマッピング

図 12.7 に、二つのプロセスの仮想アドレス空間に同じファイルの同じ部分をローカルにマップした例を示す。UNIX の mmap システムコールで MAP_PRIVATE フラグを使用した場合に相当する。

- 「ファイルデータ 1」領域

プロセスの仮想アドレス空間にマップされ、かつ、プロセスに参照された。参照された時点でフレームに読み込まれプロセスから見える状態になっている。

- 「ファイルデータ 2」領域

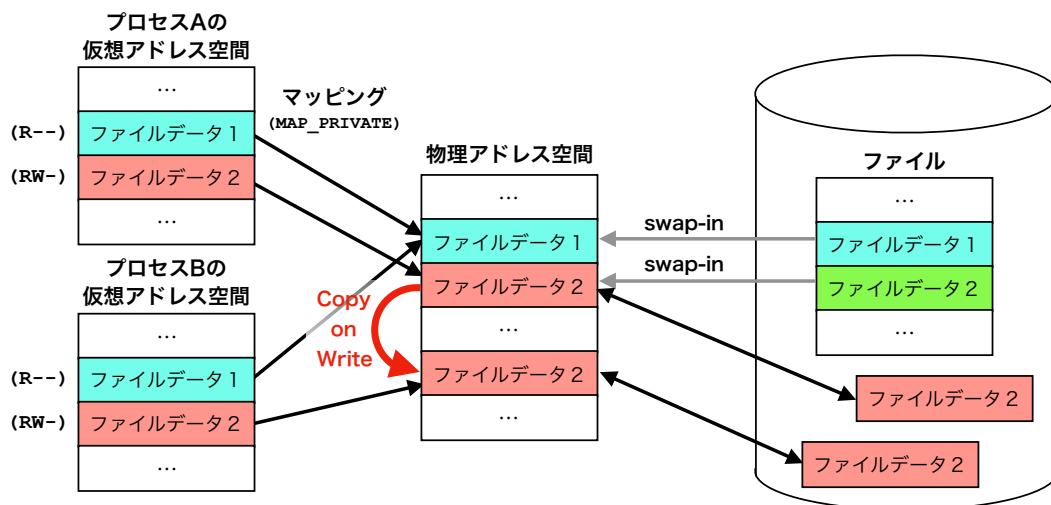


図 12.7: プロセスにローカルなメモリマップドファイル

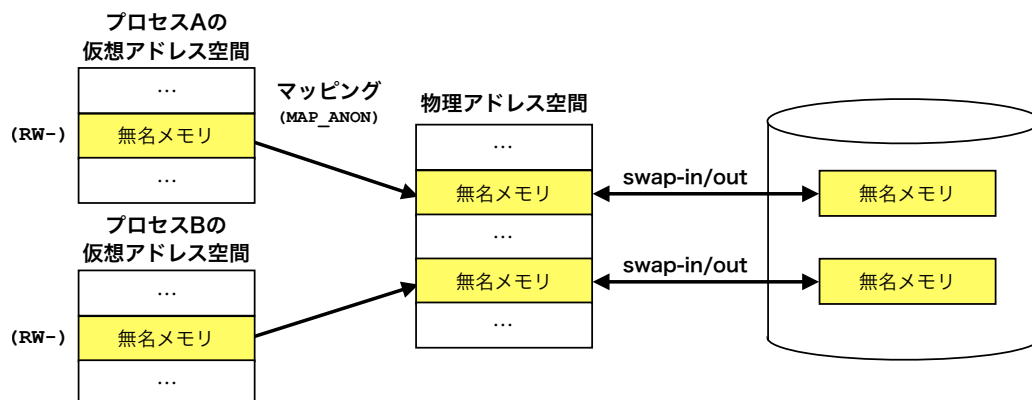


図 12.8: 無名メモリのマッピング例

一旦、「ファイルデータ 1」のように参照されフレームに読み込まれた。その後、プロセスが値を書き換えた。MAP_PRIVATE の場合は他のプロセスやファイルに変更が反映されない。Copy on Write 方式でコピーが作られ、プロセス毎に別のコピーを参照するようにマッピングする。

12.4.5 無名メモリのマッピング

図 12.8 に、無名メモリのマッピング例を示す。無名メモリはファイルと関連付けられないが、最初に内容がファイルからロードされるのではなくゼロでクリアされることを除いて・メモリマップドファイルのプロセスにローカルなマッピングと同様な管理を受ける。UNIX の mmap システムコールでは MAP_ANON フラグを使用して無名メモリを割り付ける。

12.4.6 プログラムの実行とメモリマップドファイル

図 12.2 で見たデマンドページングによるプログラムの実行は、以下のようにメモリマップドファイルを用いると実現できる。BSD UNIX では実際にメモリマップドファイルと同じ仕組みを利用して

いる [47].

- 実行形式ファイルをメモリにマッピングする.
 - プログラムは, R-X でマッピングする^{*5}*6. (プログラムはプロセス間で共用される)
 - 初期化データは, RW-, MAP_PRIVATE でマッピングする. 書き込みが起きた時点でバッキングストアと結びつける.
- 非初期化データ, ヒープ, スタックには, 無名メモリ (RW-, MAP_ANON) を割当てる.

12.5 ページ置き換えアルゴリズム

ページングによる仮想記憶では, 以下の三つの重要なアルゴリズムを定める必要がある.

1. **ページ読み込みアルゴリズム**: いつページを swap-in するか決める. 12.2 で既に学んだデマンドページングを用いる.
2. **ページ置き換えアルゴリズム**: フレーム不足時に, どのページを再利用するか決める. 本節で学ぶ.
3. **フレーム割り付けアルゴリズム**: どのフレームを使用するか決める. 12.6 で学ぶ.

デマンドページングによりページが読み込まれるにつれ, システムの空きフレームが少なくなっていく. 大量のメモリを使用するプロセスや, 同時に多数のプロセスが実行される状況では, やがて空きフレームが枯渇してしまう.

更にページを読み込む必要が生じた時, **どれかのプロセスのどれかのページ**を再利用する. 再利用することに決めたページは, ロードされた後で内容が変更されていればバッキングストアに書き出 (swap-out) し, フレームをプロセスから取り上げる. このフレームを再利用することで実行を継続する. **ページ置き換えアルゴリズム**は, どのプロセスの, どのページを解放するか決めるアルゴリズムである. 将来, 使用されないフレームをうまく選択しないと, swap-out したページが直後に swap-in されることになり, システムの性能が著しく低下する.

12.5.1 局所性・ワーキングセット・フェーズ化

プログラムの実行中, 全てのページが均等にアクセスされ続けることは稀である. 普通は一部のページにアクセスが集中し, また, アクセスが集中するページは時刻によって変化していく. その様子を図 12.9 に示す.

局所性 短い時間に着目すると, 一部の連続したページが集中的にアクセスされる. これを**空間的局所性**と呼ぶ. また, あるページに着目すると一部の連続した時刻にアクセスが集中している. これを**時間的局所性**と呼ぶ.

ワーキングセット プログラム実行中のある時間にアクセスされるページの集合を, その時間の**ワーキングセット**と呼ぶ. 同時に実行するプロセスを増やすとワーキングセットが大きくなる. ワーキングセットが利用可能なフレームの集合より大きくなる (ワーキングセットがメモリに入り切らなくなる) とページ不在が多発する. swap-in/out が繰り返され多くのプロセスがディスク I/O 待ちに

^{*5} R-X なら MAP_SHARED でも MAP_PRIVATE でも同じ.

^{*6} BSD UNIX では, デバッガがブレークポイントを設定できるように, RWX, MAP_PRIVATE でマッピングしている [47].

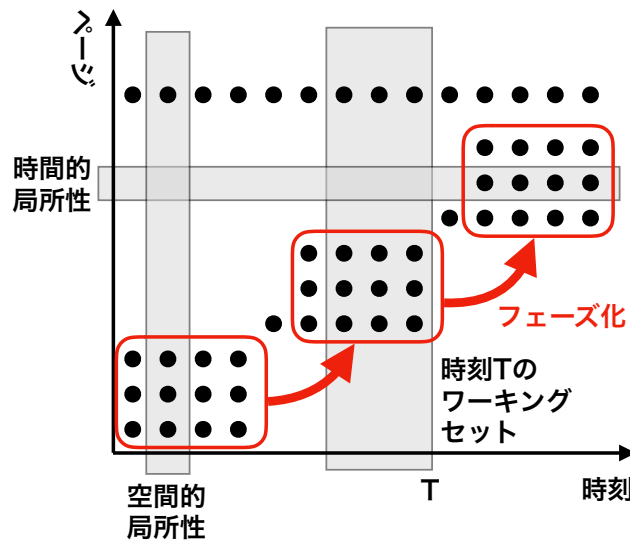


図 12.9: 局所性・ワーキングセット・フェーズ化

なり、システムの性能が急激に低下する。この状態を**スラッシング**と呼ぶ。

フェーズ化現象 プログラム実行中、時期によりワーキングセットが急激に変化する現象を**フェーズ化現象**と呼ぶ。例えば、まず、プログラムはデータを入力する。この時点では入力処理を含むページがワーキングセットになる。次に、プログラムは入力したデータを使用して計算を行う。この時点では計算処理を含むページがワーキングセットになる。最後に、プログラムは計算結果を出力する。この時点では出力処理を含むページがワーキングセットになる。フェーズが遷移する時は局所性が失われ、ページ不在が集中的に発生する。

ページ置き換えアルゴリズムは、プログラムのこれら性質に着目した様々な方式が提案されている。

12.5.2 LRU (Least Recently Used) アルゴリズム

「最近アクセスされていないページは、この先もアクセスされる可能性が低い」との仮定に基づく方式である。時間的局所性をプログラムが持っているなら最良の方式である。しかし、現実的な実装が困難とされている。もしも実装するとすると図 12.10 のようなハードウェアが必要になる。CPU はメモリアクセス毎に値がインクリメントされる十分に長いカウンタ^{*7}を備える。ページテーブルにはカウンタの値を保存できる「最終アクセス時刻」フィールドが追加されている。

1. メモリアクセス毎に、アクセスしたページのページテーブルエントリにカウンタの値を書き込む。
2. ページ不在が発生し空きフレームが無いなら、ページテーブルをスキャンし最も古いページを見つける。
3. 見つけたページを swap-out し、代わりに目的のページを swap-in する。

この方式の問題は、ハードウェアのコストと、ページ不在時の処理の重さである。ページ不在は頻繁

^{*7} 64bit のカウンタなら毎秒 1Gi 回のメモリアクセスがあったとしたとしても、500 年以上オーバーフローしない。

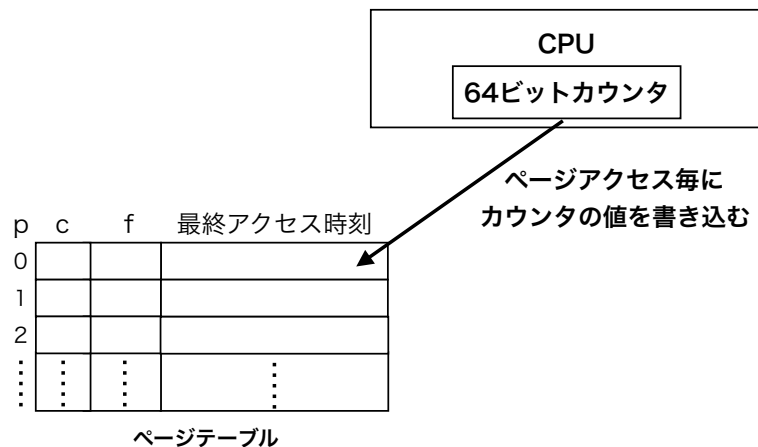


図 12.10: LRU 方式のためのハードウェア

に発生する^{*8}ので、その度にページテーブル全体をスキャンすることは現実的ではない。

12.5.3 LFU (Least Frequently Used) アルゴリズム

LRU の近似方式の一種であり、ページテーブルの R ビット (表 11.1 参照) と、フレーム毎のカウントだけを用いてソフトウェアで実現できる。NFU (Not Frequently Used) と呼ばれる。次のようなアルゴリズムである。

1. ページテーブルの R ビットとフレームのカウンタをゼロにクリアする。
2. 定期的 (例えば TICK=20ms 毎) にページテーブルをスキャンする。R=1 のエントリを見つけたら対応するフレームのカウンタをインクリメントし、R をゼロにクリアする。
3. ページ不在時にフレームが不足したなら、カウンタの値が最小のフレームを置き換える。

この方式の問題点は、ページ不在時にページテーブルのスキャンが必要なことと、一度カウンタの値が大きくなったフレームは使用されなくなっても値が大きいままで、置き換えられ難いことである。この問題を解決するために、定期的にページテーブルをスキャンする際のカウンタの更新方法を次のように改良したエージングアルゴリズムが提案された。この改良により、過去の R ビットの影響が徐々に小さくなる。

R=1 のフレーム $cnt \leftarrow cnt \div 2 + 0x8000$ (カウンタは 16bit と仮定)

R=0 のフレーム $cnt \leftarrow cnt \div 2$

12.5.4 FIFO (First-In First-Out) アルゴリズム

「長くメモリに滞在しているページは役割を終えている」との仮定に基づく。特別なハードウェアを用いることなく、ソフトウェアだけで実現できる。図 12.11 に示すリストを用いるアルゴリズムである。

1. swap-in する度にフレームをリストの最後 (図の右側) に追加していく。
2. ページ不在時にフレームが不足したなら、リストの先頭のフレームを置き換える。

^{*8} macOS の `vm_stat` コマンドを用いると、毎秒数千回のページ不在が発生する様子を見ることができる。

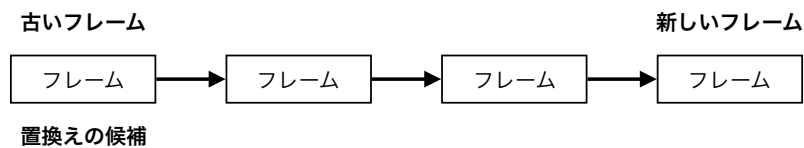


図 12.11: FIFO アルゴリズムが用いるリスト

Belady の異常な振る舞いの例

FIFO アルゴリズムを用い、ページ参照ストリング (W : 1 2 3 4 1 2 5 1 2 3 4 5) の場合

- フレーム数 (m=3) の場合 (ページ不在 9 回)

W	1	2	3	4	1	2	5	1	2	3	4	5
	*1	*2	*3	*4	*1	*2	*5	5	5	*3	*4	4
S		1	2	3	4	1	2	2	2	5	3	3
			1	2	3	4	1	1	1	2	5	5

- フレーム数 (m=4) の場合 (ページ不在 10 回)

W	1	2	3	4	1	2	5	1	2	3	4	5
	*1	*2	*3	*4	4	4	*5	*1	*2	*3	*4	*5
S		1	2	3	3	3	4	5	1	2	3	4
			1	2	2	2	3	4	5	1	2	3
				1	1	1	2	3	4	5	1	2

メモリが多い方 (m=4) のページ不在回数が多い。

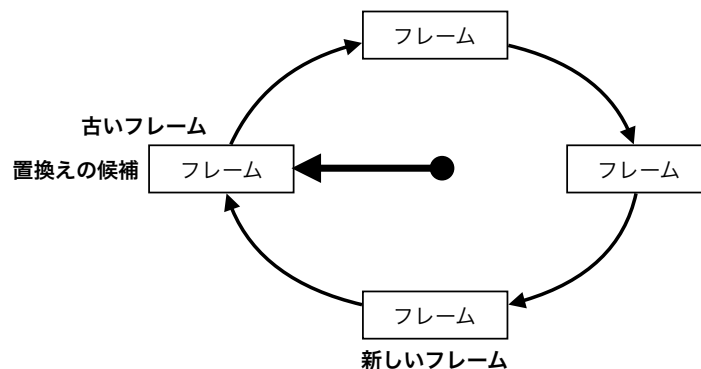


図 12.12: Clock アルゴリズムが用いる環状リスト

このアルゴリズムはページテーブルのスキャンが不要なので非常に軽い。しかし、常時使用される重要なページも時間が経過すると swap-out される問題がある。また、Belady の異常な振る舞いをすることがある。Belady の異常な振る舞いとは、メモリが多い場合の方がページ不在の回数が増える現象である。

12.5.5 Clock アルゴリズム

図 12.12 に示す FIFO のリストを環状にしたデータ構造を用いる。データ構造に加えてページテーブルの R ビットも使用する。次のようなアルゴリズムである。

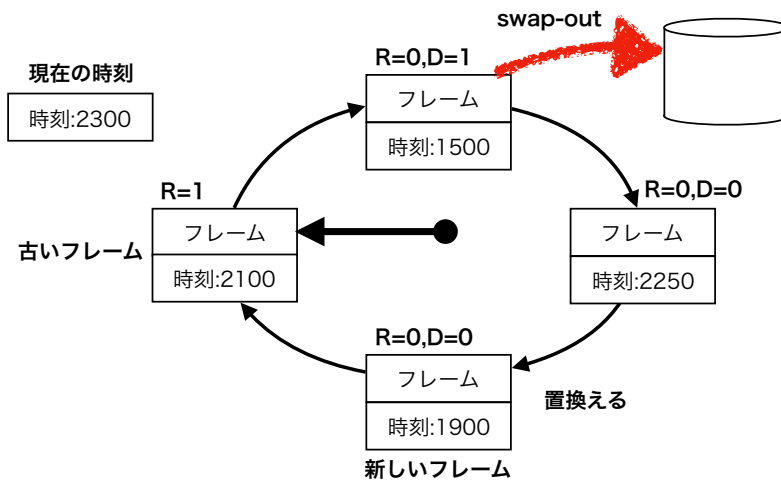


図 12.13: WSClock アルゴリズムが用いる環状リスト

1. swap-in する度にフレームを環状リストに挿入していく。挿入位置は最も古いフレームの一つ手前である。最も古いフレームは時計の針に当たるポインタが指している。
2. 定期的（例えば TICK=20ms 毎）に全ページテーブルエントリの R ビットをゼロにクリアする。
3. ページ不在時にフレームが不足したなら、時計の針が指しているフレームのページテーブルエントリの R ビットを調べる。

R=0 の場合 ページは古く、かつ、最近アクセスされていないので置き換える。

R=1 の場合 ページは古いが最近アクセスされている。R ビットをクリアして時計の針を一つ進め、次のフレームについて同じ処理を行う。

最悪でも時計の針が一周回ると R=0 のページが見つかる。

12.5.6 WSClock アルゴリズム

ワーキングセットを考慮した Clock アルゴリズムである。単純でパフォーマンスが良いので、広く使用されている [48]。図 12.13 に示す環状リストを用いる。リストのノードにはフレームが最近アクセスされた時刻が記録してある。現在時刻と比較して時刻が古くなっているフレームは、ワーキングセットから外れたと判断する。また、ページテーブルの R ビットと D ビットも使用する。次のようなアルゴリズムである。

1. swap-in する度にフレームを環状リストに挿入していく。挿入位置は最も古いフレームの一つ手前である。最も古いフレームは時計の針に当たるポインタが指している。
2. 定期的（例えば TICK=20ms 毎）に全ページテーブルエントリの R ビットをゼロにクリアする。その際、R=1 だったフレームだけに現在時刻を記録する。
3. ページ不在時にフレームが不足したなら、時計の針が指しているフレームを調べる。

R=1 の場合 R ビットをクリアして次のフレームに進む。

時刻が新しい場合 ページはワーキングセットに含まれている。次のフレームに進む。

時刻が古い場合 ページはワーキングセットに含まれていない。

D=1 の場合 内容が変更されているので swap-out を予約し、次のフレームに進む。

D=0 の場合 このフレームを置き換える。

12.6 フレーム割り付けアルゴリズム

ページングシステムでは全てのフレームが同等なので、どのフレームを、どのプロセスの、どのページに使用しても良い。任意の空きフレームを使用すれば良いのでフレーム割り付けは問題にならない。CPU が複数ある場合でも、図 2.1 のような SMP システムであれば全てのフレームが均質である。

しかし、図 2.4 に示したサーバ用の SMP システムでは事情が少し異なっている。このようなシステムは、CPU とメモリからなるノードが相互接続された構造になっている。CPU と異なるノードのメモリは、CPU と同じノードのメモリより低速なアクセスしかできない。そこで可能な限り、同じプロセスのフレームは同じノードのメモリを使用し、かつ、同じプロセスのスレッドは同じノードの CPU が実行するようにする。

12.7 まとめ

本章ではページングに基づく仮想記憶を学んだ。ページング機構では、ページテーブルの V ビットを用いてフレームが割当てられないページを表現できた。V=0 のページがアクセスされるとページ不在割込み (page fault) が発生するので、その時点で内容をバッキングストアから swap-in する。一方で利用頻度の低いページは swap-out する。この手法でメモリより大きなプログラムを実行可能にする。

データをコピーする際、複数のプロセスがフレームを共有する状態を作った後、ページに書込みがあった時点でフレームをコピーしプロセス夫々が異なる内容を持つことを可能にする。このように、書込みがある時点でコピーを行う方式を Copy on Write と呼ぶ。

メモリマップドファイルは、仮想アドレス空間にファイルをマッピングすることで、ファイルを読み書きする手段を提供する。本章ではメモリマップドファイルを使用する UNIX プログラムの例を示した。メモリマップドファイルを用いると、マッピング完了後はシステムコールを使用することなくファイルの内容を変更できる。

フレームが不足した際に、swap-out するページを選択するアルゴリズムをページ置き換えアルゴリズムと呼ぶ。プログラム実行中のページ参照に局所性があり、ワーキングセットの変化によりフェーズ化現象が生じる。ワーキングセットがメモリに入り切らないとスラッシングが発生する。ページ置き換えアルゴリズムはこれらの性質を考慮して決定される。LRU アルゴリズム、LFU アルゴリズム、FIFO アルゴリズム、Clock アルゴリズム、WSClock アルゴリズムを紹介した。

練習問題

12.1 次の言葉の意味を説明しなさい.

- 仮想記憶
- バッキングストア
- ページ不在割込み (page fault)
- デマンドページング (demand paging)
- swap-in, swap-out
- Copy on Write
- メモリマップドファイル
- 局所性
- ワーキングセット
- フェーズ化
- スラッシング
- ページ読み込みアルゴリズム
- ページ置き換えアルゴリズム
- フレーム割り付けアルゴリズム
- LRU, LFU, FIFO, Clock, WSClock アルゴリズム
- Belady の異常な振る舞い

12.2 リスト 12.1 のプログラムを実際に実行してみなさい. なお, ソースプログラムは以下から入手可能である.

<https://github.com/tctsigemura/OSTextBook/blob/master/SampleCode/Mmap>

12.3 「Belady の異常な振る舞いの例」で示したページ参照ストリングとフレーム数を用い, LRU, FIFO アルゴリズムを適用した場合をトレースしなさい.

第Ⅳ部

ファイル管理

第 13 章

二次記憶装置（ストレージ）

容量が小さく揮発性の主記憶だけではコンピュータを実用的に使用することができない。二次記憶装置をプログラムやデータを格納したファイルの永続的な置き場として使用する。ファイルを永続的に記憶するためには、大容量で不揮発性の二次記憶装置が適している。

13.1 記憶装置の階層

現代のコンピュータは、様々な種類の記憶装置を使用している。図 13.1 にコンピュータの記憶装置の関係を簡単に示す。図では、上の層にあるものほど高価で高速なメモリである。

1. **レジスタ**は CPU レジスタのことを表す。容量は小さい^{*1}が最も高速な記憶装置である。
2. **キャッシュ**は最近アクセスした主記憶のデータを、再度のアクセスに備えて、ハードウェアにより自動的に記録する高速なメモリである。ユーザプログラムからキャッシュの存在は見えない。
3. **主記憶（メモリ）**は数十ナノ秒程度の時間でアクセスできる高速な記憶装置である。コンピュータはプログラムやデータを主記憶にロードして実行する。主記憶の容量は数 Gi バイト～数十 Gi バイト程度であり、オペレーティングシステムと全てのアプリケーションを格納するには小さすぎる。
4. **二次記憶装置（ストレージ）**は、近年では、ハードディスクや SSD（Solid State Drive）のことである。容量は大きい^{*2}が、主記憶と比べるとアクセス時間がとても長い^{*2}。しかし、二次記憶装置には電源を切ってもデータが消えない特性がある^{*3}。この特性は**不揮発性**と呼ばれる。

各記憶装置には以上のような特性があるので、夫々の特性に合った使い方をする必要がある。オペレーティングシステム、アプリケーションプログラム、データの全てを永続的に格納するには、**大容量で不揮発性の二次記憶装置**が適している。

13.2 接続方式

図 13.2 に示すように、CPU と主記憶やホストコントローラは、バスによって直接に接続される。これらは、CPU が直接にアクセスすることができる。一方で二次記憶装置は、ホストコントローラ配下

^{*1} 数十～数百バイト程度である。

^{*2} ハードディスクの場合だと数ミリ秒～数十ミリ秒もかかる。

^{*3} ハードディスクなら磁氣的に記録しているので消えない。SSD ならフラッシュメモリに記録しているので消えない。

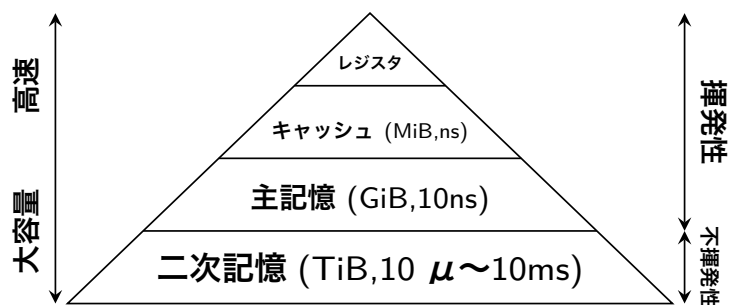


図 13.1: 記憶の階層

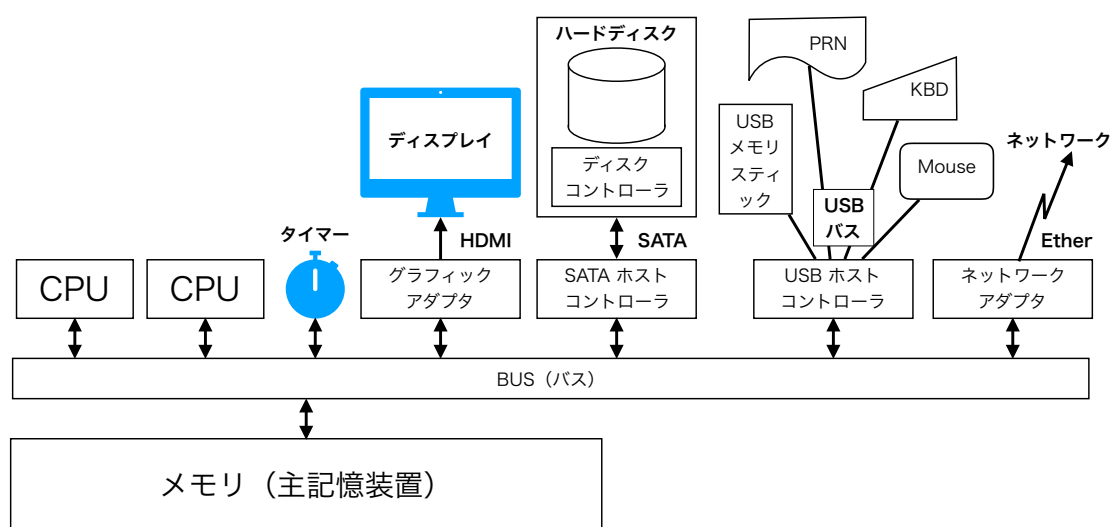


図 13.2: ハードウェア構成（再掲）

のバス等に接続される。CPU はホストコントローラにコマンドを送り、ホストコントローラが二次記憶装置と通信する。CPU は、二次記憶装置を直接にアクセスすることができない。

USB バスに接続されたメモリスティックやハードディスクは、PC 稼働中に接続・取り外しが可能である。また、CD-ROM などの光ディスクも取り外し可能である。これらは、データ交換用やバックアップ用に都合が良い。

13.3 記憶媒体

二次記憶装置は大きくテープ型の装置とディスク型の装置に分類できる。テープ型装置はデータのバックアップやデータの輸送用に用いられてきたが、最近では使用されることが少なくなっている。ハードディスクに代表されるディスク型装置は最もよく使用される二次記憶装置である。

1. テープ型装置

図 13.3 に磁気テープの写真を示す^{*4}。カセットの中に 1 本の長いテープが巻き取られた状態で

^{*4} 様々な磁気テープが用いられてきたが最近見かけることが少なくなっている。写真は、デジタルデータ記録用の 8mm 磁



図 13.3: 磁気テープ

入っている。データは磁気的にテープに記録される。データは先頭から順に（シーケンシャルに）書き込むことしかできない。読み出す場合も先頭から順に読み出すことしかできない。

シーケンシャルアクセスしかできないため読み出すデータの位置まで進むために数分かかることもある。しかし、一度、データの転送が始まるとハードディスク並のデータ転送速度になる。一般に記録できるデータあたりのメディア（磁気テープ）の値段が安いので、滅多に使用することが無いバックアップデータを保存するために用いられてきた。

2. ディスク型装置

ディスク型装置の代表はハードディスクである。図 13.4 に蓋を開けた状態のハードディスクの写真を示す^{*5}。写真のハードディスクは4枚の円盤が重なっており、各円盤の表裏（合計8面）にデータが記録できる。データは回転する円盤上に磁気的に記録される。

ディスク型装置の最大の特長は、データブロックのアドレスを指定して途中からでも自由に読み書きできることである。このようなアクセスの仕方は**ランダムアクセス**と呼ばれる。フロッピーディスク、CD-ROM、DVD-ROM、Blu-Ray Disk 等も円盤にデータを記録する方式なのでディスク型装置である。一方で、SSD や、USB メモリ、メモ리카ードは円盤にデータを記録する方式では無いが、ランダムアクセスが可能でハードディスクと同様に扱うことができる。そこで本書では、これらもディスク型装置として扱う。

13.4 ハードディスク

ハードディスクは、システムの起動ドライブとして使用される。システム起動後も、オペレーティングシステムの追加モジュールやアプリケーションはハードディスクから読み込まれるし、仮想記憶シス

気テープである。図 13.2 に磁気テープを示してないのは最近見かけなくなったためである。

^{*5} 3.5 インチのハードディスクの蓋を開けた状態である。普通、蓋を開けるとハードディスクは壊れるので、写真のハードディスクはこわれている。



図 13.4: ハードディスク

テムがバッキングストアとしても使用する。また、アプリケーションがデータを格納する場合も、第一にハードディスクが選択される。

このようにハードディスクが最も頻繁に使用されるので、ハードディスクを上手く管理できるかどうかにより、オペレーティングシステムの性能や使い勝手は大きく左右される。そのためファイル管理機構はハードディスクを管理することを前提にしている^{*6}。また、ハードディスク以外のディスク型装置はハードディスクと同様に扱えるように作ってある。そこで、ハードディスクについて少しだけ詳しく解説する。

13.4.1 セクタ・トラック・シリンダ

回転する円盤に同心円のトラックを作り磁気的にデータを記録する。一周のトラックに記録できるデータは大きすぎるので、トラックを幾つかのブロックに分割する。このブロック（サイズは 512B か 4KiB）のことをセクタと呼ぶ。データの読み書きはセクタ単位で行われる。同じ半径のトラックは円盤の面の数だけ存在することになる。各円盤面に散らばった同じ半径のトラックを集めたものをシリンダと呼ぶ。

PC 用のハードディスクが世の中に出てきた最初からセクタのサイズは 512 バイトであった。しかし、ハードディスクの大容量化に伴い 2009 年頃からセクタサイズを 4Ki バイトにした製品が出回るようになってきた。最近のオペレーティングシステムはセクタサイズが 512 バイト以外でも効率よく働くように改良されている。

13.4.2 セクタのアドレッシング

ハードディスク上の特定のセクタを指定するために、以下の二つの方式のどちらかが使用される。

CHS (Cylinder Head Sector) シリンダ番号，トラック番号，セクタ番号の組で 1 つのセクタを特定

^{*6} 最近では Apple の APFS [49] のように、SSD を重視している場合もある。

できる。長い間、ハードディスクの読み書きは、これら3つの番号を使用したセクタアドレスを用いて行われてきた。PCではトラックをトラックに対応する読み書きヘッドで置換えシリンダ (Cylinder)、ヘッド (Head)、セクタ (Sector) の組でセクタアドレスを表現してきた。このセクタアドレスの表現方式を **CHS方式**と呼ぶ。

LBA (Logical Block Addressing) 本来ハードディスクのセクタアドレスは、ハードディスクの物理構造を反映したシリンダ番号、トラック番号、セクタ番号の組で表すものである。オペレーティングシステムは、同一ファイルのデータをなるべく同じシリンダに置くなどして、ファイルアクセスの効率化を行っていた。しかし、現代のハードディスクはブラックボックスになってしまった。ディスクコントローラにハードディスクの構造を問い合わせても嘘の情報が返されるようになったのである^{*7}。そのため、従来の3次元のアドレッシングは煩雑なだけになってしまった。現在では全てのセクタに通し番号 (1次元のセクタアドレス) をふり、この番号でセクタを指定するアドレッシングが一般的である。このアドレッシングを **LBA方式**と呼ぶ。

13.5 フォーマットニング

二次記憶装置の使用を開始する前に、記憶媒体を初期化する必要がある^{*8}。ハードディスクを例に初期化の手順を以下に示す。

1. 低レベルのフォーマットニング (物理フォーマットニング) を行う。

低レベルのフォーマットニングはディスクの表面にトラックを磁氣的に描いていく作業である。20年以上前の製品ではユーザが行うことが可能であったが、最近は製造時に工場で物理フォーマットニングを行いユーザにさせない。

2. 必要に応じてディスクをパーティション (区画) に分割する。

ディスク全体を一つのボリューム^{*9}として使用しても良いが、システム領域とユーザ領域のように分けて使用したい場合や、一台のディスクに複数のオペレーティングシステムをインストールする場合は分割する必要がある。

図 13.5 に4つのパーティションに分割したハードディスクの内部を示す^{*10}。MBR (*Master Boot Record*) は、ハードディスクの最初のセクタ (LBA0) に格納され、ブートプログラムとパーティションテーブルを記録する。図 13.6 に MBR の内容を簡単に示す。パーティションテーブルに各パーティションの位置と大きさ等が記録される。シグネチャはハードディスクが初期化済みかどうかを表すデータである。この2バイトに 55H AAH が書き込まれていれば、初期化済みである。

パーティションテーブルの例を図 13.7 に示す。この例は、最初に2つパーティションが存在し (Type ≠ 00H)、残りのエントリは使用されていない (Type=00H) 場合を示している。図中の???等は使用されないエントリの内容か、CHS で表現した値が格納される部分である。CHS 表現は煩

^{*7} ディスクの容量を大きくするために外側トラックのセクタ数を内側トラックより多くするなど、従来の考え方では表現できない物理構造になってしまった等の事情がある。

^{*8} USB メモリやポータブルハードディスクは初期化済みの状態で販売されている場合が多い。ほとんどの場合、PC は内蔵ハードディスクを初期化した上でオペレーティングシステムをインストールした状態で販売されている。

^{*9} Windows の用語ではドライブと呼ぶ。一つのボリュームが一つのファイルシステムを格納する。

^{*10} この例は PC で MBR 方式を使用した場合のものである。

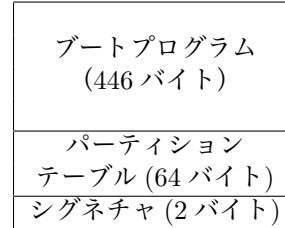
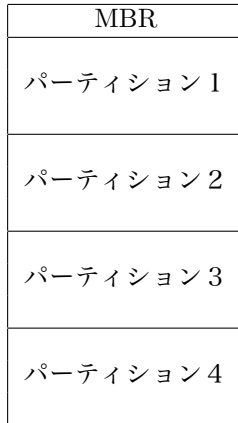


図 13.6: PC の MBR（合計 512 バイト）

図 13.5: ハードディスクのパーティション

Flag (1)	Start CHS (3)	Type (1)	End CHS (3)	Start LBA (4)	Size (4)
80H	???	06H	???	0000003FH	00003F00H
00H	???	A5H	???	00003F3FH	0000BD00H
00H	???	00H	???	?????????	???????
00H	???	00H	???	?????????	???????

図 13.7: パーティションテーブルの例

表 13.1: パーティションテーブルのエントリ

項目	バイト数	意味
Flag	1	80H アクティブ/ 00H インアクティブ
Start CHS	3	開始アドレス (CHS 表現)
Type	1	ファイルシステムの種類
End CHS	3	終了アドレス (CHS 表現)
Start LBA	4	開始アドレス (LBA 表現)
Size	4	セクタ数 (LBA 表現)

表 13.2: Type フィールドの意味

Type	意味
00H	空き
01H	FAT12
04H	FAT16(小)
06H	FAT16(大)
07H	NTFS
0BH	FAT32
83H	Linux(ext2)
A5H	FreeBSD

雑になるので省略した。

パーティションテーブルエントリの内容は表 13.1 の通りである。エントリサイズは 16 バイトである。Flag フィールドが 80H のパーティションは、オペレーティングシステムの起動パーティションである。Type フィールドの意味は表 13.2 の通りである。ここに示したものは一部である^{*11}。

3. 論理フォーマットを行う。

論理フォーマットは、ボリューム（パーティション）に空の状態のファイルシステムを作る作業である。空のファイルシステムを表現する管理データをディスクに書き込む。

^{*11} 詳しくは“Partition type”, https://en.wikipedia.org/wiki/Partition_type 等を参照のこと。

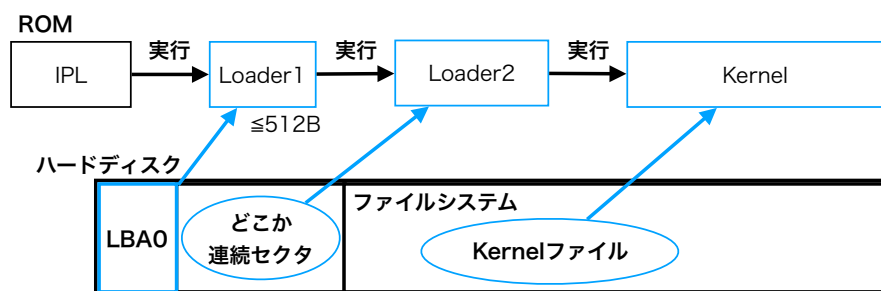


図 13.8: ハードディスクからの OS のブート手順

13.6 ブートストラップ

オペレーティングシステムは、コンピュータに内蔵されたハードディスクにインストールされる。オペレーティングシステムを起動するためには、ハードディスクからオペレーティングシステムを主記憶にロードし、実行を開始する必要がある。この作業をブートストラップ（略してブート）と呼ぶ。

多くの場合オペレーティングシステム本体（カーネル）は、ファイルシステム上にファイルとして格納されている。つまり、これから起動するオペレーティングシステムのファイルシステムの構造を解釈し、カーネルファイルを見つけ出す必要がある。しかし、どのオペレーティングシステムがインストールされるかは PC 製造時には分からない。そのため、予め PC にオペレーティングシステムのファイルシステムを解釈する機能を組込むことはできない。そこで次のように、いくつかの段階を経てオペレーティングシステムを起動する方式を用いる。図 13.8 にブート手順を模式的に表す。

1. IPL (Initial Program Loader)

PC 本体の ROM に IPL 呼ばれるプログラムが格納されている。PC の電源が投入されると IPL が自動的に実行を開始する。IPL はシステム用ハードディスクの最初のセクタ（LBA0）を主記憶にロードしそれをプログラムと見做し実行する。

2. ブートローダ（第1段階）

LBA0 に次段階のブートプログラムが書き込んである。これをブートローダ（Loader1）と呼ぶ。従来、1セクタのサイズは 512 バイトであったので、小さな Loader1 でファイルシステムを解釈しカーネルをロードすることはできない。そこで Loader1 は、ハードディスクのどこか連続セクタに格納された、第2段階の高機能なブートローダ（Loader2）をロードし制御を移す。

3. ブートローダ（第2段階）

第2段階のブートローダ（Loader2）がファイルシステムを解釈しカーネルファイルを探し出し、カーネルをロード・実行する。Loader1, Loader2 はオペレーティングシステム毎に異なるので、オペレーティングシステムと同時にインストールされる。

4. ブートセクタ（ブートマネージャ）

ハードディスクがパーティションに分割されている場合は、図 13.9 に示すように LBA0（MBR）にブートセクタ（ここでは Boot と呼ぶ）が格納される。Boot は MBR にパーティションテーブル等と同時に格納されるので 446 バイト以内でなければならない。Boot はパーティションの一

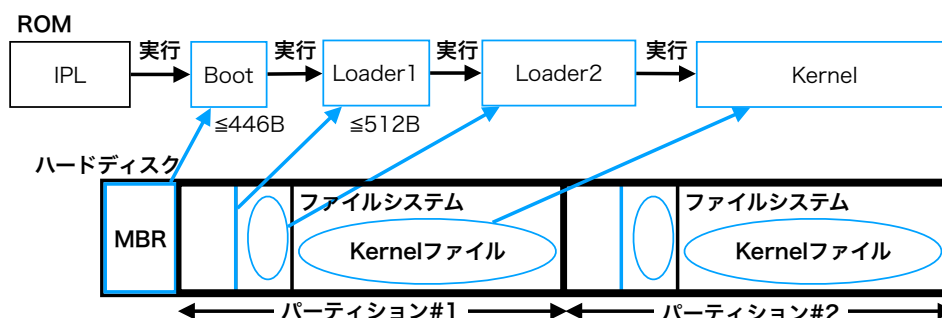


図 13.9: 複数パーティションを格納するハードディスクからの OS ブート手順

つを選択し^{*12}パーティションの先頭にインストールされている Loader1 相当のプログラムをロードし制御を移す。

以上がブートストラップの原理である。ブートローダもハードディスクにインストールされるので、同じ PC で様々なオペレーティングシステムをブートすることができる。実際は、Loader2 が更に高機能なローダを読み込む場合もあり、色々なアレンジメントがあり得る^{*13}。しかし、原理的には上の方式でオペレーティングシステムのブートが可能である。

13.7 実装例

TacOS に、ディスク型装置をアクセスするデバイスドライバと、パーティションテーブルを解析するプログラムが含まれる。TacOS は LBA 方式でハードディスク代替のマイクロ SD カードをアクセスする。23.7 では、TacOS のマイクロ SD カードのデバイスドライバを紹介している。リスト 23.19 がデバイスドライバのソースプログラムである。パーティションテーブルを解析して目的のパーティションの位置を調べるプログラムの例は、23.6 のリスト 23.14 に掲載している。

13.8 まとめ

この章では、二次記憶装置について学んだ。二次記憶装置の特徴は、**大容量**、**不揮発性**、**低速**なことである。二次記憶装置は、大きく**テープ型装置**と**ディスク型装置**に分類される。テープ型装置は**シーケンシャルアクセス**しかできないが、ディスク型装置は**ランダムアクセス**が可能である。本書では、メモリカード等の本来はディスク型ではない装置も、ランダムアクセス可能なものはディスク型装置と呼ぶことにした。

ハードディスクは、ディスク型装置の代表的なものである。ファイルシステムの多くは、ハードディスクを上手く管理することを目的としている。そこで、ハードディスクの構造について少しだけ詳しく学んだ。セクタのアドレッシングには、ハードディスクの物理構造と関係の無い **LBA 方式**と、物理構造を意識した **CHS 方式**があった。

ハードディスク全体を一つの**ボリューム**としてしてもよいが、**パーティション（区画）**に分割し夫々

^{*12} メニューを表示し、ユーザにキーボードから選択させるなどの方法を使う。

^{*13} 高機能なローダはファイルシステムに格納され、自身の設定ファイルをファイルシステム内に持つような場合もある。

をボリュームとして扱うこともできる。ディスクの先頭セクタ *MBR* に格納されたパーティションテーブルから、パーティションの位置、大きさ、タイプを知ることができる。

PC の製造時には、どのオペレーティングシステムがインストールされるか分からない。オペレーティングシステムのブート手順は、オペレーティングシステムがインストールされるまで分からない。そこで、PC の ROM に格納される *IPL* は、ハードディスクの先頭セクタを読み出し、そこに含まれるプログラム（ブートローダ）に制御を移す機能しか持たないものとする。ブートローダはオペレーティングシステムのインストール時に書き込まれ、オペレーティングシステム固有のブート手順を知っている。

練習問題

13.1 次の言葉の意味を説明しなさい。

- (a) 二次記憶装置
- (b) 揮発性・不揮発性
- (c) 記憶の階層
- (d) テープ型装置・ディスク型装置
- (e) シーケンシャルアクセス・ランダムアクセス
- (f) セクタ・トラック・シリンダ
- (g) CHS・LBA
- (h) ボリューム
- (i) パーティション
- (j) MBR
- (k) IPL
- (l) ブートストラップ

13.2 次のディスクに付いて答えなさい。

1 台全体 1,024 シリンダ
1 シリンダ 8 トラック
1 トラック 128 セクタ
1 セクタ 512 バイト

- (a) ディスクの容量をセクタ単位で答えなさい。
- (b) ディスクの容量をバイト単位で答えなさい。
- (c) 最後のセクタのアドレスを LBA で答えなさい。
- (d) 最後のセクタのアドレスを CHS で答えなさい。
(但し, C:0 以上, H:0 以上, S:1 以上である.)

13.3 図 13.7 に付いて答えなさい。

- (a) 第 1 パーティションの位置を LBA で答えなさい。
- (b) 第 1 パーティションのサイズをセクタ数で答えなさい。
- (c) 第 1 パーティションの種類を表 13.2 を参照して答えなさい。
- (d) 第 2 パーティションの位置を LBA で答えなさい。
- (e) 第 2 パーティションのサイズをセクタ数で答えなさい。
- (f) 第 2 パーティションの種類を表 13.2 を参照して答えなさい。

13.4 PC 用の高機能なブートローダ GRUB について調査しなさい。

第 14 章

ファイルシステムの概念

ファイルシステムは二次記憶装置（ストレージ）を管理・抽象化・仮想化し、使いやすいファイルをユーザに提供する。ファイルは、名前が付けられた一次元のバイト列（**バイトストリーム**）である。オペレーティングシステムはバイト列の使い方を規定しない。名前で一つのファイルを指定し、その中のバイト位置でデータを指定することができる。

14.1 ファイルの名前付け

ファイルは木構造の**ディレクトリシステム**に格納される^{*1}。図 14.1 に木構造の例を示す。ディレクトリ^{*2}は、他のディレクトリやファイルの名前とファイル本体へのポインタ^{*3}の組を記録する特殊なファイルである。木構造の中から一つのファイルを特定するために、階層構造を持った名前（パス）を用いる。パスには絶対パスと相対パスの二種類がある。

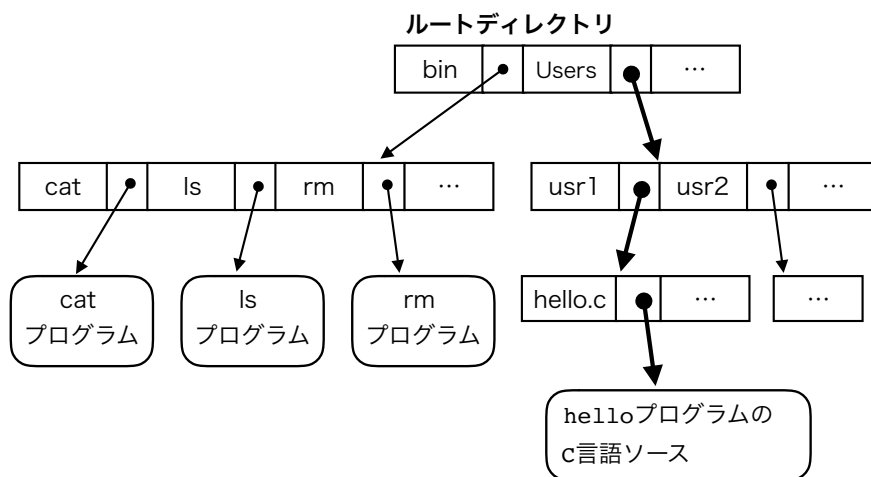


図 14.1: 木構造のディレクトリシステム

*1 現代のオペレーティングシステムでは、ほとんどの場合、そうである。

*2 Windows や macOS ではフォルダとも呼ぶ。

*3 多くの場合はファイル本体に付けられたユニークな番号である。

- 絶対パス

木構造の根にあたるルートディレクトリを起点に目的のファイルへ辿り着く道順を書き表したものを絶対パスと呼ぶ。例えば、図 14.1 の「hello プログラムの C 言語ソース」ファイルの絶対パスは、`/Users/usr1/hello.c` である。絶対パスは「/」から書き始める。

- 相対パス

プロセスは、現在の操作対象になる一つのワーキングディレクトリ（カレントディレクトリ）を持つ。相対パスは、ワーキングディレクトリを起点に目的のファイルへ辿り着く道順を書き表したものである。例えば、図 14.1 の `/Users` ディレクトリがワーキングディレクトリの場合、「hello プログラムの C 言語ソース」ファイルの相対パスは `usr1/hello.c` になる。相対パスは「/」以外から書き始める。

14.2 ファイルの別名

ファイルを別名で参照できると便利ことがある。例えば、その日の作業記録ファイルを、毎日、作成するシステムがあるとする。このシステムではファイル名の一部に年月日を埋込むことで区別し、過去 3 日分を消さずに残すものとする。しかし、最新のファイルはいつも同じ名前でアクセスできると便利だ。そこで、最新のファイルに綴りが変化しない別名を付ける。次のような状態である。

<code>2017_06_30.log</code>	2017 年 6 月 30 日のファイル
<code>2017_07_01.log</code>	2017 年 7 月 1 日のファイル
<code>2017_07_02.log</code>	2017 年 7 月 2 日のファイル
<code>today.log</code>	現時点では 2017 年 7 月 2 日のファイルの別名

このような別名の仕組みとして大きく 3 つの方式が考えられる。

- ハードリンク

主に UNIX で使用される方式である^{*4}。ファイルシステムの仕組みとして OS カーネルに組込む。図 14.1 で、同一のファイル本体を指すポインタが複数のディレクトリ・エントリに存在する状態である^{*5}。ファイル本体には何ヶ所から指されているか管理するリンクカウントを設置し、リンクが削除されカウントがゼロになった時点でファイル本体を削除する。

原理的にはディレクトリファイルをリンクすることも可能であるが、リンクのループを作ることが可能になってしまう^{*6}ので UNIX では許されていない。ループを許可すると、ルートディレクトリから分離した離れ小島状態の部分木が出来た時、リンクカウントが永遠にゼロにならない問題が生じる。ループの検出はコストが高い処理なので最初からディレクトリのリンクを禁止する。

- シンボリックリンク

主に UNIX で使用される方式である^{*7}。ファイルシステムの仕組みとして OS カーネルに組込む。

^{*4} macOS や Windows でも使用できる。

^{*5} ファイル本体が複数の箇所から指されるので、ディレクトリシステムが木構造ではなく非循環グラフになってしまう。

^{*6} ディレクトリシステムが一般グラフになる。

^{*7} macOS や Windows でも使用できる。

リスト 14.1: HFS+ ファイルシステム上の macOS のエイリアス

```

1 $ ls -l@ a.txt*
2 -rw-r--r--  1 sigemura  admin    5 Jun 27 10:19 a.txt
3 -rw-r--r--@ 1 sigemura  admin 1012 Jun 27 10:19 a.txtのエイリアス
4           com.apple.FinderInfo      32

```

シンボリックリンクは他のファイルのパスをデータとして格納した特別なファイルだと考えられる。シンボリックリンクファイルは OS カーネルが特別な扱いをする。

シンボリックリンクは、使用時に格納されたパスを評価し直すので、オリジナルファイルの名前を変更するとリンク切れ状態になる。同じ名前でも新たに別のファイルが作られると、それへのリンクが変わる。リンク先が存在しないシンボリックリンクを作ることでもある。シンボリックリンクの特徴は、リンクが切れて新しいファイルに勝手に接続されることである。

・ファイルシステムの外で実装されるリンク

Windows のショートカットや macOS のエイリアスがこれにあたる。ハードリンクやシンボリックリンクは OS カーネル内で処理され、リンクの存在がアプリケーションからは透過（透明）なので、とてもスマートな仕組みに見える。しかし、現代のオペレーティングシステム（例えば macOS）では、オペレーティングシステムはローカルハードディスクの HFS+ ファイルシステムにインストールし、ユーザのホームディレクトリはネットワークドライブに格納され SMB プロトコルでアクセスし、デジカメのデータをマイクロ SD から読み込む時は FAT ファイルシステムを使用する。使用するファイルシステムが何種類もあるので、統一的に使用できるリンクの仕組みが保証されない^{*8}。そこで、アプリケーションやライブラリ（もしかしたら OS カーネル内）等、ファイルシステムの外にリンクの代替となる仕組みを準備していることがある。

－ HFS+ 上の macOS のエイリアスの例

HFS+ ファイルシステム上でエイリアスは拡張属性（14.4 参照）を持った普通のファイルとして格納される。ファイルシステムが提供する汎用的な機構である拡張属性をエイリアスの実現に利用しているだけで、エイリアス専用の機能をファイルシステムが持つ訳ではない。リスト 14.1 に HFS+ ファイルシステム上のエイリアスの例を示す。3, 4 行からファイル「a.txt のエイリアス」が 32 バイトの拡張属性 com.apple.FinderInfo を持つことが分かる。

－ FAT 上の macOS のエイリアスの例

FAT ファイルシステム上ではエイリアス本体と拡張属性を格納する二つの通常ファイルとして作成される。ファイルの内容を解釈する時点で拡張属性を実現している。リスト 14.2 に FAT ファイルシステム上のエイリアスの例を示す。4, 5 行からファイル「a.txt のエイリアス」が 32 バイトの拡張属性 com.apple.FinderInfo を持つことが分かる。リスト 14.2 の 2 行のように FAT ファイルシステムには、「.a.txt のエイリアス」という名前の隠しファイル^{*9}ができています。リスト 14.2 の 6 行で隠しファイルを消すと、9 行のように拡張属性が消え

^{*8} 例えば FAT ファイルシステムにはハードリンクやシンボリックリンクの仕組みはない。

^{*9} 名前が「.」で始まるファイルは、普通は表示されない。

リスト 14.2: FAT ファイルシステム上の macOS のエイリアス

```

1 $ ls -la@ .* a.txt*
2 -rwxrwxrwx  1 sigemura  staff  4096 Jun 27 09:55 ._a.txtのエイリアス
3 -rwxrwxrwx  1 sigemura  staff    5 Jun 27 09:55 a.txt
4 -rwxrwxrwx@ 1 sigemura  staff  1040 Jun 27 09:55 a.txtのエイリアス
5      com.apple.FinderInfo      32
6 $ rm ._a.txtのエイリアス
7 $ ls -la@ a.txt*
8 -rwxrwxrwx  1 sigemura  staff    5 Jun 27 09:55 a.txt
9 -rwxrwxrwx  1 sigemura  staff  1040 Jun 27 09:55 a.txtのエイリアス

```

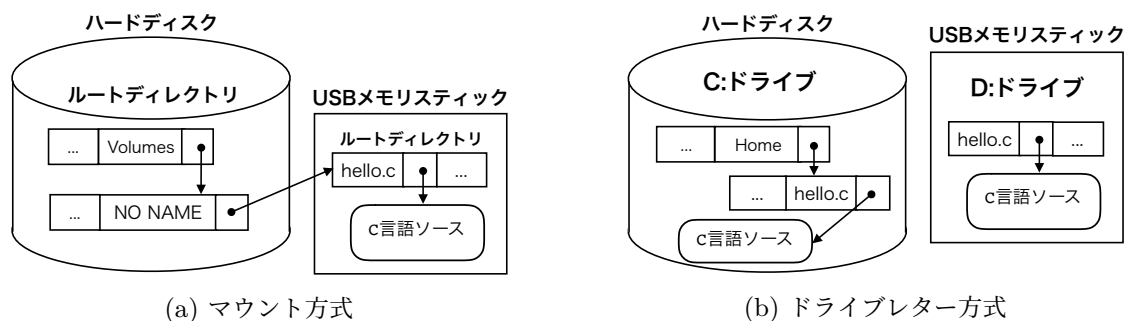


図 14.2: マウント方式とドライブレター方式

た。FAT ファイルシステム上では隠しファイルを使用して拡張属性を真似し、更に、拡張属性を使用してエイリアスを表現している。

14.3 ボリュームのマウント

ハードディスクが複数台ある場合、ハードディスクが複数のパーティションに分割されている場合、ネットワークドライブを使用する場合、一時的にメモリカード等を使用する場合などに、ルートディレクトリがあるのとは別の**ボリューム**にアクセスする必要がある。別のボリュームのファイルもパスで指定できる必要がある。図 14.2 に以下で説明する二種類の方式を模式的に示す。

(a) マウント方式

UNIX や macOS では新しいボリュームに格納されたファイルシステムを、既存のディレクトリに接続する（マウントする）方式が使用される。例えば macOS に USB メモリを接続した場合、自動的に `/Volumes/VolName*10` の位置に USB メモリの内容が見えるようにマウントされる。新しいボリュームが追加されても、単一の木に全てが格納される。図 14.2a の例では「C 言語のソース」ファイルは、`/Volumes/NO NAME/hello.c` のパスで参照できる。

^{*10} VolName は USB メモリを初期化した時に決めたボリューム名である。購入時点で既に NO NAME や UNTITLED の名前を付けて初期化されていることが多い。

リスト 14.3: 拡張属性の例

```

1 $ ls -l@ b.txt*
2 -rw-r--r--  2 sigemura  staff      123 Jun 25 19:38 b.txt
3 -rw-r--r--@ 1 sigemura  staff      836 Jun 25 19:39 b.txtのエイリアス
4           com.apple.FinderInfo      32
5 $ xattr -l b.txtのエイリアス
6 com.apple.FinderInfo:
7 00000000  61 6C 69 73 4D 41 43 53 80 00 00 00 00 00 00 |alisMACS.....|
8 00000010  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|

```

(b) ドライブレター方式

Windows ではボリューム毎に新しい木を作りドライブレターで木を区別する。図 14.2b の例では、オペレーティングシステムがインストールされたドライブを C ドライブ、USB メモリを D ドライブのように決め、C ドライブのファイルは C:\Home\hello.c のようなパス^{*11}で、D ドライブのファイルは D:\hello.c のようなパスで表現する。

14.4 ファイルの属性

ファイルが持つ属性の例を以下に示す。どのファイルシステムでも同じ属性を持っているとは限らない。ここで示すのは一般的な例である。

- **名前**：図 14.1 ではファイルを格納するディレクトリがファイル名を記録していたが、ファイル名もファイル属性の一つと考え、ファイル本体に記録する場合もある。また、FAT ファイルシステム（第 15 章参照）のように、全ての属性（ファイル名も含む）をディレクトリに記録する場合もある。
- **識別子**：ファイルシステム中でファイルを一意に識別できる番号などのこと。
- **型（タイプ）**：OS のカーネルがサポートしているファイルの種類のこと。UNIX では、通常ファイル、ディレクトリファイル、シンボリックリンク、文字デバイス、ブロックデバイス等のファイル型が定義されている。
- **保護**：`rw-rw-rw-` 等のアクセス制御情報のことである。（次の節で詳しく説明する。）
- **日時**：作成日時、最終変更日時などのこと。
- **所有者**：所有者やグループ等を識別する情報のこと。
- **位置**：ディスク上でデータが記録されている場所を表す情報のこと。
- **サイズ**：ファイルが格納するデータの大きさをバイト単位で表す。
- **拡張**：そのファイルを開く時に使用するアプリケーションの名前や、セキュリティに関する追加属性のような情報のこと。OS カーネルが使い方を定めておらず、アプリケーション等が名前を付けて書き込める小さなデータである。リスト 14.3 に macOS の例を示す。

^{*11} Windows ではパスの区切りに使用する記号が「/」ではなく「\」になる。
（日本語版の Windows では「\」が「¥」になる。）

リスト 14.4: macOS で ACL を操作した例

```
1 $ ls -le a.txt
2 -rw-r--r--  1 sigemura  staff  4 Jul  5 21:55 a.txt
3 $ chmod +a "group:admin allow write" a.txt
4 $ chmod +a "group:admin deny delete" a.txt
5 $ ls -le a.txt
6 -rw-r--r--+ 1 sigemura  staff  4 Jul  5 21:55 a.txt
7  0: group:admin deny delete
8  1: group:admin allow write
```

1 行の `ls -l@` コマンドは拡張属性の一覧も表示する。4 行から「b.txt のエイリアス」ファイルに `com.apple.FinderInfo` という名前の 32 バイトの拡張属性があることが分かる。5 行の `xattr` コマンドで拡張属性の内容を表示してみた。

14.5 アクセス制御

ファイルの保護属性に基づいたアクセス制御ができる。UNIX では `rw-rw-rw-` の 9 ビットでファイルの「所有者」、「グループ」、「その他のユーザ」の三者が Read/Write/eXecute のどれをして良いか表現する。

より一般的な方式として *ACL* (*Access Control List*) がある。ファイル毎にどのユーザ（またはグループ）が何（`rw` より詳細）をできるか（できないか）記録した順番付けされたリストを ACL と呼ぶ。リスト 14.4 に macOS の例を示す。

3, 4 行の `chmod` コマンドでファイル `a.txt` に ACL を追加している。ACL の設定状況は `ls -le` コマンドで確認できる。ACL を持つファイルでは `rw` の右に `+` が表示され、次の行から ACL の内容がリストされる。ACL はリストの最初から順に、許可・不許可が決まるまで評価される。

最近の UNIX 系オペレーティングシステム（macOS 含む）では ACL と `rw` 方式を組合せて使用することができる。その場合は、まず細かな制御が可能な ACL を用いてチェックを行う。ACL でアクセスを許可するかどうか決まらない場合に `rw` を用いる。

14.6 ファイルの種類

ファイルの型属性（14.4 参照）は、ファイルシステム（OS カーネル）が定めるファイルの種類を表現する。「通常ファイル」、「ディレクトリ」、「シンボリックリンク」等の種類がある。型が「通常ファイル」のファイルにはデータを格納することができる。OS カーネルは通常ファイルのデータがバイトストリームであることは定めているが、バイトストリームの中身には関与しない^{*12}。

ファイル名の一部でファイルの種類を表現することが、多くのオペレーティングシステムで慣例になっている。ファイル名の最後が `.xxx` のような文字列で終わっているのを誰でも見たことがあると思う。これを拡張子と呼び、ファイルに格納されたデータの種類を表現するために使用する。多くの OS

*12 実行形式プログラムは例外である

表 14.1: よく見かけるファイルの拡張子

拡張子	意味
.c, .java, .s 等	ソース・プログラム (C 言語, Java 言語, アセンブリ言語)
.py, .pl, .php 等	スクリプト言語のプログラム (python, perl, PHP)
.txt, .html, .xml 等	プレーンテキスト, マークアップ言語
.jpg, .png, .bmp 等	画像データ
.mp3, .m4a, .wma 等	音声データ
.mpg, .mp4, .wmv 等	動画データ
.pdf, .ps, .eps 等	印刷・表示用の文書ファイル
.zip, .tar, .tbz 等	アーカイブファイル
.exe, .app, 拡張子無し	実行形式プログラム (Windows, macOS, UNIX)
.doc, .docx	MS Word 文書

表 14.2: 求められるディレクトリ操作

機能	対応する UNIX の API
ファイルの作成	creat, open(... O_CREAT ...) システムコール
ディレクトリの作成	mkdir システムコール
ファイルの削除	unlink システムコール
ディレクトリの削除	rmdir システムコール
リンクの作成	link, symlink システムコール
リンクの削除	unlink システムコール
名前の変更 (移動)	rename システムコール
ディレクトリエントリの読出し	opendir, readdir, closedir 関数

で拡張子は単にファイル名の一部である^{*13}。表 14.1 によく使う拡張子と意味をまとめる^{*14*15}。

14.7 ファイルシステムの操作

ユーザがファイルを作ったり、ファイルのデータを読み書きするために必要な操作を紹介する。

14.7.1 ディレクトリ操作

ユーザがファイルやディレクトリを自由に作ったり削除したりするために、表 14.2 に示すディレクトリ操作ができることが求められる。表の右半分は UNIX で使用可能な API の例を示している。

14.7.2 ファイルアクセス

ユーザがファイルの内容や属性を読み書きするために表 14.3 の操作ができることが望ましい。

オープン open システムコールは、ファイルのパスとファイルに行う操作 (読む・書く) 等を引数に発行される。ファイルの保護属性と照らし合わせ、要求された操作が可能な場合のみファイルをオープンする。

読み書き ファイルをオープンした後、read/write システムコールを使用してファイルを先頭から最

^{*13} FAT ファイルシステムでは拡張子が特別扱いされている。

^{*14} 表中 .app 拡張子だけは macOS でディレクトリ名に付加される。

^{*15} 表の最初の 3 行 (.c から .xml) と .ps, .eps はテキストファイルの一種である。

表 14.3: 求められるファイル操作

機能	対応する UNIX の API
ファイルを開く	open システムコール
データを読む	read システムコール
データを書く	write システムコール
読み書き位置を移動	lseek システムコール
ファイルを閉じる	close システムコール
ファイルの切り詰め	truncate, open(... O_TRUNC) システムコール
ファイルのプログラムを実行	execve システムコール
ファイルの属性変更	chmod, chown, chgrp, utimes システムコール
ファイル属性の読出し	stat システムコール

後に向けてシーケンシャルアクセスすることができる。読み書き位置を自由に変更できる lseek システムコールを組合せることで、read/write システムコールはランダムアクセスにも使用できる。

クローズ close システムコールを用いてファイルをクローズする。ファイルがオープンされている間は、ファイル本体のリンクカウントが 1 増加したのと同じ状態になる。ファイルを削除しディレクトリからファイルが見えなくなってもファイルの本体は削除されない。ファイルを消してもディスクの空き領域が増えない場合は、どこかプロセスがファイルをオープンしている可能性がある。

切り詰め truncate システムコールや O_TRUNC フラグ付きで実行した open システムコールは、ファイルの長さを短く切り詰める。

プログラムの実行 execve システムコールはファイルに格納されているプログラムを実行する。ファイルのフォーマットは execve システムコールが理解できるものである必要がある^{*16}。

属性の読み書き chmod システムコール等でファイルの属性を書き換えることができる。また、stat システムコールはファイル属性の読み出しに使用できる。

14.7.3 ファイルの共有とロック

ファイルを複数のプロセスで安全に共有するためにロックのメカニズムが求められる。「5.4.5 リーダ・ライタ問題」でも紹介した**共有ロック** (*shared lock*) と**排他ロック** (*exclusive lock*) をファイルに掛ける UNIX の仕組みを紹介する。

UNIX ではファイルをロックするために flock システムコールが準備されている。flock は、引数に定数 LOCK_SH を渡すと共有ロックを、定数 LOCK_EX を渡すと排他ロックをファイルに掛ける。共有ロックは複数のプロセスが同時に掛けることができる。排他ロックはファイルが全くロックされていない場合のみ掛けることができる。排他ロックされている間は、他のプロセスはどちらのロックも掛けることができなくなる。ロックが掛けられない時、flock システムコールがブロックしないようにするには、上記の定数に LOCK_NB フラグを（ビット毎の論理和で）合わせて flock に渡す。以下に macOS の flock の書式を示す。

^{*16} ファイルは UNIX の実行可能な機械語形式かインタプリタに渡すデータである。ファイルの先頭が#!path で始まる場合は path で指定されたインタプリタを起動し、ファイルの内容を解釈・実行させる。


```
#include <sys/file.h>
#define LOCK_SH 1 // 共有ロック
#define LOCK_EX 2 // 排他ロック
#define LOCK_NB 4 // ブロックしない
#define LOCK_UN 8 // ロック解除
int flock(int fd, int operation);
```

また、open システムコールを使用してファイルにロックを掛けることもできる。open システムコールは、引数に O_SHLOCK フラグを指定すると共有ロックを、引数に O_EXLOCK フラグを指定すると排他ロックを、ファイルのオープン時に自動的に掛ける。

14.7.4 ワーキングディレクトリの変更

ワーキングディレクトリ（カレントディレクトリ）はプロセス毎に決められるので、プロセスの操作に分類するほうが正しいかもしれないがここで紹介しておく。UNIX では chdir システムコールを用いてプロセスが自身のワーキングディレクトリを変更する。初期のワーキングディレクトリは親プロセスから引き継がれる。以下に chdir の書式を示す。

```
#include <unistd.h>
int chdir(const char *path);
```

14.8 ファイルシステムの健全性

停電、OS のクラッシュ、ハードウェアの故障等により、ファイルシステムが壊れてしまうことがある。ファイルシステムの一貫性をチェックし必要に応じて修復する方法と、壊れ難いファイルシステムについて紹介する。

14.8.1 一貫性チェック

コンピュータが異常停止してしまった場合、次の起動時にファイルシステムの一貫性をチェックする。例えば UNIX では、正常なシステム終了時にはファイルシステムに「正常にファイルシステムがアンマウントされた」印が残る。次のシステム起動時に、印が付いていないファイルシステムについて fsck^{*17} コマンドが自動的に実行される。

fsck コマンドは、使用中の i-node やディレクトリの内容等を突き合わせ矛盾がないか確認する。例えば、使用中の i-node がどのディレクトリからも参照されていない（リンクカウントが間違っている）、同じデータブロックが複数の i-node から参照されている等の矛盾が予想される。fsck コマンドは、チェック結果からファイルシステムの修復を行う。

この方式はメタデータ^{*18}の矛盾を解消するが元通りにするわけではない。メタデータに矛盾があったファイルやディレクトリが失われたり、更新したはずのファイルが更新途中の状態になったりする可能性がある。また、fsck が終了するまで^{*19}システムが起動しない。

^{*17} UNIX の fsck にあたるコマンドは、Windows では chkdsk や scandisk、macOS では Disk First Aid 等である。

^{*18} ファイルシステムの構造を管理するデータのこと。FAT ファイルシステムのディレクトリや FAT、UNIX の i-node やディレクトリエントリ等が該当する。

^{*19} 一貫性のチェックには数分かかる場合も多い。その間、システムが使用できない。

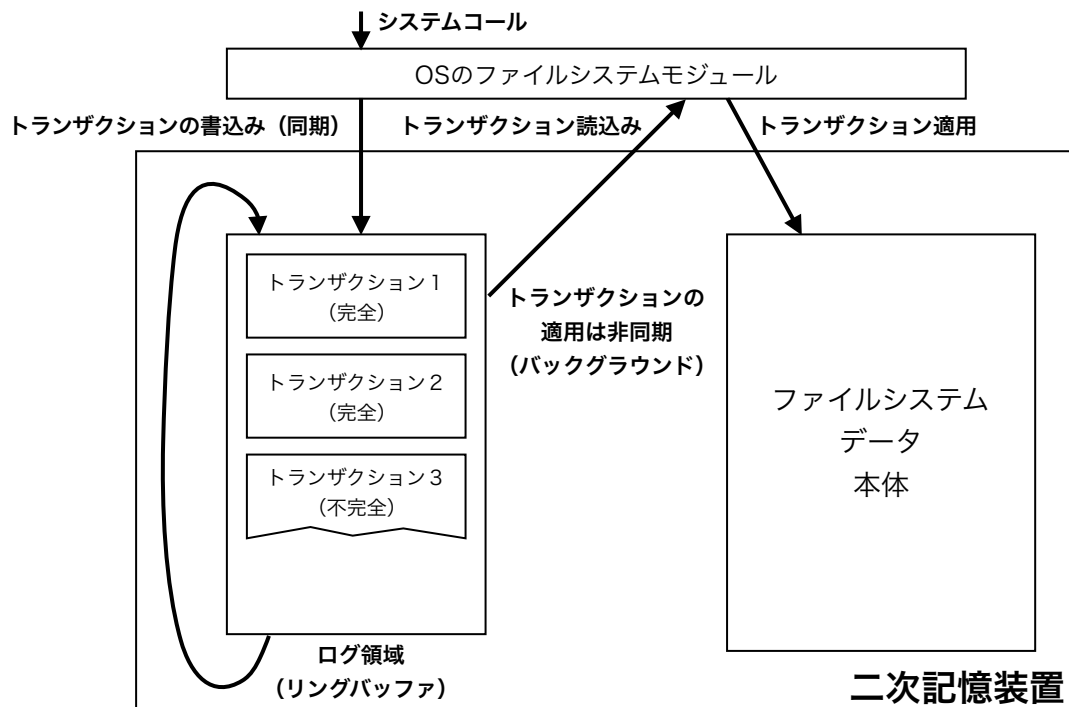


図 14.3: ジャーナリングファイルシステムの仕組み

14.8.2 ジャーナリング・ファイルシステム

データベースで使用された WAL (Write Ahead Logging) をファイルシステムに応用したものである。NTFS^{*20}、ext3、ext4^{*21}、HFS+^{*22}等はジャーナリングファイルシステムである。

システムがクラッシュした後も、ジャーナリングファイルシステムの状態は、システムコールが完了した後かシステムコールを実行し始める前か、どちらかに落ち着く。システムコールを実行する途中の中途半端な状態になりファイルシステムが壊れることはない。図 14.3 にジャーナリングファイルシステムの仕組みを、以下におおよその動作原理を示す。

1. システムコールによる一連の操作はトランザクションとしてログ領域^{*23}に記録する。
2. トランザクションの書き込み完了でシステムコールは完了し、ユーザプロセスは次の処理を開始できる。
3. OS はバックグラウンド処理でログ領域からトランザクションを順に取り出し、ファイルシステム本体に適用する。
4. システムがクラッシュした場合、ログ領域への書き込みが完了していたトランザクションは再実行しファイルシステム本体に完全に反映する。書き込み途中だったトランザクションは無視する。

^{*20} Windows のファイルシステムである。

^{*21} Linux のファイルシステムである。

^{*22} macOS のファイルシステムである。

^{*23} 同一ディスクの別領域の場合と別ディスクの場合がある。

トランザクションはログ領域にシーケンシャルに書き込まれる。ファイルシステム本体の操作はランダムアクセスが必要なので時間がかかるが、シーケンシャルアクセスだけで完了するトランザクション書込みは短時間に終わる。システムコールを短時間に終わらせることができる。

なお、ログ領域を通して操作するのはメタデータだけのシステムが多い。ファイルシステムの構造が不整合を起こすことは予防できるが、ファイル内のデータを守ることはできない。

14.9 まとめ

この章ではファイルシステムが備えるべき機能や概念に付いて学んだ。ファイルシステムは、二次記憶装置を抽象化・仮想化したファイルユーザに提供するオペレーティングシステムの仕組みである。本章の多くの部分で UNIX（または macOS）を例にしたが、基本は Windows 等でも共通である。

現代のオペレーティングシステムは、ファイルを木構造のディレクトリシステムに格納する方式を採用している。ディレクトリシステムから一つのファイルを選択するためにパスを用いる。パスにはルートディレクトリを起点とする絶対パスと、ワーキングディレクトリを起点とする相対パスがある。

ファイルに別名を付ける方法として、ハードリンク、シンボリックが知られている。これらはファイルシステムの仕組みとして実装され、オペレーティングシステムのカーネル内で処理される。これらの他に、ファイルシステムの外で実装される別名の仕組みもある。Windows のショートカットや macOS のエイリアスがそれに該当する。

複数のボリュームがある時、二つ目以降のボリュームを既存のディレクトリに接続（マウント）する方式をマウント方式と呼ぶ。一方で、ボリューム毎にボリュームを表す文字を決めて区別する方式をドライブレター方式と呼ぶ。

ファイルは、保護、最終変更時刻、所有者等の属性を持つ。本章では代表的な属性を紹介した。また、ファイルは保護属性に基づいたアクセス制御ができることが望ましい。ファイルはファイルシステムが定めた型（タイプ）を表す属性を持っている。この属性により、通常ファイル、ディレクトリ、シンボリックリンク等を区別する。ファイルが、どのアプリケーションのデータを格納しているかは、ファイル名の一部（拡張子）で区別する。

ファイルシステムは、ファイルの読み・書き、ファイルの作成・削除など、いくつかの基本的な機能を備える必要がある。また、これらの機能をユーザプログラムが呼び出して使用できるようにシステムコールを提供する必要がある。本章では代表的な操作（システムコール）を紹介した。

練習問題

14.1 次の言葉の意味を説明しなさい。

- (a) ディレクトリシステム
- (b) パス、絶対パス、相対パス
- (c) ディレクトリ、ファイル
- (d) ハードリンク、シンボリックリンク
- (e) ショートカット、エイリアス
- (f) マウント、ドライブレター
- (g) 拡張属性、ACL

- (h) 拡張子
- (i) 共有ロック・排他ロック
- (j) 一貫性チェック
- (k) ジャーナリング・ファイルシステム

14.2 自分が使用しているオペレーティングシステムについて調査しなさい.

(GUIではなく、CLIのコマンドを用いるとより詳しい観察ができる場合がある.)

- (a) ショートカット (Windows), エイリアス (macOS)
- (b) ファイルの属性 (保護, 日時, 所有者, サイズ等)
- (c) 拡張属性が使用できるオペレーティングシステムか?
- (d) ACL が使用できるオペレーティングシステムか?
- (e) USB メモリにはどのようなパスで到達できるか?
- (f) ファイルシステムの一貫性をチェックするコマンドは何か?

14.3 自分が使用しているオペレーティングシステムで試してみなさい.

- (a) ショートカット (Windows) やエイリアス (macOS) を作成し, 予定通りに働くか GUI と CLI の両方で試してみなさい.

```

1  # macOSの場合の実行例
2  $ echo aaa > a.txt
3  $ open a.txt
4  $ open a.txtのエイリアス      <--- エイリアスはGUIで作る
5  $ cat a.txt
6  $ cat a.txtのエイリアス

```

- (b) UNIX や macOS で実行して結果が異なる理由を考察しなさい.

# ハードリンクの場合	# シンボリックリンクの場合
\$ echo aaa > a.txt	\$ echo aaa > a.txt
\$ echo bbb > b.txt	\$ echo bbb > b.txt
\$ ln a.txt c.txt	\$ ln -s a.txt c.txt
\$ mv a.txt d.txt	\$ mv a.txt d.txt
\$ mv b.txt a.txt	\$ mv b.txt a.txt
\$ cat c.txt	\$ cat c.txt

- (c) ショートカットやエイリアスの振る舞いを調べる.
(リンク先ファイルを削除・移動・別ファイルに置換えた場合など)
- (d) ACL の追加・削除とその効果を確認する.

第 15 章

FAT ファイルシステム

ファイルシステムの事例として FAT ファイルシステムを紹介する。FAT ファイルシステムは、MS-DOS（1981 年）から Windows ME（2000 年）までで、OS のシステムディスクに使用された。それ以降の Windows ではシステムディスク用には使用されていないが、仕様が公開^{*1}されているので、USB メモリやメモ리카ードのファイルシステムとして広く使用されている。

15.1 特徴

PC だけでなく、様々な電子機器でも用いられ、広く普及しているファイルシステムである。Windows, Linux, macOS 等も FAT ファイルシステムをサポートしている。USB メモリ等を用いて、異なるオペレーティングシステムや電子機器間でファイルをやり取りできるは、そのおかげである。PC 以外でも音楽プレーヤ、デジカメ、デジタルテレビ、カーナビ、電子楽器、計測機器等が、データの記録やファームウェアのバージョンアップ用にサポートしている。

ファイル名は、半角 8 文字に加え 3 文字の拡張子を合わせた最大 11 文字で表現する。英字のアルファベットは大文字のみ使用できる。デジカメの写真データが IMG_1234.JPG^{*2}のようなファイル名になっているのは、FAT ファイルシステムの仕様に合わせたためと考えられる。

表 15.1 のような、四種類の FAT ファイルシステムがある。FAT12, FAT16, FAT32 の三つは仕様が無料で入手できるが exFAT はそうではない。また、VFAT と呼ばれる規格と合わせて使用すると長いファイル名が使用できる。以下では VFAT を含まない FAT16 の場合を中心に述べる。

表 15.1: FAT ファイルシステムの種類

種類	最大ボリュームサイズ	最大ファイルサイズ	ファイル名
FAT12	32MiB	32MiB	8+3 文字
FAT16	2GiB	2GiB	8+3 文字
FAT32	2TiB	4GiB	8+3 文字
exFAT	16EiB	16EiB	255 文字

^{*1} <http://www.microsoft.com/whdc/system/platform/firmware/fatgen.mspx> 等から入手できる。

^{*2} キヤノンのデジカメはこのような名前を使用する。

BPB (512 バイト)
FAT1
FAT2
ルートディレクトリ
データ領域

図 15.1: FAT ボリュームの構造

フォーマットに関する パラメータ (62 バイト)
ブートプログラム (448 バイト)
シグネチャ (2 バイト)

図 15.2: BPB (合計 512 バイト)

15.2 ボリューム内部の配置

FAT ファイルシステムは、ボリューム（パーティション）内部に図 15.1 のように配置される。ここで各領域の意味は次の通りである。

- *BPB (BIOS Parameter Block)*

ボリュームの先頭セクタに配置される。内容は図 15.2 に示すように、FAT ファイルシステムを初期化する時に決めたパラメータと、ブートプログラムである。表 15.2 に BPB に格納される主要な情報を示す。この表に掲載した値の例は、約 2GiB のボリュームをクラスタサイズ 32KiB の FAT16 フォーマットに初期化した例である。MBR のブートプログラムは BPB をロードし実行する。BPB の先頭には初代 PC の CPU である Intel 8086 の JMP 機械語命令が置いてあり、BPB の後半に格納されたブートプログラムへジャンプする。

- *FAT (File Allocation Table)*

FAT ファイルシステムにとって大変重要なデータなので多重化してある。FAT の数（2 重化なら 2）、FAT のサイズ（FAT あたりのセクタ数）は BPB から知ることができる。

- *ルートディレクトリ*

FAT 領域の直後に固定領域が確保される。ルートディレクトリ以外のディレクトリはデータ領域にファイルとして記録される。ルートディレクトリサイズは BPB の rootDir サイズにエントリ数として記録されている。1 エントリは 32 バイトなので、表 15.2 の例ではルートディレクトリサイズは $32B \times 512 = 16KiB$ になる。これをセクタ数に換算すると $16KiB \div 512B = 32$ セクタ になる。

- *データ領域*

ボリュームの残り領域はファイルの内容データを記録するために使用される。データ領域のセクタは**クラスタ**と呼ばれるブロックで扱われる^{*3}。なお、データ領域に配置されるクラスタの番号は 2 から始まる。

^{*3} 表 15.2 の例では 1 クラスタを 64 セクタ (32KiB) で構成している。この例では 1 バイトのファイルでも 32KiB のデータ領域を使用することになりセクタを無駄遣いするが、扱えるボリュームサイズを大きくするために、このようになっている。

表 15.2: BPB に格納される主要な情報

パラメータ	意味	位置	長さ	値の例
ジャンプ命令	ジャンプ機械語命令	0	3	0xeb 0x3e 0x90
セクタサイズ	1 セクタのバイト数	11	2	512 バイト
クラスタサイズ	1 クラスタのセクタ数	13	1	64 セクタ
予約セクタ数	予約セクタ数 (BPB を含む)	14	2	1 セクタ
FAT 数	FAT を何重に記録するか	16	1	2 個
rootDir サイズ	ディレクトリエントリ数	17	2	512 エントリ
総セクタ数 16	ボリュームのサイズ	19	2	0
FAT サイズ	FAT のセクタ数	22	2	245 セクタ
総セクタ数 32	ボリュームのサイズ	32	4	3,999,681 セクタ
ボリュームラベル	ボリュームの名前	43	11	"MICRODRIVE"
ブートプログラム	ブートプログラム	62	448	
シグネチャ	フォーマット済みマーク	510	2	0x55 0xaa

(位置と長さの単位はバイト)

(値の例はボリュームサイズ 2GiB, クラスタサイズ 32KiB, FAT16 の場合)

('総セクタ数 16' で表現できない場合は '総セクタ数 32' を使用する)

Bytes	8	3	1	10	2	2	2	4
	FileName	Ext	Atr	Reserved	Time	Date	Cls	Size

図 15.3: ディレクトリエントリ

15.3 ディレクトリエントリ

ルートディレクトリとディレクトリファイルで共通な 32 バイトのデータ構造が使用される。図 15.3 にディレクトリエントリの構造を図示する。各部の意味は次の通りである。

- FileName は 8 文字以内のファイル名である。

左づめで格納し、余ったバイトはスペース (0x20) で埋める。FileName の第一バイトが 0x00 の場合は、そのエントリーと以降のエントリーが使用されていないことを表す^{*4}。0xe5 の場合は、エントリーが削除されていることを表す。0x05 の場合は、本来の第一バイトが文字コード 0xe5 であることを表す。リスト 15.1 に TacOS のソースプログラム^{*5}中で、エントリからファイル名を読む部分を示す。文字コード 0x05 の扱いを確認してほしい。

- Ext は 3 文字以内のファイル名の拡張子である。
- Atr にはファイルの属性を格納する。

read-only (0x01), hidden (0x02), system-file (0x04), archive (0x20), directory (0x10) 等の属性がある。directory はそのファイルがディレクトリファイルであることを、archive は前回のバックアップ後にファイルが変更されたことを表す。ビットを組合せて属性を表現する。例えば

^{*4} ディレクトリファイルの EOF を表現している。

^{*5} <https://github.com/tctsigemura/TacOS/blob/master/os/fs/dirAccess.cmm>

リスト 15.1: ファイル名を読み出すプログラム

```
// readFname : キャッシュから DIR エントリの名前("abcdefgh.txt"形式)を読み出す
// 引数 s : 抜き出した名前を格納する領域
// ofs : DIR エントリの位置
void readFname(char[] s, int ofs) {
    for (int i=0; i<11; i=i+1) // ファイル名を抜き出す
        s[i] = dirCache[ofs+i]; // 1文字ずつコピーする
    if (s[0]!='\x05') s[0]='\xe5'; // 本当はSJIS漢字コードの0xe5
    s[11] = '\0'; // C++ 文字列として完成させる
}
```

0x03 は、読み出し専用の隠しファイルの意味になる。

- Reserved は未使用の領域である。
- Time はファイルの最終変更時刻を 2 秒の精度で表現する^{*6}。
- Date はファイルの最終変更日を表現する^{*7}。
- CIs はファイルのデータが格納されている先頭クラスタの番号である。
- Size はバイト単位で表したファイルのサイズである。ディレクトリファイルの場合は 0 にする。

15.4 FAT (File Allocation Table)

FAT は、図 15.4 のような表である。図 15.5 のように、FAT のエントリとデータ領域のクラスタが、一対一に対応する。エントリには、ファイル中で次のクラスタの番号を格納する。同一ファイルのデータ領域は、FAT 中に表現されたクラスタのチェーンにより辿ることができる。エントリが 12 ビットの場合を FAT12、16 ビットの場合を FAT16、28 ビットの場合を FAT32 と呼ぶ。以下では *FAT16* の場合を例に説明する。

• エントリ値の意味

FAT エントリに書き込まれる値の意味を表 15.3 にまとめる。0x0000 は、そのエントリが使用されていないことを表す。0x0001 は、何かの意味を割当てるために予約されている^{*8}。0x0002～0xffff6 が普通のクラスタ番号である。よって、最大 65,525 個^{*9}のクラスタに番号を付けることができる。0xffff7 は不良クラスタ^{*10}を表す。0xffff8～0xfffff は、クラスタチェーンの終わり（ファイルの終わり）を表している。FAT エントリの 0x0000 が未使用クラスタ（空きクラスタ）を表すので、FAT は空き領域管理の役割も担っている。

• クラスタチェーン

^{*6} 下位ビットから順に、秒 ÷ 2 (5 ビットなので 2 秒単位)、分 (6 ビット)、時 (5 ビット) で表現する。

^{*7} 下位ビットから順に、日 (5 ビット)、月 (4 ビット)、年 - 1980 (7 ビット) で表現する。2108 年問題を含んでいる。

^{*8} TacOS の内部処理では、ルートディレクトリのクラスタ番号として利用している。

^{*9} $0xFFFF6 - 1 = 0xFFFF5 = 65,525$

^{*10} 何かの理由で正しく読み書きできないセクタが含まれるクラスタのこと。

0	0x0000	← 使用不可
1	0x0000	← 使用不可
2	0x0004	← 次は第4 クラスタ
3	0xffff7	← 不良クラスタ
4	0x0005	← 次は第5 クラスタ
5	0xfffff	← 終了クラスタ
6	0x0000	← 未使用クラスタ
...	...	

図 15.4: FAT の仕組み

表 15.3: FAT エントリ値の意味

値	意味
0x0000	未使用クラスタ
0x0001	予約クラスタ
0x0002 ~ 0xffff6	普通のクラスタ
0xffff7	不良クラスタ
0xffff8 ~ 0xfffff	終了クラスタ

FAT エントリに**次クラスタの番号**を書き込むことにより、**クラスタチェーン**を作る。例えば図 15.4 は、第 2、第 4、第 5 クラスタからなるチェーンを含んでいる。第 5 クラスタの 0xffff はチェーンの終わりを表している。一つのチェーンがファイルに属するクラスタのリストを表現している。チェーンの先頭（ファイルの先頭）はディレクトリエントリの CIs から分かる。

FAT ファイルシステムはクラスタチェーンを作るので、データブロックを**リンク方式**で管理していると言える。FAT を順に調べることでランダムアクセスができるが、**順に調べる必要がある**。

15.5 ディレクトリファイル

ルートディレクトリ以外の（サブ）ディレクトリは、Atr の directory (0x10) 属性が ON になったファイルである。これを**ディレクトリファイル**と呼ぶ。ディレクトリファイルにはルートディレクトリと同じ形式のディレクトリエントリを記録する。ディレクトリファイルもクラスタチェーンでデータ領域を割り付けられるので、表 15.2 の構成なら最低でも 32KiB の大きさになる。

リスト 15.2 に、macOS で FAT16 ファイルシステム上にディレクトリ A を作成し、ディレクトリファイル A を 16 進ダンプした例を示す。00000080 以降は全てのデータが 00 なので*が表示され省略されているが、最後の行の 00008000 からファイルサイズが 32KiB であることが分かる。また、00000000 からカレントディレクトリ「.」を表すエントリ、00000020 から親ディレクトリ「..」を表すエントリが格納されている。00000040 からディレクトリ DIR を表すエントリ、00000060 からファイル A.TXT を表すエントリが格納されている。図 15.3 と比較しながら解析して欲しい。

15.6 FAT ファイルシステムの全体像を示す例

表 15.2 に例示したパラメータで初期化された FAT ファイルシステムに 65KiB の \ABCDEFGH.TXT と、1KiB の \SAMPLE.DAT の二つのファイルが書き込まれた状態を図 15.5 に示す。

• ルートディレクトリ

表 15.2 の例ではルートディレクトリは 512 エントリからなり、32 セクタ使用することになっている。リスト 15.2 ではカレントディレクトリ「.」と親ディレクトリ「..」が格納されていたが、ルートディレクトリには存在しない。図 15.5 では、先頭の 2 エントリを用い 2 つのファイルが登録されている。通常ファイルなので Atr は 0x00 である。最終変更日時は 1980 年 1 月 1 日 0 時を

リスト 15.2: FAT ファイルシステムのディレクトリの 16 進ダンプ結果

```

$ cd /Volumes/NO\ NAME
$ mkdir A
$ mkdir A/DIR
$ echo AAA > A/A.TXT
$ hexdump -C A
00000000  2e 20 20 20 20 20 20 20 20 20 20 30 00 aa 7d 98  |.          0..}.|
00000010  e4 4c e4 4c 00 00 a9 98 e4 4c 21 00 00 00 00 00  |.L.L....L!....|
00000020  2e 2e 20 20 20 20 20 20 20 20 20 10 00 aa 7d 98  |..          ...}.|
00000030  e4 4c e4 4c 00 00 7d 98 e4 4c 00 00 00 00 00 00  |.L.L..}..L.....|
00000040  44 49 52 20 20 20 20 20 20 20 20 10 00 31 a2 98  |DIR          ..1..|
00000050  e4 4c e4 4c 00 00 a2 98 e4 4c 31 00 00 00 00 00  |.L.L....L1.....|
00000060  41 20 20 20 20 20 20 20 54 58 54 20 00 13 a9 98  |A          TXT ....|
00000070  e4 4c e4 4c 00 00 a9 98 e4 4c 40 00 04 00 00 00  |.L.L....L@.....|
00000080  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |.....|
*
00008000

```

表す値になっている。C1s に格納された値が FAT 上のクラスタチェーン開始エントリを指している。Size は 65KiB を表す 0x00010400 と 1KiB を表す 0x00000400 になっている。3 つ目のエントリ以降は使用されていないので FileName の第 1 バイトが空きを意味する 0x00 になっている。

- *FAT*

表 15.2 の例では、FAT は 245 セクタ使用することになっている。セクタサイズが 512 バイト、FAT エントリは 16 ビット (2 バイト) なので 1 セクタに 256 エントリ格納できる。FAT 全体のエントリ数は $245 \times 256 = 62,720$ になり、FAT のエントリ番号は 0 から 62,719 の範囲である。

ルートディレクトリの ABCDEFGH.TXT エントリから、このファイルは第 2 クラスタから始まるクラスタチェーンに格納されることが分かる。FAT の内容を確認すると第 2, 第 4, 第 5 クラスタからなる長さ 3 のチェーンになっているので、ABCDEFGH.TXT ファイルのデータは、これら 3 クラスタを使用して格納される。3 クラスタの合計容量は 96KiB なので 65KiB のファイルを格納するには十分である^{*11}。

ルートディレクトリの SAMPLE.DAT エントリから、このファイルは第 6 クラスタから始まるクラスタチェーンに格納されることが分かる。FAT の内容を確認すると第 6 クラスタがチェーンの最終クラスタなのでチェーンの長さは 1 である。SAMPLE.DAT ファイルのデータは第 6 クラスタに格納される。

- *データ領域*

表 15.2 の例では、ボリューム全体で 3,999,681 セクタである。BPB (1 セクタ), FAT1 (245 セクタ), FAT2 (245 セクタ), ルートディレクトリ (32 セクタ) であるので、残り $3,999,681 - 1 - 245 \times 2 - 32 = 3,999,158$ セクタがデータ領域として使用できる。

^{*11} 2 クラスタでは 64KiB までしか格納できない。

クラスタサイズは 64 セクタなので、データ領域は $3,999,158 \div 64 = 62,486$ クラスタ（余り 54 セクタ^{*12}）になる。データ用クラスタの番号は 2 番から始めていることになっている（表 15.3 参照）ので、2 番から 62,487 番までがデータ領域のクラスタ番号になる。FAT エントリは 62,719 番まで用意されているが、62,488 番から 62,719 番は使用されない。

ABCDEFGH.TXT ファイルのクラスタチェーンは、第 2、第 4、第 5 クラスタなので、ファイルデータが先頭から 32KiB ずつ第 2、第 4 クラスタに置かれる。第 5 クラスタは、先頭にファイル末尾の 1KiB が置かれ残り 31KiB は使用されない。

15.7 実装例

第 23 章に TacOS のファイルシステムサーバ (fs) の実装例を示す。この実装例は、FAT16 専用のファイルシステム管理プログラムを C++ 言語で記述したものである。

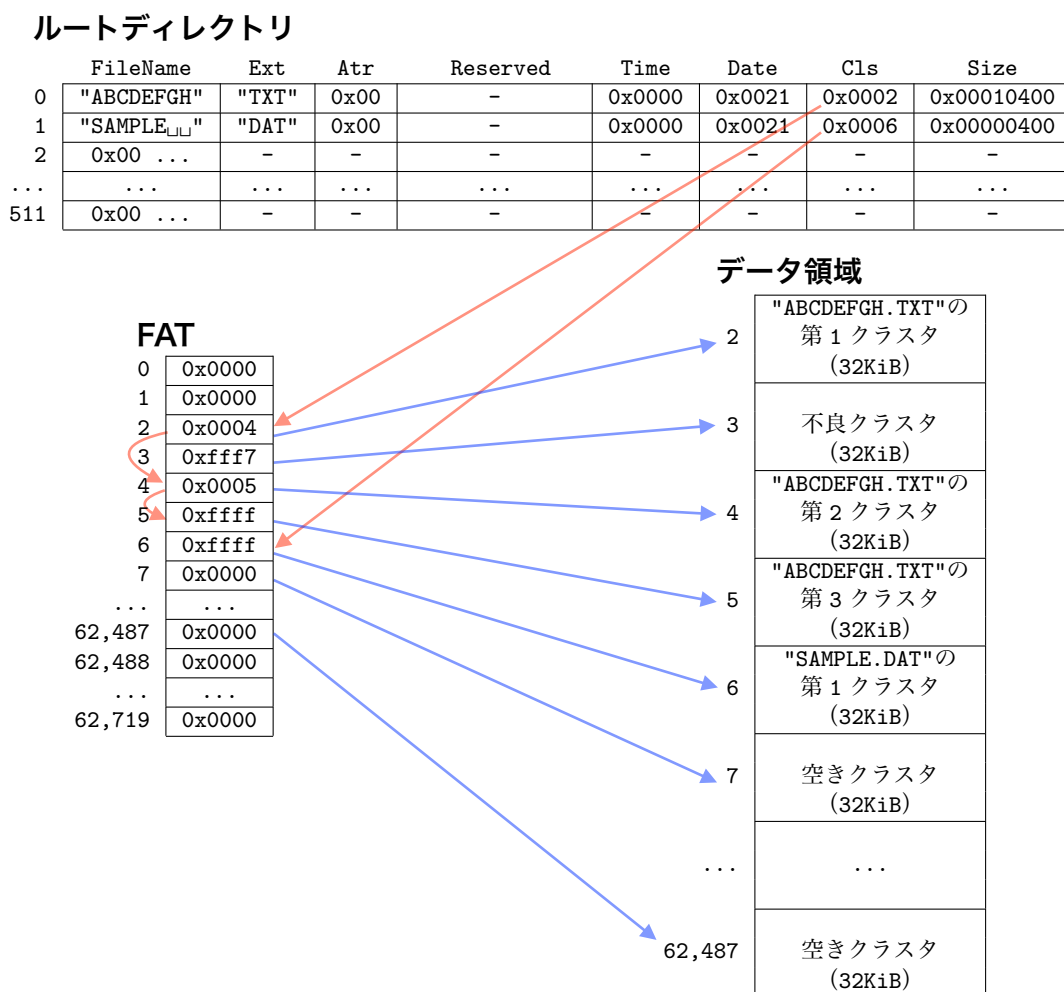


図 15.5: FAT ファイルシステムにファイルを格納した例

*12 余り 54 セクタは使用できない。

15.8 まとめ

FAT ファイルシステムは、USB メモリやメモリカードで使用されている。多くのオペレーティングシステムや電子機器が FAT ファイルシステムをサポートしているので、オペレーティングシステムや機器の種類を越えてデータを交換するために盛んに使用されている。

この章では特に FAT16 ファイルシステムについてかなり踏み込んだ解説を行った。ボリューム内部の配置、ディレクトリエントリ、FAT、ディレクトリファイルを具体的な値を示しながら説明した。また、第 23 章には TacOS の FAT16 ファイルシステムサーバの実装例を掲載している。

練習問題

15.1 次の言葉の意味を説明しなさい。

- (a) BPB
- (b) ルートディレクトリ
- (c) クラスタ
- (d) ディレクトリエントリ
- (e) FAT
- (f) クラスタチェイン
- (g) ディレクトリファイル

15.2 リスト 15.2 を図 15.3 と比較しながら解析しなさい。

15.3 図 15.5 の ABCDEFGH.TXT ファイルの第 0x00002000 バイトが格納されるクラスタの番号を答えなさい。

15.4 前問と同様に、第 0x00004000, 0x00008000, 0x00010000 バイトに付いて答えなさい。

第 16 章

UNIX ファイルシステム

ファイルシステムの事例として *UFS (UNIX File System)* を紹介する。UFS は UNIX で使われてきたファイルシステムを指し様々なバージョンがある。階層構造のディレクトリシステムを持ち、マルチユーザで利用できるファイルシステムである。ここでは、UFS の概念が分かりやすいように、UFS の特徴を表す仕組みを特定のバージョンに拘らずに紹介する^{*1}。

16.1 概要

UFS は、1979 年にリリースされた Version 7 Unix のファイルシステムと、それを改良した多くのファイルシステムのことである。第 14 章では、木構造のディレクトリシステム、ハードリンク、シンボリックリンク、ボリュームをマウントする方式、ファイルの属性、ファイルシステムの操作等で「UNIX の場合」を基本に解説を行ったが、「UNIX の場合」とは「UFS の場合」であった。

また、Windows の NTFS、macOS の HFS+ や APFS、Linux の ext3 や ext4 等のファイルシステムは、ハードリンクや、ファイル名の大小文字の区別、アクセス権限等で、UFS と同じ構造を持っているようにユーザに見せることができる。

16.2 ボリューム内部の配置

UFS ボリューム（パーティション）の内部は、例えば図 16.1 のような配置になっている。この例は、1 セクタ 512 バイト、1 ブロック 16 セクタ (8KiB)、i-node サイズ 128 バイトのものである。

- **ブートブロック**

ブートプログラムが格納される。PC の場合、ブートブロックのブートプログラムは、MBR のブートプログラムによってロード実行される。

- **スーパーブロック**

ボリュームのサイズ、ブロックのサイズ、i-node リストのサイズ等、ファイルシステムを初期化した時に決めたパラメータや、最終変更日時、最終マウントポイント、空きブロック数等の運用中に変更される値が格納される。「ファイルシステムが正常にアンマウントされた」印^{*2}もスーパー

^{*1} ここで説明していることの多くは概念である。実際の構造はバージョンによっては全く異なる場合もある。

^{*2} 「14.8.1 一貫性チェック」で紹介した。

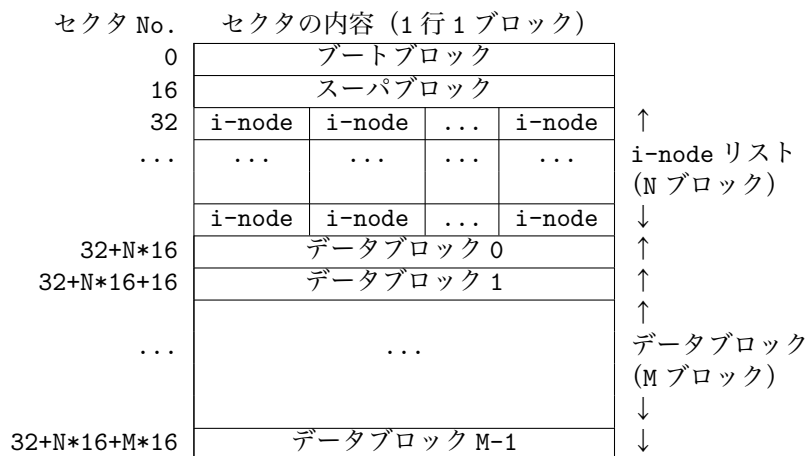


図 16.1: UFS ボリュームの構造

ブロックに含まれる。

- **i-node リスト**

図 16.1 の例では、128 バイトの i-node を 512 バイトのセクタに 4 個格納している。16 セクタのブロックには $4 \times 16 = 64$ 個の i-node が格納できる。N ブロックの i-node リスト領域全体で $N \times 64$ 個の i-node が格納される。1 つの i-node が 1 つのファイルを管理するので、初期化時に決定した i-node リストの大きさによって、このファイルシステムに作成できるファイルの最大数が決まる。

- **データブロック**

ファイルのデータ本体を格納する領域である。データブロック領域全体では $M \times 16$ セクタを使用している。

16.3 i-node (index node)

1 つの i-node が 1 つのファイルを管理する。図 16.2 に i-node の構造を簡単に表したものを示す^{*3}。

- **タイプ・モード**

タイプ・モードは type/sst/rwxrwxrwx の 16 ビットから構成される。type 4 ビットでファイルの型を表現する。ファイルの型には、通常ファイル、ディレクトリ、シンボリックリンク、キャラクタ型デバイス、ブロック型デバイス、パイプ、ソケット等がある。

ss の 2 ビット (Set-uid, Set-gid) は、ファイルにプログラムが格納されている場合、プログラムがファイル所有者やグループの権限で実行されることを表す。例えば macOS の /bin/ps プログラムは、プロセス情報を収集するためにシステム管理者の権限で実行される必要があるため、これら

^{*3} 図 16.2 は、FreeBSD4 のソースコード <https://github.com/freebsd/freebsd/blob/stable/4/sys/ufs/ufs/dinode.h> を参考に、一部を簡単化して描いた。

i-node

<----- 32 ビット ----->		
タイプ・モード／リンク数	タイプ・モード	ハードリンク数
ファイルサイズ	バイト単位 (64 ビット)	
ファイルアクセス時刻	1970/1/1 からの経過秒数 小数点以下 (ns 単位)	
ファイル変更時刻	1970/1/1 からの経過秒数 小数点以下 (ns 単位)	
i-node 変更時刻	1970/1/1 からの経過秒数 小数点以下 (ns 単位)	
直接ブロック [0]	ブロック番号	
直接ブロック [1]	ブロック番号	
直接ブロック [2]	ブロック番号	
直接ブロック [3]	ブロック番号	
直接ブロック [4]	ブロック番号	
直接ブロック [5]	ブロック番号	
直接ブロック [6]	ブロック番号	
直接ブロック [7]	ブロック番号	
直接ブロック [8]	ブロック番号	
直接ブロック [9]	ブロック番号	
直接ブロック [10]	ブロック番号	
直接ブロック [11]	ブロック番号	
1 重間接ブロック	ブロック番号	
2 重間接ブロック	ブロック番号	
3 重間接ブロック	ブロック番号	
所有者 ID	ユーザ番号	
グループ ID	グループ番号	
* 以下省略 *	...	

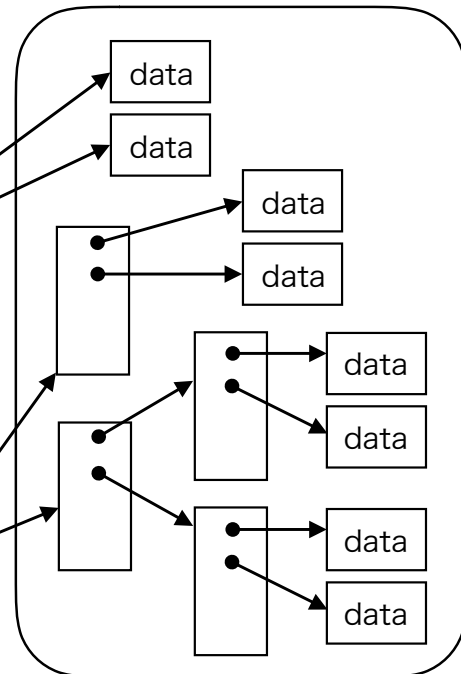
データブロック

図 16.2: i-node の構造

2 ビットが 10_2 に設定されている^{*4}。t ビットは UNIX のバージョンによって解釈が異なるので、ここでは解説しない。rwxrwxrwx の 9 ビットは、おなじみのファイルの保護モードである。

- **リンク数**

リンク数はファイルが幾つ名前を持つか（ハードリンクされているか）管理するカウンタである。リンクを削除しリンク数が 0 になった時にファイルが削除される（i-node が解放される）。

- **ファイルサイズ**

ファイルのサイズをバイト単位で表現する。図 16.2 の例ではファイルサイズは 64 ビットである。

- **3つの時刻**

ファイルの最終アクセス時刻、変更時刻、i-node 変更時刻が記録される。各時刻は、次の 2 つの 32 ビット整数で表現する。1 つ目の 32 ビット整数は、1970 年 1 月 1 日午前 0 時（UTC）からの経過秒数を表現する^{*5}。2 つ目の 32 ビット整数は、秒の小数点以下をナノ秒単位で表現する。

- **直接ブロック**

ファイル本体のデータを格納した 12 個のデータブロックの番号である。図 16.2 はブロック番号が 32 ビットの例である。データブロックサイズが図 16.1 のように 8KiB とすると、最大で

^{*4} macOS で `ls -l /bin/ps` を実行するとファイルのモードが `-rwsr-xr-x` のように表示される。これは、ps プログラムがファイルの所有者の権限で実行されることを表している。

^{*5} 32 ビットの経過秒数は 2038 年にオーバーフローする（負の数になる）。UNIX 時間の 2038 年問題と言われる。

$8KiB \times 12 = 96KiB$ のファイルまでを表現できる。ファイルの第 0 バイトから第 $96Ki - 1$ バイトまでは、いつも直接ブロックで管理される。

- **1 重間接ブロック**

直接ブロックだけでは表現できない大きなファイルに用いる。ここに番号を格納したデータブロックを間接ブロックと呼び、他のデータブロックの番号を格納するために使用する。

1 ブロックが $8KiB$ 、ブロック番号が 32 ビット (4 バイト) と仮定すると、1 つの 1 重間接ブロックに $8KiB \div 4B = 2Ki$ 個のブロック番号が格納できる。2Ki 個のデータブロックでは $8KiB \times 2Ki = 16MiB$ のデータを記録できる。ファイルの第 $96Ki$ バイトから第 $96KiB + 16MiB - 1$ バイトの範囲が、1 重間接ブロックで管理される。

- **2 重間接ブロック**

1 重間接ブロックでも表現できない大きなファイルに用いる。ここに番号を格納したデータブロックを 2 重間接ブロックと呼び、1 重間接ブロックのブロック番号を格納する。

1 ブロックが $8KiB$ 、ブロック番号が 32 ビットと仮定すると、1 重間接ブロックを用いて $16MiB$ のデータを記録できた。2 重間接ブロックを用いると 1 重間接ブロックを 2Ki 個格納できるので、 $16MiB \times 2Ki = 32GiB$ のファイルデータを管理できる。

- **3 重間接ブロック**

2 重間接ブロックを 2Ki 個管理できるので、 $32GiB \times 2Ki = 64TiB$ のデータを管理できる。

- **所有者 ID**

ファイル所有者のユーザ番号を格納する。(マルチユーザに対応)

- **グループ ID**

ファイルのグループ番号を格納する。

i-node がデータブロックを管理する方式を**インデクス方式**と呼ぶ。ランダムアクセスをする場合、インデクス (直接ブロック、間接ブロック等) から素早く目的のデータブロックを見つけることができる。(リスト方式ではブロックを順に調べる必要があった。)

「システム内には小さなファイルが多く大きなファイルは少ない」との仮定が成立てば、小さなファイルを効率よく扱えるこの方式は合理的である。間接ブロックを用いる大きなファイルの場合は少し効率が悪くなるが、使用頻度が低いので我慢できる。平均的なファイルサイズは時代によりシステムにより大きく変化するので、ファイルシステム初期化時にデータブロックサイズを適切に決める必要がある。

内容が全て 0x00 のデータブロックは割り付けを省略しても良い。また、ランダムアクセスで途中を飛ばしてデータを書き込んだ場合、途中のデータが書き込まれたことがないデータブロックも省略できる。このような途中に穴が空いたファイルのことを**スパースファイル** (*sparse file*) と呼ぶ。スパースファイルはデータブロックを消費することなく広いアドレス空間を提供できる。

16.4 ディレクトリファイル

ディレクトリファイルは i-node の type が**ディレクトリ**になっているファイルである。ディレクトリファイルのデータブロックには、ファイル名と i-node の対応表が記録される。対応表の 1 行をディ

32bit	16bit	8bit	8bit	l_2 バイト	詰め物
i-node 番号	l_1	型	l_2	ファイル名	\0 ... \0
<-----	l_1 バイト			----->	

図 16.3: ディレクトリエントリの構造

レクトリエントリと呼ぶ。ディレクトリエントリは、図 16.3 に示す可変長のものである^{*6}。エントリの各フィールドの意味は次の通りである。

- **i-node 番号**は、ファイル名とリンクされるファイル本体の i-node 番号である。
- l_1 は、ディレクトリエントリの長さをバイト単位で表す。 l_1 は 4 の倍数でなければならない。
- **型**はファイル本体の型を格納する。また、エントリが削除された時、空エントリを表現する値を格納する。
- l_2 は、ファイル名の長さをバイト単位で表す。 l_2 が 8 ビットなので、ファイル名は 255 バイト以内に制限される。
- **ファイル名**は、 l_2 バイトのファイル名を格納する領域である。
- **詰め物**には、ディレクトリエントリの長さが 4 の倍数になるように 0x00 のバイトを書き込む。

16.5 パス名と i-node の対応付け

以下では OS が /etc/passwd ファイルを探索する手順^{*7}を図 16.4 を用いて説明する。

- (1) 与えられたパスが絶対パスなので、ルートディレクトリから探索を開始する。この例では、ルートディレクトリの i-node 番号は必ず 2 と決められているものとしている。
- (2) ルートディレクトリの i-node から、データブロック 3 にルートディレクトリの内容が格納されていることが分かる。データブロック 3 を見に行く。
- (3) データブロック 3 に格納されているディレクトリエントリを解析する^{*8}。ファイル名 `etc` は 3 番のエントリに見つかる。このエントリから 12 番の i-node が `etc` に対応することが分かるので、i-node リストの 12 番のエントリを見に行く。
- (4) 12 番の i-node から `etc` はディレクトリファイルであること、ディレクトリファイルの内容がデータブロック 123 に格納されていることが分かる。データブロック 123 を見に行く。
- (5) データブロック 123 に格納されているディレクトリエントリを解析する。ファイル名 `passwd` は 3 番のエントリに見つかる。このエントリから 45 番の i-node が `passwd` に対応することが分かるので、i-node リストの 45 番のエントリを見に行く。
- (6) 45 番の i-node から `passwd` は普通のファイルであること、ファイルの内容がデータブロック 456 に格納されていることが分かる。データブロック 456 を読みに行く。

^{*6} 図 16.3 は、FreeBSD4 のソースコード <https://github.com/freebsd/freebsd/blob/stable/4/sys/ufs/ufs/dir.h> を参考に描いた。

^{*7} 例えば `open` システムコールが、渡されたパス名をもとに目的ファイルを探索する手順のこと。

^{*8} ディレクトリファイルに「`.`」や「`..`」も格納されていることも確認して欲しい。

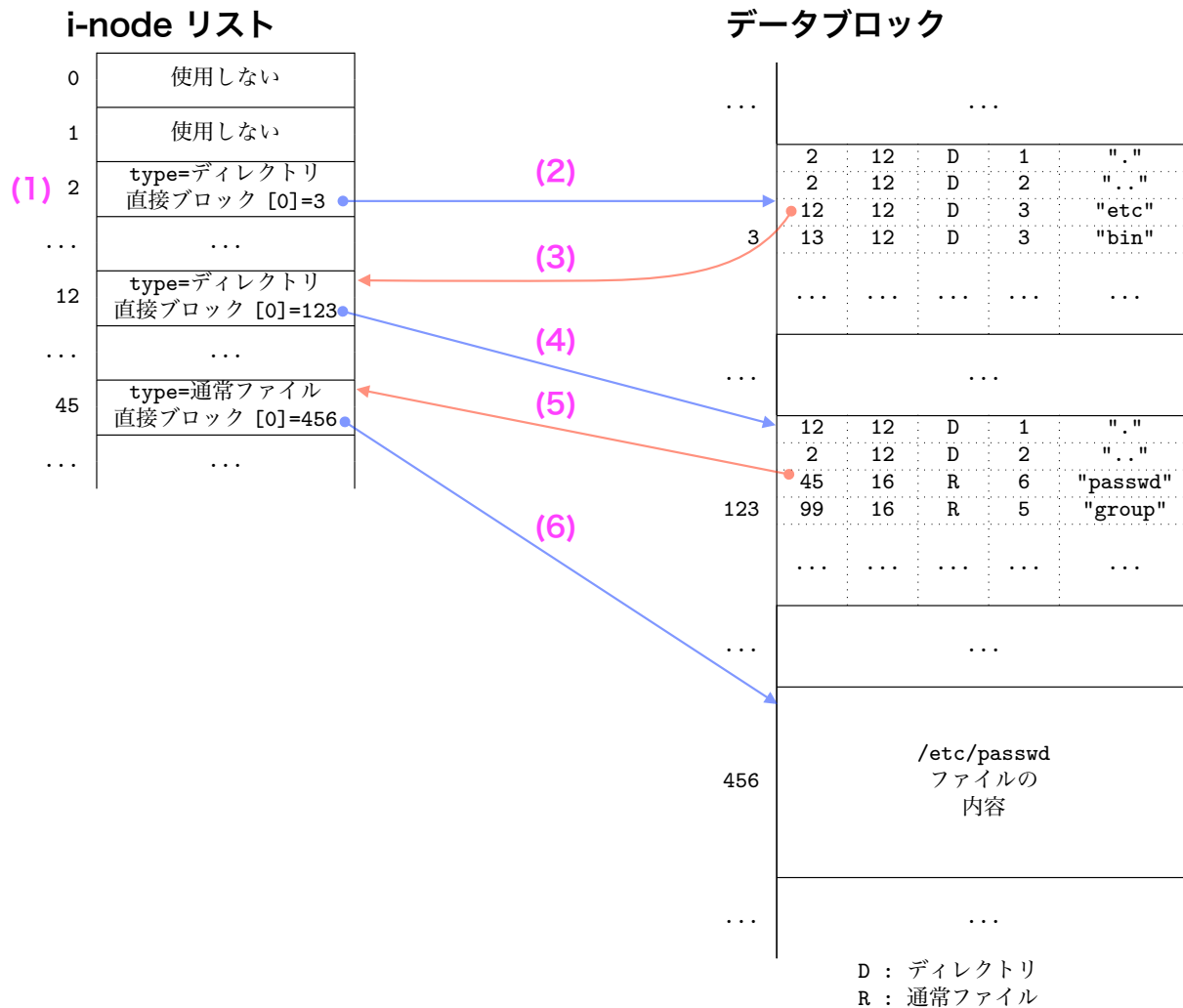


図 16.4: UFS で/etc/passwd ファイルを探索する手順

16.6 まとめ

UFS は、Version 7 Unix のファイルシステムと、それを改良した多くのファイルシステムのことを指す。様々な改良がされた多数のバージョンが存在する。また、Windows の NTFS, macOS の HFS+ や APFS, Linux の ext3 や ext4 等のファイルシステムは UFS ではないが、ハードリンクを始め、本章で説明した UFS と同じ構造であるような振る舞いを行うことができる。

この章では、UFS ボリュームの構造は概念のみを示し、i-node とディレクトリエントリは FreeBSD4 を参考にフォーマットを示し内容を解説した。最後に、UFS 上でパスを解析し特定のファイルを見つける手順を、例を用いて説明した。

練習問題

16.1 次の言葉の意味を説明しなさい.

- (a) UFS (UNIX File System)
- (b) ブートブロック
- (c) スーパーブロック
- (d) i-node
- (e) i-node リスト
- (f) インデクス方式
- (g) スパースファイル
- (h) ディレクトリファイル
- (i) ディレクトリエントリ
- (j) 直接ブロック
- (k) 間接ブロック

16.2 ブロックサイズが8セクタ(4KiB)の場合, 直接ブロックだけ用いて表現できるファイルの最大サイズを答えなさい.

16.3 ブロックサイズが8セクタ(4KiB)の場合, 1重間接ブロックを用いることによって, 直接ブロックだけの場合と比較して, ファイルサイズを最大でどれだけ大きくできるか答えなさい.

16.4 ブロックサイズが8セクタ(4KiB)の場合, 2重間接ブロックを用いることによって, 直接ブロックと1重間接ブロックだけ使用する場合と比較して, ファイルサイズを最大でどれだけ大きくできるか答えなさい.

16.5 図 16.2 の例がスパースファイルを表現しているとする. また, ブロックサイズ等は「16.2 ボリューム内部の配置」で示したものと同一とする. 次のアドレスはデータブロックが割当てられているか答えなさい.

- (a) 第 0x00000000 バイト
- (b) 第 0x00001000 バイト
- (c) 第 0x00010000 バイト
- (d) 第 0x00100000 バイト
- (e) 第 0x01000000 バイト
- (f) 第 0x10000000 バイト

第 17 章

ZFS

ファイルシステムの事例として、従来のファイルシステムとは異なる新しいアイデアを取り入れた ZFS を紹介する。ZFS は、2005 年にサン・マイクロシステムズ（Sun Microsystems、現在は Oracle の一部）が OpenSolaris に実装して公開し、オープンソースで開発されているファイルシステムである。その後、FreeBSD、Linux 等に移植され Solaris 以外の OS でも使用できるようになっている。

17.1 特徴

ZFS は、大きな主記憶と高速なマルチプロセッサシステムを前提に設計され、以下のような特徴を持っている。

- *COW (Copy On Write)* でデータやメタデータをハードディスク（以下ではデバイス）に書き込む。デバイスのブロックを上書きすることが無いので、書換え途中でシステムがクラッシュしてもファイルシステムが壊れない。
- 一連の書き込みが終了した時点で、変更を反映するための最後の書き込み（Uberblock の更新）がされる。Uberblock の書き込み前なら変更前の完全な状態、Uberblock の書き込み後なら変更後の完全な状態になり、変更途中の不完全な状態になることはない。
- チェックサムにより高い信頼性が確保されている。ファイルシステムのメタデータだけでなく、全てのブロックのチェックサムが、そのブロックを管理する 1 階層上のデータ構造に記録されている。その様子を図 17.1 に示す。チェックサムの不整合が見つかった場合、データの 2 重化（ミラー）がされていれば、自動的にミラーからデータを修復する。
- **スナップショット^{*1}**や**クローン^{*2}**の作成は、図 17.1 において最上位のブロックのコピーを作るような作業である。一瞬で作成が完了する。その後は COW の手法を使用し、コピーとオリジナルに違いが出た時点で、違いが出たブロックとその親だけがコピーされる。デバイスの容量も無駄にならない。
- ボリュームの代わりにストレージプールと呼ばれるソフトウェアの層をデバイスとファイルシステムの間にはさんでいる。従来のボリュームを模式的に表したものを図 17.2 に、ZFS のストレージ

^{*1} ある時点のファイルシステムをコピーして凍結したもの。変更ができない。

^{*2} ある時点のファイルシステムをコピーしたもの。変更ができる。

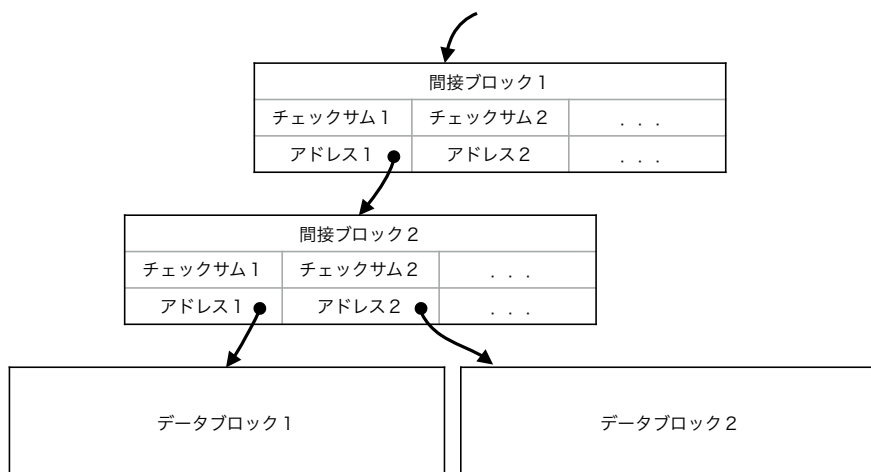


図 17.1: 全ブロックにわたるチェックサムのイメージ図

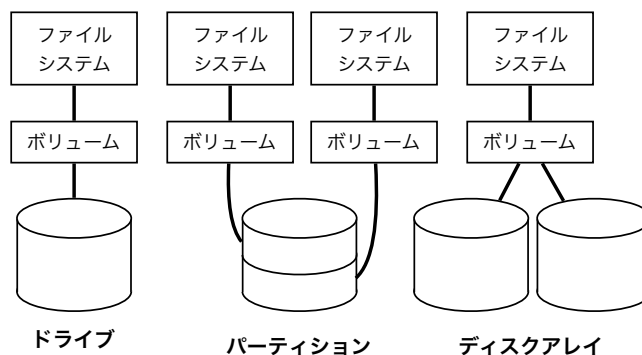


図 17.2: 従来のボリューム

ジプールを模式的に表したものを図 17.3 に示す。従来は、ファイルシステムの初期化以前にボリュームを決定し、後で変更することはできなかった。ZFS のストレージプールは沢山のデバイスを収容し、ファイルシステムからの要求に応じてブロックを割り付ける。C 言語プログラムが `malloc()` や `free()` を使用して必要な時にメモリを割当て方式に似ている。また、ストレージプールには後でデバイスを追加することも可能である。

- ファイルサイズ等の制約が事実上無くなった。ファイルサイズは最大 2^{64} バイト、ストレージプールサイズは最大 2^{70} バイト^{*3}である。
- ストレージプールは、ミラーや RAID-Z^{*4}[50] 等によりデバイスの故障に対する信頼性・可用性を向上する仕組みを持っている。
- ストレージプールは、データ圧縮や重複除去の機能を持っている。データを圧縮することで読み書きするデータの量が減少するので、ファイルの読み書き性能が高くなることもある^{*5}。

^{*3} Zetta = 2^{70} が ZFS の名前に関係しているらしい。

^{*4} RAID-5 の変形版のことである。

^{*5} ディスクに比較すると CPU はとても速い。

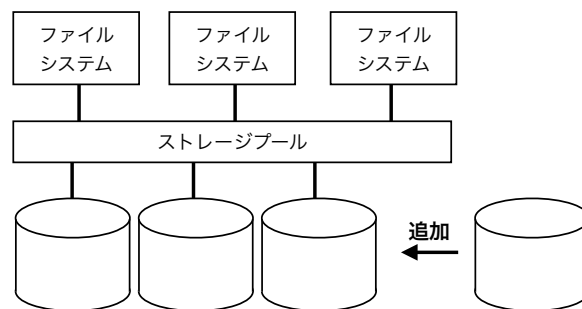


図 17.3: ストレージプール

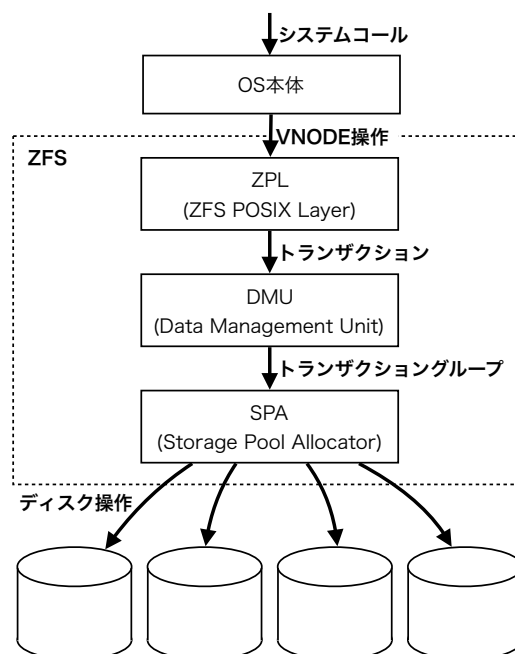


図 17.4: ZFS のソフトウェアモジュール

逆に，弱点としては次が挙げられる．

- 仮想記憶のページキャッシュと統合されていない．
- CPU やメモリの利用率が高い． 64 ビット CPU でないと ZFS に十分なメモリを提供できない．

17.2 ZFS のソフトウェア構成

図 17.4 に ZFS を構成する主要なソフトウェアモジュールの関係を，図 17.5 にソフトウェアモジュール間をデータが受け渡される様子を示す．次にシステムコールが処理される手順を説明する．

1. アプリケーションが発行したシステムコールは，OS カーネル本体に含まれる上位ファイルシステムにより VNODE 操作に変換される．VNODE 操作は UNIX ファイルシステムの `i-node` の操作

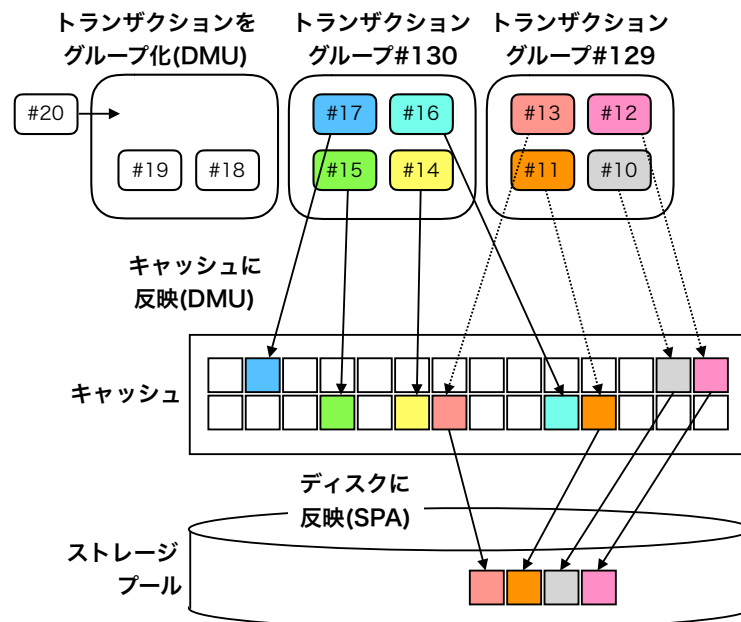


図 17.5: トランザクションが書き込まれるまで

を仮想化したものである。VNODE インタフェースは ZFS 以前から用いられてきたものである。このインタフェースを用いて ZFS のソフトウェアモジュールが OS に接続される。

2. ZPL (ZFS POSIX Layer) は VNODE 操作を ZFS のトランザクションに変換する。
3. DMU (Data Management Unit) は複数のトランザクションをひとまとめた**トランザクショングループ**を作る。DMU は、トランザクショングループ毎に、SPA が提供するストレージプールのキャッシュに書き込みを行う。
4. SPA (Storage Pool Allocator) は、DMU がトランザクショングループをキャッシュに書き込み終わると、キャッシュの内容をデバイスに反映させる。その際、なるべくデバイス上の連続セクタを使用するようにし、一度の書き込みで反映が終わるようにする^{*6}。

17.3 ストレージプールの構造 (概要)

ストレージプールは一般に複数のデバイスから構成されるが、ここでは話を簡単にするために一台のデバイスだけでストレージプールが構成される場合を考える。図 17.6 は、ストレージプールを構成するデバイスの内部を示している。256KiB のボリュームラベル (VL) が、安全のためデバイス先頭に 2 箇所、末尾に 2 箇所、合計 4 箇所に記録されている。

ボリュームラベル内部の構造は図 17.7 のようになっている。前半にはデバイスに関する情報（デバイスの名前など）が格納される。後半は 128 個の Uberblock を格納する配列である。トランザクショ

^{*6} 一度の書き込みで終わるほうが複数回の書き込みに分けるより速い。ホストコントローラがスキッタギャザー (scatter gather) 機能を持っていれば、メモリ上でバラバラに配置されたデータでも一度のコマンドで連続セクタにバースト書き込みすることができる。

VL_1 (256KiB)
VL_2 (256KiB)
ブートコード (3.5MiB)
データ領域
VL_3 (256KiB)
VL_4 (256KiB)
VL_n : ボリュームラベル

図 17.6: デバイス内部の配置

デバイス情報など 名前/値ペア (128KiB)
Uberblock[0] (1KiB)
Uberblock[1] (1KiB)
Uberblock[2] (1KiB)
...
Uberblock[127] (1KiB)

図 17.7: ボリュームラベルの内容

ングループがプールに書き込まれると、最後に Uberblock がトランザクショングループ番号とともに書き込まれる。書き込まれる位置はトランザクショングループ番号を 128 で割った余りで計算できる。

クラッシュや突然の停電によりシステムが正常にシャットダウンされなかった場合でも、全てのボリュームラベルの Uberblock 配列から、トランザクショングループ番号を手がかりに最も新しい Uberblock を見つけ出せば、最後に書き込みが完了した時点のストレージプールの状態を再現できる。

17.4 ストレージプールの更新

図 17.1 に示したような^{*7}木構造を用いて、メタデータブロックやデータブロックが記録される。この木構造はストレージプールに記録される。木構造のルートの位置は Uberblock に記録される。トランザクショングループの操作がストレージプールに反映される様子を図 17.8 に模式的に示す。

1. 初期状態

デバイス上のストレージプールには、Uberblock を起点にする木構造でストレージプールのメタデータブロックやデータブロックが記録されている。

2. データブロック更新

ストレージプールのデータブロックに変更あった場合、そのブロックを上書きするのではなく、別のブロックを確保し新しい内容をそこに書き込む (COW)。

3. メタデータブロック更新

変更のあったブロックを指すメタデータブロック中のポインタは、新しいブロックを指すように変更されなければならない。別に新しいブロックを確保し、ポインタを更新した新しい内容をそこに書き込む (COW)。この作業を木構造のルートまで繰り返す。

4. Uberblock 更新

木構造の新しいルートを指すように Uberblock を変更する。この際も、既存の Uberblock を上書

^{*7} 図 17.1 はファイルのデータブロックを管理している様子である。ストレージプール全体ではもっと複雑になる。

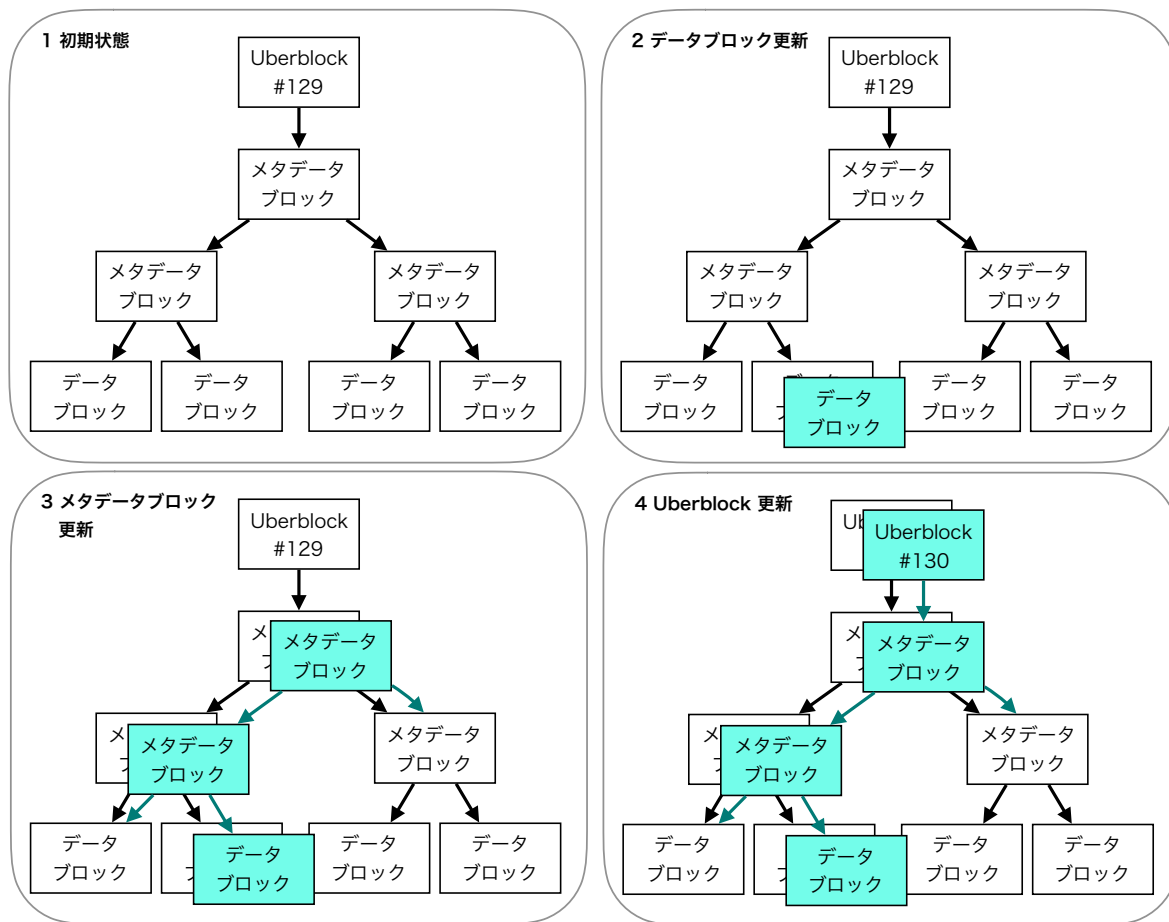


図 17.8: ストレージプールの COW による更新のイメージ図

きするのではなく、新しい Uberblock を使用する。Uberblock にトランザクショングループ番号を書き込むことで、どの Uberblock が最新のものと区別できる。トランザクショングループ番号は 64 ビットなので、システムが廃棄されるまでオーバーフローする心配はない。

以上の手順では、Uberblock が正常に更新されるまで新しいデータは全く反映されない。どの段階でシステムがクラッシュしても、ストレージプールの状態はトランザクションが適用される前か、トランザクションが完了した後のどちらかになる。Uberblock が正常に更新されどこからも指されなくなったブロックは再利用される。

17.5 ストレージプールの構造

17.5.1 ブロックポインタ

図 17.1 で、「チェックサム」、「アドレス」と表現した部分は**ブロックポインタ**と呼ばれる 128 バイトのデータ構造である。ブロックポインタはデータ多重化のために最大 3 組のアドレスを記録できる。ブロックポインタには以下の情報が含まれる。

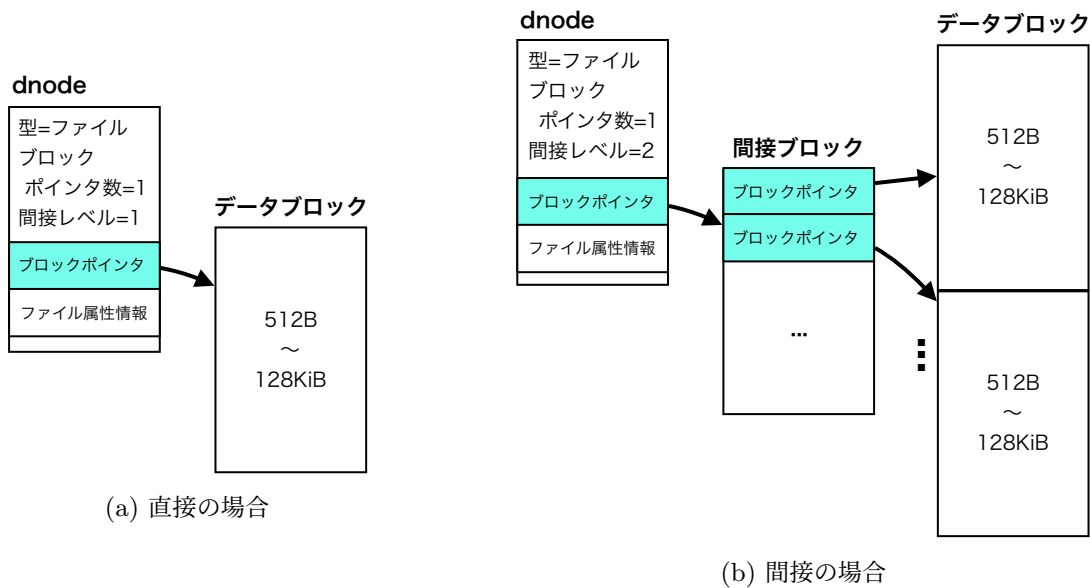


図 17.9: ファイルを表現する dnode の例

- ・ **サイズ**: ブロックの大きさに関する情報である。
- ・ **チェックサム** (256 ビット): ブロックのチェックサムである。
- ・ **ブロックのアドレス**: ブロックのストレージプール内での格納位置に関する情報である。(最大 3 個)
- ・ **タイムスタンプ**: ブロックを作成したトランザクショングループの番号である。ファイルシステムからブロックが削除される時、ファイルシステムのスナップショットのタイムスタンプと比較する。スナップショットより古いブロックはスナップショットで使用されているので、削除できない。
- ・ **その他**: チェックサム計算に使用するアルゴリズムの種類、データ圧縮に使用するアルゴリズムの種類、圧縮後のサイズなど、ここでは紹介しない情報が含まれる。

17.5.2 Dnode

dnode はストレージプール内のあらゆるオブジェクトを表現する 512 バイトのデータ構造である。UFS の *i-node* に似ているが *dnode* はファイルやディレクトリだけでなく、ファイルシステムや、スナップショット、クローンなどの表現にも用いられる。*dnode* は、表現するオブジェクトや付属するデータの大きさによって幾つかの形式を取るが、ファイルを表する場合の例を図 17.9 に示す。

以下では、図 17.9 について説明する。図 17.9 はファイルを表する *dnode* の例であるが、ファイル以外のオブジェクトも同様な方法でデータを格納する。

- ・ *dnode* は三つ以内のブロックポインタを格納することができる。図は一つのブロックポインタを格納した例である。
- ・ *dnode* は表現するオブジェクトに応じたデータを格納する領域を持っている。この領域はブロックポインタと共用になっているので、ブロックポインタの数が多い場合は小さくなる。図の例は

dnode がファイルを表現する場合なので、ファイルの属性情報（時刻や保護属性など）を格納している。

- データの大きさが 128KiB 以内の場合は図 17.9a に示すように dnode のブロックポインタがデータブロックを直接参照する。
- 大きさが 128KiB を超える場合は、図 17.9b に示すように dnode のブロックポインタが間接ブロックを指すようになる。最大 128KiB の間接ブロックは 128B のブロックポインタを最大 1Ki 個格納できる。
- $128KiB \times 1Ki = 128MiB$ より大きなデータを表現する時は、UFS のように多重の間接ブロックを用いる。間接レベルは 6 までサポートされており、 2^{64} バイト以上のファイルが表現できる。UFS ではファイル後方のデータブロックだけが間接ブロックになったが、すべてのブロックが同じ間接レベルで扱われる。間接レベルは dnode に記録される。

17.5.3 全体像

図 17.10 にストレージプールの構造を模式的に描いたものを示す。Objset は dnode の配列を管理する 2KiB^{*8}のデータ構造である。Objset に埋め込まれた dnode が、dnode 配列を格納するブロックを管理する。Objset には最大三つの dnode を埋め込むことができる。

MOS (Meta Object Set) layer

MOS layer はストレージプール全体に関わるオブジェクトや、ファイルシステムやスナップショット、クローンなどの根になる dnode を格納する。オブジェクトの種類は dnode の型から分かる。

- dnode 配列の先頭は *master node* である。master node はストレージプールのコンフィグ、プロパティ、エラーログ、統計情報などを記録する。
- dnode 配列の最後には *space map* が格納される。space map はストレージプール内のブロックの割付を管理する。
- dnode 配列の他の要素は、Object-set layer のファイルシステムやスナップショット、クローンなどの根になる dnode を格納する。

Object-set layer

Object-set layer はファイルシステムやスナップショット、クローンなどの実体を格納する。図 17.10 は二つのファイルシステムが Object-set layer に格納されている様子を表している。

- ファイルシステムは Objset が管理する dnode リストにより表現される。
- この dnode リストが UFS の *i-node* リストに相当する。
- dnode リスト先頭の master node は root ディレクトリの dnode 番号やファイルシステムのバージョン番号などを格納している。
- dnode リストの二番目の dnode は通常ファイルの例である。
- dnode リストの三番目の dnode はディレクトリファイルの例である。ディレクトリファイルは

^{*8} https://people.freebsd.org/~gibbs/zfs_doxygenation/html/annotated.html に公開されている FreeBSD ZFS の実装を前提にしている。

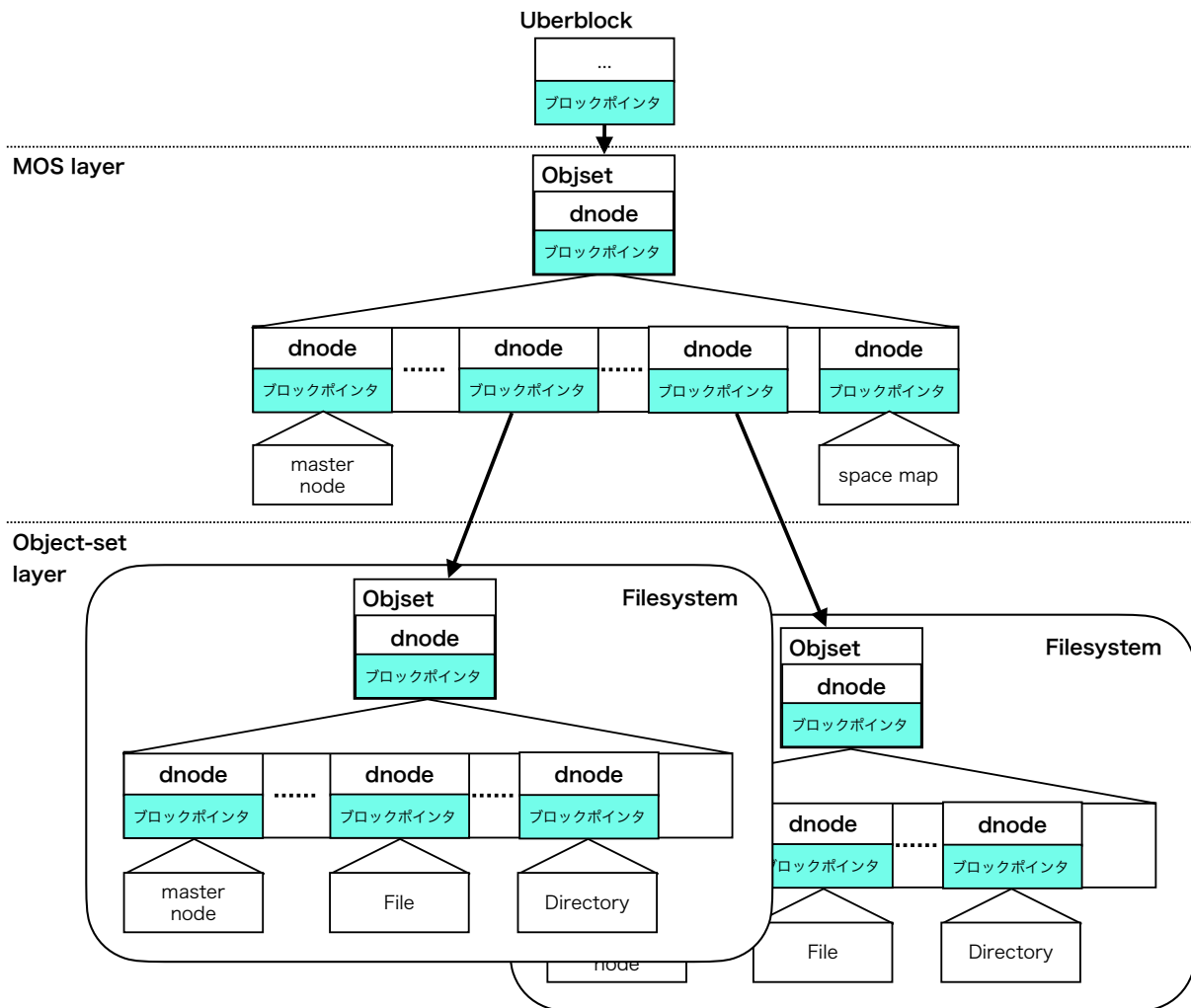


図 17.10: ストレージプールの構造

ファイル名と *dnode* 番号の対応表を格納する。

- 図では Objset はファイルシステム本体だけ管理しているが、実際はユーザやグループ毎の記憶領域の使用量を管理するオブジェクトや、ファイルシステムでは使用されなくなったがスナップショットが使用しているために解放できないブロックのリスト (deadlist) など管理している。

17.6 スナップショットとクローン

ある時点のファイルシステムの状態をコピーして凍結したものを**スナップショット**と呼ぶ。スナップショットをもとに変更可能にしたものを**クローン**と呼ぶ。

17.6.1 スナップショットの作成

図 17.11 は ZFS がスナップショットやクローンを作る原理を説明している。スナップショットの作成は次の二つのステップでできる。

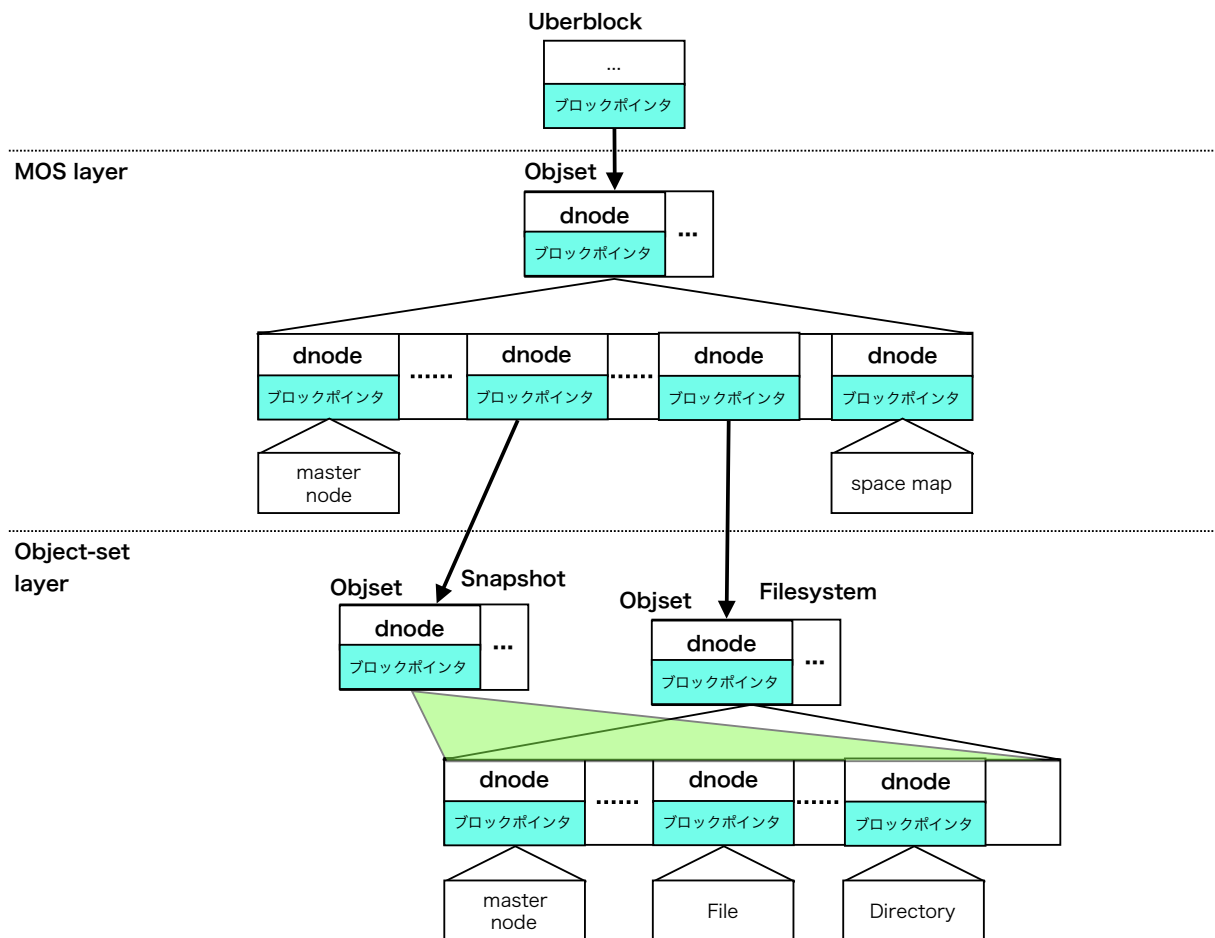


図 17.11: スナップショットの作成

1. Object-set layer に格納されているファイルシステムの Objset のコピーを作る。
2. MOS layer の dnode 配列に新しい dnode を追加し、Objset のコピーを参照するようにする。

スナップショットの作成は一瞬で完了する。スナップショットは読み出し専用なので Objset 以下は変化しない。しかし、ファイルシステムは変化する。ファイルシステムが変化した時は COW を用いて必要最小限の範囲のブロックだけ書き換える。

17.6.2 ブロックの解放

通常、COW により新しいコピーに役割を譲ったストレージプールのブロックは解放される。しかし、スナップショットがある場合、ブロックがスナップショットからも参照されている可能性があるので解放しても良いか判断が難しくなる。そこで、ブロックポインタにリンクカウントを設ける方法が考えられる。しかし、この方法ではスナップショットを作るたびにファイルシステムの全てのブロックポインタを書き換える必要が生じるので、ZFS は別の方式を採用している。

スナップショットと deadlist の構造

ZFS ではブロックポインタのタイムスタンプとスナップショットのタイムスタンプを比較することで、解放しても良いブロックかどうか判断している。そのために MOS layer で、同じファイルシステ

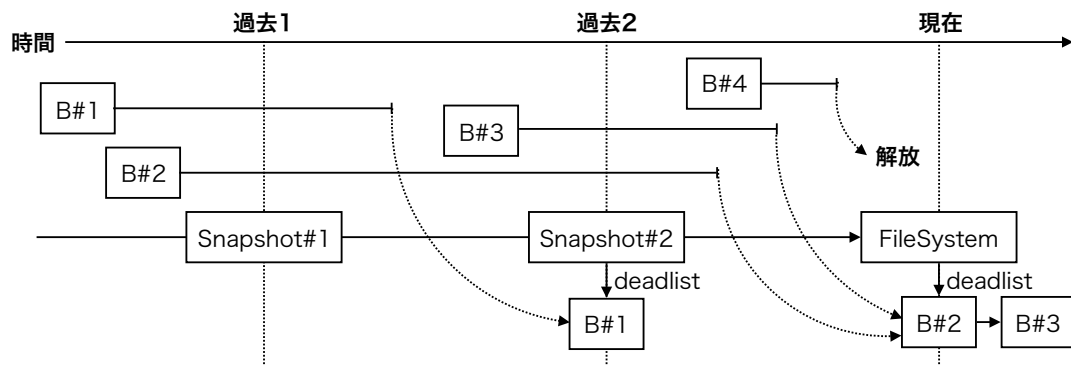


図 17.12: スナップショットと deadlist

ムから過去に作成したスナップショットがどれか分かるように管理している。タイムスタンプには、現実の時刻ではなく、トランザクショングループ番号が用いられる。また、ファイルシステムの Objset は、現在は使用していないがスナップショットで使用されているので解放できないブロックのリスト (deadlist) も管理している。その様子を図 17.12 に示す。

- B#N はブロックを表している。ブロックを指すブロックポインタが、タイムスタンプを格納する。ブロックから右に伸びる直線は、ブロックが割当てられてからファイルシステムで使用されなくなるまでの時間を表している。
- Snapshot#N はスナップショットを表している。図では、「過去1」、「過去2」の二回スナップショットが作成されている。スナップショット作成時点では使用されていなかったが、過去のスナップショットから参照されているため解放できないブロックのリストである deadlist を持っている。
- FileSystem は現在アクティブなファイルシステムを表している。現在は使用されていないがスナップショットから参照されているブロックのリストである deadlist を持っている。

使用されなくなったブロック

図 17.12 は、「過去1」から「現在」までの間に B#1, B#2, B#3, B#4 の四つのブロックが使用されなくなった場合を説明してる。

- B#1 は Snapshot#1 と Snapshot#2 の間で使用されなくなったが、Snapshot#1 よりも前から使用されていた。Snapshot#1 が使用しているので解放できない。その時点でアクティブな FileSystem の deadlist に入れる。Snapshot#2 作成時に deadlist はスナップショット側に残される。FileSystem の deadlist は空になる。
- B#2 と B#3 は Snapshot#2 より前に割当てられていた。これらのブロックも過去のスナップショットが使用しているので、現在のファイルシステムで使用されなくなっても解放できない。FileSystem の deadlist に入れる。
- B#4 は Snapshot#2（最新のスナップショット）より後に割当てられ、短時間の内に使用されなくなったブロックである。このブロックは、どのスナップショットでも使用されていないので解放で

練習問題

17.1 次の言葉の意味を説明しなさい。また、分からない言葉について調べなさい。

- (a) COW (Copy On Write)
- (b) メタデータ
- (c) チェックサム
- (d) スナップショット
- (e) クローン
- (f) ボリューム
- (g) ストレージプール
- (h) ZPL (ZFS POSIX Layer)
- (i) DMU (Data Management Unit)
- (j) SPA (Storage Pool Allocator)
- (k) VNODE
- (l) Uberblock
- (m) ブロックポインタ
- (n) Dnode
- (o) MOS (Meta Object Set) layer
- (p) Object-set layer
- (q) space map
- (r) master node

17.2 トランザクショングループ番号は 64 ビットです。毎秒 100 トランザクショングループを処理したとして、トランザクショングループ番号がオーバーフローするまでに約何年かかるか計算しなさい。

17.3 ZFS で使用できるチェックサム計算アルゴリズムについて調べなさい。

17.4 ファイルを表現する dnode が間接レベル 2 の時、最大のファイルサイズは何バイトになるか計算しなさい。

17.5 ブロックにリンクカウントを設け、スナップショットからの参照数を管理することで、ブロックの解放を判断するアイデアの利点と問題点を挙げなさい。

第 V 部

TacOS の実装例

第 18 章

TaC と TacOS

TaC (Tokuyama Advanced educational Computer) は, TeC7 (Tokuyama Educational Computer Ver.7)^{*1}に内蔵された 16bit のコンピュータである。TeC7 基板上のジャンパ設定により TaC モードに切り換える。図 18.1 に写真を示す。TaC は, ディスプレイ, キーボード, マイクロ SD カードを接続することで, 1980 年代前半の 8bit パソコン程度の能力を発揮する。コンピュータサイエンスを学ぶ大学や高専の学生が, 実際に動作する PC の例として使用したり, 仕組みを解析する目的で設計してある。

TaC 上では C--言語^{*2}で記述された TacOS^{*3} が動作する。本書では TacOS をオペレーティングシステムの実装例として参照する。



(a) TeC7 の写真



(b) TaC としての使用例

図 18.1: TeC7 と TaC

^{*1} 詳細は <https://github.com/tctsigemura/TeC7> を参照のこと。

^{*2} C 言語に似た言語, 詳細は <https://github.com/tctsigemura/C--/blob/master/doc/cmm.pdf> を参照のこと。

^{*3} 詳細は <https://github.com/tctsigemura/TacOS> を参照のこと。

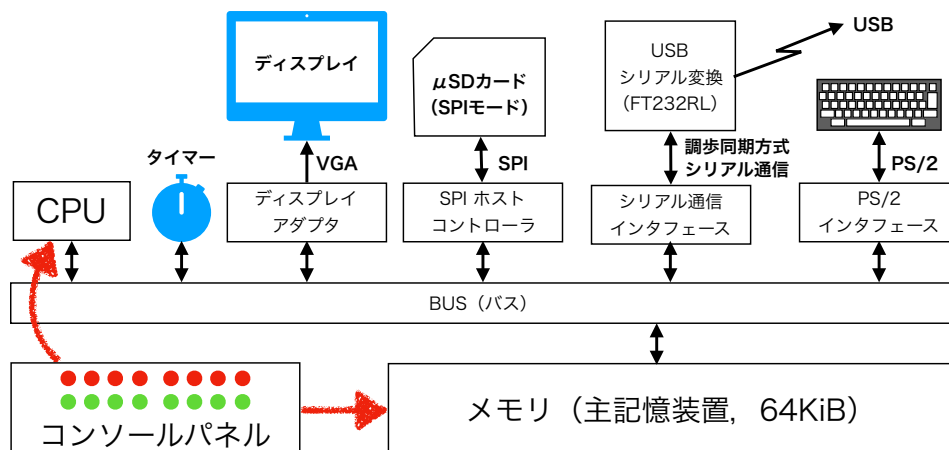


図 18.2: TaC のハードウェア構成

18.1 ハードウェア構成

図 18.2 に TaC のハードウェア構成を示す。16 ビットのシングルプロセッサ（CPU が一つ）、主記憶 64KiB の非常に単純なシステムである。単純なのでオペレーティングシステムの構築も容易である。TaC に関する資料を付録 A にまとめる。

• コンソールパネル

図 18.1a で、TeC7 本体右半分のランプやスイッチで構成される部分をコンソールパネルと呼ぶ。コンソールパネルは CPU や主記憶と直接接続されており、CPU を停止した状態で、CPU や主記憶の内容を操作したり観察したりすることができる。また、機械語命令を一命令毎に実行するステップ実行機能や、ある番地の命令を実行した時点でプログラムを停止するブレークポイント機能が利用できる。コンソールパネルの機能はハードウェアで実現されているので、オペレーティングシステムの内部をステップ実行することも可能である。TacOS の開発では、コンソールパネルがデバッグに活用された。

• CPU

図 A.2 に示すような CPU レジスタと PSW を持つ 16 ビット CPU である。PSW のフラグに実行モードを表す P ビットを持ち、カーネルモードとユーザモードを切り換えることができる。機械語命令は、図 A.3 に示す 46 種類が準備されている。機械語命令のアドレッシングモードは 8 種類ある。

• メモリ

メモリは図 A.4 に示す構成である。メモリ空間全体で 64KiB、自由に使用できるメモリが 56KiB、2KiB の VRAM と 4KiB の IPL、32B の割込みベクタからなる。メモリは 8 ビット単位、または、16 ビット単位で読み書きできる。16 ビット単位の場合は偶数アドレスを用いる。

• タイマー

1 ミリ秒から $2^{16} - 1$ ミリ秒までの間隔で割込みを発生する二つのインターバルタイマーが利用で

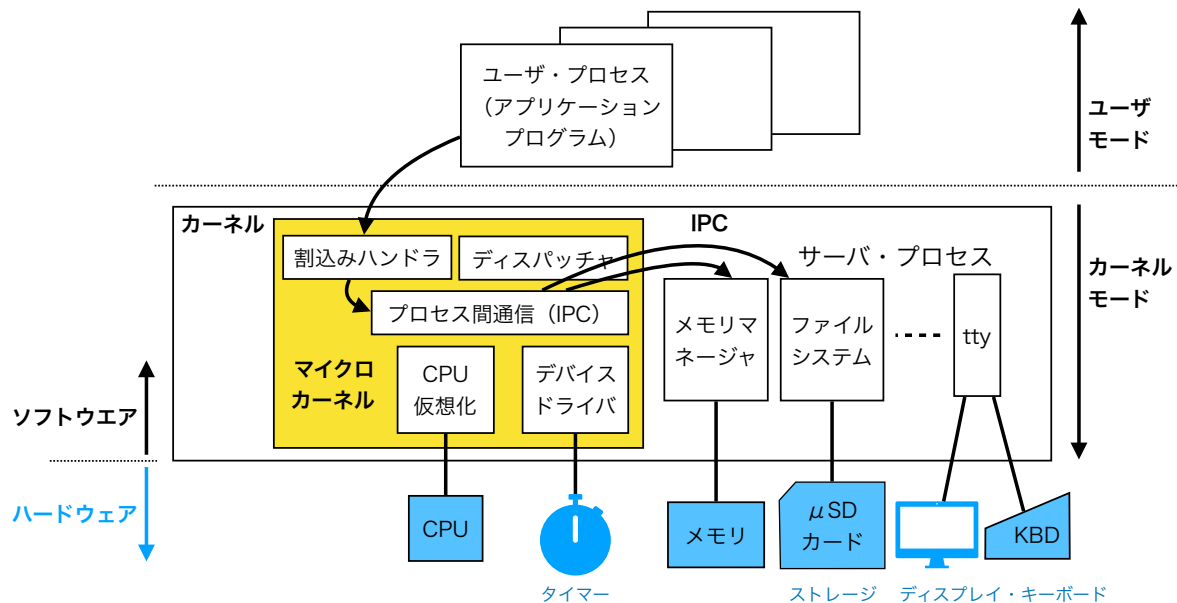


図 18.3: TacOS の構成

きる。

- **ディスプレイアダプタ**

80 文字 × 24 行の文字を VGA ディスプレイに表示する。メモリ空間の E000h から配置される VRAM に書き込んだ ASCII コードと対応する文字をディスプレイに表示する。E000h 番地がディスプレイの左上隅に対応する、E001h 番地が一行目の 2 文字の位置、E04Fh 番地が一行目の 80 文字の位置、E050h 番地が二行目の 1 文字の位置に対応する。

- **SPI ホストコントローラ**

スロットに挿入されたマイクロ SD カードを SPI モードに切換え読み書きを行う。SPI ホストコントローラに初期化コマンドを発行すると、マイクロ SD カードを SPI モードに切換える。ブロックアドレスとメモリアドレスを設定して読み出しコマンドを発行すると、マイクロ SD カードの指定したブロックから 512 バイトのデータを CPU を介さずに (DMA : Direct Memory Access を用いて) メモリに読み出す。書込みコマンドを発行すると、メモリから指定ブロックにデータを書き込む。

- **シリアル通信インタフェース**

調歩同期方式、9,600Baud の通信インタフェースである。USB シリアル変換 IC を通して PC 等のシリアルターミナルと通信できる。1 バイト転送する毎に割り込みを発生する。

18.2 TacOS

図 18.3 に TaC 用の OS である TacOS の構造を示す。マイクロカーネルがプロセス間通信 (IPC) 機能を提供し、サーバプロセスがメモリ管理やファイルシステム機能を提供する。図 2.7 の一般的なマイクロカーネル方式と異なり、サーバプロセスがカーネルモードで動作しハードウェアに直接アクセス

する。また、サーバプロセスはマイクロカーネルとリンクされ一つのプログラムモジュールになる。このプログラムモジュールを**カーネル**と呼ぶことにする。同じプログラムモジュール内なので、サーバプロセスはマイクロカーネル内ルーチンを CALL 機械語命令で直接に呼び出すことができる。

割込みや SVC 命令の実行が原因で、ユーザプロセスはカーネルモードに切り換わりマイクロカーネル内の割込みハンドラが呼び出される。割込みハンドラで割込み原因を判断し、マイクロカーネル内のルーチンを呼び出したり、サーバプロセスの機能を IPC を用いて呼び出したりする。

18.3 まとめ

TaC は、本書でオペレーティングシステムの実装例として使用する TacOS を稼働させるコンピュータである。コンソールパネルを持ち、TacOS のカーネル内までステップ実行によるトレースが可能である。*TacOS* はマイクロカーネル方式の簡単なオペレーティングシステムである。本書では、しばしば TacOS のソースコードを実装例として参照する。

第 19 章

TacOS の CPU 仮想化

CPU 仮想化の実例として TacOS^{*1}の例を紹介する。TacOS はマルチプロセスのオペレーティングシステムである。以下では CPU の時分割多重に必要なプロセス切換え機構を紹介する。

19.1 PCB

PCB はプロセス切換え機構にとっても最も重要なデータ構造である。PCB は、リスト 19.1 に示す PCB 構造体として定義されている^{*2}。PCB 構造体の内容を順に説明する。

- 仮想 CPU(sp)
プロセスのコンテキストのほとんどをカーネルスタック上に保存する。そして、保存位置を表すスタックポインタ (SP) だけを PCB に保存する。カーネルスタックはプロセス毎に準備されている。PCB に保存されるのは仮想 CPU の一部 (SP) だけである。
- プロセス番号 (pid)
- 状態 (stat)
プロセスの状態は以下の三つである。
 1. P_RUN
Running と Ready の二つを兼用している。プロセスは実行可能プロセスの待ち行列 (実行可能列) に挿入される際に P_RUN 状態になる。実行中も P_RUN 状態のまま変更しない。
 2. P_WAIT
Waiting 状態のことである。
 3. P_ZOMBIE
プロセスが終了したが、終了ステータスを親プロセスに渡していない状態である。終了処理の途中状態と考えるとよい。
- 優先度 (nice, enice)
ゼロが最も高い優先度を表す。優先度には、本来の優先度 (nice) と、実質の優先度 (enice) の二つがある。現在の実装では、この二つは同じ値を持つ。将来、動的に変化する優先度を採用する

^{*1} TacOS の詳細は <https://github.com/tctsigemura/TacOS> を参照のこと。

^{*2} <https://github.com/tctsigemura/TacOS/blob/master/os/kernel/process.hmm> の一部である。

リスト 19.1: TacOS の PCB 宣言ソースプログラム

```

1 struct PCB {                // PCB を表す構造体
2     int sp;                  // コンテキスト(他の CPU レジスタと PSW は
3                             // プロセスのカーネルスタックに置く)
4     int pid;                 // プロセス番号
5     int stat;                // プロセスの状態
6     int nice;                // プロセスの本来優先度
7     int enice;               // プロセスの実質優先度(将来用)
8     int idx;                 // この PCB のプロセステーブル上のインデクス
9
10    // プロセスのイベント用セマフォ
11    int evtCnt;                // カウンタ(>0:sleep中, ==-1:wait中, ==0:未使用)
12    int evtSem;                // イベント用セマフォの番号
13
14    // プロセスのアドレス空間(text, data, bss, ...)
15    char[] memBase;           // プロセスのメモリ領域のアドレス
16    int memLen;                // プロセスのメモリ領域の長さ
17
18    // プロセスの親子関係の情報
19    PCB parent;                // 親プロセスへのポインタ
20    int exitStat;              // プロセスの終了ステータス
21
22    // オープン中のファイル一覧
23    int[] fds;                 // オープン中のファイル一覧
24
25    // プロセスは重連結環状リストで管理
26    PCB prev;                  // PCB リスト(前へのポインタ)
27    PCB next;                  // PCB リスト(次へのポインタ)
28    int magic;                 // スタックオーバーフローを検知
29 };

```

場合に、`enice` の値が変化するようにする。

- プロセステーブルのインデクス (`idx`)
この PCB が登録されているプロセステーブル内の位置である。プロセスが消滅する際にプロセステーブルから PCB を削除するために使用する。
- イベント用カウンタとセマフォ (`evtCnt`, `evtSem`)
セマフォはプロセス間の同期に使用する基本的な機構である^{*3}。タイマー待ち、子プロセスの終了待ち等で、このセマフォを使用してプロセスを待ち状態にする。カウンタはタイマーの待ち時間を計るため等に使用される。
- プロセスのアドレス空間 (`memBase`, `memLen`)

^{*3} 詳しくは第 20 章で解説する。

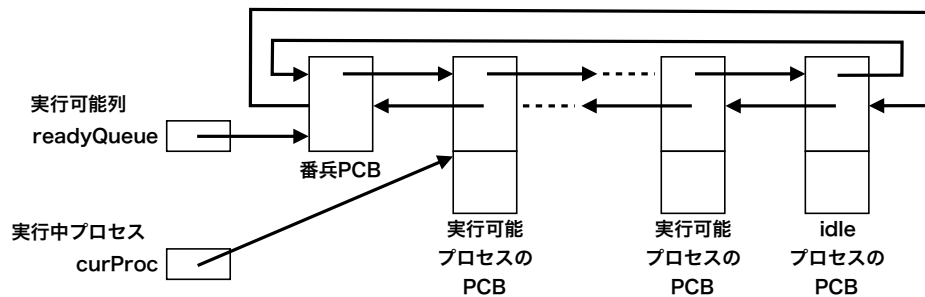


図 19.1: TacOS の実行可能列

仮想記憶のような高度な機構は無い。各プロセスは、物理メモリの領域をオペレーティングシステムによって割付けられる。memBase はオペレーティングシステムがプロセスに割当てたメモリ領域の開始アドレス、memLen はメモリ領域のバイト数である。

- プロセスの親子関係の情報 (parent, exitStat)
プロセスは親プロセスを記憶している。parent は親プロセスの PCB を指すポインタである。exitStat は P_ZOMBIE 状態になった時、親に渡すべき終了ステータスを保存する領域である。
- オープン中のファイル一覧 (fds)
プロセスがオープンしたファイルの一覧を記憶する配列である。ここには、システム全体で一意的なファイルディスクリプタ (番号) が記録される。close システムコールは、クローズするファイルディスクリプタが正当なものか調べるために、この配列を使用する。exit システムコールは、プロセスを終了する前にプロセスの全オープンファイルをクローズするために、この配列を使用する。
- PCB リストの管理 (prev, next)
プロセスのリストを PCB のリストとして表現する。PCB リストは番兵付きの双方向循環リストである (図 19.1 参照)。prev, next はリスト上で前後のプロセスの PCB を指すポインタである。
- スタックオーバーフローの検知 (magic)
PCB の直後にプロセスのカーネルスタックを配置する。万一、カーネルスタックがオーバーフローすると PCB が後ろから破壊される。magic はそれを検知するために使用される。PCB を初期化する際に magic に 0xabcd を格納する。カーネルスタックがオーバーフローすると、まず、magic 領域が破壊される。magic の値が変化していないかチェックすることで、カーネルスタックのオーバーフローを検知することができる。

19.2 実行可能列

実行可能列は、図 19.1 に示すような PCB の番兵付き双方向循環リストである。番兵の次のプロセスが、最も優先度が高い実行可能なプロセスである。プロセスをディスパッチする際は、番兵の次のプロセスを選択する。選択されたプロセスを curProc が指すように変更してからディスパッチする。プロセス実行時も、PCB は実行可能列に置いたままにする。プロセスがブロックする際は、curProc を実行可能列からイベントの待ち行列に移動する。実行可能列の末尾には、常に idle プロセスが置かれてい

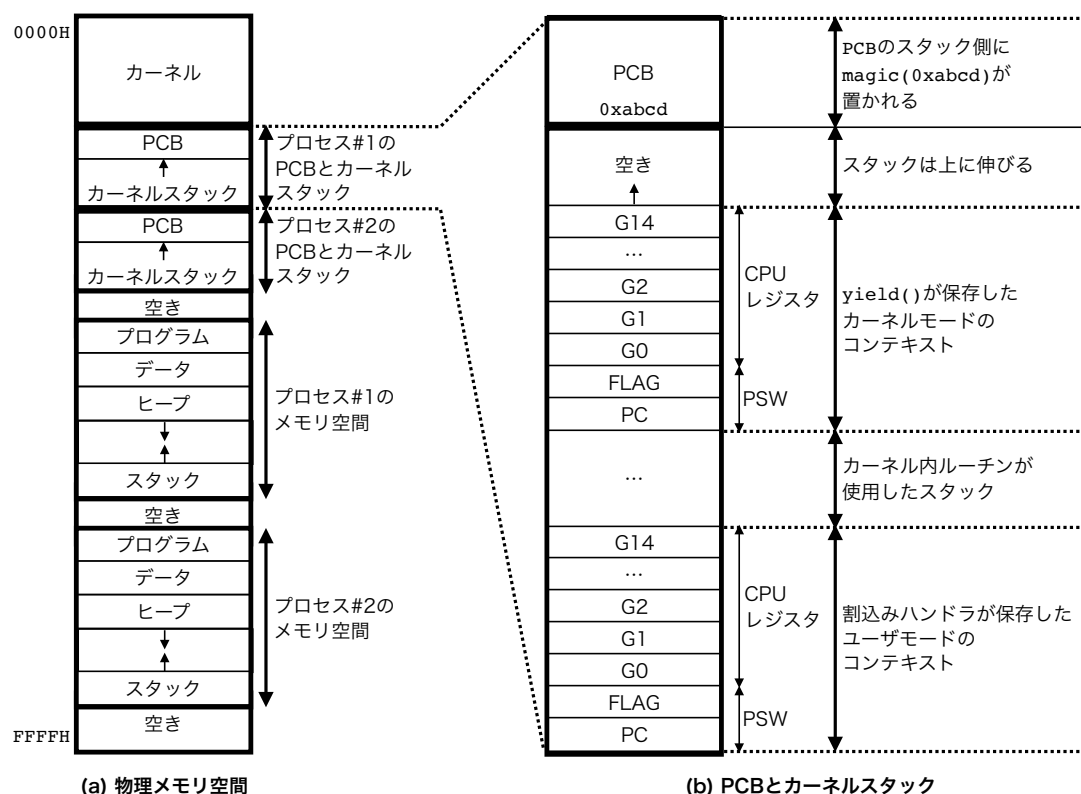


図 19.2: TacOS のメモリ配置

る。実行可能列が空になることは無い。

19.3 メモリ配置

図 19.2 に TacOS 実行時のメモリマップを示す。図は二つのプロセスが実行中の例である。まず「物理メモリ空間」の配置について、次に「PCB とカーネルスタック」について説明する。

(a) 物理メモリ空間

- カーネル

マイクロカーネルとサーバプロセスを一つのプログラムにリンクしたものを**カーネル**と呼ぶことにする。カーネルのプログラムとデータ（変数）がこの領域に配置される。

- プロセス#1の PCB とカーネルスタック

プロセス1の PCB とカーネルスタックが隣接して配置される。

- プロセス#2の PCB とカーネルスタック

プロセス毎に PCB とカーネルスタックが準備される。

- プロセス#1のメモリ空間

プロセス1のプログラム、データ、ヒープ、スタック領域が配置される。ユーザモードのプロセスは、この範囲以外のメモリをアクセスできないように保護すべきである。しかし、TaC はメモリ保護機構を持っていない。

- プロセス # 2 のメモリ空間
プロセス毎にメモリ空間が準備される。

(b) PCB とカーネルスタック

PCB とカーネルスタックは隣接して配置される。ユーザプロセス実行中はカーネルスタックの内容は空になる。カーネルモードで動作するサーバプロセスはカーネルスタックを使用する。割込みが発生すると PSW と CPU レジスタをカーネルスタックに保存した後、マイクロカーネル内ルーチンの実行が始まる。マイクロカーネル内ルーチンはカーネルスタックを使用する。万一、カーネルスタックが伸びすぎて PCB を破壊した場合は、PCB のスタック寄りに置いたマジックナンバー (0xabcd) が書き換わる。プロセススイッチの際にマジックナンバーを調べ、PCB が破壊されていないことを確認する。

19.4 割込み処理

TaC はベクタ方式の割込み機構を持っており、割込み原因毎に割込みハンドラを登録することが可能である。割込みが発生すると TaC の CPU は次の処理を自動的に行う。

1. カーネルスタックに PSW を保存する。
2. CPU をカーネルモードかつ割込み禁止に設定する。
3. 対応する割込みベクタに登録されているハンドラにジャンプする。

以下では、タイマー割込みを例に割込みハンドラの振舞いと、割込み発生時のカーネルスタックの状態を説明する。リスト 19.2^{*4}に TacOS のタイマー管理プログラム部分を示す。

- 割込みハンドラの初期化

tmrInit() 関数は OS 起動時に実行され、割込みハンドラ (tmrIntr()) 関数を 0xffe0 番地から始まる割込みベクタに登録し、タイマーのハードウェアを起動する。

- 割込みハンドラ

interrupt 型の tmrIntr() 関数が割込みハンドラである。図 19.2 の右半分に示すように、C++ 言語の interrupt 型関数は、自動的にコンテキストをスタックに保存する。図 19.2 では分かりやすさのために「ユーザモードのコンテキスト」としている。

tmrIntr() 関数はプロセステーブルの全てのプロセスについて (8 行)、タイマーの残り時間が 0 以下になるものを起こす (14 行)。タイマー待ちのプロセスは PCB の evtSem セマフォを用いてブロックしているので、iSemV() 関数^{*5}を用いてプロセスを起こす。

最後に、プロセス切換えの可能性があるなら yield() 関数^{*6}を呼び出し、プロセスを切り換えるチャンスを作る。プロセスが切り換わった場合は、20 行を実行中の状態でプロセスがブロックする。21 行で interrupt 関数が終了し、コンテキストが復元され、割込まれたプログラムが再開する。

^{*4} <https://github.com/tctsigemura/TacOS/blob/master/os/kernel/kernel.cmm> の一部である。

^{*5} 「20.5 V 操作ルーチン」を参照すること。

^{*6} 19.5 で詳しく説明する。

リスト 19.2: TacOS のタイマー管理プログラム

```

1  #define TICK 10                                // 割り込みは 10ms 単位
2
3  // タイマー割り込みハンドラ(10ms 毎に割り込みによって起動される)
4  interrupt tmrIntr() {
5      boolean disp = false;                      // ディスパッチの必要性
6
7      // 起きないといけないプロセスを起こしてまわる
8      for (int i=0; i<PRC_MAX; i=i+1) {
9          PCB p = procTbl[i];
10         if (p!=null && p.evtCnt>0) {             // タイマー稼働中なら
11             int cnt = p.evtCnt - TICK;           // 残り時間を計算
12             if (cnt<=0) {                         // 時間が来たら
13                 cnt = 0;                          // タイマーを停止し
14                 disp = iSemV(p.evtSem) || disp;    // プロセスを起こす
15             }
16             p.evtCnt = cnt;
17         }
18     }
19
20     if (disp) yield();                          // 必要ならディスパッチ
21 }
22
23 // タイマー初期化 : 割り込みベクタとハードウェアを初期化する
24 void tmrInit() {
25     int[] VECTOR = _ItoA(0xffe0);
26     VECTOR[0] = addrof(tmrIntr);                 // タイマー0 のベクタ初期化
27     out(0x0000, TICK);                          // タイマー0 に周期をセット
28     out(0x0002, 0x8001);                         // タイマー0 スタート
29 }

```

19.5 プロセス切換えプログラム

プロセス切換えプログラムは、コンテキストを保存する `yield()` 関数^{*7}と復旧する `dispatch()` 関数^{*8}からなる。`dispatch()` 関数は実行可能列の先頭にあるプロセスを選択しコンテキストを復旧する。

19.5.1 `yield()` 関数

リスト 19.3^{*9}に TaC のアセンブリ言語で記述した `yield()` 関数を示す。`yield()` 関数がカーネルモードのコンテキストをカーネルスタックに保存する。保存位置は図 19.2 に示した通りである。

19.4 で述べたように割込が発生すると、まず、割込みハンドラがユーザモードのコンテキストをカー

^{*7} 高級言語から `yield()` 関数を呼び出すと、アセンブリ言語の `_yield` ルーチンが実行される。

^{*8} 高級言語から `dispatch()` 関数を呼び出すと、アセンブリ言語の `_dispatch` ルーチンが実行される。

^{*9} <https://github.com/tctsigemura/TacOS/blob/master/os/kernel/dispatcher.s> の一部である。

リスト 19.3: TacOS のコンテキスト保存プログラム

```

1  _yield
2      ;--- G13(SP)以外の CPU レジスタと FLAG をカーネルスタックに退避 ---
3      push    g0                ; FLAG の保存場所を準備する
4      push    g0                ; G0 を保存
5      ld      g0,flag           ; FLAG を上で準備した位置に保存
6      st      g0,2,sp           ;
7      push    g1                ; G1 を保存
8      push    g2                ; G2 を保存
9      push    g3                ; G3 を保存
10     push    g4                ; G4 を保存
11     push    g5                ; G5 を保存
12     push    g6                ; G6 を保存
13     push    g7                ; G7 を保存
14     push    g8                ; G8 を保存
15     push    g9                ; G9 を保存
16     push    g10               ; G10 を保存
17     push    g11               ; G11 を保存
18     push    fp                ; フレームポインタ(G12)を保存
19     push    usp               ; ユーザモードスタックポインタ(G14)を保存
20     ;
21     ;----- G13(SP)を PCB に保存 -----
22     ld      g1,_curProc        ; G1 <- curProc
23     st      sp,0,g1           ; [G1+0] は PCB の sp フィールド
24     ;
25     ;----- [curProc の magic フィールド]をチェック -----
26     ld      g0,30,g1          ; [G1+30] は PCB の magic フィールド
27     cmp     g0,#0xabcd        ; P_MAGIC と比較、一致しなければ
28     jnz     .stkOverflow       ; カーネルスタックがオーバーフローしている

```

ネルスタックに保存する。次に、カーネルモードでマイクロカーネル内のプログラムが実行される。この時点では、割込み前のプロセスの一部として実行されている。最後に、マイクロカーネル内のプログラムがプロセスを切換えるために `yield()` を呼び出す。`yield()` はカーネルモードのコンテキストをカーネルスタックに追加保存し、新しいプロセスの実行に切換える。次に、リスト 19.3 の内容を解説する。

- 1 行 プロセスを切換える時にマイクロカーネル内で呼び出される `yield()` 関数の入口である。`yield()` 関数は、現在プロセスのカーネルモードのコンテキストを保存し CPU を解放する。
- 2～19 行 プロセスのカーネルモードのコンテキストをスタックに保存する処理である。`yield()` が (CALL 命令で) 呼び出された時点で PC はスタックに格納されている。後で RETI 命令で PC と FLAG を同時に復旧するので PC の次に FLAG を格納している (6 行)。
- 21～23 行 プロセスのカーネルモードのコンテキストを保存したスタックの位置を PCB に保存す

る。_curProc 変数には現在のプロセスの PCB を指すポインタが保存されている (図 19.1 参照)。PCB 先頭の sp 領域 (リスト 19.1 参照) にスタックポインタを保存する。ここまでの処理でコンテキストの保存が完了した。

25~28 行 カーネルスタックがオーバーフローしていないか調べる。PCB の magic フィールドの値をチェックし 0xabcd 以外の値になっていたら、カーネルスタックが隣接する PCB まで伸びた (オーバーフローした) と判断する。この場合、.stkOverflow ルーチンにジャンプしシステムを停止する。カーネルのエラーなので復旧は諦める。オーバーフローが検知されない場合は次の行に進む。次の行はリスト 19.4 の 1 行に示す _dispatch である。jnz でジャンプしなかった場合はディスパッチャに進み次のプロセスを再開する。

19.5.2 dispatch() 関数

リスト 19.4^{*10}に TaC のアセンブリ言語で記述した dispatch() 関数のソースプログラムを示す。dispatch() 関数がカーネルモードのコンテキストをカーネルスタックから復旧する。復旧するコンテキストは図 19.2 に示したように保存されている。次にリスト 19.4 の内容を解説する。

1 行 プロセスに CPU を割付けるディスパッチャ (dispatch() 関数) の入口である。

2~6 行 まず、実行可能列 (_readyQueue) の先頭 PCB のアドレスを _curProc 変数にセットする。実行可能列は番兵付きの双方向循環リストなので、番兵の次が先頭の PCB である (図 19.1 参照)。実行可能なプロセスが無い場合は idle プロセスが選択される。_curProc が更新されたので、新しいプロセスが現在のプロセスになった。次に、現在のプロセスのスタックポインタ (sp) を PCB から復旧する。

8~22 行 スタックポインタが復旧されたので、スタックから CPU レジスタを復旧する。

24~25 行 RETI 命令を用いて PSW (FLAG と PC) を復旧し、前回プロセスが yield() を呼び出した位置に戻る。yield() を呼び出した位置に戻るために RET 命令ではなく RETI 命令を使用するのは、プロセス生成時は例外的に、この RETI で実行モードを切換えてユーザプログラムの実行を開始するからである。

19.6 スケジューラ

実行可能になったプロセスをスケジューリングするプログラムをスケジューラと呼ぶ。スケジューラの例をリスト 19.5 に示す^{*11}。このプログラムは、新しく実行可能になったプロセスを実行可能列に挿入する。その際、実行可能列がプロセスの優先度順にソートされるようにする。dispatch() 関数は実行可能列の先頭のプロセスを実行するので、プロセスは優先度が高い順に実行される。実行可能列は PCB の番兵付き双方向循環リストとして表現する (図 19.1 参照)。次にリスト 19.5 の内容を解説する。

1 行 schProc() 関数がスケジューラである。

2 行 setPri() 関数は PSW の割込み許可フラグを操作する^{*12}。schProc() 関数はプロセス間の共

^{*10} <https://github.com/tctsigemura/TacOS/blob/master/os/kernel/dispatcher.s> の一部である。

^{*11} <https://github.com/tctsigemura/TacOS/blob/master/os/kernel/kernel.cmm> の一部である。

^{*12} 詳しくは「20.6 setPri() 関数」で説明する。

リスト 19.4: TacOS のコンテキスト復旧プログラム

```

1  _dispatch
2      ;----- 次に実行するプロセスの G13(SP)を復元 -----
3      ld      g0,_readyQueue  ; 実行可能列の番兵のアドレス
4      ld      g0,28,g0        ; [G0+28] は PCB の next フィールド(先頭の PCB)
5      st      g0,_curProc     ; 現在のプロセス(curProc)に設定する
6      ld      sp,0,g0         ; PCB から SP を取り出す
7      ;
8      ;----- G13(SP)以外の CPU レジスタを復元 -----
9      pop     usp             ; ユーザモードスタックポインタ(G14)を復元
10     pop     fp              ; フレームポインタ(G12)を復元
11     pop     g11             ; G11 を復元
12     pop     g10             ; G10 を復元
13     pop     g9              ; G9 を復元
14     pop     g8              ; G8 を復元
15     pop     g7              ; G7 を復元
16     pop     g6              ; G6 を復元
17     pop     g5              ; G5 を復元
18     pop     g4              ; G4 を復元
19     pop     g3              ; G3 を復元
20     pop     g2              ; G2 を復元
21     pop     g1              ; G1 を復元
22     pop     g0              ; G0 を復元
23     ;
24     ;----- PSW(FLAG と PC)を復元 -----
25     reti                     ; RETI 命令で一度に POP して復元する

```

リスト 19.5: TacOS のスケジューラ・ソースプログラム

```

1 public void schProc(PCB proc) {
2     int r = setPri(DI|KERN);           // 割り込み禁止、カーネル
3     int enice = proc.enice;
4     PCB head = readyQueue.next;        // 実行可能列から
5     while (head.enice<=enice)           // 優先度がより低い
6         head = head.next;               // プロセスを探す
7     insProc(head,proc);                 // 見つけたプロセスの
8     setPri(r);                           // 直前に挿入する
9 }                                       // 割り込み状態を復元する

```

有資源である実行可能列を操作するので、クリティカルセクションである。schProc() 関数は、サーバプロセスから割込み許可状態で呼ばれることがある。プロセスがプリエンブションしないように、ここで割込み禁止にしている。

3 行 enice がプロセスの優先度である。enice は値が小さい方が優先度が高い。

4 行 スケジューラは、実行可能列 (readyQueue) を番兵 PCB の次の PCB から探索する。

5~6 行 挿入するプロセスの enice より大きいものを探す。実行可能列の最後には、常時 idle プロセスの PCB が置かれている (図 19.1 参照)。idle の enice は最大値に設定されているのでループは必ず正常に終了する。

7 行 大きいものを見つけたら insProc() を使用して、見つけた PCB の直前に新しいプロセスの PCB を挿入する。

現在の実装では、enice はプロセス生成時に nice と同じ値に設定され、その後は変化しない。TacOS は静的優先度を用いる優先度順スケジューリング方式を用いる。将来、enice の値を動的に変化させるように変更すれば、動的優先度方式になる。

19.7 まとめ

TacOS が時分割多重方式で CPU を仮想化する方法について解説した。PCB とカーネルスタックを合わせてコンテキストの保存が行われる。カーネルスタックはプロセス毎に割当てられコンテキストの大半が保存される。PCB にはプロセスの SP だけを保存する。

また、タイマー処理を例に、割込みが発生してプロセスが切替わるまでの処理概要を説明した。ユーザプログラム実行中に割込みが発生すると、カーネルモードに切替わり割込みハンドラに制御が移る。ハンドラはユーザモードのコンテキストをプロセスのカーネルスタックに保存し、その後マイクロカーネル内で必要な処理を行う。処理の結果によりプロセスを切替える場合は yield() 関数を呼ぶ。

更に、プロセス切替えプログラムについて仕組みを解説した。yield() 関数はアセンブリ言語で記述されており、プロセスのカーネルモードのコンテキストを保存する。dispatch() 関数もアセンブリ言語で記述されており、プロセスのカーネルモードのコンテキストを復旧する。コンテキストを復旧すると、以前に yield() 関数を呼び出した次の行からプロセスの実行が再開される。

最後に、スケジューラの例を紹介した。schProc() 関数は、実行可能列のプロセスが優先度順 (enice 順) の線形リストになるように、新しいプロセスをリストに追加する。

第 20 章

TacOS のセマフォ

TacOS ではプロセス同期の基本機構としてセマフォを用いる。セマフォ機構はマイクロカーネルが提供する。今のところ TacOS でセマフォを利用できるのは、カーネルモードで実行されるサーバプロセスとマイクロカーネルだけである。

20.1 データ構造

セマフォはリスト 20.1 に示す構造体である^{*1}。また、図 20.1 にセマフォ関連データの構造を示す。semTbl はセマフォの一覧である。システム起動時に SEM_MAX 個（30 個）のセマフォを準備し semTbl に登録する。semInUse はセマフォが使用中かどうかを記録する論理型の配列である。セマフォが必要になった時に、一覧の中から空きセマフォを選んで使用する。セマフォは一覧のインデクス（セマフォ番号）で識別するので、P 操作や V 操作を行う関数の引数がセマフォ番号になる。

セマフォ構造体（Sem 構造体型）は、セマフォの値（cnt）とプロセスの待ち行列（queue）を持っている。cnt はカウンタなので、TacOS のセマフォはカウンティングセマフォである。セマフォの数は 30 個に固定されており、システム起動時に Sem 構造体と番兵 PCB で初期化される。プロセスの待ち行列は PCB の双方向循環リストとして表現される。待ち行列は到着順にソートされ FIFO を構成するので、セマフォ待ちプロセスは *FCFS 方式のスケジューリング* を受ける。次に、図 20.1 で表している三つのセマフォについて説明する。

リスト 20.1: TacOS のセマフォ構造体

```
#define SEM_MAX 30          // セマフォは最大 30 個

struct Sem {                // セマフォを表す構造体
    int cnt;                 // カウンタ
    PCB queue;               // 待ち行列
};
```

^{*1} <https://github.com/tctsigemura/TacOS/blob/master/os/kernel/process.hmm> の一部である。

リスト 20.2: TacOS でのセマフォの架空の使用例

```

1 #include <kernel.h>
2 int      account;           // スレッド間の共有変数 (残高)
3 int      accSem;            // accountのロック用セマフォの番号
4 void initProc() {           // プロセスの初期化ルーチン
5     accSem = newSem(1);      // 初期値1のセマフォを確保する
6 }
7 void receiveThread() {      // 入金管理スレッド
8     for ( ; ; ) {           // 入金管理スレッドは以下を繰り返す
9         int receipt = receiveMoney(); // ネットワークから入金を受信する
10        semP( accSem );       // account変数をロックするための P 操作
11        account = account + receipt; // account変数を変更する(クリティカルセクション)
12        semV( accSem );       // account変数をロック解除するための V 操作
13    }
14 }
15 void payThread() {          // 引落し管理スレッド
16     for ( ; ; ) {           // 引落し管理スレッドは以下を繰り返す
17         int payment = payMoney(); // ネットワークから入金を受信する
18         semP( accSem );       // account変数をロックするための P 操作
19         account = account - payment; // account変数を変更する(クリティカルセクション)
20         semV( accSem );       // account変数をロック解除するための V 操作
21    }
22 }

```

20.3 割当て

リスト 20.3 にセマフォ割当てと解放ルーチンを示す^{*2}.

データ構造 1 行の `semTbl`, 2 行の `semInUse` は, 図 20.1 に描かれている「セマフォ一覧」と「使用中のセマフォ」のことである. `semTbl` は TacOS の起動時に「Sem 構造体」や「番兵 PCB」で初期化される.

割込み禁止による相互排除 5 行の `newSem()` 関数が `semTbl` から未使用のセマフォを探す. `newSem()` 関数や後述の `semP()`, `semV()` 関数は, 複数のプロセスから並列に呼び出され `semTbl` や `semInUse` をアクセスする. これらのデータ構造はプロセス間の共有データである. `newSem()` 関数の内部はこれら共有データのクリティカルセクションに当たるので相互排他が必要である. TaC はシングルプロセッサシステムなので, 5.3.1 で紹介した「割込み禁止による相互排除」を行う.

6 行では, 現在のフラグ^{*3}の値を `r` に保存した後, 「割込み禁止 (DI)」にしている. `setPri()` 関数はフラグの値を読み出し, 同時に引数値をフラグにセットするアセンブリ言語ルーチンである^{*4}.

^{*2} <https://github.com/tctsigemura/TacOS/blob/master/os/kernel/kernel.cmm> の一部である.

^{*3} CPU の PSW のフラグのこと.

^{*4} `setPri()` 関数の詳細は「20.6 `setPri()` 関数」を参照のこと

リスト 20.3: TacOS のセマフォ割当て解放ルーチン

```

1 Sem[] semTbl=array(SEM_MAX); // セマフォの一覧表
2 boolean[] semInUse=array(SEM_MAX); // どれが使用中か (falseで初期化)
3
4 // セマフォの割当て
5 public int newSem(int init) {
6     int r = setPri(DI|KERN); // 割り込み禁止、カーネル
7     for (int i=0; i<SEM_MAX; i=i+1) { // 全てのセマフォについて
8         if (!semInUse[i]) { // 未使用のものを見つけたら
9             semInUse[i] = true; // 使用中に変更し
10            semTbl[i].cnt = init; // カウンタを初期化し
11            setPri(r); // 割り込み状態を復元し
12            return i; // セマフォ番号を返す
13        }
14    }
15    panic("newSem"); // 未使用が見つからなかった
16    return -1; // ここは実行されない
17 }
18
19 // セマフォの解放
20 // (書き込み1回で仕事が終わるので割り込み許可でも大丈夫)
21 public void freeSem(int s) {
22     semInUse[s] = false; // 未使用に変更
23 }

```

`newSem()` 関数はカーネルモードで呼び出すので、実行モードが変化しないように「カーネルモード (KERN)」も指定している。

7行からのループで使用されていないセマフォを探す。割り込み禁止で実行するので探索の途中でプリエンブションは発生しない。未使用のセマフォが見つかったら12行でその番号を返す。

クリティカルセクションが終わるので、通常は割り込みを許可するが、`newSem()` 関数を呼び出す前から割り込み禁止だった場合もある。11行では6行で保存した `r` を用いてフラグの状態を復旧している。もともと `newSem()` 関数が割り込み許可状態で呼び出された場合だけ割り込み許可状態に戻る。

エラー処理 未使用のセマフォが見つからなかった場合は、15行で `panic()` 関数を呼び出す。現在の TacOS では、セマフォを使用できるのはマイクロカーネルとサーバプロセスだけである。セマフォが不足するようならオペレーティングシステムのバグである。`panic()` 関数はエラーメッセージを表示した後、CPU を停止する。`panic()` 関数は戻ってこないなので16行は実行されない。

解放ルーチン 21行の `freeSem()` は割当てられていたセマフォを解放する。共有変数 `semInUse` 配列の書き換えは、単一のストア機械語命令で終了するので割り込み禁止にする必要はない。

リスト 20.4: TacOS の P 操作ルーチン

```

1 public void semP(int sd) {
2     int r = setPri(DI|KERN);                // 割り込み禁止、カーネル
3     if (sd<0 || SEM_MAX<=sd || !semInUse[sd]) // 不正なセマフォ番号
4         panic("semP(%d)", sd);
5
6     Sem s = semTbl[sd];
7     if(s.cnt>0) {                            // カウンタから引けるなら
8         s.cnt = s.cnt - 1;                    // カウンタから引く
9     } else {                                  // カウンタから引けないなら
10        delProc(curProc);                     // 実行可能列から外し
11        curProc.stat = P_WAIT;                // 待ち状態に変更する
12        insProc(s.queue, curProc);            // セマフォの行列に登録
13        yield();                              // CPUを解放し
14    }                                          // 他プロセスに切替える
15    setPri(r);                                // 割り込み状態を復元する
16 }

```

20.4 P 操作ルーチン

リスト 20.4 に P 操作ルーチンを示す^{*5}。P 操作ルーチンは semP() 関数のことである。

割り込み禁止による相互排除 semP() 関数も、semInUse や、semTbl の配下の Sem 構造体、PCB 構造体等の共有データをアクセスするので相互排除を必要とする。semP() 関数の内部は 2 行と 15 行の setPri() 関数を用いて、割り込み禁止による相互排除を行っている。

セマフォ番号からセマフォ構造体への変換 3 行で引数のセマフォ番号が正当なものかチェックしている。不正なものが渡されるようならオペレーティングシステムのバグなので panic() 関数を用いてシステムを停止させる。セマフォ番号が正しい場合は、6 行で semTbl 配列から目的のセマフォを見つける。

セマフォ値のデクリメント 7 行でセマフォの値を調べ、1 以上なら 8 行で値を 1 減らす。この場合は 15 行で割り込み許可フラグを復元して semP() 関数を終了する。

Block(事象待ち) 7 行でセマフォの値を調べ、1 未満なら 10 行に進み現在のプロセスをブロック^{*6}する。ブロックの手順は次の通りである。

1. delProc() 関数を用いて現在のプロセスを実行可能列から外す。
2. 現在のプロセスの状態を「待ち状態 (P_WAIT)」に変更する。
3. 現在のプロセスをセマフォの待ち行列の最後に追加する^{*7}。
4. yield() 関数を呼び出し CPU を解放する。後でセマフォが V 操作されプロセスが実行可能

^{*5} <https://github.com/tctsigemura/TacOS/blob/master/os/kernel/kernel.cmm> の一部である。

^{*6} プロセスのブロック (Block: 事象待ち) については、「3.2 プロセスの状態」を参照のこと。

^{*7} insProc() 関数を用いて番兵 PCB の直前に挿入する。循環リストで番兵 PCB の直前は最後尾のことになる。

リスト 20.5: TacOS の PCB リスト操作関数

```

1 // プロセスキューでp1の前にp2を挿入する p2 -> p1
2 void insProc(PCB p1, PCB p2) {
3     p2.next=p1;
4     p2.prev=p1.prev;
5     p1.prev=p2;
6     p2.prev.next=p2;
7 }
8
9 // プロセスキュー(実行可能列やセマフォの待ち行列)で p を削除する
10 void delProc(PCB p) {
11     p.prev.next=p.next;
12     p.next.prev=p.prev;
13 }

```

になったら、yield() 関数から実行が再開される。

なお、ここで使用している delProc() と insProc()^{*8}は、リスト 20.5 のような PCB リスト操作関数である。yield() 関数はリスト 19.3 に示したプロセス切換えプログラムである。

20.5 V 操作ルーチン

リスト 20.6 に V 操作ルーチンを示す^{*9}。V 操作ルーチンは iSemV() と semV() の二種類がある。iSemV() 関数はセマフォに V 操作だけ行う。semV() 関数はセマフォに V 操作を行った後で、プロセス切換えを試みる。semV() 関数を用いると、V 操作によって実行可能になったプロセスの優先度が現在のプロセスの優先度より高い場合に、プロセスが切り換わる。iSemV() はマイクロカーネル内部でプリエンプションを避けたい場合に使用する。

割込み禁止による相互排除 iSemV() 関数や semV() 関数も相互排除を必要とする。semV() 関数は 28 行と 32 行の setPri() 関数を用いて、割込み禁止による相互排除を行っている。iSemV() 関数は、呼び出し側で割込み禁止にして使用する。

セマフォ番号からセマフォ構造体への変換 5 行でセマフォ番号の妥当性をチェックしてから、9 行で semTbl 配列から目的のセマフォを見つける。

セマフォ値のインクリメント 12 行で待ち行列の状態を調べる。番兵 PCB (q) と番兵直後の PCB (p) が同じなら待ち行列は空である^{*10}。待ち行列が空の場合は 13 行でセマフォの値を 1 増やし false を返り値として iSemv() 関数を終了する。

Complete(事象完了) 12 行で待ち行列を調べ空でないなら 15 行に進み、待ち行列の先頭のプロセスを

^{*8} <https://github.com/tctsigemura/TacOS/blob/master/os/kernel/kernel.cmm> の一部である。

^{*9} <https://github.com/tctsigemura/TacOS/blob/master/os/kernel/kernel.cmm> の一部である。

^{*10} 図 20.1 の「Sem 構造体 (#29)」を参照のこと。

リスト 20.6: TacOS の V 操作ルーチン

```

1 // ディスパッチを発生しないセマフォのV操作
2 // (V 操作をしたあとまだ仕事があるとき使用する)
3 // (kernel 内部専用、割込み禁止で呼出す)
4 boolean iSemV(int sd) {
5     if (sd<0 || SEM_MAX<=sd || !semInUse[sd]) {           // 不正なセマフォ番号
6         panic("iSemV(%d)", sd);
7     }
8     boolean ret = false;                                     // 起床するプロセスなし
9     Sem s = semTbl[sd];                                     // 操作するセマフォ
10    PCB q = s.queue;                                         // 待ち行列の番兵
11    PCB p = q.next;                                          // 待ち行列の先頭プロセス
12    if(p==q) {                                               // 待ちプロセスが無いなら
13        s.cnt = s.cnt + 1;                                   // カウンタを足す
14    } else {                                                 // 待ちプロセスがあるなら
15        delProc(p);                                          // 待ち行列から外す
16        p.stat = P_RUN;                                     // 実行可能に変更
17        schProc(p);                                          // 実行可能列に登録
18        ret = true;                                          // 起床するプロセスあり
19    }
20    return ret;                                              // 実行可能列に変化があった
21 }
22
23 // セマフォの V 操作
24 // 待ちプロセス無し : カウンタを1増やす
25 // 待ちプロセス有り : 待ち行列からプロセスを1つ外して実行可能にした後、
26 //                      ディスパッチャを呼び出す
27 public void semV(int sd) {
28     int r = setPri(DI|KERN);                                // 割り込み禁止、カーネル
29     if (iSemV(sd)) {                                        // V 操作し必要なら
30         yield();                                             // プロセスを切り替える
31     }
32     setPri(r);                                              // 割り込み状態を復元する
33 }

```

起床させる。先頭のプロセスは Complete(事象完了)^{*11}の状態遷移をする。15 行でセマフォの待ち行列から先頭プロセスを外し、16 行でプロセスの状態を実行可能 (P_RUN) に変更し、17 行でスケジューラ (schProc() 関数)^{*12}に依頼し実行可能列の適切な位置に挿入する。この場合は true を返り値として iSemv() 関数を終了する。

プロセスの切換え semV() 関数は、V 操作により実行可能列に新しいプロセスが追加された場合

^{*11} プロセスの Complete(事象完了) については、「3.2 プロセスの状態」を参照のこと。

^{*12} スケジューラ (schProc() 関数) はリスト 19.5 で定義されている。

リスト 20.7: TacOS のフラグ操作ルーチン

```

1 ;; CPU のフラグの値を返すと同時に新しい値に変更
2 _setPri
3     ld      g0,2,sp      ; 引数の値を G0 に取り出す
4     push    g0           ; 新しい状態をスタックに積む
5     ld      g0,flag      ; 古いフラグの値を返す準備をする
6     reti                    ; reti は FLAG と PC を同時に pop する

```

(iSemv() 関数が true で返った場合) に yield() 関数を呼び出す。実行可能列に現在のプロセスより優先度の高いものがあった場合、プロセスの切換えが起こる。

TacOS のプロセス同期機構は全てセマフォに基づいて構成される。例えば、メッセージ通信機構もセマフォを利用して構築されている。

20.6 setPri() 関数

割込み禁止による相互排除で使った setPri() 関数のソースプログラムをリスト 20.7 に示す^{*13}。setPri() 関数は CPU の PSW のフラグを参照・操作し、呼び出し前のフラグ状態を呼び出し側に返すと同時に、フラグを新しい値に変更する。フラグに CPU の割込許可ビットがあるので、setPri() 関数は、CPU の割込みの許可・不許可状態を変更するために使用できる。

setPri() 関数は TaC のアセンブリ言語で記述してある。C--言語から setPri という名前で参照されるためには、アセンブリ言語では _setPri というラベルを宣言する必要がある。2 行は setPri() 関数の入口になるラベルを宣言している。

C--言語プログラムは関数引数をスタックに積んで渡す^{*14}。3 行では C--言語が setPri() 関数に渡した引数を G0 に読み出している。4 行で読み出した値をスタックに積み直す。

5 行では現在のフラグ値を G0 にコピーする。flag は PSW のフラグを意味している。C--言語では関数の戻り値を G0 レジスタに入れて返すので^{*15}、この値は setPri() 関数の戻り値になる。6 行の reti 機械語命令は、スタックからフラグと PC の値を取り出し、setPri() 関数を呼び出した場所に制御を戻す。この時、4 行でスタックに積んだ値が PSW のフラグに読み出される。

以上の仕組みで、setPri() 関数は引数の値をフラグにセットすると同時に、以前のフラグ値を呼び出し側に返している。

20.7 まとめ

本章では、TacOS のセマフォが、どのように実装されているかを学んだ。セマフォ機構はマイクロカーネルによって提供され、サーバプロセスとマイクロカーネルが使用できる。セマフォのユーザは、セマフォ番号を用いてセマフォを指定する。マイクロカーネル内部でセマフォは Sem 構造体で表現され

^{*13} <https://github.com/tctsigemura/TacOS/blob/master/os/util/crt0.s> の一部である。

^{*14} C 言語などで関数に引数を渡す仕組みは同様である

^{*15} C 言語などで関数値を返す仕組みは同様である

る。Sem 構造体は、カウンタと、PCB の待ち行列を保持する。TacOS のセマフォはカウンティングセマフォである。プロセスの待ち行列は PCB の双方向循環リストであり FIFO を構成する。

セマフォを用いて相互排除を行う例を示した。newSem() 関数は確保したセマフォの番号を返す。semP() 関数と semV() (iSemV()) 関数はセマフォ番号を引数にし、セマフォを操作する。

newSem() 関数、semP() 関数、semV() (iSemV()) 関数のソースコードを示し内容を解説した。これらの関数の内部では割込み禁止による相互排除が行われていた。P 操作を行う semP() 関数は、ブロックするプロセスの PCB をセマフォの待ち行列の最後に追加する。V 操作を行う semV() (iSemV()) 関数は待ち行列の先頭のプロセスを実行可能列に移動する。セマフォの待ち行列は FIFO 方式になっている。

練習問題

- 20.1 TaC をマルチプロセッサシステムに進化させた時、「リスト 20.4 TacOS の P 操作ルーチン」をどのように改造する必要があるか？ (Sem 構造体を変更しない場合)
- 20.2 TaC をマルチプロセッサシステムに進化させた時、「リスト 20.4 TacOS の P 操作ルーチン」をどのように改造する必要があるか？ (Sem 構造体も変更して良い場合)

第 21 章

TacOS のメッセージ通信

TacOS ではマイクロカーネルがメッセージ通信機構を提供し、ユーザ・プロセスとサーバプロセス、サーバプロセスとサーバプロセスの通信にメッセージを用いる。

21.1 メッセージ通信機構

TacOS のメッセージ通信機構は、クライアントプロセスがサーバプロセスの機能を利用する、クライアント・サーバの通信に特化したランデブー方式である。図 21.1 に TacOS のメッセージ通信の様子を示す。メッセージ通信の手順は次のようになる。

1. サーバプロセスがリンクを所有し他プロセスからの通信を待ち受ける。
2. クライアントプロセスは `sndrec()` 関数を用いてリンクに処理内容をメッセージとして送信する。
3. サーバプロセスは `receive()` 関数を用いてメッセージを受信する。
4. サーバプロセスはメッセージの内容に合った処理を行う。
5. サーバプロセスは処理結果を `send()` 関数を用いて返信する。
6. `sndrec()` 関数が完了し、クライアントプロセスは処理結果を受取る。

TacOS のメッセージ通信機構は、**間接指定方式**、**固定長**、**ランデブー方式**と言える。クライアントプロセスとサーバプロセスが並列に処理することができないが、サービスモジュールをサーバプロセスにすることができ、オペレーティングシステムを構築するためのプログラミングを容易にしている。

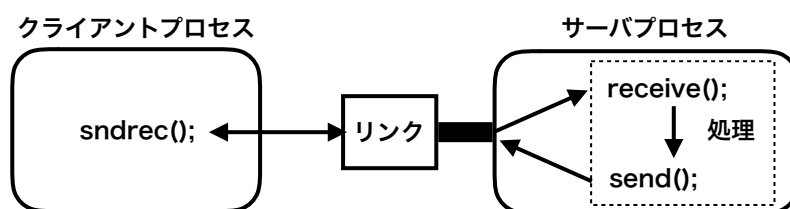


図 21.1: TacOS のメッセージ通信

リスト 21.1: TacOS のリンク構造体

```

1 #define LINK_MAX 5          // リンクは最大 5 個
2
3 struct Link {               // リンクを表す構造体
4     PCB server;             // リンクを所持するサーバ
5     PCB client;             // リンクを使用中のクライアント
6     int s1;                 // サーバがメッセージ受信待ちに使用するセマフォ
7     int s2;                 // クライアント同士が相互排除に使用するセマフォ
8     int s3;                 // クライアントがメッセージ返信待ちに使用するセマフォ
9     int op;                 // メッセージの種類
10    int prm1;                // メッセージのパラメータ1
11    int prm2;                // メッセージのパラメータ2
12    int prm3;                // メッセージのパラメータ3
13 };

```

21.2 リンク構造体

リスト 21.1 にリンク構造体の宣言を示す^{*1}。Link 構造体はランデブー用のリンクを定義している。server はリンクを所有するサーバプロセスの PCB、client はリンクを使用中のクライアントプロセスの PCB である。s1, s2, s3 は相互排除と同期のために使用されるセマフォである。TacOS のリンクはセマフォを基盤にしている。op, prm1, prm2, prm3 が固定長のメッセージ本体になる。

21.3 リンクの作成

リスト 21.2 に、TacOS のマイクロカーネル内のリンク作成ルーチンを示す^{*2}。newLink() 関数はサーバプロセスがリンクを作り所有するために呼び出す。TacOS のサーバプロセスはカーネルモードで実行され、マイクロカーネル内ルーチンを呼び出すことができる。複数のプロセスが newLink() 関数を呼び出す可能性があるので、6 行から 19 行の範囲は割込み禁止による相互排除を行っている。

空きリンクは 7 行で管理している。リンクの廃棄手段は準備されていないので、空きリンクの管理は単純である。15 行でリンクを所有するサーバプロセスを記録する。16, 17, 18 行で、三つのセマフォをリンクに割当てている。20 行では作成したリンクの番号を返している。

21.4 サーバ用のメッセージ通信ルーチン

マイクロカーネル内にある、サーバプロセス用のメッセージ通信プログラムをリスト 21.3 に示す^{*3}。2 行の receive() 関数はメッセージの受信に使用する。引数 num は newLink() が返したリンク番号である。4 行でリンクの所有者を調べている。所有者が自身ではないならオペレーティングシステムのバグなので panic() 関数を用いてシステムを停止する。5 行で初期値 0 のセマフォ (s1) に P 操作を

^{*1} <https://github.com/tctsigemura/TacOS/blob/master/os/kernel/process.hmm> の一部である。

^{*2} <https://github.com/tctsigemura/TacOS/blob/master/os/kernel/kernel.cmm> の一部である。

^{*3} <https://github.com/tctsigemura/TacOS/blob/master/os/kernel/kernel.cmm> の一部である。

リスト 21.2: TacOS のリンク作成ルーチン

```

1 Link[] linkTbl = array(LINK_MAX);           // リンクの一覧表
2 int linkID = -1;                             // リンクの通し番号
3
4 // リンクを生成する (サーバが実行する)
5 public int newLink() {
6     int r = setPri(DI|KERN);                 // 割り込み禁止、カーネル
7     linkID = linkID + 1;                     // 通し番号を進める
8 #ifdef DEBUG
9     printf("newLink:ID=%d,SERVER=%d\n",linkID,curProc.pid);
10 #endif
11     if (linkID >= LINK_MAX)                  // リンクが多すぎる
12         panic("newLink");
13
14     Link l = linkTbl[linkID];                // 新しく割り当てるリンク
15     l.server = curProc;                      // リンクの所有者を記憶
16     l.s1 = newSem(0);                        // server が受信待ちに使用
17     l.s2 = newSem(1);                        // client が相互排他に使用
18     l.s3 = newSem(0);                        // client が返信待ちに使用
19     setPri(r);                               // 割り込み復元
20     return linkID;                           // 割当てたリンクの番号
21 }

```

行い、クライアントがリンクにデータを書き込むのを待つ。6行でデータが書き込まれたリンクを返す。

10行の `send()` 関数は、クライアントプロセスにメッセージを返信するために使用する。引数 `num` はリンク番号、`res` は返信するデータである。サーバが行った処理の結果を16ビット（2バイト）で表現する。12行では `receive()` 関数と同様にリンクの所有者を調べている。13行で処理結果をリンクに書き込み、14行でクライアントが待ち合わせているセマフォ（`s3`）にV操作を行う。これでクライアントが処理結果を受取り処理を再開する。

21.5 サーバプロセスの例

リスト 21.4 にサーバプロセスの例として、プロセスマネージャのメインルーチンを示す^{*4}。プロセスマネージャは `exec` システムコール等を処理するサーバプロセスである。3行でリンクを作成し `pmLink` グローバル変数^{*5}に記録する。5行でクライアントプロセスからのメッセージを待ち受ける。メッセージを受信したら6行に進み、リンクに書き込まれていた内容とクライアントプロセスのPCBを引数に、プロセスマネージャのシステムコール処理ルーチンを実行する。処理結果は7行の `send()` 関数を用いてクライアントプロセスに返信する。

^{*4} <https://github.com/tctsigemura/TacOS/blob/master/os/pm/pm.cmm> の一部である。

^{*5} <https://github.com/tctsigemura/TacOS/blob/master/os/pm/pm.hmm> で宣言されている。

リスト 21.3: TacOS のメッセージ通信ルーチン (サーバ用)

```

1 // サーバ用の待ち受けルーチン
2 public Link receive(int num) {
3     Link l = linkTbl[num];
4     if (l.server != curProc) panic("receive");           // 登録されたサーバではない
5     semP(l.s1);                                           // サーバをブロック
6     return l;
7 }
8
9 // サーバ用の送信ルーチン
10 public void send(int num, int res) {
11     Link l = linkTbl[num];
12     if (l.server != curProc) panic("send");             // 登録されたサーバではない
13     l.op = res;                                           // 処理結果を書込む
14     semV(l.s3);                                           // クライアントを起こす
15 }

```

リスト 21.4: TacOS のメッセージ通信使用例 (サーバ側)

```

1 // プロセスマネージャサーバのメインルーチン
2 public void pmMain() {
3     pmLink = newLink();                                  // リンクを生成する
4     while (true) {                                       // システムコールを待つ
5         Link l = receive(pmLink);                       // システムコールを受信
6         int r=pmSysCall(l.op,l.prm1,l.prm2,l.prm3,l.client); // システムコール実行
7         send(pmLink, r);                                 // 結果を返す
8     }
9 }

```

21.6 クライアント用のメッセージ通信ルーチン

リスト 21.5 に TacOS のクライアントプロセス用のメッセージ通信プログラム (sndrec()) を示す*6。sndrec() 関数はサーバプロセスのリンクにメッセージを書込み、サーバプロセスに処理を依頼する。サーバプロセスが処理を完了したら、sendrec() 関数は処理結果を返り値として終了する。

4 行では初期値 1 のセマフォ (s2) を用いてリンクをロックし、他のクライアントプロセスとの相互排除を行っている。6 行から 9 行でリンクにメッセージを書き込む。iSemV() 関数を使用するために、10 行から 13 行まで割込み禁止による相互排除を行っている。11 行でメッセージを書き込んだことをサーバに知らせ、12 行で初期値 0 のセマフォ (s3) に P 操作を行いサーバが処理を終了するのを待つ。サーバの処理が終了したら 14 行に進みサーバがリンクに書き込んだ処理結果を取り出す。15 行でリン

*6 <https://github.com/tctsigemura/TacOS/blob/master/os/kernel/kernel.cmm> の一部である。

リスト 21.5: TacOS のメッセージ通信ルーチン (クライアント用)

```

1 // クライアント用メッセージ送受信ルーチン
2 public int sndrec(int num, int op, int prm1, int prm2, int prm3) {
3     Link l = linkTbl[num];           // 他のクライアントと相互
4     semP(l.s2);                      //  排除しリンクを確保
5     l.client = curProc;              // リンク使用中プロセス記録
6     l.op = op;                       // メッセージを書込む
7     l.prm1 = prm1;
8     l.prm2 = prm2;
9     l.prm3 = prm3;
10    int r = setPri(DI|KERN);          // 割り込み禁止、カーネル
11    iSemV(l.s1);                      // サーバを起こす
12    semP(l.s3);                      // 返信があるまでブロック
13    setPri(r);                       // 割り込み復元
14    int res = l.op;                  // 返信を取り出す
15    semV(l.s2);                      // リンクを解放
16    return res;
17 }

```

リスト 21.6: TacOS のメッセージ通信使用例 (クライアント側)

```

1 public int exec(char[] path, char[][] argv) {
2     int r=sndrec(pmLink,EXEC,_Atoi(path),_Atoi(argv),0);
3     return r;                       // 新しい子の PID を返す
4 }

```

クのロックを解除し 16 行で処理結果を持って関数を終了する。

21.7 クライアントプロセスの例

リスト 21.6 にメッセージ通信機構のクライアント側の例として、プロセスマネージャ (サーバプロセス) に `exec` システムコールの処理を依頼するプログラムを示す^{*7}。TacOS の `exec` システムコールは、新しいプロセスを作ってプログラムを実行させる。引数はプログラムファイルのパス名 (`path`) と、新しいプログラムの `main()` 関数に渡すコマンド行引数 (`argv`) である。

1 行はクライアントプロセスの `exec` システムコールの入口になる。カーネルモードで動作する他のサーバプロセスは `exec()` 関数を直に呼び出す。ユーザモードで動作するユーザプロセスは SVC 機械語命令で割り込みを発生し、SVC 割り込みハンドラから `exec()` 関数を呼び出す。割り込みハンドラは現在のプロセスのコンテキストで実行されるので、`exec()` 関数はカーネルモードに切り換わった状態のユーザプロセスによって実行されることになる。

2 行でプロセスマネージャ (サーバプロセス) とランデブーを行う。 `pmLink` はリスト 21.4 でプロセ

^{*7} <https://github.com/tctsigemura/TacOS/blob/master/os/pm/pm.cmm> の一部である。

スマネージャが生成したリンクである。EXEC がシステムコールの種類を表している。システムコールの二つの引数は `_Atoi()` 関数を用いて `int` 型に変換して渡している。処理結果は子プロセスのプロセス番号 (PID) である。3 行で PID を呼び出し側に返す。

21.8 まとめ

TacOS のメッセージ通信について、それを実現するマイクロカーネル内プログラムと利用例を示した。

練習問題

21.1 TacOS のメッセージ通信機構について正しいか正しくないか答えなさい。

- (a) メッセージの形式に柔軟性がある。
- (b) リンクに三つのセマフォが含まれる。
- (c) TacOS のメッセージ通信機構は相互排除と同期にセマフォだけを用いている。
- (d) 複数生産者と複数消費者の問題の解に使用できる。

第 22 章

TacOS のメモリ管理

TacOS はファーストフィットで可変区画方式のメモリ管理を行う。メモリ管理はメモリマネージャサーバが担当する。メモリマネージャサーバは、OS がプロセス領域等を割り付けるために使用するサーバプロセスである。メモリマネージャサーバのソースコードは、<https://github.com/tctsigemura/TacOS/blob/master/os/mm/mm.cmm> から入手可能である。プロセス内部でヒープ領域を管理するプログラム (`malloc()`、`free()`) のアルゴリズムも基本は同じである。

TacOS のサーバプロセスはマイクロカーネルにリンクされ一つのプログラムモジュールになる (図 18.3 参照)。このプログラムモジュールをカーネルと呼ぶ。メモリマネージャサーバもカーネルの一部である。

22.1 データ構造の初期化

メモリ管理用構造体 (`MemBlk`) の宣言と、初期化プログラムをリスト 22.1 に示す。変数 `memPool` と番兵 (`MemBlk` 構造体) は、カーネルがメモリにロードされた時点で、カーネルのデータ領域に初期化された状態で置かれる。 `_end` はカーネルが使用している領域の最後のアドレス (空き領域の先頭のアドレス) を知るために用いる特殊な名前である。 `mmInit()` 関数はカーネルが起動する際に一度だけ実行されデータ構造の初期化を行う。 `mmInit()` 関数は、まず、空き領域の先頭部分を `MemBlk` 構造体と見做し番兵の `next` がこの構造体を指すようにする (18 行)。次に空き領域サイズを計算し、この構造体の `size` に代入する (19 行)。更に、この構造体がリストの最後になるように `next` に `null` を代入する (20 行)。

以上の初期化処理が完了した時点のデータ構造を図 22.1 に示す。 `memPool` 変数を起点に番兵付きで長さが 1 の空き領域リストが完成している。カーネルはメモリの 0000H 番地から配置される。この領域にカーネルのプログラムとデータがロードされる。図では分かりにくいですが、 `memPool` 変数と番兵はこの領域に配置される。最初は、カーネルの直後からメモリ最後の使用不可領域の直前までが一つの空き領域になっている。空き領域の先頭に `MemBlk` 構造体を置いたと見做し、番兵がそこを指すように初期化する。空き領域先頭の `MemBlk` 構造体が空き領域サイズを記憶し、E000H 番地まで空き領域が続いていることを表現している。

リスト 22.1: データ構造と初期化

```

1  #define MBSIZE sizeof(MemBlk)           // MemBlk のバイト数
2  #define MAGIC  (memPool)               // 番兵のアドレスを使用する
3
4  // 空き領域はリストにして管理される
5  struct MemBlk {                        // 空き領域管理用の構造体
6      MemBlk next;                      // 次の空き領域アドレス
7      int     size;                     // 空き領域サイズ
8  };
9
10 //-----
11 // 初期化ルーチン
12 //-----
13 // メモリ管理の初期化
14 MemBlk memPool = {null, 0};            // 空き領域リストの番兵
15 public int _end();                     // カーネルの BBS 領域の最後
16
17 void mmInit() {                        // プログラム起動前の初期化
18     memPool.next = _ItoA(addrrof(_end)); // 空き領域
19     memPool.next.size = 0xe000 - addrrof(_end); // 空きメモリサイズ
20     memPool.next.next = null;
21 }

```

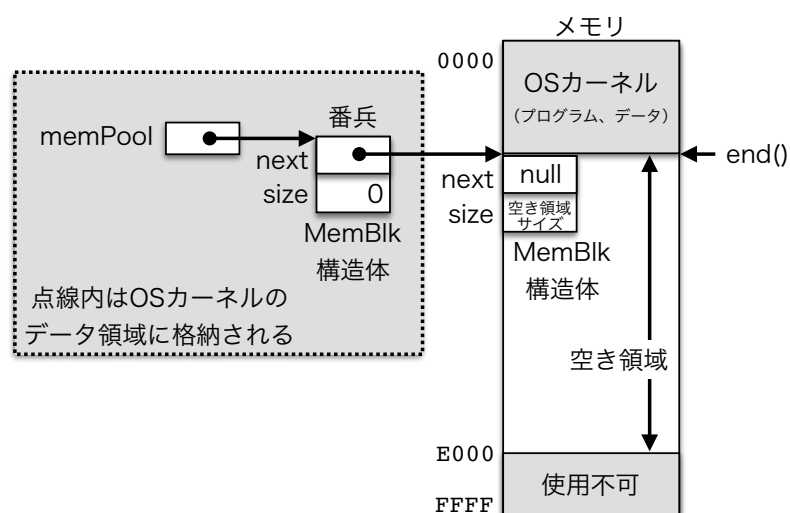


図 22.1: 初期化直後のデータ構造

リスト 22.2: 1KiB の領域を三つ割り付ける

```

1  a = mmAlloc( 1024 );    // 1KiB の領域を割り付ける
2  b = mmAlloc( 1024 );    // 1KiB の領域を割り付ける
3  c = mmAlloc( 1024 );    // 1KiB の領域を割り付ける

```

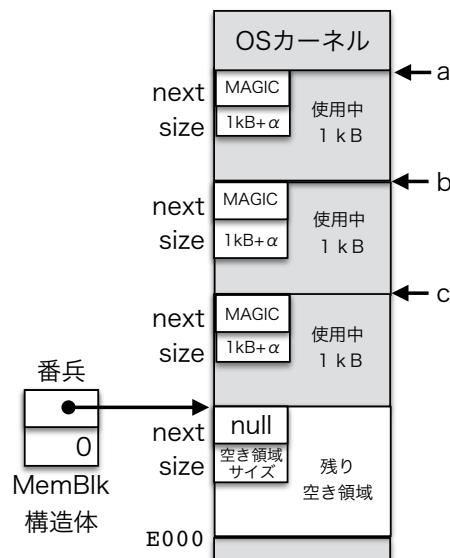


図 22.2: 三つの領域を割り付けた状態

22.2 メモリの割り付け

メモリ領域の割り付けは `mmAlloc()` 関数が行う。 `mmAlloc()` 関数は引数に与えられたバイト数の領域を割り付け、領域の先頭アドレスを返す。リスト 22.2 の手順で 1KiB の領域を三つ割り付けた状態を図 22.2 に示す。 `mmAlloc()` は、空き領域の前半に、要求された大きさの領域を割り付ける。リスト 22.2 では `mmAlloc()` が 3 回実行され、メモリの先頭に 1KiB の使用中領域を三つ割り付けている。その結果、空き領域が小さくなっている。

リスト 22.3 に `mmAlloc()` 関数の本体を示す。 `mmAlloc()` 関数は領域の要求サイズ (2 行目の `size`) を引数に呼び出され、割り付けた領域のアドレスを整数 (`int` 型) で返す。実際に割り付ける領域は、要求されたサイズに `MemBlk` 構造体のサイズを加えた後に偶数に切上げた大きさである (3 行目)。TaC では 16bit データ (2 バイトデータ) をメモリに格納する時、連続した $2i$ 番地と $2i+1$ 番地 (i は適当な整数) を使う決まりになっているので、領域は常に偶数番地から始まるようにする必要がある^{*1}。

`mmAlloc()` 関数は、空き領域リストを探索し割り付けるサイズ以上の領域を見つける (7 行〜)。サイズ比較に使用される `_uCmp()` 関数は符号なし整数用の大小比較関数である。領域を探す間はポインタ `m` が目的の領域を、ポインタ `p` が一つ前の領域を指している。リストの最後に達した場合は、適切な領

^{*1} 割当てた領域が、2 バイト (16 ビット) データの配列として使用される場合を想像して欲しい。

リスト 22.3: メモリ割り付けプログラム

```

1 // メモリを割り付ける
2 int mmAlloc(int siz) {                                // メモリ割り当て
3     int s = (siz + MBSIZE + 1) & ~1;                  // 制御データ分大きい偶数に
4     MemBlk p = memPool;                               // 直前の領域
5     MemBlk m = p.next;                                // 対象となる領域
6
7     while (_uCmp(m.size,s)<0) {                         // 領域が小さい間
8         p = m;                                         // リストを手繰る
9         m = m.next;
10        if (m==null) return 0;                         // メモリが不足する場合は
11    }                                                  // エラーを表す null ポインタ
12
13    if (_uCmp(m.size ,s+MBSIZE+2)<=0) {                 // 分割する価値がない領域サイズ
14        if (memPool.next==m && m.next==null)          // リストの長さがゼロにならない
15            return 0;                                  // ようにする
16        p.next = m.next;                               // リストから外す
17    } else {                                           // 領域を分割する価値がある
18        MemBlk n = _addrAdd(m, s);                     // 残り領域
19        n.next = m.next;
20        n.size = m.size - s;
21        p.next = n;
22        m.size = s;
23    }
24    m.next = MAGIC;                                    // マジックナンバー格納
25    return _AtoI(_addrAdd(m, MBSIZE));                // 管理領域を除いて返す
26 }

```

域が見つからなかったことになる。mmAlloc() 関数は 0 を返して終了する。

適切な領域 (m) が見つかったら、それを使用領域と空き領域に分割するべきか判断する (13 行)。ちょうどピッタリか少しだけ大きい領域なら分割しない。分割しない場合は領域をリストから外す (16 行)。

分割する場合は領域の前半 (m) を使用領域、残りを空き領域とする。空き領域のアドレスは、アドレス用の足算関数 _addrAdd() で n に求める (18 行)。領域 m をリストから外し代わりに領域 n をリストに挿入する (19 行～21 行)。領域 n の大きさを設定する (22 行)。

最後に、領域 n が正当に割当てられたことを表すマジックナンバー (MAGIC) を next に書込む (24 行)。mmAlloc() 関数が返すアドレスは MemBlk 構造体直後である (25 行)。MAGIC は mmFree() 関数が領域を解放する時に、正当に割当てられた領域かどうかチェックするために使用される。

22.3 メモリの解放

mmFree(b); を実行し領域 b を開放した状態を図 22.3 に示す。続けて mmFree(c); を実行し領域 c も開放した状態を図 22.4 に示す。図 22.3 では、領域 b が開放され空き領域が二つになり、空き領域リス

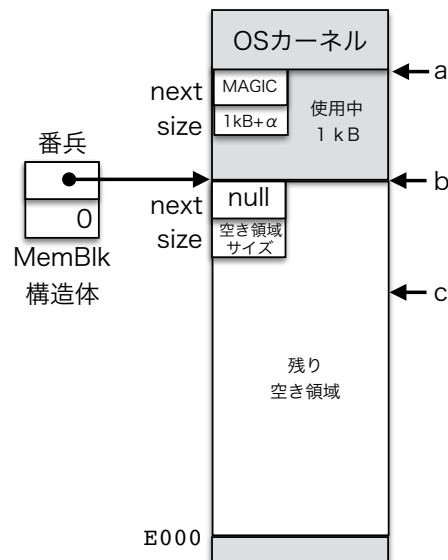
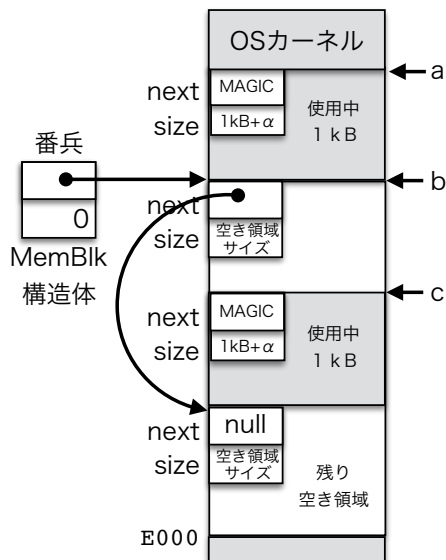


図 22.3: 領域 b を開放した状態

図 22.4: 領域 c も開放した状態

トの長さが 2 になっている。図 22.4 では、領域 c が開放され空き領域を一つに合体することができたので、空き領域リストの長さが 1 になっている。

メモリの解放を行う `mmFree()` 関数の本体をリスト [22.4](#) に示す. `mmFree()` 関数は解放する領域 `mem` を引数に実行される (2 行). 領域の `MemBlk` 構造体に `MAGIC` が格納されていない場合は `mmAlloc()` 関数で割り付けられた正当な領域では無いのでエラーを表示してシステムを停止する (8 行)^{*2}.

解放された領域は新しい空き領域になる。空き領域リストがアドレス順になるように、新しい空き領域を挿入すべき位置を探す処理を行う (10 行～)。

解放する領域が直前の空き領域に重なっていたり、直後の空き領域に重なっていたりしていないかチェックしている (19 行)^{*3}.

解放する領域が直前の空き領域に隣接している場合は、直前の空き領域サイズを大きくすることで一つの空き領域にする（23 行）。直後の空き領域とも隣接している場合は、直前の空き領域サイズを更に大きくし直後の空き領域も一つの領域にする（25 行）。この時は、空き領域が一つにまとめられたので、空き領域リストから直後の空き領域を削除する（26 行）。

更に、直後の空き領域だけと隣接している場合 (28 行), どの空き領域とも隣接していない場合 (32 行) の処理が続いている.

22.4 まとめ

本章では、TacOS のメモリ管理プログラムを例に、ファーストフィット方式を用いる可変区画方式のメモリ割り付けプログラムを紹介した。

^{*2} このプログラムは OS 内部で動くものである。このような事象が発生するのは OS のバグが原因と考えられるのでシステムを停止する。

*3 これも OS のバグ以外では発生しない。OS 自身がバグを含んでいないかチェックする機会として利用している。

リスト 22.4: メモリ解放プログラム

```

1 // メモリを解放する
2 int mmFree(void[] mem) {                                // 領域解放
3     MemBlk q = _addrAdd(mem, -MBSIZE);                 // 解放する領域
4     MemBlk p = memPool;                                 // 直前の空き領域
5     MemBlk m = p.next;                                  // 直後の空き領域
6
7     if (q.next!=MAGIC)
8         badaddr();                                     // 領域マジックナンバー確認
9
10    while (_aCmp(m, q)<0) {                               // 解放する領域の位置を探る
11        p = m;
12        m = m.next;
13        if (m==null) break;
14    }
15
16    void[] ql = _addrAdd(q, q.size);                     // 解放する領域の最後
17    void[] pl = _addrAdd(p, p.size);                     // 直前の領域の最後
18
19    if (_aCmp(q,pl)<0 || m!=null&&_aCmp(m,ql)<0)         // 未割り当て領域では？
20        badaddr();
21
22    if (pl==q) {                                          // 直前の領域に隣接している
23        p.size = p.size + q.size;
24        if (ql==m) {                                     // 直後の領域とも隣接してる
25            p.size = p.size + m.size;
26            p.next = m.next;
27        }
28    } else if (ql==m) {                                  // 直後の領域に隣接している
29        q.size = q.size + m.size;
30        q.next = m.next;
31        p.next = q;
32    } else {
33        p.next = q;
34        q.next = m;
35    }
36    return 0;
37 }

```


第 23 章

TacOS のファイルシステム

TacOS のファイルシステムサーバ (fs)^{*1}のプログラムを用いて、FAT ファイルシステムの実装例を示す。

23.1 ファイルシステムサーバ

ファイルシステムサーバのクラス図を図 23.1 に示す。ファイルシステムサーバは、サーバプロセスのメインルーチン (fs クラス^{*2})、システムコールの処理プログラム (fatSys クラス^{*3})、オープン済みファイル毎にバッファを割付けバイト単位の操作を提供する上位のファイルシステム (file クラス^{*4})、ディレクトリの管理を行う (dirAccess クラス^{*5})、FAT を管理しセクタ単位の操作を提供する下位のファイルシステム (blkFile クラス^{*6})、デバイスドライバ (mmcsapi クラス^{*7}) からなる。

ファイルシステムサーバは、カーネルモードで実行されるプロセスである。他のプロセスは、メッセージ通信を用いてファイルシステムサーバに処理を要求する。ファイルシステムサーバは要求された処理を行い、結果を要求元に返信する。ファイルシステムサーバ以外のプロセスは、ファイル操作に関するデータ構造やハードウェアなどの資源にアクセスしない。ファイルシステムサーバは、排他制御なしにこれらの資源にアクセスできる。第 22 章では触れなかったが、メモリマネージャプロセスも同様にメモリ管理のデータ構造を排他制御なしにアクセスしている。

23.2 fs クラス

fs クラスはファイルシステムサーバのメインループと、他のプロセスが呼び出すシステムコールのスタブルーチンを持っている。

• サーバプロセス

リスト 23.1 に示す fsMain() がサーバプロセスのメインルーチンである。リンクを作成した後、無

*1 <https://github.com/tctsigemura/TacOS/tree/master/os/fs>

*2 <https://github.com/tctsigemura/TacOS/blob/master/os/fs/fs.cmm> (fs.hmm)

*3 <https://github.com/tctsigemura/TacOS/blob/master/os/fs/fatSys.cmm> (fatSys.hmm)

*4 <https://github.com/tctsigemura/TacOS/blob/master/os/fs/file.cmm> (file.hmm)

*5 <https://github.com/tctsigemura/TacOS/blob/master/os/fs/dirAccess.cmm> (dirAccess.hmm)

*6 <https://github.com/tctsigemura/TacOS/blob/master/os/fs/blkFile.cmm> (blkFile.hmm)

*7 <https://github.com/tctsigemura/TacOS/blob/master/os/fs/mmcsapi.cmm> (mmcsapi.hmm)

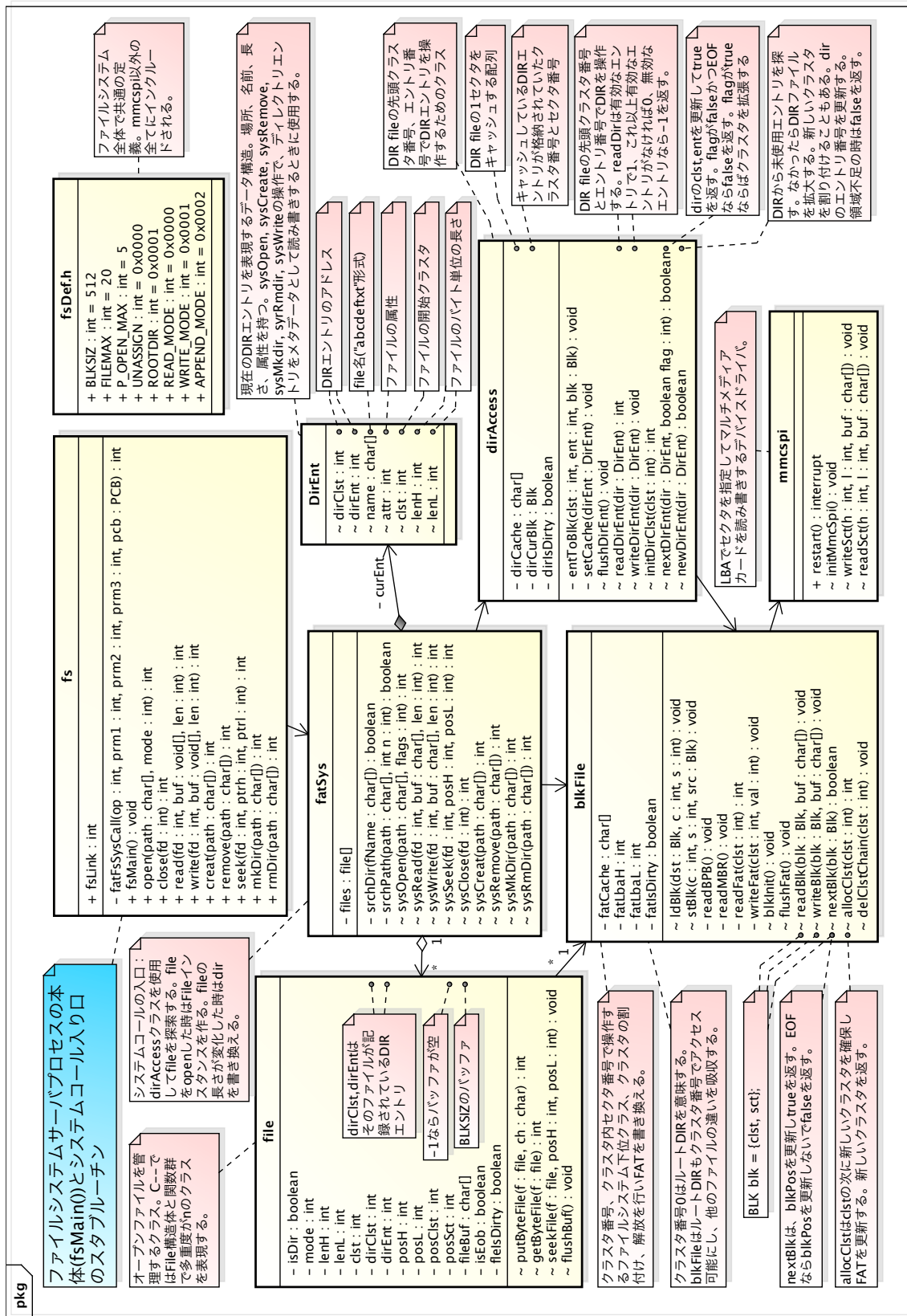


図 23.1: TacOS のファイルシステムサーバの構造

リスト 23.1: ファイルシステムサーバのメインループ (fsMain())

```
// ファイルシステムサーバのメインルーチン
public void fsMain() {
    fsLink = newLink();                // リンク生成
    blkInit();                         // ドライバの初期化等
    while (true) {                     // FS のメインループ
        Link l = receive(fsLink);      // システムコール受付
        int r=fatFsSysCall(l.op, l.prm1, l.prm2, l.prm3, // システムコール実行
                           l.client);
        send(fsLink ,r);              // 結果を返す
    }
}
```

リスト 23.2: システムコールによって分岐する (fatFsSysCall())

```
int fatFsSysCall(int op, int prml, int prm2, int prm3, PCB pcb) {
    int rs;                            // 実行結果(返り値)
    if (op==CREAT) {
        rs = sysCreat(_ItoA(prm1));    // creat(path)
    } else if (op==REMOVE) {
        rs = sysRemove(_ItoA(prm1));  // remove(path)
    }
}
```

リスト 23.3: クライアントプロセスが呼び出すスタブルーチンの例 (open())

```
// open システムコール
public int open(char[] path, int mode) {
    int fd = sndrec(fsLink, OPEN, _AtoI(path), mode, 0);
    return fd;
}
```

限ループでメッセージの受信と返信を繰り返す。receive(), send() は第 21 章で紹介した TacOS のメッセージ通信機構である。fatFsSysCall() は、システムコールの種類に応じて fatSys クラス内のシステムコール処理関数を呼び出す。fatFsSysCall() の一部をリスト 23.2 に示す。

- スタブルーチン

open()～rmDir() は、システムコールを発行するクライアントプロセスが呼び出す。これらの関数はシステムコールを実際に処理をするのではなく、ファイルシステムサーバにメッセージを送信するだけのスタブである。スタブの例として open() 関数をリスト 23.3 に示す。sndrec() はサーバプロセスの receive(), send() と通信する TacOS のメッセージ通信機構の関数である。

リスト 23.4: ディレクトリファイル内を探索する (srchDir())

```

1 // srchDir   : ファイルをディレクトリから探し、ファイルの情報を curEnt に格納
2 // 返回值    : true=見つかった、false=見つからなかった
3 // 引数 name : 探すファイルの名前
4 boolean srchDir(char[] name) {
5     int r;
6     do {
7         if ((r=readDirEnt(curEnt))==1    &&           // 有効なエントリで
8             (curEnt.attr & 0x0a)==0    &&           // 通常ファイルかディレクトリ
9                                     // system,volumeLabel,vfat以外
10            cmpFname(curEnt.name, name)==0)         // ファイル名が一致するなら
11             return true;                          // ファイルが見つかった
12     } while (r!=0 && nextDirEnt(curEnt, false));    // 有効なエントリが続く間
13     return false;                                 // 見つからなかった
14 }

```

23.3 fatSys クラス

ファイルシステムのシステムコールを処理するクラスである。file クラス、dirAccess クラス、blkFile クラスの機能と、fatSys クラス自身が持つパスの解析機能とオープン中ファイルの管理機能を使用して、システムコールを実行する（図 23.1 参照）。

• ディレクトリファイル内の探索

リスト 23.4 の srchDir() 関数は、ディレクトリファイルを指定しファイル名でエントリを探索する。srchDir() 関数は、現在の着目位置から始めて指定された名前のエントリを探索する。着目位置は、DirEnt 型（リスト 23.11 参照）のオブジェクト curEnt に、ディレクトリファイルのクラスタ番号とディレクトリエントリの番号として記録している。

dirAccess クラスの readDirEnt() 関数は、着目位置のディレクトリエントリを curEnt に読み込む。nextDir() 関数は curEnt の着目位置を一つ進める。

• パスの解析

TacOS にはカレントディレクトリ概念が無いので、パスの探索はいつもルートディレクトリから開始する。パスの探索はリスト 23.5 の srchPath() 関数が行う。引数は解析するパス (path) と解析する文字数 (n) である。ファイルが登録されているディレクトリを操作する場合などは、パスの途中までを解析することがあるので文字数が必要である。

ルートディレクトリはクラスタ番号が ROOTDIR（実際の値は 1）のファイルとして blkFile クラスが扱うので、curEnt は 3 行から 9 行のように初期化する。13 行でパスの先頭にある余分な「/」を取り除き、解析位置がパスの最後まで来ていたら終了である（14 行）。着目しているディレクトリファイルのクラスタ番号を、新しく見つけたファイル（次の階層のディレクトリファイル）の開始クラスタ番号で置換える（18 行）。これにより、着目しているディレクトリが次の階層のものにな

リスト 23.5: パスを解析する (srchPath())

```

1  boolean srchPath(char[] path, int n) {
2      // ルート・ディレクトリを見つけた状態にする
3      curEnt.dirClst = ROOTDIR;                // ルートの親はルート
4      curEnt.dirEnt  = 0;                      // 最初のエントリ
5      strCpy(curEnt.name, "/");                // 名称は "/"
6      curEnt.attr    = 0x10;                  // 属性はディレクトリ
7      curEnt.clst    = ROOTDIR;               // ルート・ディレクトリ
8      curEnt.lenH    = 0x0000;                // ディレクトリファイル長は
9      curEnt.lenL    = 0x0000;                // 0x0000 0000 にする
10
11     // n 文字目までパスを解析
12     for (int p=0; ;) {                        // 渡されたパスについて
13         while (p<n && path[p]!='/') p = p + 1; // 余計な '/' を読み飛ばす
14         if (p>=n) break;                     // パス解析完了
15
16         // 階層を進む
17         if ((curEnt.attr&0x10)!=0x10) return false; // 普通ファイルの中には進めない
18         curEnt.dirClst = curEnt.clst;           // 次の階層の DIR FILE を設定
19         curEnt.dirEnt  = 0;                     // エントリ番号を初期化
20
21         // ディレクトリファイル内でファイル名を探索する
22         p = getFname(path, p, n, fname);        // パスからファイル名を取り出す
23         if (!srchDir(fname)) return false;      // ファイルが見つからない
24     }
25     return true;                               // ファイルが見つかった
26 }

```

る。探索中のパス名から次の階層のファイル名を取出し (22 行)、着目しているディレクトリファイルの中を、前出の `srchDir()` 関数を用いて探索する (23 行)。もしも、名前が見つからなければ終了するし、そうでなければ 13 行に戻る。

• オープン中ファイルの管理

オープンファイル毎に File クラス (図 23.1 では file クラス) のオブジェクトを割付け file 配列に登録する。ファイルがクローズされるまで、ファイル操作は File オブジェクトを用いて行われる。open システムコールが File オブジェクトの登録を行うので、open システムコールの処理プログラム `sysOpen()` をリスト 23.6 に示す。この関数は、リスト 23.2 の `fatFsSysCall()` 関数から呼び出される。

2 行はクライアントプロセスの PCB 中に、ファイルディスクリプタを記録する場所を確保している*8。4 行は fatSys クラスの files 配列に場所を確保している。7 行で前出の `srchPath()` 関数

*8 プロセス終了時にファイルを自動的にクローズするために、プロセス (PCB) に記録を残す必要がある。

リスト 23.6: open システムコールの本体 (sysOpen())

```

1 public int sysOpen(char[] path, int mode, PCB pcb) {
2     int idx = newIdx(pcb);                // FDS のインデクス
3     if (idx<0) return EMFILE;              // プロセスごとのオープン数超過
4     int fd = newFd();                      // ファイル記述子
5     if (fd<0) return ENFILE;              // システム全体のオープン数超過
6
7     if (!srchPath(path, strLen(path)))     // ファイルを探索する
8         return ENOENT;                    // 見つからなかったらエラー
9     if (isOpened(curEnt.dirClst, curEnt.dirEnt)) // ファイルがオープン済みなら
10        return EOPENED;                   // エラー
11    boolean isDir = (curEnt.attr&0x10)!=0;   // 見つけたファイルは DIR か?
12    if (isDir && mode!=READ_MODE) return EMODE; // DIR かつ READ でない
13
14    // File オブジェクトを生成する
15    File f = malloc(sizeof(File));          // File オブジェクトを生成
16    if (f==null) return ENOMEM;             // メモリ不足
17    char[] fb = malloc(BLKSIZ);            // バッファを生成
18    if (fb==null) { free(f); return ENOMEM; } // メモリ不足
19
20    // File オブジェクトを初期化する
21    f.isDir   = isDir;                     // ディレクトリファイル
22    f.mode    = mode;                      // オープンモード
23    f.lenH    = curEnt.lenH;                // ファイルサイズ(上位16bit)
24    f.lenL    = curEnt.lenL;                // ファイルサイズ(下位16bit)
25    f.clst    = curEnt.clst;                // ファイルの先頭クラスタ番号
26    f.dirClst = curEnt.dirClst;             // DIR FILE の先頭クラスタ番号
27    f.dirEnt  = curEnt.dirEnt;              // DIR エントリのエントリ番号
28    f.fileBuf = fb;                        // バッファを記録
29    f.isEob   = f.fileIsDirty = false;     // フラグを下ろす
30
31    // f.posXX は seekFile() で初期化される
32    if (mode==APPEND_MODE) {                // APPEND_MODE なら
33        seekFile(f, f.lenH, f.lenL);        // ファイルの最後に移動
34    } else {                                // READ_MODE か WRITE_MODE なら
35        seekFile(f, 0, 0);                  // ファイルの先頭に移動
36    }
37    files[fd] = f;                          // ファイル一覧に登録
38    pcb.fds[idx] = fd;                      // FDS にファイル記述子を登録
39    return idx;                             // FDS のインデクスを返す
40 }

```

リスト 23.7: read システムコールの本体 (sysRead())

```

1 public int sysRead(PCB pcb, int idx, char[] buf, int len) {
2     File f = chkIdx(pcb, idx);                // IDX と FD が正しい値か確認
3     if (len<0) return EINVAL;                // 引数が不正
4     if (f==null) return EBADF;               // エラー
5     if (f.mode!=READ_MODE) return EMODE;     // 読み込みモードでない
6     if (!f.isDir && f.clst==UNASSIGN) return 0; // 新規ファイルは読まない
7         // !isDir && clst==0 作成直後のファイル (新規ファイル)
8         // !isDir && clst!=0 データ書き込み済みのファイル
9         // isDir && clst==0 存在しない
10        // isDir && clst!=0 通常DIRはsysMkDir()でクラスタを割付け済み
11    int i,r;
12    for(i=0; i<len; i=i+1) {
13        if ((r=getByteFile(f))<0) break;
14        buf[i] = chr(r);
15    }
16    return i;                                // 読み込んだバイト数を返す
17 }

```

を用いて path を最後まで解析し、目的ファイルのディレクトリエントリを探す。見つからない場合はエラーになる^{*9}。12 行でディレクトリファイルの「書き込みオープン」を禁止している。

14～18 行では File オブジェクトとセクタを格納するバッファを生成している。malloc(), free() 関数は、メモリマネージャサーバのシステムコールを呼び出すスタブルーチンである。

20～29 行では File オブジェクトを初期化している。21 行ではオープンファイルがディレクトリファイルであることを表すフラグをセットしている。26～27 行ではオープンファイルが格納されているディレクトリエントリを記録している^{*10}。

33 行, 35 行では File オブジェクトを引数に、file クラスの seekFile() 関数を使用してる。C++ 言語で、file クラスのような複数のインスタンスを持つクラスを操作する場合、インスタンスを引数にクラスの関数を呼び出す。

• オープン中ファイルの操作

オープン中のファイルは File オブジェクトを用いて操作する。例として read システムコールの処理プログラム sysRead() をリスト 23.7 に示す。この関数は、リスト 23.2 の fatFsSysCall() 関数から呼び出される。

2 行の chkIdx() はプロセスのファイルディスクリプタ番号から、File クラスのオブジェクトを求める。6 行は作成後まだクラスタが割当てられていないファイルの処理である。11～15 行では File クラスの getByteFile() を用いて、ファイルオブジェクトを通してファイルのデータを buf [] に

^{*9} TacOS の open システムコールにはファイルの作成機能は無い。ファイルの作成は creat システムコールで行う。

^{*10} FAT ファイルシステムではファイルサイズなどをディレクトリエントリに記録しているので、ファイルをオープン後もディレクトリエントリにアクセスする必要がある。

リスト 23.8: remove システムコールの本体 (sysRemove())

```

1 // delFile : ファイルの本体を削除する
2 void delFile() {
3     delClstChain(curEnt.clst);           // クラスタチェーンを解放する
4     curEnt.name[0] = '\xe5';           // エントリを「削除」に変更
5     writeDirEnt(curEnt);               // DIR エントリを書き込む
6     flushDirEnt();                     // 念のためにフラッシュ
7     flushFat();                         // 念のためにフラッシュ
8 }
9
10 // remove システムコール
11 public int sysRemove(char[] path) {
12     if (!srchPath(path, strlen(path))) return EPATH; // path が見つからなかった
13     if ((curEnt.attr&0x10)!=0) return EFATTR;      // ディレクトリならエラー
14     if (isOpened(curEnt.dirClst, curEnt.dirEnt)) // ファイルがオープン
15         return EOPENED;                          //   されていたらエラー
16     delFile();                                     // ファイルの本体を削除
17     return 0;
18 }

```

読み出している。

- ディレクトリの書き換え

ディレクトリ書き換え操作の例として、ファイルを削除する remove システムコールの処理プログラムをリスト 23.8 に示す。この関数は、リスト 23.2 の fatFsSysCall() 関数から呼び出される。12 行で srchPath() を用いて削除するファイルのディレクトリエントリを探索する。14 行の isOpened() は、システム中の全オープンファイルについて調べ、削除するファイルがオープン中かどうか調べる。オープン中のファイルは削除できない。削除しても構わないなら、16 行で delFile() を用いてファイルを消す。delFile() はディレクトリの削除 (rmdir システムコール) からも使用される。3, 7 行で使用される delClstChain(), flushFat() は blkFile クラスの関数、5, 6 行の writeDirEnt(), flushDirEnt() は dirAccess クラスの関数である。

23.4 File クラス (file クラス)

オープン中のファイルを管理・操作するためのクラスである。File クラスはセクタの内容を保持するバッファを持ち、バイト単位のファイル操作機能を提供する。file.hmm にインタフェース、file.cmm に実装が書いてある。

このクラスはオープンファイルの数だけインスタンスが生成される。オブジェクト指向言語ではない C++ では、file.hmm 中に宣言されている File 構造体でインスタンスの属性を表現し、file.cmm 中の関数に引数として属性を渡して処理させることで、インスタンスの操作を実現している。インスタンスが複数ある場合は、複数の File 構造体インスタンスが生成される。なお、インスタンスを一つしか持

リスト 23.9: File クラスの外部インタフェース (file.hmm)

```

1 struct File {                                     // オープンファイルを表す構造体
2     // 不変な情報
3     boolean isDir;                                // ディレクトリかどうか
4     int mode;                                     // オープンモード
5     int lenH;                                     // ファイルサイズ(上位16bit)
6     int lenL;                                     // ファイルサイズ(下位16bit)
7     int clst;                                     // 先頭クラスタ番号
8     int dirClst;                                  // DIR FILE のクラスタ#
9     int dirEnt;                                   // DIR FILE のエントリ#
10
11     // 現在の情報
12     int posH;                                     // 参照位置の上位バイト位置
13     int posL;                                     // 参照位置の下位バイト位置
14     int posClst;                                  // バッファ中のクラスタ#
15     int posSct;                                  // バッファ中のセクタ#
16     char[] fileBuf;                              // ファイルごとのバッファ
17     boolean isEob;                                // セクタの境界にあるかどうか
18     boolean fileIsDirty;                          // ダーティーフラグ
19 };
20
21 public int putByteFile(File f, char ch);
22 public int getByteFile(File f);
23 public void seekFile(File f, int posH, int posL);
24 public void flushBuf(File f);

```

たないクラスの場合は、属性をスタティック変数として表現すれば十分である。

- **File クラスの仕様**

リスト 23.9 に File クラスの仕様にあたる file.hmm を示す。1～19 行の構造体が属性を、21～24 行のプロトタイプ宣言が操作を表現する。3～9 行はファイルシステムに書き込まれるファイルの静的な属性である。12～18 行はファイルがクローズされると必要なくなる一時的な属性である。21～24 行の関数はファイルオブジェクトを引数に実行される。

- **ファイルの読み書き**

例として、ファイルからデータを読み出す getByteFile() をリスト 23.10 に示す。6 行の fillBuf() は、ファイルオブジェクトのバッファに、次のバイトを含むセクタが格納済みか調べ、格納されていなかったら blkFile クラスの機能を利用して読み込む。

23.5 dirAccess クラス

dirAccess クラスは blkFile クラスの機能を利用して、ディレクトリエントリを読み書きする機能を提供する。fatSys クラスはパスの解析や、ファイルの作成・削除、ファイルサイズの変更等でディレク

リスト 23.10: ファイルからバイト単位でデータを読む (getBytesFile())

```

1 // getBytesFile : ファイルから Byte 単位でデータを読み込む
2 // 返回值      : 0>:ファイルから読み出した値 -1:EOF
3 // 引数 f      : データを読み込む File オブジェクト
4 public int getBytesFile(File f) {
5     if (!f.isDir && isEof(f)) return -1;           // EOF
6     if (!fillBuf(f)) return -1;                   // 目的のセクタを読む
7                                                     // DIR ファイルはここで EOF 判定
8     int r = ord(f.fileBuf[f.posL%BLKSIZ]);        // データを 1Byte 取り出す
9     nextPos(f);                                    // 現在位置を 1Byte 進める
10    return r;                                       // バイトを 16bit で返す
11 }

```

リスト 23.11: ディレクトリエントリ構造体 (DirEnt)

```

1 struct DirEnt {                                     // DIR エントリを表す構造体
2     int    dirClst;                                // DIR FILE の先頭クラスタ番号
3     int    dirEnt;                                 // DIR エントリのエントリ番号
4     char[] name;                                   // ファイル名("abcdefgh.txt"形式)
5     int    attr;                                   // ファイルの属性
6     int    clst;                                   // ファイルの開始クラスタ番号
7     int    lenH;                                   // ファイルサイズ(上位)
8     int    lenL;                                   // ファイルサイズ(下位)
9 };

```

トリの操作で, dirAccess クラスの機能を利用する (図 23.1 参照).

• ディレクトリエントリのキャッシュ

dirAccess クラスはディレクトリエントリを含むセクタを dirCache にキャッシュし, メディアに対するアクセス回数を減らす工夫をしている. dirCurBlk, dirIsDirty (図 23.1 参照) は, キャッシュを管理するためのデータである.

• ディレクトリエントリ構造体

リスト 23.11 に示す DirEnt 構造体はディレクトリエントリの読み書きに使用される. dirClst, dirEnt がディレクトリファイルとファイル内の位置であり, ディレクトリエントリのアドレスを表現している. 4 行以下はディレクトリエントリの内容である.

• ディレクトリエントリの読み書き

例として, ディレクトリファイル (またはルートディレクトリ) からエントリを読み出す readDirEnt() をリスト 23.12 に示す.

readDirEnt() 関数は, DirEnt 型の引数 dirEnt にディレクトリファイルの位置とディレクトリファイル内のエントリ番号を格納して呼び出される. 5 行で, 目的のディレクトリエントリが

リスト 23.12: ディレクトリファイルからエントリを読む (readDirEnt())

```

1 // readDirEnt : DIR FILE から DIR エントリの情報を DirEnt に読み込む
2 // 返回值      : 1:有効 0:これ以上有効なエントリはない -1:無効なエントリ
3 // 引数 dirEnt : 目的 DIR エントリの dirClst と dirEnt が設定された DirEnt
4 public int readDirEnt(DirEnt dirEnt) {
5     setCache(dirEnt); // 目的のセクタをキャッシュする
6     int curEnt=(dirEnt.dirEnt%(BLKSIZ/32))*32; // セクタ中の着目エントリ
7     char c = dirCache[curEnt]; // 着目エントリの先頭
8     if (c=='\x00') return 0; // これ以上有効なエントリはない
9     if (c=='\xe5') return -1; // 無効なエントリ
10    readFname(dirEnt.name, curEnt); // ファイル名を読み込む
11    dirEnt.attr = ord(dirCache[curEnt+11]); // 属性を読み込む
12    dirEnt.clst = wordLE(dirCache, curEnt+26); // 先頭クラスタ番号を読む
13    dirEnt.lenH = wordLE(dirCache, curEnt+30); // ファイルサイズ (上位) を読む
14    dirEnt.lenL = wordLE(dirCache, curEnt+28); // ファイルサイズ (下位) を読む
15    return 1;
16 }

```

キャッシュに存在しない時、ディレクトリエントリを含むセクタを dirCache 配列 (図 23.1 参照) に読む。6 行は、目的のエントリの dirCache 配列中の位置を curEnt に計算する。

7 行は、ファイル名の第 1 バイトを c に読む。c の値によりエントリの状態を確認し (8, 9 行)、有効なエントリの場合は内容を引数の dirEnt 構造体を読み込む (10 行以降)。ファイル名の読み出しにはリスト 15.1 の readFname() 関数を用いる。11 行で使用している ord() は文字型を整数型に変換する C++ 言語の演算子である。

FAT ファイルシステムはリトルエンディアンである。12 行から 14 行では、文字 (バイト) 配列からリトルエンディアンの 16 ビットデータを読取るために、wordLE() マクロを使用している。C++ 言語には 32 ビット整数型がないので、ファイル長は 2 つの 16 ビット整数 (lenH, lenL) で表現する。リトルエンディアンなので、下位桁の方がバイト位置では前になっている (13, 14 行)。

23.6 blkFile クラス

mmcspi クラスの機能を利用して、クラスタチェーンを辿りながらセクタ単位でファイルを読み書きする機能と、FAT にクラスタを割当てたりクラスタチェーンを削除する機能を、dirAccess クラス、file クラス、fatSys クラスに提供する (図 23.1 参照)。また、特別なクラスタ番号 (ROOTDIR=0x0001) を用いてルートディレクトリをファイルのように読み書きする機能も提供する。

・初期化

FAT の位置、ルートディレクトリの位置、データ領域の位置等の基本情報 (リスト 23.13) を、readMBR() (リスト 23.14) と readBPB() (リスト 23.15) を使用して求める。これらの位置は 32 ビットの LBA なので、int 型の配列で表現する。

リスト 23.13: BPB 基本情報 (blkFile.cmm)

```

1  int sctPrClst;                // 1クラスタ当たりのセクタ数
2  int sctPrFAT;                // 1FAT 当たりのセクタ数
3  int[] bpbLba = { 0, 0 };     // BPB 領域の開始 LBA
4  int[] fatLba = { 0, 0 };     // FAT 領域の開始 LBA
5  int[] rootLba = { 0, 0 };    // ルートディレクトリの開始 LBA
6  int[] dataLba = { 0, 0 };    // データ領域の開始 LBA

```

リスト 23.14: MBR を読む (readMBR())

```

1  void readMBR() {
2      char[] buf = malloc(BLKSIZ);        // 1セクタ分のバッファを確保
3      readSct(0,0,buf);                   // MBR を読み込む
4
5      for (int i=446; i<510; i=i+16) {    // パーティションテーブルについて
6          // int active = ord(buf[i]);     // アクティブフラグ
7          int fType = ord(buf[i+4]);       // ファイルシステムタイプ
8
9          if(/*(active & 0x80)!=0 &&*/fType==0x06){ //アクティブなFAT16パーティション
10             ld32(bpbLba,wordLE(buf,i+10),wordLE(buf,i+8)); // パーティションの開始 LBA
11             free(buf);                   // バッファを解放して
12             return;                     // 戻る
13         }
14     }
15     panic("readMBR");                  // 最後まで行くとエラー
16 }

```

- readMBR() はマイクロ SD カードの LBA0 (MBR) を読み、パーティションテーブルを解析して TacOS が扱える FAT16 パーティションを探す。パーティションテーブルエントリの構造は、図 13.7 や表 13.1 に示した。見つけたパーティションの位置をリスト 23.13 の bpbLba に格納する。

2 行 buf にセクタを読み出すための 512 バイトのバッファを確保する。

3 行 mmcspi クラスの readSct() 関数を用いて buf に MBR を読み込む。

5 行 MBR の第 446 バイトからパーティションテーブルが始まる。パーティションテーブルエントリのサイズは 16 バイトである。

6 行 エントリの先頭の Flag を読む。

7 行 エントリの 4 バイト目の Type を読む。

9 行 TacOS は FAT16 パーティションにインストールされているので探す。

10 行 FAT16 パーティションが見つかったら、エントリの 8 バイト目から格納されている 32 ビットの Start LBA を読み出す。エントリに LBA はリトルエンディアンで格納されて

リスト 23.15: BPB を読む (readBPB())

```

1 void readBPB() {
2     char[] buf = malloc(BLKSIZ);           // 1セクタ分のバッファを確保
3
4     readSct(bpbLba[0], bpbLba[1], buf);     // BPB を読み込む
5     if (wordLE(buf, 11) != 512)             // セクタ長は512バイトだけサポート
6         panic("BLKSIZ!=512");
7
8     sctPrClst = ord(buf[13]);                // クラスタあたりのセクタ数
9
10    // FAT 開始位置(セクタ)の計算
11    ld32(fatLba, 0, wordLE(buf, 14));        // fat <= 予約セクタ数
12    _add32(fatLba, bpbLba);                 // fat <= fat + bpb
13
14    // ルートディレクトリ位置(セクタ)の計算
15    sctPrFAT = wordLE(buf, 22);              // sctPrFAT <= FATあたりセクタ数
16    if (sctPrFAT == 0)                     // sctPrFAT==0 は FAT32
17        panic("FAT32?");
18
19    ld32(rootLba, 0, ord(buf[16]));          // root <= FAT数
20    _add32(_mul32(rootLba, sctPrFAT), fatLba); // root <= root * sctPtFAT + fat
21
22    // データの開始位置(セクタ)の計算
23    ld32(dataLba, 0, wordLE(buf, 17) / 16); // data <= ルートディレクトリサイズ
24    _add32(dataLba, rootLba);               // data <= data + root
25
26    free(buf);                             // バッファを解放
27 }

```

いるので、上位ワードを 10 バイト目、下位ワードを 8 バイト目から読み出している。読み出した LBA は bpbLba に格納する。

11, 12 行 FAT16 パーティションを見つけたのでバッファを解放して終了する。

- readBPB() は bpbLba を参照し FAT の BPB 領域の位置を知り、BPB の内容を解析する。BPB の構造は、表 15.2 に示した。解析結果は、リスト 23.13 の fatLba, rootLba, dataLba に格納する。

2 行 buf にセクタを読み出すための 512 バイトのバッファを確保する。

4 行 mmcspi クラスの readSct() 関数を用いて buf に BPB を読み込む。

5 行 BPB の 11 バイト目からの 2 バイトを読みセクタ長を調べる。

8 行 13 バイト目からクラスタあたりのセクタ数が分かる。

11, 12 行 14 バイト目からの 2 バイト（予約セクタ）は、パーティション先頭から FAT 開始位置の間のセクタ数である。最低でも BPB を格納するために 1 セクタが予約されてい

リスト 23.16: ファイルのデータセクタを読む (readBlk())

```

1 // readBlk : 1セクタ読み出す
2 // 引数 blk : 読み込み対象の(クラスタ番号、セクタ番号)
3 //      buf : データを読み出すバッファ
4 public void readBlk(Blk blk, char[] buf) {
5     if (badBlk(blk)) panic("readBlk");          // 読めない場所を読もうとしている
6
7     if (blk.clst==ROOTDIR) {                     // ルートディレクトリ
8         ld32(dTmp, rootLba[0], rootLba[1]);      // ルートディレクトリ先頭の先頭
9         ld32(sTmp, 0, blk.sct);                  // ルートディレクトリ内
10        _add32(dTmp, sTmp);                        // セクタ番号を足す
11    } else {                                       // ルートディレクトリ以外
12        ld32(dTmp, 0, blk.clst-2);                // クラスタ番号を
13        _mul32(dTmp, sctPrClst);                  // セクタ番号に変換し
14        _add32(dTmp, dataLba);                    // データ領域開始位置を足す
15        ld32(sTmp, 0, blk.sct);                  // クラスタ内
16        _add32(dTmp, sTmp);                      // セクタ番号を足す
17    }
18    readSct(dTmp[0], dTmp[1], buf);              // セクタを読み込む
19 }

```

る。bpbLba に加え FAT の位置を計算する。

15 行 FAT の大きさをセクタ単位で求める。

19,20 行 FAT 数を読み出し FAT のセクタ数 (sctPrFAT) を掛ける。その結果を fatLba に加えることで、ルートディレクトリの位置 rootLba を計算する。

23,24 行 17 バイト目からの 2 バイトからルートディレクトリのエントリ数分かる。エントリは 32 バイトなのでセクタ (512 バイト) に 16 エントリ格納できる。16 で割ってセクタ数に換算し rootLba に加えることでデータ領域の開始位置 dataLba を求める。

26 行 バッファを解放して終了する。

• クラスタの読み書き

例としてクラスタを読む readBlk() をリスト 23.16 に示す。クラスタ番号とクラスタ内セクタ番号 (Blk 構造体で表現) をアドレスとして用い、セクタ単位でファイル本体を読む機能を提供する。クラスタ番号が ROOTDIR (値は 0x0001) の場合は、クラスタではなくルートディレクトリ領域のセクタを読む (7~10 行)。

普通のクラスタを読む場合はクラスタ番号からセクタ番号へ変換する計算を行う (12~16 行)。クラスタ番号から 2 を引いている (12 行) のは、普通のクラスタ番号は 2 から始まる (表 15.3 参照) からである。

セクタ番号が決まったら、mmcsapi クラス (デバイスドライバクラス) の readSct() を用いてセクタを buf に読み込む。

• クラスタチェーンの操作

リスト 23.17: データセクタのアドレスを進める (nextBlk())

```

1 // nextBlk : 次のセクタを求める
2 // 返回值 : true(blk を更新している)、false(EOF のため blk を更新していない)
3 // 引数 blk : 読み込み対象直前の(クラスタ番号、セクタ番号)
4 public boolean nextBlk(Blk blk) {
5     int clst = blk.clst;
6     int sct = blk.sct + 1;           // セクタを進める
7     if (clst==ROOTDIR) {           // ルートディレクトリの場合
8         if (sct>=ROOTSCT_MAX) return false; // 固定長のセクタ以上なら EOF
9     } else if (sct>=sctPrClst) {    // クラスタの最後まで来たら
10        clst = readFat(clst);        // クラスタを進める
11        if (_uCmp(clst,ENDCLST)>0) return false; // 終端番号なら EOF
12        sct = 0;                    // セクタ番号はリセット
13    }
14    blk.clst = clst;
15    blk.sct = sct;
16    return true;
17 }

```

FAT 上のクラスタチェーンを操作する三つの機能を準備している。allocClst() は、新しいクラスタをクラスタチェーンの最後に追加する。delClstChain() は先頭クラスタを指定してチェーン全体を削除する。nextBlk() は Blk 構造体のアドレス（セクタアドレス）を一つ進める。

クラスタチェーンを操作する例として、Blk 構造体のアドレスを一つ進める役割を持つ nextBlk() をリスト 23.17 に示す。6 行でセクタ番号を一つ進める。もしもセクタ番号がクラスタの最後まで進んでいれば、FAT を参照して次のクラスタ番号を求める。10 行の readFat() が FAT を読む関数である。

• FAT のキャッシュ

多くの FAT ファイルシステムの実装では FAT 全体を主記憶に常駐させている。しかし、メモリ容量が限られる TacOS では、FAT のセクタを一つだけキャッシュに読み込む方法をとっている。fatCache, fatLbaH, fatLbaL, fatIsDirty (図 23.1 参照) がキャッシュを管理するデータである。FAT のキャッシュに関連するプログラムの例として、readFat() 関数のソースプログラムをリスト 23.18 に示す

セクタサイズが 512 バイトの時、1 セクタに 16 ビット (2 バイト) のクラスタ番号を 256 個まで記録できる。6 行ではクラスタ番号から、そのクラスタ番号に対応するセクタ番号に変換している。7, 8 行でセクタ番号に FAT の開始アドレスを加算する 32 ビット演算を行った。

必要なセクタがキャッシュに格納されていない場合は 11 行に進む。flushFat() は、FAT のキャッシュがダーティな（読み込み時と変化している）場合だけ、キャッシュの内容をセクタに書き戻す。その後、12 行で目的のセクタをキャッシュに読み込む。

17 行でキャッシュ上のクラスタ番号の格納バイト位置を計算し、18 行でリトルエンディアンの 16

リスト 23.18: クラスタ番号で指定して FAT を読む (readFat())

```

1 // readFat   : FAT を読み、次のクラスタ番号を返す
2 // 返回值    : 次のクラスタ番号
3 // 引数 clst : 現在のクラスタ番号
4 int readFat(int clst) {
5     // clst が指すセクタアドレスに変換する
6     int s = (clst >> 8) & 0xff;           // FAT内セクタ番号
7     ld32(sTmp, 0, s);                     // セクタ番号を 32bit にする
8     _add32(sTmp, fatLba);                  // FAT の開始アドレスを加える
9
10    if(sTmp[0] != fatLbaH || sTmp[1] != fatLbaL){ // キャッシュにあるものと違うなら
11        flushFat();                         // キャッシュをフラッシュ
12        readSct(sTmp[0], sTmp[1], fatCache); // セクタをキャッシュに読み込む
13        fatLbaH = sTmp[0];                 // キャッシュ中のアドレスを更新
14        fatLbaL = sTmp[1];                 // キャッシュ中のアドレスを更新
15    }
16
17    int offs = (clst & 0xff) << 1;          // セクタ内オフセット
18    return wordLE(fatCache, offs);          // FAT から次のクラスタ番号を求める
19 }

```

ビットデータとして読み出す。

23.7 mmcspi クラス

マイクロ SD カードのデバイスドライバ・クラスである。カードの読み書き機能を提供する。リスト 23.19 にプログラムを示す。

restart() 関数 3 行は SPI ホストコントローラ (図 18.2 参照) の割込みハンドラである。C++ 言語は割り込みハンドラ用の関数型 interrupt をサポートしている。interrupt 型の関数は、割り込まれたプロセスのコンテキストを破壊しない。4 行の semV() は、セマフォを待ち合わせているプロセス (ファイルシステムサーバ) を起床させる。

initMmcSpi() 関数 8 行はデバイスドライバの初期化関数である。この関数はファイルシステムサーバの起動時に実行される。10 行で割込みハンドラ restart() を割込みベクタの 8 番目 (図 A.4 の uSD と表記されている部分) に登録している。11 行は割込みハンドラとプロセスの間で同期を取るために使用する初期値 0 のセマフォを割当てている。12 行では、I/O アドレスを指定して 16bit データを I/O ポートに出力する out() 関数を用いて、ハードウェアに初期化コマンドを発行している。13 行ではセマフォを用いてハードウェアの初期化が終わるのを待つ。初期化の終了は割込みで通知され 3 行の割込みハンドラが実行される。割込みハンドラは 4 行でセマフォを待っているプロセスを起こす。

readSct() 関数 20 行はデバイスドライバのセクタ読み出しルーチンである。セクタのアドレスは

リスト 23.19: マイクロ SD カードのデバイスドライバ

```

1  int sem;                                // 生成したセマフォ番号が入る
2
3  interrupt restart() {                   // 割込が発生したら
4      semV(sem);                          // 待っていたプロセスを起こす
5  }
6
7  // initMmcSpi : microSD カードの初期化
8  public void initMmcSpi() {
9      int[] Vector = _ItoA(0xffe0);      // 割り込みベクタ登録
10     Vector[8]=addrof(restart);
11     sem=newSem(0);                       // セマフォを生成
12     out(SD_CTRL, INIT | INT_ENA);       // microSD カードの初期化、割り込み許可
13     semP(sem);                          // ブロック
14 }
15
16 // readSct : ブロックを読み込む
17 // 引数 h : 読み込むブロックの上位ブロックアドレス
18 //      l : 読み込むブロックの下位ブロックアドレス
19 //      buf : データを読み込むバッファ
20 public void readSct(int h, int l, void[] buf) {
21     out(MEM_ADDR, _AtoI(buf));          // buf のアドレスを MEM_ADDR に格納
22     out(BLK_ADDR_H, h);                 // BLK_ADDR にブロックアドレスを格納
23     out(BLK_ADDR_L, l);
24     out(SD_CTRL, READ | INT_ENA);       // 読み込み開始指示、割り込み許可
25     semP(sem);                          // ブロック
26 }
27
28 // writeSct : ブロックを書き込む
29 // 引数 h : 書き込むブロックの上位ブロックアドレス
30 //      l : 書き込むブロックの下位ブロックアドレス
31 //      buf : 書き込む内容が格納されているバッファ
32 public void writeSct(int h, int l, void[] buf) {
33     out(MEM_ADDR, _AtoI(buf));
34     out(BLK_ADDR_H, h);
35     out(BLK_ADDR_L, l);
36     out(SD_CTRL, WRITE | INT_ENA);      // 書き込み開始指示、割り込み許可
37     semP(sem);                          // ブロック
38 }

```

LBA 方式で、上位 16bit (h) と下位 16bit (l) とに分けて渡される。buf はデータを読み出すバッファである。21 行でバッファのアドレスを SPI ホストコントローラのアドレスレジスタ (MEM_ADDR) に書き込む。22, 23 行で LBA を SPI ホストコントローラのレジスタに書き込み、24 行で SPI ホストコントローラの動作を開始させる。SPI ホストコントローラが DMA (16 ページ参照) を用いて、CPU の力を借りることなく目的のデータをメモリに転送する。その間、デバイスドライバを呼び出したプロセス (ファイルシステムサーバ) は 25 行で CPU を解放して割り込みを待つ。

writeSct() 関数 セクタ書き込みルーチンである。readSct() 関数とよく似ているので説明は省略する。

23.8 まとめ

本章では、TacOS のファイルシステムサーバを例に、FAT ファイルシステムの実装例を UML 図とプログラムリストを使って示した。ファイルシステムサーバはカーネルモードで実行されるプロセスであり、UML 図に示す幾つかのクラスにより構成される。

fs クラスは、サーバプロセスのメインループとスタブルーチンからなる。サーバプロセスは、メッセージを受信して処理を行い結果を返信する。クライアントプロセスは、ファイル操作のシステムコールを発行するために、スタブルーチンを呼び出す。スタブルーチンはサーバプロセスにメッセージを送り、システムコールの処理を依頼する。

fatSys クラスは、システムコールの処理を行うクラスである。open システムコール、read システムコール、remove システムコールを例に、プログラムリストを示して内容を解説した。

File クラスは、オープン中のファイルを管理・操作するためのクラスである。File クラスは、セクタの内容を保持するバッファを持ち、バイト単位のファイル操作機能を提供する。File クラスのインスタンスは、オープン中のファイル毎に作られる。オブジェクト指向言語ではない C++ 言語では、関数 (操作) にインスタンスの属性を表す File 構造体を渡す。

dirAccess クラスは、ディレクトリエントリの読み書き機能を提供する。DirEnt 構造体に目的のディレクトリエントリの位置を管理し、セクタのキャッシュを通してエントリ内容进行操作する。

blkFile クラスは、セクタを単位とするファイルの操作と、FAT の管理を行う。fatSys クラス、File クラス、dirAccess クラスは、blkFile クラスの機能を用いてバッファにファイルのセクタを読み出した後、FAT を操作したりする。

mmcspi クラスは、マイクロ SD カードを読み書きするデバイスドライバである。blkFile クラスが使用する。

第 VI 部

資料と文献

付録 A

TaC に関する資料

A.1 CPU の概要

TaC で使用できるデータの形式，CPU 内部のレジスタ構成，機械語命令について説明する．

A.1.1 データ形式

図 A.1 に TaC が扱うことができるデータ形式を示す．16 ビットの整数データと，16 ビットのアドレスデータの他に，8 ビットのデータを扱うことができる．16 ビットのデータは CPU の内部でもメモリや I/O でも使用できる．メモリや I/O の 16 ビットデータにアクセスする場合は偶数番地を用いる．8 ビットデータはメモリと I/O の読み書きだけに使用できる．メモリや I/O の 8 ビットデータにアクセスする場合は，CPU レジスタの下位 8 ビットが使用される．

A.1.2 CPU レジスタと PSW

図 A.2 に CPU 内部のレジスタなどを示す．レジスタはどれも 16 ビット幅である．CPU レジスタは，汎用の G0 から G11，フレームポインタとして使用する FP，カーネルモード用のスタックポインタ SSP，ユーザモード用のスタックポインタ USP からなる．これらは全て計算用にもアドレス用にも使用できる．FP，SSP，USP は，以下に説明する特別な意味も持っている．

FP はフレームポインタ相対アドレッシングモードで使用できる．このアドレッシングモードを用いると，スタックフレーム内のローカル変数や関数引数へ，1 ワードの機械語命令でアクセスできる．

SSP はカーネルモードで SP の位置にマップされスタックポインタとして使用される．USP はユーザモードで SP の位置にマップされスタックポインタとして使用される．USP は最後のレジスタとして常時マップされており，カーネルモードでも USP をアクセスすることができる．

PSW は PC と FLAG からなる．PC はプログラムカウンタのことである．FLAG には，計算結果で変化する V，C，S，Z と，割込み許可 E，カーネルモード P の各ビットがある．割込みが発生する

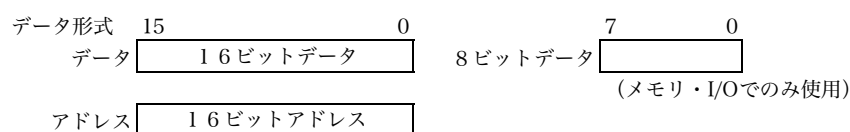


図 A.1: データ形式

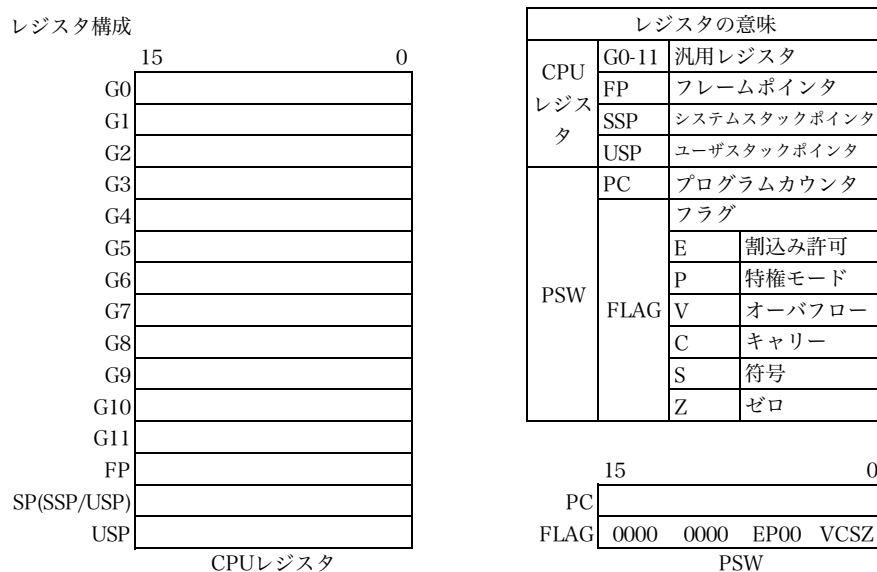


図 A.2: CPU 内部の記憶装置

と PC と FLAG が順にカーネルスタックに PUSH された後で、割込みが禁止されカーネルモードに切り換わる（E ビットが 0, P ビットが 1 になる）。

A.1.3 機械語命令

図 A.3 に TaC の機械語命令の一覧表を示す。IN, OUT, RETI, EI, DI, HALT は、カーネルモードでしか使用できない**特権命令**である。SVC 命令はシステムコールを発行するために SVC 割込みを発生する。

ほとんどの転送命令と計算命令で 8 種類のアドレッシング・モードが使用できる。Direct, Indexed, Immediate の三つのアドレッシング・モードを使用する場合は 2 ワードの機械語命令になる。他のアドレッシング・モードの場合は全て 1 ワード命令である。

Byte Register Indirect アドレッシング・モードだけが、メモリの 8 ビットデータをアクセスする。Byte Register Indirect アドレッシング・モードの ST 命令は、CPU レジスタの下位 8 ビットをメモリに書き込む。ST 以外の命令は、メモリから読み出した 8 ビットデータの上位に 00h を付加した 16 ビットデータに変換して使用する。

命令	ニーモニック	オペコード	アドレッシングモード (数値はステート数)										フラグ	説明
	命令 オペラント	OP Rd Rx	Drct	Index	Imm	FP Rlt	Reg	Imm4	Indr	B Indr	Othr	変化		
No Operation	NO	00h 0h 0h	--	--	--	--	--	--	--	--	3	×	何もしない	
Load	LD Rd,EA	08h Rd EA	7	7	5	7	4	4	6	6	--	×	Rd ← [EA]	
Load	LD Rd,FLAG	14h Rd 0h	--	--	--	--	--	--	--	--	4	×	Rd ← FLAG	
Store	ST Rd,EA	10h Rd EA	6	6	--	6	--	--	5	5	--	×	[Dsp] ← EA	
Add	ADD Rd,EA	18h Rd EA	7	7	5	7	5	4	6	6	--	○	Rd ← Rd + [EA]	
Subtract	SUB Rd,EA	20h Rd EA	7	7	5	7	5	4	6	6	--	○	Rd ← Rd - [EA]	
Compare	CMP Rd,EA	28h Rd EA	7	7	5	7	5	4	6	6	--	○	Rd - [EA]	
Logical And	AND Rd,EA	30h Rd EA	7	7	5	7	5	4	6	6	--	○	Rd ← Rd and [EA]	
Logical Or	OR Rd,EA	38h Rd EA	7	7	5	7	5	4	6	6	--	○	Rd ← Rd or [EA]	
Logical Xor	XOR Rd,EA	40h Rd EA	7	7	5	7	5	4	6	6	--	○	Rd ← Rd xor [EA]	
Add with Scale	ADDS Rd,EA	48h Rd EA	8	8	6	8	6	5	7	7	--	○	Rd ← Rd + [EA]*2	
Multiply	MUL Rd,EA	50h Rd EA	57	57	55	57	55	54	56	56	--	○	Rd ← Rd × [EA]	
Divide	DIV Rd,EA	58h Rd EA	73	73	71	73	71	70	72	72	--	○	Rd ← Rd / [EA]	
Modulo	MOD Rd,EA	60h Rd EA	73	73	71	73	71	70	72	72	--	○	Rd ← Rd % [EA]	
Multiply Long	MULL Rd,EA	680h Rd EA	57	57	55	57	55	54	56	56	--	○	(Rd+1,Rd) ← Rd × [EA]	
Divide Long	DIVL Rd,EA	70h Rd EA	73	73	71	73	71	70	72	72	--	○	Rd ← (Rd+1,Rd) / [EA], Rd+1 ← (Rd+1,Rd) % [EA]	
Shift Left Arithmetic	SHLA Rd,EA	80h Rd EA	8+n	8+n	6+n	8+n	6+n	5+n	7+n	7+n	--	○	Rd ← Rd << [EA]	
Shift Left Logical	SHLL Rd,EA	88h Rd EA	8+n	8+n	6+n	8+n	6+n	5+n	7+n	7+n	--	○	Rd ← Rd << [EA]	
Shift Right Arithmetic	SHRA Rd,EA	90h Rd EA	8+n	8+n	6+n	8+n	6+n	5+n	7+n	7+n	--	○	Rd ← Rd >> [EA]	
Shift Right Logical	SHRL Rd,EA	98h Rd EA	8+n	8+n	6+n	8+n	6+n	5+n	7+n	7+n	--	○	Rd ← Rd >>> [EA]	
Jump on Zero	JZ EA	A0h 0h EA	4/5	4/5	--	--	--	--	4/5	--	--	×	If (Z) PC ← EA	
Jump on Carry	JC EA	A0h 1h EA	4/5	4/5	--	--	--	--	4/5	--	--	×	If (C) PC ← EA	
Jump on Minus	JM EA	A0h 2h EA	4/5	4/5	--	--	--	--	4/5	--	--	×	If (S) PC ← EA	
Jump on Overflow	JO EA	A0h 3h EA	4/5	4/5	--	--	--	--	4/5	--	--	×	if (V) PC ← EA	
Jump on greater than	JGT EA	A0h 4h EA	4/5	4/5	--	--	--	--	4/5	--	--	×	If (not (Z or (S xor V))) PC ← EA	
Jump on greater or equal	JGE EA	A0h 5h EA	4/5	4/5	--	--	--	--	4/5	--	--	×	if (not (S xor V)) PC ← EA	
Jump on less or equal	JLE EA	A0h 6h EA	4/5	4/5	--	--	--	--	4/5	--	--	×	If (Z or (S xor V)) PC ← EA	
Jump on less than	JLT EA	A0h 7h EA	4/5	4/5	--	--	--	--	4/5	--	--	×	If (S xor V) PC ← EA	
Jump on Non Zero	JNZ EA	A0h 8h EA	4/5	4/5	--	--	--	--	4/5	--	--	×	If (not Z) PC ← EA	
Jump on Non Carry	JNC EA	A0h 9h EA	4/5	4/5	--	--	--	--	4/5	--	--	×	If (not C) PC ← EA	
Jump on Non Minus	JNM EA	A0h Ah EA	4/5	4/5	--	--	--	--	4/5	--	--	×	If (not S) PC ← EA	
Jump on Non Overflow	JNO EA	A0h Bh EA	4/5	4/5	--	--	--	--	4/5	--	--	×	If (not V) PC ← EA	
Jump on higher	JHI EA	A0h Ch EA	4/5	4/5	--	--	--	--	4/5	--	--	×	If (not (Z or C)) PC ← EA	
Jump on lower or same	JLS EA	A0h Eh EA	4/5	4/5	--	--	--	--	4/5	--	--	×	If (Z or C) PC ← EA	
Jump	JMP EA	A0h Fh EA	5	5	--	--	--	--	5	--	--	×	PC ← EA	
Call subroutine	CALL EA	A8h 0h EA	6	6	--	--	--	--	6	--	--	×	[--SP] ← PC, PC ← EA	
Input	IN Rd,EA	B0h Rd EA	7	--	--	--	--	--	6	6	--	×	Rd ← IO[EA]	
Output	OUT Rd,EA	B8h Rd EA	6	--	--	--	--	--	5	5	--	×	IO[EA] ← Rd	
Push Register	PUSH Rd	C0h Rd 0h	--	--	--	--	--	--	--	--	5	×	[--SP] ← Rd	
Pop Register	POP Rd	C4h Rd 0h	--	--	--	--	--	--	--	--	6	×	Rd ← [SP++]	
Return from Subroutine	RET	D0h 0h 0h	--	--	--	--	--	--	--	--	6	×	PC ← [SP++]	
Return from Interrupt	RETI	D4h 0h 0h	--	--	--	--	--	--	--	--	9	×	FLAG ← [SP++], PC ← [SP++]	
Enable Interrupt	EI	E0h 0h 0h	--	--	--	--	--	--	--	--	5	×	割込み許可	
Disable Interrupt	DI	E4h 0h 0h	--	--	--	--	--	--	--	--	5	×	割込み禁止	
Supervisor Call	SVC	F0h 0h 0h	--	--	--	--	--	--	--	--	12	×	システムコール	
Halt	HALT	FFh 0h 0h	--	--	--	--	--	--	--	--	5	×	CPU停止	

アドレッシングモード (上の表中EAの詳細) に付いて

アドレッシングモード	略記	ニーモニック (EA部分の標記方法)	命令フォーマット		EA(実効アドレス)の決め方	
			第1ワード	第2ワード	略記	解説
Direct	Drct	OP Rd,Dsp	OP+0 Rd0h	Dsp	[Dsp]	Dsp番地
Indexed	Index	OP Rd,Dsp,Rx	OP+1 RdRx	Dsp	[Dsp+Rx]	(Dsp+Rxレジスタの内容) 番地
Immediate	Imm	OP Rd,#Imm	OP+2 Rd0h	Imm	Imm	Immそのもの
FP Rerative	FP Rlt	OP Rd,Dsp4,FP	OP+3 RdD4	--	[Dsp4+FP]	(D4を符号拡張した値×2 + FPレジスタの内容)番地(D4=Dsp4/2)
Register	Reg	OP Rd,Rs	OP+4 RdRs	--	Rs	Rsレジスタの内容
4bit Signed Immediate	Imm4	OP Rd,#Imm4	OP+5 RdI4	--	Imm4	I4を符号拡張した値そのもの
Register Indirect	Indr	OP Rd,@Rx	OP+6 RdRx	--	[Rx]	Rxレジスタの内容番地
Byte Register Indirect	B Indr	OP Rd,@Rx	OP+7 RdRx	--	[Rx]	Rxレジスタの内容番地 (但し番地の内容は8bitデータ)
Other	Othr	OP Rd	OP Rd0h	--		なし
		OP	OP 0h0h	--		なし

注 4

※アセンブリ言語でDspとDsp4、ImmとImm4の標記は同じ (値によりアセンブラが自動判定)。

※FP相対で、Dsp4は-16～+14の偶数

注 1 : MULL、DIVL命令ではRdは偶数番号のレジスタ

注 2 : D4はDsp4(4bitディシプレースメント)の1/2の値

注 3 : I4はImm 4 (4bit即値)のこと

注 4 : アドレッシングモードによりOPの値が変化する

図 A.3: 命令表

A.2 メモリマップと I/O マップ

図 A.4 に TaC のメモリマップと I/O マップを示す。メモリや I/O は 8 ビット毎にアドレス付けされており、8 ビットデータ、16 ビットデータのどちらも読み書きできる。アドレッシング・モードによって、8 ビットデータと 16 ビットデータの区別をする。16 ビットデータは偶数アドレスを指定してアクセスしなければならない。

A.2.1 メモリ空間

TaC のメモリ空間は 0000h から FFFFh の 64KiB である。16 ビットデータは偶数アドレスからの 2 バイトに配置され、偶数アドレスを指定してアクセスする。16 ビットデータにアクセスするには、Byte Register Indirect モード**以外**のアドレッシング・モードを用いる。8 ビットデータにアクセスするには、Byte Register Indirect モードを用いる。

メモリ空間の最初から 56KiB は自由に使用できるメモリであり、ここに TacOS のカーネルやユーザプロセスがロードされる。E000h から EFFFh までは VRAM が配置されている。VRAM に ASCII コードを書き込むと対応する文字がディスプレイに表示される。VRAM のアドレスがディスプレイの表示位置に対応する。F000h から FFDFh は IPL (ROM) が配置される。IPL はマイクロ SD カードから TacOS を読み出して起動する。FFE0h から FFFFh は割込みベクタ領域である。16 種類の割込みに対応するハンドラの入口番地を TacOS がセットする。

A.2.2 I/O 空間

TaC の I/O 空間は 00h から FFh の 256B である。I/O 空間のアドレス幅は 8 ビットだが、IN, OUT 命令では I/O アドレスが 16 ビットで表現される。I/O アドレスの上位 8 ビットは 00h になるようにする。上位 8 ビットが 00h 以外になった場合の動作は保証されない。

メモリ空間と同様に 8 ビットデータと 16 ビットデータの両方を読み書きできる。8 ビットデータと 16 ビットデータの区別は、IN, OUT 命令のアドレッシングモードにより行う。I/O の 8 ビットデータにアクセスするには、Byte Register Indirect モードを用いる。

メモリマップ

+0番地		+1番地	
0000h 0002h 0004h ... DFFEh	RAM(56kB)		RAM
E000h ... EFFEh	予約 (アトリビュート)	VRAM(2kB)	
F000h ... FFDEh	IPL(4064B)		
FFE0h	Timer0		割り込みリンクタ
FFE2h	Timer1		
FFE4h	INT2		
FFE6h	INT3		
FFE8h	SIO 受信		
FFEAh	SIO 送信		
FFECh	PS2 受信		
FFEEh	PS2 送信		
FFF0h	uSD		
FFF2h	ADC		
FFF4h	不正 (奇数) アドレス		
FFF6h	上下限アドレス違反		
FFF8h	ゼロ除算(※1)		
FFFAh	特権違反 (※1)		
FFFCh	未定義命令 (※1)		
FFFEh	SVC (※1)		

※1：マイクロプログラムにより発生

I/Oマップ

+0番地		+1番地	
00h	Timer0(In:現在値/Out:周期)		I/O装置
02h	Timer0(In:フラグ/Out:コントロール)		
04h	Timer1(In:現在値/Out:周期)		
06h	Timer1(In:フラグ/Out:コントロール)		
08h	00H	SIO-Data	
0Ah	00H	SIO-Stat/Ctrl	
0Ch	00H	PS2-Data	
0Eh	00H	PS2-Stat/Ctrl	
10h	00H	uSD-Stat/Ctrl	
12h	uSD-MemAddr		
14h	uSD-BlkAddrH		
16h	uSD-BlkAddrL		
18h	00H	拡張ポート(In/Out)	
1Ah	00H	ADC参照電圧(Out)	
1Ch	00H	I/Oポート (予約)	
1Eh	00H	モード(In)	
20h	00H	ADC(CH0)	
22h	00H	ADC(CH1)	
24h	00H	ADC(CH2)	
26 h	00H	ADC(CH3)	
28h	空き	空き	
...	...		
F4h	下限アドレス		
F6h	上限アドレス		
F8h	データレジスタ(Out)/データSW(IN)		
FAh	アドレスレジスタ (IN)		
FCh	00H	ロータリー-SW(IN)	
FEh	00H	機能レジスタ(IN)	
			メモリ 保護
			コンソール

IPLルーチンのエントリーポイント

番地	関数	意味
F000h	_ipl0	IPLに戻る

IOポートの詳細

番地	名称	ビット構成	説明
02h	Timer0 コントロール	1000 ... 000S	I=Enable Interrupt, S=Start
04h	Timer1 コントロール	1000 ... 000S	I=Enable Interrupt, S=Start
0Bh	SIO-Stat (in)	TR00 0000	T=Transmitter Ready, R=Reciver Ready
0Bh	SIO-Ctrl (out)	TR00 0000	T=Enable Transmitter Interrupt, R=Enable Reciver Interrupt
0Fh	PS2-Stat (in)	TR00 0000	T=Transmitter Ready, R=Reciver Ready
0Fh	PS2-Ctrl (out)	TR00 0000	T=Enable Transmitter Interrupt, R=Enable Reciver Interrupt
11h	uSD-Ctrl	0000 EIRW	E=INT_ENA, I=INIT, R=READ, W=WRITE
13h	uSD-Stat	0000 IE00	I=IDLE, E=ERROR
1Fh	モード	0000 00MM	MM: 00=TeC, 01=TaC, 10=DEMO1, 11=DEMO2
FDh	ロータリー-SW(IN)	000S SSSS	sssss: 0=G0, 1=G1, ..., 11=G11, 12=FP, 13=SP, 14=PC, 15=FLAG, 16=MD, 17=MA
FFh	機能レジスタ(IN)	0000 FFFF	FFFF: 0=ReadReg, 1=WriteReg, 13=ReadMem, 14=WriteMem

図 A.4: メモリマップと I/O マップ

付録 B

TacOS のファイルフォーマット

TacOS と C--言語サポートユーティリティ^{*1}が使用する 3 種類のバイナリ形式ファイルの内容について解説する。C--言語サポートユーティリティは、クロスアセンブラ `as--`、クロスリンカ `ld--`、クロスローダ `objbin--`、実行形式ファイル作成プログラム `objexe--` からなる。

B.1 .o 形式ファイル

`as--`が出力する再配置可能な機械語ファイル形式である。`ld--`は、複数の.o 形式ファイルを入力し一つに結合する。結合されたファイルも同じ.o 形式ファイルになる。

B.1.1 ファイル形式

図 B.1 に.o 形式ファイルのフォーマットを示す。なお、TaC の 1 ワードは 16 ビットである。

ヘッダ (8 ワード)
テキストセグメント
データセグメント
テキストリロケーション
データリロケーション
シンボルテーブル
文字列テーブル

図 B.1: .o ファイルフォーマット

^{*1} 詳細は <https://github.com/tctsigemura/Util--> を参照のこと。

B.1.2 ヘッダ

.o 形式ファイルのヘッダは次の構造体により定義される。ただし、ここで `uint` 型は符号無しの 16 ビット整数型である。

```
struct ObjHdr {
    uint magic;        // マジックナンバー(0x0107)
    uint text;         // テキストセグメントサイズ(バイト単位)
    uint data;         // 初期化データセグメントサイズ(バイト単位)
    uint bss;          // 非初期化データセグメント(BSS)サイズ(バイト単位)
    uint syms;         // シンボルテーブルサイズ(バイト単位)
    uint entry;        // 常に 0
    uint trsize;       // テキストリロケーションサイズ(バイト単位)
    uint drsize;       // データリロケーションサイズ(バイト単位)
};
```

B.1.3 リロケーションレコード

テキストリロケーション、データリロケーション領域には、再配置情報を記録したリロケーションレコードの表が格納される。リロケーションレコードは次の構造体で定義される。レコードは 2 ワード長で、第 1 ワードが再配置時に書き換えが必要なポインタのセグメント中アドレス、第 2 ワードは上位 2 ビットがポインタの種類 (type) データ、下位 14 ビットがポインタ値を格納するシンボルのシンボルテーブル上の添字 (idx) データである。

```
struct ObjRel {
    uint addr;         // 再配置すべきポインタのセグメント内アドレス
    uint type: 2,      // ポインタの型
        idx: 14;      // シンボルテーブルのポインタが登録されている位置
};

// type の意味
#define UNDEF 0      // 未定義
#define TEXT 1      // テキストセグメント
#define DATA 2     // データセグメント
#define BSS 3       // コモン
```

B.1.4 シンボルテーブル

シンボルテーブルは、シンボルとアドレスを対応付けを行う。アセンブラが処理したシンボル (ラベル) の中で EQU ラベルを除くもの全てがシンボルテーブルに出力される。また、アセンブラのソースプログラムのファイル名もシンボルテーブルに出力され、`objexe--`、`objbin--` プログラムがファイル名とともにエラー表示ができるようにしている。シンボルは 1 文字目によって意味付けがされる。意味は、「@: ファイル名」、「.: ローカル名」、それ以外はグローバルな名前になる。シンボルテーブルを構成するシンボルレコードの構造を次に示す。

シンボルレコードは 2 ワード長である。第 1 ワードの上位 2 ビットがシンボルの種類 (type) を表

```

struct Symbol {
    uint type: 2,      // シンボルの型
        sIdx:14;      // シンボル名称の文字列テーブル上の位置
    uint val;          // シンボルの値
};

// type の意味
#define UNDEF 0      // 未定義
#define TEXT  1      // テキストセグメント
#define DATA 2      // データセグメント
#define BSS   3      // コモン

```

し、下位 14 ビットが文字列テーブル上でシンボルの綴が格納されている場所を表す添字データ (sIdx) を格納する。第 2 ワード (val) はシンボルの値をセグメント内オフセットで表す。ただし、未定義 (UNDEF) シンボルの場合は 0、コモン (BSS) シンボルの場合は領域のサイズを格納する。

ld--は、複数の入力ファイル中に同名のコモンシンボルを発見した場合、それらを一つの領域に重ね合わせる。このとき、領域のサイズは重ね合わせたシンボルの中で最大のものになる。また、一つのデータセグメントシンボルと一つ以上のコモンシンボルが見つかった場合は、データセグメントシンボルに集約する。未定義シンボル同士は一つの未定義シンボルに、未定義シンボルと他の種類のシンボルは未定義ではない方のシンボルに集約する。これ以外に同名のシンボルが見つかった場合は、エラー (シンボルの 2 重定義) になる。

B.1.5 文字列テーブル

文字列テーブルはシンボルの綴を格納する。文字列テーブルの内容は C--言語文字列の繰返である。C--言語文字列は、「\0」で終端された 8 ビットの文字コード配列である。ヘッダに文字列表のサイズは格納されていないので、ファイルサイズから文字列テーブルのサイズを知る必要がある。

シンボルテーブルに同じ綴のシンボルが複数ある場合 (「.」で始まるローカルシンボルは同じ綴の可能性はある) は、メモリの節約のため、複数のシンボルレコードで同じ文字列表エントリーを共用する。C--コンパイラが自動的に生成するローカルラベルは、毎回、同一のパターンなので、多くのシンボルレコードで文字列表エントリーが共用される。

B.2 .exe 形式ファイル

TacOS の実行可能なアプリケーションプログラムファイルである。内容は、再配置可能な機械語ファイル形式から、実行時に不要な情報を取り除いたものである。プログラムをロードする処理を簡単にしカーネルを小さくするために、.o を単純化したファイル形式を準備した。objexe--は、.o 形式のファイルの一つを入力して、.exe ファイルの一つを出力する。未定義シンボルを含む.o 形式ファイルは、.exe 形式ファイルに変換することができない。

B.2.1 ファイル形式

図 B.2 に.exe 形式ファイルのフォーマットを示す。.o 形式ファイルを単純化したファイル形式である。

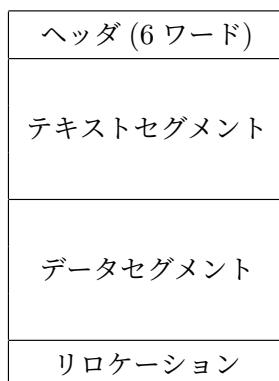


図 B.2: .exe ファイルフォーマット

B.2.2 ヘッダ

.exe 形式ファイルのヘッダは次の構造体により定義される。

```
struct ExeHdr {  
    uint magic;      // マジックナンバー(0x0108)  
    uint text;       // テキストセグメントサイズ(バイト単位)  
    uint data;       // 初期化データセグメントサイズ(バイト単位)  
    uint bss;        // 非初期化データセグメント(BSS)サイズ(バイト単位)  
    uint rsize;      // リロケーションサイズ(ワード単位)  
    uint stkSize;    // ユーザスタックサイズ(バイト単位)  
};
```

B.2.3 リロケーションレコード

リロケーション領域には、再配置情報を記録したリロケーションレコードの表が格納される。リロケーションレコードは次の構造体で定義される。レコードは 1 ワード長で、再配置時に書き換えが必要なポインタの格納アドレスを表現する。

```
struct ExeRel {  
    uint addr;       // 再配置すべきポインタのアドレス(バイト単位)  
};
```

B.3 .bin 形式ファイル

ロードアドレスが確定した機械語プログラムを格納するためのファイル形式である。この形式のプログラムはメモリにロードするだけで実行できる。objbin--プログラムによって.o形式ファイルを.bin形式ファイルに変換する。未定義シンボルを含む.o形式ファイルは、.bin形式ファイルに変換することができない。

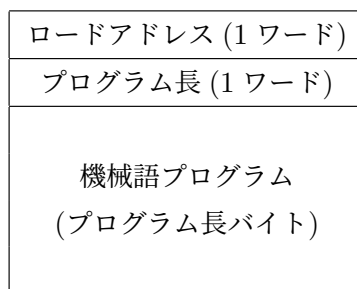


図 B.3: .bin ファイルフォーマット

B.3.1 ファイル形式

8 ビット版の TeC で使用してきた .bin 形式ファイルを単純に 16 ビットに拡張した形式のファイルである。図 B.3 にファイルの形式を示す。第 1 ワードが機械語プログラムのロードアドレス、第 2 ワードが機械語プログラムの長さ (バイト単位) を表現する。第 3 ワードから先は機械語プログラム本体である。プログラム本体は、テキストセグメント、初期化データセグメント、非初期化セグメントを結合したものである。非初期化セグメントは、0x00 で満たされている。

.bin 形式ファイルはリロケーションレコードを持たず、ロードアドレスが確定した機械語プログラムを格納する。IPL プログラム (ipl.bin)、OS カーネルプログラム (kernel.bin) が、この形式のプログラムファイルである。

参考文献

- [1] 重村哲至, 古川達也, 相知政司, 林 敏浩: コンソールパネルを持つ機械語教育用マイコンの開発と授業への応用, 情報処理学会論文誌, Vol.48, No.9, pp.3318–3327 (2007) .
- [2] ウキペディア, OS/360, <https://ja.wikipedia.org/wiki/OS/360> (2017.10.03 閲覧)
- [3] ウキペディア, MVS, https://ja.wikipedia.org/wiki/Multiple_Virtual_Storage (2017.10.03 閲覧)
- [4] ウキペディア, OS/390, <https://ja.wikipedia.org/wiki/OS/390> (2017.10.03 閲覧)
- [5] ウキペディア, z/OS, <https://ja.wikipedia.org/wiki/Z/OS> (2017.10.03 閲覧)
- [6] ウキペディア, UNIX (「UNIX および UNIX 系システムの系統図」を含む), <https://ja.wikipedia.org/wiki/UNIX> (2017.10.03 閲覧)
- [7] ウキペディア, Solaris, <https://ja.wikipedia.org/wiki/Solaris> (2017.10.03 閲覧)
- [8] ウキペディア, AIX, <https://ja.wikipedia.org/wiki/AIX> (2017.10.03 閲覧)
- [9] ウキペディア, Mach, <https://ja.wikipedia.org/wiki/Mach> (2017.10.03 閲覧)
- [10] ウキペディア, BSD の子孫, <https://ja.wikipedia.org/wiki/BSD%E3%81%AE%E5%AD%90%E5%AD%AB> (2017.10.03 閲覧)
- [11] ウキペディア, BSD, <https://ja.wikipedia.org/wiki/BSD> (2017.10.04 閲覧)
- [12] ウキペディア, 386BSD, <https://ja.wikipedia.org/wiki/386BSD> (2017.10.04 閲覧)
- [13] ウキペディア, FreeBSD, <https://ja.wikipedia.org/wiki/FreeBSD> (2017.10.03 閲覧)
- [14] ウキペディア, FreeNAS, <https://ja.wikipedia.org/wiki/FreeNAS> (2017.10.03 閲覧)
- [15] ウキペディア, NEXTSTEP, <https://ja.wikipedia.org/wiki/NEXTSTEP> (2017.10.03 閲覧)
- [16] ウキペディア, Classic Mac OS, https://ja.wikipedia.org/wiki/Classic_Mac_OS (2017.10.03 閲覧)
- [17] ウキペディア, ダイナブック, <https://ja.wikipedia.org/wiki/ダイナブック> (2017.10.03 閲覧)
- [18] ウキペディア, macOS, <https://ja.wikipedia.org/wiki/MacOS> (2017.10.03 閲覧)
- [19] ウキペディア, iOS (アップル), [https://ja.wikipedia.org/wiki/IOS_\(アップル\)](https://ja.wikipedia.org/wiki/IOS_(アップル)) (2017.10.03 閲覧)
- [20] ウキペディア, Linux, <https://ja.wikipedia.org/wiki/Linux> (2017.10.03 閲覧)
- [21] ウキペディア, Andriod, <https://ja.wikipedia.org/wiki/Android> (2017.10.03 閲覧)
- [22] ウキペディア, MS-DOS, <https://ja.wikipedia.org/wiki/MS-DOS> (2017.10.03 閲覧)

- [23] ウキペディア, Microsoft Windows (「Windows ファミリー系統図」含む), https://ja.wikipedia.org/wiki/Microsoft_Windows (2017.10.03 閲覧)
- [24] ウキペディア, IBM PC, https://ja.wikipedia.org/wiki/IBM_PC (2017.10.04 閲覧)
- [25] ウキペディア, UNIX System V, https://ja.wikipedia.org/wiki/UNIX_System_V (2017.10.04 閲覧)
- [26] 重村哲至, 情報電子工学科電算機室における PC-UNIX の歴史, <http://www2.tokuyama.ac.jp/giga/Sigemura/Public/IeNet/history.html> (2017.10.03 閲覧)
- [27] Linux kernel release 1.0, <https://www.kernel.org/pub/linux/kernel/v1.0/linux-1.0.tar.gz> (2017.10.04)
- [28] Andrew S. Tanenbaum, Herbert Bos: “The Third Generation(1965–1980):ICs and Multiprogramming”, Modern Operating Systems (4th Edition), pp.9-14, Pearson Education,Inc (2014).
- [29] Andrew S. Tanenbaum, Herbert Bos: “The Fourth Generation(1980–Present):Personal Computers”, Modern Operating Systems (4th Edition), pp.15–19, Pearson Education,Inc (2014).
- [30] Alan C. Kay: “A Personal Computer for Children of All Ages”, Proceeding ACM '72 Proceedings of the ACM annual conference - Volume 1 Article No 1 (1972).
- [31] アラン・ケイ:すべての年齢の「子供たち」のためのパーソナルコンピュータ, 阿部和広, 小学生からはじめるわくわくプログラミング, pp.130–141, 日経 BP 社 (2013).
- [32] アラン・ケイ: Dynabook とは何か? 「すべての年齢の「子供たち」のためのパーソナルコンピュータ」の後日談, 阿部和広, 小学生からはじめるわくわくプログラミング, pp.142–149, 日経 BP 社 (2013).
- [33] 師尾 潤他:スーパーコンピュータ「京」のオペレーティングシステム, <http://img.jp.fujitsu.com/downloads/jp/jmag/vol63-3/paper07.pdf> (2017.10.03 閲覧), 富士通 (2012).
- [34] Marshall Kirk McKusick, George V. Neville-Neil, Robert N. M. Watson: “The Zettabyte Filesystem, ” The Design and Implementation of the FreeBSD Operating System Second Edition, Pearson Education,Inc,pp.523-550 (2015).
- [35] Andrew S. Tanenbaum, Herbert Bos: “INTRODUCTION”, Modern Operating Systems (4th Edition), pp.1-3, Pearson Education,Inc (2014).
- [36] Andrew S. Tanenbaum, Herbert Bos: “VIRTUALIZATION AND THE CLOUD”, Modern Operating Systems (4th Edition), pp.471-516, Pearson Education,Inc (2014).
- [37] ヴィエムウェア株式会社: “VMware 徹底入門 第3版”, 廣済堂 (2012).
- [38] 仮想ハードディスクイメージのダウンロード, <https://www.ubuntulinux.jp/download/ja-remix-vhd> (2017.10.19 閲覧), Ubuntu Japanese Team (2012).
- [39] Andrew S. Tanenbaum, Herbert Bos: “Thread Usage”, Modern Operating Systems (4th Edition), pp.97-102, Pearson Education,Inc (2014).
- [40] ウキペディア, ハイパースレッディング・テクノロジー, <https://ja.wikipedia.org/wiki/%E3%83%8F%E3%82%A4%E3%83%91%E3%83%BC%E3%82%B9%E3%83%AC%E3%83%83%E3%83%87%E3%82%A3%E3%83%B3%E3%82%B0%E3%83%BB%E3%83%86%E3%82%AF%E3%83%8E%E3%83%AD%E3%82%B8%E3%83%BC> (2017.11.02 閲覧)

-
- [41] B.H.Liskov, S.N.Zilles : “Programming with Abstract Data Type”, SIGPLAN Notices, 9, 4, pp.50-59 (1974).
- [42] Abraham Silberschatz, Peter Baer Galvin, Greg Gagne : “Memory Allocation”, Operating System Concepts (9th Edition), pp.362-363, John Wiley & Sons,Inc (2013).
- [43] John H. Crawford, Patrick P. Gelsinger : “ベースとリミット”, 80386 プログラミング, 工学社, pp.413-414 (1988) .
- [44] John H. Crawford, Patrick P. Gelsinger : “セグメント部：セグメント・レジスタ”, 80386 プログラミング, 工学社, pp.48-50 (1988) .
- [45] John H. Crawford, Patrick P. Gelsinger : “デスクリプタ用の裏レジスタ”, 80386 プログラミング, 工学社, pp.420-421 (1988) .
- [46] Albert Chang, Mark F. Mergen : “801 storage: architecture and programming”, ACM Transactions on Computer Systems, 6, 1, pp.28-50 (1988) .
- [47] Marshall Kirk McKusick, George V. Neville-Neil, Robert N. M. Watson : “Execution of a File”, The Design and Implementation of the FreeBSD Operating System Second Edition, Pearson Education,Inc,pp.262-263 (2015).
- [48] Andrew S. Tanenbaum, Herbert Bos : “The WSClock Page Replacement Algorithm”, Modern Operating Systems (4th Edition), pp.219-221, Pearson Education,Inc (2014).
- [49] ウキペディア, Apple File System, https://ja.wikipedia.org/wiki/Apple_File_System (2018.09.11 閲覧)
- [50] Marshall Kirk McKusick, George V. Neville-Neil, Robert N. M. Watson : “RAIDZ”, The Design and Implementation of the FreeBSD Operating System Second Edition, Pearson Education,Inc,pp.540-541 (2015).
- [51] Marshall Kirk McKusick, George V. Neville-Neil, Robert N. M. Watson : “Deduplication”, The Design and Implementation of the FreeBSD Operating System Second Edition, Pearson Education,Inc,pp.545-546 (2015).

オペレーティングシステム (PDF 版)

発行日 2018年10月15日 初版
2025年9月3日 Ver. 1.0.13

著 者 重村 哲至

発 行 徳山工業高等専門学校 情報電子工学科
〒745-8585 山口県周南市学園台 3538
0834-29-6304
sigemura@tokuyama.ac.jp

ISBN978-4-9910528-1-1 C3055